

aggregator.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Aggregator
5 # Description:
6 #   Централизованный агрегатор телеметрии, синхронизирующий фо
7 #   новый сбор данных
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.aggregator import T
11 #   elemetryAggregator
12 #   service = TelemetryAggregator()
13 #
14 # File: aggregator.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Централизованный агрегатор телеметрии, синхронизирующий фон
24 овый сбор данных"""
25
26 import os
27 import threading
28 import time
29 from datetime import datetime, timedelta, timezone
30 from typing import Any, Dict, List, Optional
```

```

30
31 try:
32     from logger_ai.logger import logger
33 except ImportError:
34     from logger import logger
35 from apps.windows.telemetry.telemetry_config import TelemetryC
onfigManager
36 from apps.windows.telemetry.sensor_collector import SensorColl
ector
37 from apps.windows.telemetry.file_collector import FileCollecto
r
38 from apps.windows.telemetry.json_logger import TelemetryJsonLo
gger
39 from apps.windows.telemetry.sqlite import TelemetryStorage
40
41
42 class TelemetryAggregator:
43     """Централизованный агрегатор телеметрии и сенсорного сбор
а в базу данных и лог.
44
45     Осуществляет периодический сбор аппаратных метрик (через :
class:`SensorCollector`)
46     и файловых событий (через :class:`FileCollector`) с сохран
ением сырых измерений
47     в базу данных SQLite (:class:`TelemetryStorage`), а также
выполняет периодическую
48     почасовую агрегацию метрик.
49
50     Примечание:
51     Экземпляр :class:`TelemetryJsonLogger` сохранён в `__i
nit__` (атрибут `self.logger`)
52     для поддержки явного внедрения зависимостей (DI) и всп
омогательного/резервного
53     файлового JSON-логирования, в то время как основным по
стоянным хранилищем
54     телеметрии выступает SQLite база данных (:class:`Telem
etryStorage`).
55     """

```

```

56 |
57 |     def __init__(
58 |         self,
59 |         config_manager: Optional[TelemetryConfigManager] = Non
60 |     e,
61 |         log_dir: Optional[str] = None,
62 |         storage: Optional[TelemetryStorage] = None,
63 |         sensor_collector: Optional[SensorCollector] = None,
64 |         file_collector: Optional[FileCollector] = None,
65 |         telemetry_logger: Optional[TelemetryJsonLogger] = None
66 |     ,
67 |     ) -> None:
68 |         """Инициализирует агрегатор телеметрии.
69 |
70 |         Args:
71 |             config_manager: Менеджер конфигурации (если None,
72 | создается новый).
73 |             log_dir: Директория для логов (по умолчанию: %APPD
74 | ATA%\AI-Breadboard\apps\logs).
75 |             storage: Экземпляр постоянного SQLite хранилища Te
76 |lemetryStorage.
77 |             sensor_collector: Коллектор аппаратных сенсоров.
78 |             file_collector: Коллектор файловых событий.
79 |             telemetry_logger: Логгер JSON-потока телеметрии.
80 |         """
81 |         self.config_manager = config_manager or TelemetryConfi
82 |gManager()
83 |         self.log_dir = log_dir or self._get_default_log_dir()
84 |         self.storage = storage or TelemetryStorage.get_instanc
85 |e()
86 |
87 |         # Инициализация коллекторов с поддержкой явного внедре
88 |ния зависимостей (DI)
89 |         self.sensor_collector = sensor_collector or SensorColl
90 |ector(config_manager=self.config_manager)
91 |         self.file_collector = file_collector or FileCollector(
92 |             watch_dirs=self.config_manager._config.get("watch_
93 | directories", ["C:\\Users\\"])

```

```

84         )
85         self.logger = telemetry_logger or TelemetryJsonLogger(
86             log_dir=self.log_dir,
87             filename=self.config_manager._config.get("log_file
name", "ai_sensors_polls.json"),
88             max_file_size_mb=self.config_manager._config.get("
max_file_size_mb", 100),
89         )
90
91         self._running = False
92         self._thread: Optional[threading.Thread] = None
93         self._stop_event = threading.Event()
94
95         self._last_measurements: Dict[str, Any] = {}
96         self._measurement_count = 0
97         self._start_time = datetime.now(timezone.utc)
98         self._last_aggregation = self._start_time
99         self._last_audit_check = self._start_time
100        self._audit_check_interval = 3600.0 # Проверка аудито
в раз в час
101
102        # Ленивая инициализация чекера аудитов
103        self._audit_checker = None
104
105        @staticmethod
106        def _get_default_log_dir() -> str:
107            """Возвращает путь к директории логов по умолчанию.
108
109            Returns:
110                str: Путь к лог-директории.
111            """
112            appdata = os.environ.get("APPDATA", os.path.expanduser
("~\\AppData\\Roaming"))
113            return os.path.join(appdata, "AI-Breadboard", "apps",
"windows", "telemetry", "logs")
114
115        def start(self) -> bool:
116            """Запускает фоновый поток сбора телеметрии.

```

```
117 |
118 |     Returns:
119 |         bool: True если агрегатор успешно запущен.
120 |     """
121 |     if self._running:
122 |         logger.warning("Агрегатор телеметрии уже запущен."
123 | )
124 |         return False
125 |
126 |     self._stop_event.clear()
127 |     self._running = True
128 |
129 |     self._thread = threading.Thread(
130 |         target=self._worker_loop,
131 |         name="TelemetryAggregatorWorker",
132 |         daemon=True,
133 |     )
134 |     self._thread.start()
135 |
136 |     logger.info("Агрегатор телеметрии запущен")
137 |     return True
138 |
139 | def stop(self) -> bool:
140 |     """Останавливает фоновый поток сбора телеметрии.
141 |
142 |     Returns:
143 |         bool: True если агрегатор был остановлен.
144 |     """
145 |     if not self._running:
146 |         return False
147 |
148 |     self._running = False
149 |     self._stop_event.set()
150 |
151 |     if self._thread and self._thread.is_alive():
152 |         self._thread.join(timeout=3.0)
153 |
154 |     self._thread = None
```

```

154         self.file_collector.stop()
155         logger.info("Агрегатор телеметрии остановлен")
156         return True
157
158     def _worker_loop(self) -> None:
159         """Основной рабочий цикл агрегатора."""
160         while not self._stop_event.is_set():
161             try:
162                 self.poll_once()
163             except Exception as e:
164                 logger.error(f"Ошибка при сборе телеметрии: {e
165                                }")
166
167             min_interval = self._get_min_interval()
168             sleep_time = max(0.1, min_interval)
169
170             if self._stop_event.wait(timeout=sleep_time):
171                 break
172
173     def _get_min_interval(self) -> float:
174         """Возвращает минимальный интервал среди всех сенсоров
175         , учитывая быстрый режим.
176
177         Если с момента запуска прошло меньше fast_duration_days,
178         используется fast_interval.
179         Иначе - обычный минимум среди включенных сенсоров.
180         """
181         now = datetime.now(timezone.utc)
182         fast_days = self.config_manager.get_fast_duration_days
183         ()
184         if (now - self._start_time).days < fast_days:
185             return self.config_manager.get_fast_interval()
186         # Обычная логика
187         intervals = []
188         for sensor_name in self.config_manager.get_enabled_sen
189             sors():
190             interval = self.config_manager.get_sensor_interval
191             (sensor_name)

```

```

186         intervals.append(interval)
187     if not intervals:
188         return self.config_manager._default_interval
189     return min(intervals)
190
191     def poll_once(self) -> None:
192         """Проводит единовременный сбор всей телеметрии."""
193         self._measurement_count += 1
194         now_iso = datetime.now(timezone.utc).isoformat()
195
196         hw_snapshot = {}
197         file_events = []
198         try:
199             hw_snapshot = self.sensor_collector.get_hardware_s
200             napshot()
201             sensor_items = []
202             for s in hw_snapshot.get('sensors', []):
203                 val_entry = s.get('values', [{}])[0]
204                 sensor_items.append({
205                     'id': s.get('id'),
206                     'hardware_name': s.get('hardware_name'),
207                     'hardware_type': s.get('hardware_type'),
208                     'sensor_category': s.get('sensor_category'
209                     ),
210                     'sensor_name': s.get('sensor_name'),
211                     'unit': s.get('unit'),
212                     'value': val_entry.get('num')
213                 })
214             if sensor_items:
215                 self.storage.save_sensor_polls_batch(sensor_it
216                 ems, timestamp=now_iso)
217             except Exception as e:
218                 logger.error(f'Ошибка при сборе и сохранении сенсо
219                 ров: {e}')
220
221             try:
222                 file_events = self.file_collector.get_recent_event
223                 s()

```

```

219 |         except Exception as e:
220 |             logger.error(f'Ошибка при сборе файловых событий:
    | {e}')
221 |
222 |         self._last_measurements = {
223 |             'hardware': hw_snapshot,
224 |             'file_events': file_events,
225 |             'timestamp': now_iso,
226 |         }
227 |
228 |         # После обычного поллинга проверяем необходимость часо
    | вой агрегации
229 |         now = datetime.now(timezone.utc)
230 |         fast_days = self.config_manager.get_fast_duration_days
    | ()
231 |         if (now - self._start_time).days >= fast_days:
232 |             if (now - self._last_aggregation).total_seconds()
    | >= self.config_manager.get_aggregation_interval():
233 |                 self._aggregate_hourly()
234 |                 self._last_aggregation = now
235 |
236 |         # Периодическая проверка критических аудитов
237 |         if (now - self._last_audit_check).total_seconds() >= s
    | elf._audit_check_interval:
238 |             self._check_audits_periodic()
239 |             self._last_audit_check = now
240 |
241 |     def _check_audits_periodic(self) -> None:
242 |         """Периодическая проверка критических состояний систем
    | ы.
243 |
244 |         Проверяет:
245 |         - Pending reboot (требуется перезагрузка)
246 |         - Критическую загрузку CPU/RAM
247 |         - Проблемные драйверы (опционально)
248 |         """
249 |         try:
250 |             # Ленивая инициализация чекера

```



```

251         if self._audit_checker is None:
252             from .audit_startup_checker import AuditStartu
pChecker
253             self._audit_checker = AuditStartupChecker()
254
255             result = self._audit_checker.check_startup_health(
256                 check_integrity=True,
257                 check_performance=True,
258                 check_drivers=False,
259                 check_eventlog=False
260             )
261
262             if not result.is_healthy:
263                 logger.error(
264                     f"⚠️ AUDIT CHECK: Критические проблемы! "
265                     f"Critical: {result.critical_count}, Warni
ngs: {result.warning_count}"
266                 )
267                 for finding in result.findings:
268                     severity = finding.get('severity', 'unknow
n').upper()
269                     domain = finding.get('domain', 'unknown')
270                     title = finding.get('title', 'No title')
271                     logger.warning(f"  - [{severity}] {domain}
: {title}")
272
273                 # Сохраняем инцидент в БД
274                 try:
275                     self.storage.save_event(
276                         event_type='audit_critical',
277                         event_details={
278                             'critical_count': result.critical_
count,
279                             'warning_count': result.warning_co
unt,
280                             'findings': result.findings
281                         },
282                         severity='critical' if result.critical

```

```

    _count > 0 else 'warning'
283         )
284         except Exception as save_err:
285             logger.debug(f"Не удалось сохранить инциде
инт аудита: {save_err}")
286
287             elif result.warning_count > 0:
288                 logger.info(
289                     f"  AUDIT CHECK: Предупреждения: {result.w
arning_count} "
290                     f"({result.duration_ms}ms)"
291                 )
292             else:
293                 logger.debug(f"AUDIT CHECK: OK ({result.durati
on_ms}ms)")
294
295         except Exception as e:
296             logger.warning(f"Ошибка при периодической проверке
аудитов: {e}")
297
298     def _aggregate_hourly(self) -> None:
299         """Агрегирует данные сенсоров за последний полный час
и сохраняет их в таблицу агрегатов."""
300         # Вычисляем границы последнего полного часа
301         now = datetime.now(timezone.utc)
302         period_end = now.replace(minute=0, second=0, microseco
nd=0)
303         period_start = period_end - timedelta(hours=1)
304
305         # Запрос к базе: агрегируем по sensor_id за указанный
период
306         query = """
307             SELECT sensor_id,
308                    AVG(value) as avg_value,
309                    MIN(value) as min_value,
310                    MAX(value) as max_value,
311                    COUNT(*) as count
312             FROM sensor_polls

```

```

313         WHERE timestamp >= ? AND timestamp < ?
314         GROUP BY sensor_id
315         """
316         try:
317             with self.storage._lock, self.storage._get_connection() as conn:
318                 cursor = conn.cursor()
319                 cursor.execute(query, (period_start.isoformat(), period_end.isoformat()))
320                 rows = cursor.fetchall()
321                 aggregates = []
322                 for row in rows:
323                     sensor_id, avg_val, min_val, max_val, cnt
324                     = row
325                     aggregates.append({
326                         "sensor_id": sensor_id,
327                         "period_start": period_start.isoformat(),
328                         "period_end": period_end.isoformat(),
329                         "avg": avg_val,
330                         "min": min_val,
331                         "max": max_val,
332                         "count": cnt,
333                     })
334                 if aggregates:
335                     saved = self.storage.save_sensor_aggregates_batch(aggregates)
336                     logger.info(f"Сохраниено {saved} агрегированных записей за период {period_start} - {period_end}")
337             except Exception as e:
338                 logger.error(f"Ошибка при часовой агрегации телеметрии: {e}")
339
340     def get_status(self) -> Dict[str, Any]:
341         """Возвращает текущий статус агрегатора.
342
343         Returns:
344             Dict[str, Any]: Состояние агрегатора.

```

```
344 |         """
345 |         return {
346 |             "is_running": self._running,
347 |             "measurement_count": self._measurement_count,
348 |             "enabled_sensors": self.config_manager.get_enabled
|_sensors(),
349 |         }
350 |
351 |     def get_last_measurement(self) -> Optional[Dict[str, Any]]
|:
352 |         """Возвращает последнюю записанную телеметрию.
353 |
354 |         Returns:
355 |             Optional[Dict[str, Any]]: Последняя телеметрия или
|None.
356 |         """
357 |         return self._last_measurements
358 |
```

analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  | - Analyzer
4 # =====
5 # Description:
6 #   Аналитический движок исследования логов телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.analyzer import Tel
  | emetryResearcher
11 #
12 #       service = TelemetryResearcher()
13 #
14 # File: analyzer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Аналитический движок исследования логов телеметрии."""
24
25 import math
26 import statistics
27 import uuid
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional, Tuple, Union
30
31 try:
```

```

32     from logger import logger
33 except ImportError:
34     from logger import logger
35
36 from .charts import TelemetryChartGenerator
37 from .extractor import TelemetryDataExtractor
38 from .models import (
39     AnomalyEvent,
40     ChartConfig,
41     CorrelationMatrixItem,
42     DeepResearchReport,
43     DeviceEventSummary,
44     HypothesisResult,
45     MetricPoint,
46     MetricStats,
47     ResearchScenarioRequest,
48     TelemetryResearchReport,
49     TimeSeriesDataset,
50 )
51
52
53 class TelemetryResearcher:
54     """Исследователь и аналитик логов системной и аппаратной т
    елеметрии."""
55
56     def __init__(self, extractor: Optional[TelemetryDataExtrac
    tor] = None) -> None:
57         """Инициализация аналитического движка.
58
59         Args:
60             extractor: Экземпляр загрузчика данных телеметрии.
61         """
62         self.extractor = extractor or TelemetryDataExtractor()
63
64     def analyze(self, source: Optional[Union[str, List[Dict[str, Any]]]] = None) -> TelemetryResearchReport:
65         """Провести глубокое исследование логов телеметрии и с
    формировать отчет.

```

```

66 |
67 |         Args:
68 |             source: Источник данных (директория, путь к файлу
        или список записей).
69 |
70 |         Returns:
71 |             TelemetryResearchReport: Комплексный отчет исследо
        вания.
72 |         """
73 |         records = self.extractor.load_all_records(source)
74 |         logger.info(f"Загружено {len(records)} записей телемет
        рии для исследования.")
75 |         now_str = datetime.now(timezone.utc).isoformat()
76 |         if not records:
77 |             return TelemetryResearchReport(
78 |                 report_id=f"rep-{uuid.uuid4().hex[:8]}",
79 |                 generated_at=now_str,
80 |                 records_analyzed=0,
81 |                 health_score=100.0,
82 |                 summary_conclusions=["Логи телеметрии не содер
        жат записей для анализа."],
83 |             )
84 |
85 |         time_series = self._extract_time_series(records)
86 |         stats_map: Dict[str, MetricStats] = {}
87 |         for metric_name, ds in time_series.items():
88 |             vals = [pt.value for pt in ds.points]
89 |             if vals:
90 |                 stats_map[metric_name] = self._calculate_stats
        (vals, ds.unit)
91 |
92 |         device_summary = self._analyze_device_events(records)
93 |         anomalies = self._detect_anomalies(time_series, device
        _summary)
94 |         health_score = self._compute_health_score(stats_map, a
        nomalies, device_summary)
95 |         conclusions = self._generate_conclusions(stats_map, an
        omalies, device_summary, health_score)

```

```

96
97         ts_list = [r.get("timestamp") for r in records if r.ge
t("timestamp")]
98         start_time = ts_list[0] if ts_list else None
99         end_time = ts_list[-1] if ts_list else None
100
101         return TelemetryResearchReport(
102             report_id=f"rep-{uuid.uuid4().hex[:8]}",
103             generated_at=now_str,
104             records_analyzed=len(records),
105             time_window_start=start_time,
106             time_window_end=end_time,
107             statistics=stats_map,
108             anomalies=anomalies,
109             device_summary=device_summary,
110             health_score=health_score,
111             summary_conclusions=conclusions,
112         )
113
114     def _extract_time_series(self, records: List[Dict[str, Any
]]) -> Dict[str, TimeSeriesDataset]:
115         """Извлечь структурированные временные ряды из разноро
дных записей."""
116         datasets: Dict[str, TimeSeriesDataset] = {
117             "cpu_load": TimeSeriesDataset(name="CPU Load", uni
t="%", color="#3B82F6"),
118             "ram_used_percent": TimeSeriesDataset(name="RAM Us
age", unit="%", color="#10B981"),
119             "gpu_load": TimeSeriesDataset(name="GPU Load", uni
t="%", color="#8B5CF6"),
120             "cpu_temp": TimeSeriesDataset(name="CPU Temperatur
e", unit="°C", color="#EF4444"),
121             "gpu_temp": TimeSeriesDataset(name="GPU Temperatur
e", unit="°C", color="#F59E0B"),
122             "disk_read_mb_s": TimeSeriesDataset(name="Disk Rea
d", unit="MB/s", color="#06B6D4"),
123             "disk_write_mb_s": TimeSeriesDataset(name="Disk Wr
ite", unit="MB/s", color="#EC4899"),

```



```

124         "net_recv_mb_s": TimeSeriesDataset(name="Network R
    ecv", unit="MB/s", color="#6366F1"),
125         "net_sent_mb_s": TimeSeriesDataset(name="Network S
    ent", unit="MB/s", color="#14B8A6"),
126     }
127
128     for r in records:
129         ts = r.get("timestamp", "")
130         # Поддержка вложенного формата psutil/SystemSnapsh
    ot
131         cpu = r.get("cpu") or {}
132         if isinstance(cpu, dict):
133             val = cpu.get("total_percent") or cpu.get("loa
    d_percent")
134             if val is not None:
135                 datasets["cpu_load"].points.append(MetricP
    oint(timestamp=ts, value=float(val)))
136             temp = cpu.get("temperature_celsius")
137             if temp is not None:
138                 datasets["cpu_temp"].points.append(MetricP
    oint(timestamp=ts, value=float(temp)))
139             if "cpu_total_percent" in r and r["cpu_total_perce
    nt"] is not None:
140                 datasets["cpu_load"].points.append(MetricPoint
    (timestamp=ts, value=float(r["cpu_total_percent"])))
141             elif "cpu_percent" in r and r["cpu_percent"] is no
    t None:
142                 datasets["cpu_load"].points.append(MetricPoint
    (timestamp=ts, value=float(r["cpu_percent"])))
143             elif "cpu_usage_pct" in r and r["cpu_usage_pct"] i
    s not None:
144                 datasets["cpu_load"].points.append(MetricPoint
    (timestamp=ts, value=float(r["cpu_usage_pct"])))
145
146         mem = r.get("memory") or {}
147         if isinstance(mem, dict):
148             val = mem.get("percent")
149             if val is not None:

```

```

150         datasets["ram_used_percent"].points.append
(MetricPoint(timestamp=ts, value=float(val)))
151         if "memory_percent" in r and r["memory_percent"] i
s not None:
152             datasets["ram_used_percent"].points.append(Met
ricPoint(timestamp=ts, value=float(r["memory_percent"])))
153             elif "memory_usage_pct" in r and r["memory_usage_p
ct"] is not None:
154                 datasets["ram_used_percent"].points.append(Met
ricPoint(timestamp=ts, value=float(r["memory_usage_pct"])))
155
156         gpu = r.get("gpu") or (r.get("gpus")[0] if isinsta
nce(r.get("gpus"), list) and r.get("gpus") else {})
157         if isinstance(gpu, dict):
158             val = gpu.get("load_percent")
159             if val is not None:
160                 datasets["gpu_load"].points.append(MetricP
oint(timestamp=ts, value=float(val)))
161             temp = gpu.get("temperature_celsius")
162             if temp is not None:
163                 datasets["gpu_temp"].points.append(MetricP
oint(timestamp=ts, value=float(temp)))
164             if "gpu_load_percent" in r and r["gpu_load_percent
"] is not None:
165                 datasets["gpu_load"].points.append(MetricPoint
(timestamp=ts, value=float(r["gpu_load_percent"])))
166             if "gpu_temp_c" in r and r["gpu_temp_c"] is not No
ne:
167                 datasets["gpu_temp"].points.append(MetricPoint
(timestamp=ts, value=float(r["gpu_temp_c"])))
168
169         disk_io = r.get("disk_io") or {}
170         if isinstance(disk_io, dict):
171             rb = disk_io.get("read_bytes_per_sec", 0.0) or
(disk_io.get("read_kb_s", 0.0) * 1024.0)
172             wb = disk_io.get("write_bytes_per_sec", 0.0) o
r (disk_io.get("write_kb_s", 0.0) * 1024.0)
173             if rb > 0:

```

```

174         datasets["disk_read_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(rb / (1024 * 1024), 2)))
175         if wb > 0:
176             datasets["disk_write_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(wb / (1024 * 1024), 2)))
177             if "disk_read_bytes_sec" in r and r["disk_read_bytes_sec"] is not None:
178                 rb = float(r["disk_read_bytes_sec"])
179                 datasets["disk_read_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(rb / (1024 * 1024), 2)))
180                 if "disk_write_bytes_sec" in r and r["disk_write_bytes_sec"] is not None:
181                     wb = float(r["disk_write_bytes_sec"])
182                     datasets["disk_write_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(wb / (1024 * 1024), 2)))
183
184             # Сеть
185             net = r.get("network")
186             if isinstance(net, list) and net:
187                 n0 = net[0]
188                 if isinstance(n0, dict):
189                     rs = n0.get("bytes_recv_sec", 0.0)
190                     ss = n0.get("bytes_sent_sec", 0.0)
191                     if rs > 0:
192                         datasets["net_recv_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(rs / (1024 * 1024), 2)))
193                     if ss > 0:
194                         datasets["net_sent_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(ss / (1024 * 1024), 2)))
195             if "network_recv_bytes_sec" in r and r["network_recv_bytes_sec"] is not None:
196                 rs = float(r["network_recv_bytes_sec"])
197                 datasets["net_recv_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(rs / (1024 * 1024), 2)))
198             elif "net_bytes_recv_sec" in r and r["net_bytes_recv_sec"] is not None:

```

```

199         try:
200             rs = float(r["net_bytes_recv_sec"])
201             datasets["net_recv_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(rs / (1024 * 1024), 2)))
202         except ValueError:
203             pass
204         if "network_sent_bytes_sec" in r and r["network_sent_bytes_sec"] is not None:
205             ss = float(r["network_sent_bytes_sec"])
206             datasets["net_sent_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(ss / (1024 * 1024), 2)))
207         elif "net_bytes_sent_sec" in r and r["net_bytes_sent_sec"] is not None:
208             try:
209                 ss = float(r["net_bytes_sent_sec"])
210                 datasets["net_sent_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(ss / (1024 * 1024), 2)))
211             except ValueError:
212                 pass
213
214         # Поддержка sensor_polls (LHM / LibreHardwareMonitor)
215         sensor_name = str(r.get("sensor_name", "")).lower()
216         sensor_type = str(r.get("sensor_type", "")).lower()
217         sensor_cat = str(r.get("sensor_category", "")).lower()
218         val_raw = r.get("value")
219         if val_raw is not None and val_raw != "":
220             try:
221                 val = float(val_raw)
222                 st = f"{sensor_name} {sensor_type} {sensor_cat}"
223                 if "cpu" in st and ("load" in st or "total" in st or "usage" in st):
224                     datasets["cpu_load"].points.append(MetricPoint(timestamp=ts, value=val))

```

```

225         elif "cpu" in st and ("temp" in st or "pac
| kage" in st):
226             datasets["cpu_temp"].points.append(Met
| ricPoint(timestamp=ts, value=val))
227             elif "gpu" in st and ("load" in st or "cor
| e" in st):
228                 datasets["gpu_load"].points.append(Met
| ricPoint(timestamp=ts, value=val))
229                 elif "gpu" in st and "temp" in st:
230                     datasets["gpu_temp"].points.append(Met
| ricPoint(timestamp=ts, value=val))
231                 elif ("ram" in st or "memory" in st) and (
| "used" in st or "percent" in st or "%" in str(r.get("unit", ""
| )))
232                     datasets["ram_used_percent"].points.ap
| pend(MetricPoint(timestamp=ts, value=val))
233             except (ValueError, TypeError):
234                 pass
235
236     return {k: ds for k, ds in datasets.items() if ds.poin
| ts}
237
238     def _calculate_stats(self, values: List[float], unit: str
| = "%") -> MetricStats:
239         """Рассчитать описательную статистику по массиву значе
| ний."""
240         if not values:
241             return MetricStats(unit=unit)
242         sorted_vals = sorted(values)
243         n = len(sorted_vals)
244         min_v = sorted_vals[0]
245         max_v = sorted_vals[-1]
246         avg_v = sum(sorted_vals) / n
247         median_v = statistics.median(sorted_vals)
248         p95_idx = int(math.ceil(0.95 * n)) - 1
249         p95_v = sorted_vals[min(p95_idx, n - 1)]
250         std_v = statistics.stdev(sorted_vals) if n > 1 else 0.

```

```

251         return MetricStats(
252             count=n,
253             min_val=round(min_v, 2),
254             max_val=round(max_v, 2),
255             avg_val=round(avg_v, 2),
256             median_val=round(median_v, 2),
257             p95_val=round(p95_v, 2),
258             std_dev=round(std_v, 2),
259             unit=unit,
260         )
261
262     def _analyze_device_events(self, records: List[Dict[str, Any]]) -> DeviceEventSummary:
263         """Собрать агрегированную статистику по аппаратным событиям и сбоям."""
264         summary = DeviceEventSummary()
265         flapping_set = set()
266         for r in records:
267             ev_type = r.get("event_type")
268             dev_id = r.get("device_instance_id") or r.get("friendly_name")
269             if not ev_type and not dev_id:
270                 continue
271             summary.total_events += 1
272             if ev_type:
273                 summary.by_event_type[ev_type] = summary.by_event_type.get(ev_type, 0) + 1
274             cat = r.get("category") or r.get("device_class") or "Другое"
275             summary.by_category[cat] = summary.by_category.get(cat, 0) + 1
276             if r.get("has_problem") or r.get("problem_code", 0) > 0:
277                 summary.error_count += 1
278             flap_count = r.get("flapping_count_in_window", 0)
279             if flap_count > 0 and dev_id:
280                 flapping_set.add(f"{dev_id} (всплесков: {flap_count})")

```

```

281 |         summary.flapping_devices = list(flapping_set)
282 |         return summary
283 |
284 |     def _detect_anomalies(
285 |         self, time_series: Dict[str, TimeSeriesDataset], device_summary: DeviceEventSummary
286 |     ) -> List[AnomalyEvent]:
287 |         """Обнаружить аномальные значения метрик и сбои оборудования."""
288 |         anomalies: List[AnomalyEvent] = []
289 |         if "cpu_load" in time_series:
290 |             for pt in time_series["cpu_load"].points:
291 |                 if pt.value >= 90.0:
292 |                     anomalies.append(
293 |                         AnomalyEvent(
294 |                             timestamp=pt.timestamp,
295 |                             metric="CPU Load",
296 |                             value=pt.value,
297 |                             threshold=90.0,
298 |                             severity="critical" if pt.value >=
299 |                             98.0 else "warning",
300 |                             description=f"Пиковая нагрузка процессора достигла {pt.value}%",
301 |                         )
302 |                     )
303 |         if "ram_used_percent" in time_series:
304 |             for pt in time_series["ram_used_percent"].points:
305 |                 if pt.value >= 90.0:
306 |                     anomalies.append(
307 |                         AnomalyEvent(
308 |                             timestamp=pt.timestamp,
309 |                             metric="RAM Usage",
310 |                             value=pt.value,
311 |                             threshold=90.0,
312 |                             severity="critical" if pt.value >=
313 |                             95.0 else "warning",
314 |                             description=f"Высокое потребление оперативной памяти: {pt.value}%",

```

```

313         )
314     )
315     if "cpu_temp" in time_series:
316         for pt in time_series["cpu_temp"].points:
317             if pt.value >= 80.0:
318                 anomalies.append(
319                     AnomalyEvent(
320                         timestamp=pt.timestamp,
321                         metric="CPU Temp",
322                         value=pt.value,
323                         threshold=80.0,
324                         severity="critical" if pt.value >=
325                             90.0 else "warning",
326                         description=f"Перегрев процессора:
327                             {pt.value}°C",
328                     )
329                 )
330             if device_summary.error_count > 0:
331                 anomalies.append(
332                     AnomalyEvent(
333                         timestamp=datetime.now(timezone.utc).isofo
334                         rmat(),
335                         metric="Device Failures",
336                         value=float(device_summary.error_count),
337                         threshold=0.0,
338                         severity="warning",
339                         description=f"Зафиксировано {device_summar
340                             y.error_count} сбоев в работе подключенных устройств",
341                     )
342                 )
343             return anomalies[-50:]
344
345     def _compute_health_score(
346         self, stats_map: Dict[str, MetricStats], anomalies: Li
347         st[AnomalyEvent], device_summary: DeviceEventSummary
348     ) -> float:
349         """Рассчитать интегральный индекс здоровья системы от
350         0.0 до 100.0."""

```



```

345 |         score = 100.0
346 |         if "cpu_load" in stats_map:
347 |             avg_cpu = stats_map["cpu_load"].avg_val
348 |             if avg_cpu > 70.0:
349 |                 score -= min(30.0, (avg_cpu - 70.0) * 1.0)
350 |         if "ram_used_percent" in stats_map:
351 |             avg_ram = stats_map["ram_used_percent"].avg_val
352 |             if avg_ram > 80.0:
353 |                 score -= min(25.0, (avg_ram - 80.0) * 1.25)
354 |         if "cpu_temp" in stats_map:
355 |             max_temp = stats_map["cpu_temp"].max_val
356 |             if max_temp > 80.0:
357 |                 score -= min(20.0, (max_temp - 80.0) * 1.5)
358 |         if device_summary.error_count > 0:
359 |             score -= min(25.0, device_summary.error_count * 5.
0)
360 |
361 |         crit_count = sum(1 for a in anomalies if a.severity ==
"critical")
362 |         score -= min(20.0, crit_count * 4.0)
363 |         return max(0.0, round(score, 1))
364 |
365 |     def _generate_conclusions(
366 |         self,
367 |         stats_map: Dict[str, MetricStats],
368 |         anomalies: List[AnomalyEvent],
369 |         device_summary: DeviceEventSummary,
370 |         health_score: float,
371 |     ) -> List[str]:
372 |         """Сформировать текстовые выводы и рекомендации исслед
ования."""
373 |         conclusions = []
374 |         if health_score >= 90.0:
375 |             conclusions.append("Состояние системы стабильное:
параметры утилизации ресурсов находятся в допустимых границах.
")
376 |             elif health_score >= 70.0:
377 |                 conclusions.append("Состояние системы удовлетворит

```

```

    ельное, но выявлены эпизодические пики нагрузки.")
378         else:
379             conclusions.append("Внимание: состояние системы де
градировало из-за высокой утилизации ресурсов или сбоев оборуд
ования.")
380
381         if "cpu_load" in stats_map:
382             cpu = stats_map["cpu_load"]
383             conclusions.append(f"Процессор (CPU): средняя нагр
узка {cpu.avg_val}%, пик {cpu.max_val}% (p95: {cpu.p95_val}%).
")
384         if "ram_used_percent" in stats_map:
385             ram = stats_map["ram_used_percent"]
386             conclusions.append(f"Оперативная память (RAM): сре
днее использование {ram.avg_val}%, максимум {ram.max_val}%.")
387         if device_summary.error_count > 0:
388             conclusions.append(f"Обнаружено {device_summary.er
ror_count} аппаратных сбоев или проблемных устройств.")
389         if device_summary.flapping_devices:
390             conclusions.append(f"Нестабильные устройства (флап
пинг): {' '.join(device_summary.flapping_devices)}.")
391         return conclusions
392
393     def run_deep_research(
394         self,
395         scenario: Optional[ResearchScenarioRequest] = None,
396         chart_generator: Optional[TelemetryChartGenerator] = N
one,
397     ) -> DeepResearchReport:
398         """Выполнить полный цикл исследовательского анализа те
леметрии (с корреляциями и гипотезами)."""
399         sc = scenario or ResearchScenarioRequest()
400         source = sc.records if sc.records is not None else sc.
source_path
401         c_gen = chart_generator or TelemetryChartGenerator()
402
403         raw_records = self.extractor.load_all_records(source)
404         filtered_records = self._filter_records(

```

```
405         records=raw_records, start_time=sc.start_time, end
|_time=sc.end_time, subsystems=sc.subsystems
406     )
407     logger.info(f"Запуск исследования телеметрии: обработка
|но {len(filtered_records)} записей (из исходных {len(raw_recor
|ds)}).")
408
409     base_report = self.analyze(filtered_records)
410     time_series_map = self._extract_time_series(filtered_r
|ecords)
411     base_report.charts = c_gen.generate_chart_configs(time
|_series_map, base_report)
412
413     correlations = self._calculate_correlations(time_serie
|s_map)
414     hypotheses: List[HypothesisResult] = []
415     if sc.enable_hypotheses_check:
416         hypotheses = self._evaluate_hypotheses(base_report
|, time_series_map, correlations)
417
418     recommendations = self._generate_recommendations(base_
|report, hypotheses, correlations)
419     summary = self._build_investigation_summary(base_repor
|t, hypotheses, len(filtered_records))
420
421     report_id = f"deep_research_{datetime.now(timezone.utc
|).strftime('%Y%m%d_%H%M%S')}_{uuid.uuid4().hex[:6]}"
422     now_iso = datetime.now(timezone.utc).isoformat()
423
424     return DeepResearchReport(
425         report_id=report_id,
426         generated_at=now_iso,
427         base_report=base_report,
428         correlations=correlations,
429         hypotheses=hypotheses,
430         investigation_summary=summary,
431         actionable_recommendations=recommendations,
432     )
```

```

433
434     # Алиас для совместимости
435     analyze_deep = run_deep_research
436
437     def _filter_records(
438         self,
439         records: List[Dict[str, Any]],
440         start_time: Optional[str] = None,
441         end_time: Optional[str] = None,
442         subsystems: Optional[List[str]] = None,
443     ) -> List[Dict[str, Any]]:
444         """Фильтрует записи по временному диапазону и составу
445         подсистем."""
446         if not records:
447             return []
448         start_dt = self._parse_iso_time(start_time) if start_t
449         ime else None
450         end_dt = self._parse_iso_time(end_time) if end_time el
451         se None
452         result: List[Dict[str, Any]] = []
453
454         for r in records:
455             ts_str = r.get("timestamp") or r.get("time") or r.
456             get("created_at")
457             if ts_str and (start_dt or end_dt):
458                 rec_dt = self._parse_iso_time(str(ts_str))
459                 if rec_dt:
460                     if start_dt and rec_dt < start_dt:
461                         continue
462                     if end_dt and rec_dt > end_dt:
463                         continue
464                 if subsystems:
465                     match_subsystem = False
466                     for sub in subsystems:
467                         sub_lower = sub.lower()
468                         if sub_lower in r or any(sub_lower in str(
469                             k).lower() for k in r.keys()):
470                             match_subsystem = True

```

```

466         break
467         if sub_lower == "devices" and ("device_ins
tance_id" in r or "event_type" in r):
468             match_subsystem = True
469             break
470         if not match_subsystem:
471             continue
472         result.append(r)
473     return result
474
475     @staticmethod
476     def _parse_iso_time(time_str: Optional[str]) -> Optional[d
atetime]:
477         """Парсит строку даты и времени в ISO формате."""
478         if not time_str:
479             return None
480         clean_str = time_str.replace("Z", "+00:00")
481         try:
482             return datetime.fromisoformat(clean_str)
483         except Exception:
484             return None
485
486     def _calculate_correlations(self, time_series_map: Dict[str,
TimeSeriesDataset]) -> List[CorrelationMatrixItem]:
487         """Вычисляет коэффициенты корреляции Пирсона между пар
ами системных метрик."""
488         keys = sorted(list(time_series_map.keys()))
489         correlations: List[CorrelationMatrixItem] = []
490
491         for i in range(len(keys)):
492             for j in range(i + 1, len(keys)):
493                 key_a = keys[i]
494                 key_b = keys[j]
495                 ds_a = time_series_map[key_a]
496                 ds_b = time_series_map[key_b]
497                 pts_a = ds_a.points if hasattr(ds_a, "points")
else ds_a
498                 pts_b = ds_b.points if hasattr(ds_b, "points")

```

```

    else ds_b
499         if not pts_a or not pts_b:
500             continue
501         val_a = [p.value if hasattr(p, "value") else f
load(p[1] if isinstance(p, (list, tuple)) else p) for p in pts
_a]
502         val_b = [p.value if hasattr(p, "value") else f
load(p[1] if isinstance(p, (list, tuple)) else p) for p in pts
_b]
503         min_len = min(len(val_a), len(val_b))
504         if min_len < 3:
505             continue
506         slice_a = val_a[:min_len]
507         slice_b = val_b[:min_len]
508         coeff = self._pearson_correlation(slice_a, sli
ce_b)
509         if math.isnan(coeff):
510             continue
511         interpretation = self._interpret_correlation(c
oeff, key_a, key_b)
512         correlations.append(
513             CorrelationMatrixItem(
514                 metric_a=key_a,
515                 metric_b=key_b,
516                 coefficient=round(coeff, 3),
517                 sample_size=min_len,
518                 interpretation=interpretation,
519             )
520         )
521         return sorted(correlations, key=lambda x: abs(x.coeffi
cient), reverse=True)
522
523     @staticmethod
524     def _pearson_correlation(x: List[float], y: List[float]) -
> float:
525         """Вычисляет коэффициент линейной корреляции Пирсона д
ля двух списков чисел."""
526         n = len(x)

```

```

527         if n == 0:
528             return 0.0
529         mean_x = sum(x) / n
530         mean_y = sum(y) / n
531         diff_prod = sum((x[k] - mean_x) * (y[k] - mean_y) for
k in range(n))
532         sq_diff_x = sum((x[k] - mean_x) ** 2 for k in range(n)
)
533         sq_diff_y = sum((y[k] - mean_y) ** 2 for k in range(n)
)
534         denominator = math.sqrt(sq_diff_x * sq_diff_y)
535         if denominator == 0.0:
536             return 0.0
537         return diff_prod / denominator
538
539     @staticmethod
540     def _interpret_correlation(coeff: float, metric_a: str, me
tric_b: str) -> str:
541         """Формирует текстовое объяснение коэффициента корреля
ции."""
542         abs_c = abs(coeff)
543         direction = "прямая" if coeff > 0 else "обратная"
544         if abs_c >= 0.8:
545             strength = f"Очень сильная {direction}"
546         elif abs_c >= 0.6:
547             strength = f"Заметная {direction}"
548         elif abs_c >= 0.3:
549             strength = f"Умеренная {direction}"
550         else:
551             strength = f"Слабая {direction}"
552         return f"{strength} связь между '{metric_a}' и '{metri
c_b}' (r={coeff:.2f}). {'Изменения происходят синхронно.' if c
oeff > 0.5 else ''}"
553
554     def _evaluate_hypotheses(
555         self,
556         report: TelemetryResearchReport,
557         time_series_map: Dict[str, TimeSeriesDataset],

```

```

558         correlations: List[CorrelationMatrixItem],
559     ) -> List[HypothesisResult]:
560         """Проверяет набор стандартных системных гипотез."""
561         hypotheses: List[HypothesisResult] = []
562
563         # 1. CPU Thermal stress
564         cpu_temp_stats = report.statistics.get("cpu_temp")
565         cpu_load_stats = report.statistics.get("cpu_load")
566         h1_confirmed = False
567         h1_evidence: List[str] = []
568         h1_conf = 0.5
569         if cpu_temp_stats and cpu_temp_stats.max_val >= 82.0:
570             h1_confirmed = True
571             h1_conf = 0.85
572             h1_evidence.append(f"Пиковая температура CPU дости
573 гла {cpu_temp_stats.max_val}°C (порог 82°C)")
574             if cpu_load_stats:
575                 h1_evidence.append(f"Средняя загрузка CPU сост
576 авляла {cpu_load_stats.avg_val:.1f}%")
577             else:
578                 h1_evidence.append("Температуры CPU находятся в пр
579 еделах нормы (ниже 82°C)")
580             hypotheses.append(
581                 HypothesisResult(
582                     hypothesis_id="H_CPU_THERMAL_STRESS",
583                     title="Термический стресс и перегрев CPU",
584                     description="Проверка риска троттлинга или дег
585 радации при высоких температурах процессора.",
586                     confirmed=h1_confirmed,
587                     confidence=h1_conf,
588                     evidence=h1_evidence,
589                     recommendation="Проверить термопасту, систему
590 охлаждения и отсутствие пыли в радиаторах CPU."

```



```

591 |         # 2. RAM Pressure
592 |         ram_stats = report.statistics.get("ram_used_percent")
593 |         h2_confirmed = False
594 |         h2_evidence: List[str] = []
595 |         h2_conf = 0.5
596 |         if ram_stats and ram_stats.max_val >= 88.0:
597 |             h2_confirmed = True
598 |             h2_conf = 0.9
599 |             h2_evidence.append(f"Пиковое потребление RAM соста
вилось {ram_stats.max_val}% (порог 88%)")
600 |             if ram_stats:
601 |                 h2_evidence.append(f"Средняя загрузка RAM: {ra
m_stats.avg_val:.1f}%")
602 |             else:
603 |                 h2_evidence.append("Потребление RAM находится в бе
зопасных пределах (ниже 88%)")
604 |             hypotheses.append(
605 |                 HypothesisResult(
606 |                     hypothesis_id="H_RAM_PRESSURE",
607 |                     title="Дефицит оперативной памяти (RAM Pressur
e)",
608 |                     description="Анализ рисков нехватки оперативно
й памяти и повышенного использования файла подкачки.",
609 |                     confirmed=h2_confirmed,
610 |                     confidence=h2_conf,
611 |                     evidence=h2_evidence,
612 |                     recommendation="Закрыть неиспользуемые фоновые
приложения или расширить объем ОЗУ."
613 |                     if h2_confirmed
614 |                     else "Памяти достаточно для текущей рабочей на
грузки.",
615 |                 )
616 |             )
617 |
618 |         # 3. Hardware Flapping
619 |         dev_sum = report.device_summary
620 |         h3_confirmed = dev_sum.error_count > 0 or len(dev_sum.
flapping_devices) > 0

```

```

621         h3_evidence: List[str] = []
622         if h3_confirmed:
623             if dev_sum.error_count > 0:
624                 h3_evidence.append(f"Зафиксировано {dev_sum.er
ror_count} сбоев и ошибок оборудования в логах")
625                 if dev_sum.flapping_devices:
626                     h3_evidence.append(f"Обнаружен флаппинг устрой
ств: {' ', ' '.join(dev_sum.flapping_devices)}")
627             else:
628                 h3_evidence.append("Ошибок оборудования и флаппинг
а устройств в исследованный период не обнаружено")
629             hypotheses.append(
630                 HypothesisResult(
631                     hypothesis_id="H_DEVICE_INSTABILITY",
632                     title="Нестабильность и сбои оборудования (Har
dware Flapping)",
633                     description="Проверка наличия сбоящих шин, отв
аливающихся USB/PCI контроллеров или конфликтов драйверов.",
634                     confirmed=h3_confirmed,
635                     confidence=0.95 if h3_confirmed else 0.7,
636                     evidence=h3_evidence,
637                     recommendation="Проверить подключение кабелей,
обновить драйверы сбоящих устройств или заменить порт."
638                     if h3_confirmed
639                     else "Оборудование работает стабильно.",
640                 )
641             )
642
643         # 4. GPU Thermal
644         gpu_temp = report.statistics.get("gpu_temp")
645         h4_confirmed = False
646         h4_evidence: List[str] = []
647         if gpu_temp and gpu_temp.max_val >= 83.0:
648             h4_confirmed = True
649             h4_evidence.append(f"Пиковая температура GPU: {gpu
_temp.max_val}°C")
650         else:
651             h4_evidence.append("Температура GPU в безопасной з

```

```

    one")
652         hypotheses.append(
653             HypothesisResult(
654                 hypothesis_id="H_GPU_THERMAL",
655                 title="Термическая стабильность графического у
скорителя (GPU)",
656                 description="Оценка температурных режимов граф
ического чипа при нагрузке.",
657                 confirmed=h4_confirmed,
658                 confidence=0.85 if h4_confirmed else 0.6,
659                 evidence=h4_evidence,
660                 recommendation="Настроить кривую вентиляторов
GPU или проверить вентиляцию корпуса."
661                 if h4_confirmed
662                 else "GPU работает в штатном тепловом режиме."
663             )
664         )
665
666     return hypotheses
667
668     def _generate_recommendations(
669         self,
670         report: TelemetryResearchReport,
671         hypotheses: List[HypothesisResult],
672         correlations: List[CorrelationMatrixItem],
673     ) -> List[str]:
674         """Генерирует практические рекомендации на основе анализ
а. """
675         recs: List[str] = []
676         for h in hypotheses:
677             if h.confirmed and h.recommendation:
678                 recs.append(f"[{h.title}]: {h.recommendation}")
679         )
680         for anom in report.anomalies:
681             if anom.severity in ("critical", "high"):
682                 recs.append(f"[Аномалия {anom.metric}]: {anom.
description}")

```

```
682         if not recs:
683             recs.append("Система работает стабильно. Регулярно
        проводите профилактический аудит телеметрии.")
684         return recs
685
686     @staticmethod
687     def _build_investigation_summary(
688         report: TelemetryResearchReport, hypotheses: List[Hypo
thesisResult], total_records: int
689     ) -> str:
690         """Формирует текстовое резюме проведенного исследовани
я."""
691         confirmed_count = sum(1 for h in hypotheses if h.confir
med)
692         return (
693             f"Исследование завершено. Проанализировано {total_
records} записей телеметрии. "
694             f"Индекс здоровья системы: {report.health_score}/1
00. Обнаружено аномалий: {len(report.anomalies)}. "
695             f"Подтверждено проблемных гипотез: {confirmed_coun
t} из {len(hypotheses)}."
696         )
697
```

assistant.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Assistant
4 # =====
5 # Description:
6 #   Интеллектуальный AI-ассистент анализа системной телеметрии
  #   и построения графиков.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.assistant import Te
  #       lemetryAssistant
11 #
12 #       service = TelemetryAssistant()
13 #
14 # File: assistant.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:48:00
20 # =====
21
22 from __future__ import annotations
23 """Интеллектуальный AI-ассистент анализа системной телеметрии,
  #   ресурсов программ клиента и построения графиков."""
24
25 import asyncio
26 import json
27 from typing import Any, Dict, List, Optional, Union
28
29 try:
```

```

30     from logger import logger
31 except ImportError:
32     import logging
33     logger = logging.getLogger(__name__)
34
35 from .query_engine import TelemetryQueryEngine
36 from .client_resource_analyzer import ClientResourceAnalyzer
37
38 try:
39     from src.ai.orchestration.unified_chat import UnifiedChatModel
40 except Exception as ex:
41     logger.warning(f"Не удалось импортировать UnifiedChatModel
42 : {ex}")
43     UnifiedChatModel = None
44
45 class TelemetryAssistant:
46     """Ассистент для диалогового исследования телеметрии с генерацией графиков и SQL."""
47
48     SYSTEM_INSTRUCTION = """Ты – интеллектуальный эксперт по анализу системной телеметрии Windows и энергопотреблению компьютера в проекте AI-Breadboard.
49 Твоя задача – отвечать на вопросы пользователя четко, профессионально и на русском языке, анализировать показатели загрузки ЦП, видеокарты, дисков, температурных датчиков, энергопотребления и процессов.
50 Когда пользователю требуется визуализация (график, круговая диаграмма, столбчатая диаграмма), ты формируешь развернутый ответ с выводами, рекомендациями и структурированными данными.
51 Всегда приводи конкретные числовые значения (кВт·ч, Вт, %, ГГц, МБ) и объясняй их физический и системный смысл."""
52
53     def __init__(
54         self,
55         query_engine: Optional[TelemetryQueryEngine] = None,
56         chat_model: Optional[Any] = None,

```

```

57         client_analyzer: Optional[ClientResourceAnalyzer] = No
ne,
58     ) -> None:
59         """Инициализация ассистента.
60
61         Args:
62             query_engine: Экземпляр движка запросов telemetry.
db.
63             chat_model: Модель чата (по умолчанию UnifiedChatM
odel).
64             client_analyzer: Анализатор ресурсов программ клие
нта.
65         """
66         self.query_engine = query_engine or TelemetryQueryEngi
ne()
67         self._chat_model = chat_model
68         self.client_analyzer = client_analyzer or ClientResour
ceAnalyzer(
69             query_engine=self.query_engine, chat_model=self._c
hat_model
70         )
71
72     @property
73     def chat_model(self) -> Any:
74         """Получить активную модель LLM."""
75         if self._chat_model is None and UnifiedChatModel is no
t None:
76             try:
77                 self._chat_model = UnifiedChatModel(system_ins
truction=self.SYSTEM_INSTRUCTION)
78                 if self.client_analyzer and self.client_analyz
er._chat_model is None:
79                     self.client_analyzer._chat_model = self._c
hat_model
80             except Exception as ex:
81                 logger.warning(f"Не удалось инициализировать U
nifiedChatModel: {ex}")
82                 self._chat_model = None

```

```

83         return self._chat_model
84
85     async def handle_query(
86         self,
87         user_message: str,
88         source_path: Optional[str] = None,
89         history: Optional[List[Dict[str, Any]]] = None,
90     ) -> Dict[str, Any]:
91         """Обработать пользовательский запрос на естественном
92         языке.
93
94         Args:
95             user_message: Текст запроса пользователя.
96             source_path: Путь к выбранному файлу БД или логов.
97             history: История диалога.
98
99         Returns:
100             Dict[str, Any]: Ответ ассистента со структурирован
101             ным текстом, графиком и SQL.
102         """
103         msg_lower = user_message.strip().lower()
104
105         # 1. Распознавание интента: Ресурсы клиентских програм
106         # м и оценка Gemini
107         if self._is_client_resources_query(msg_lower):
108             return await self._handle_client_resources_query(u
109             ser_message, source_path)
110
111         # 2. Распознавание интента: График загрузки ЦПУ по вре
112         # мени
113         if self._is_cpu_timeline_query(msg_lower):
114             return await self._handle_cpu_timeline_query(user_
115             message, source_path)
116
117         # 3. Распознавание интента: Потребление энергии и суто
118         # чная круговая диаграмма
119         if self._is_power_query(msg_lower):
120             return await self._handle_power_consumption_query(

```



```

    user_message, source_path)
114
115     # 4. Распознавание интента: Топ процессов
116     if self._is_top_processes_query(msg_lower):
117         return await self._handle_top_processes_query(user
    _message, source_path)
118
119     # 5. Произвольный SELECT SQL запрос от пользователя
120     if msg_lower.startswith("select ") or msg_lower.starts
    with("with "):
121         return self._handle_direct_sql_query(user_message,
    source_path)
122
123     # 6. Общий аналитический запрос с обращением к LLM
124     return await self._handle_general_ai_query(user_messag
    e, source_path, history)
125
126     def _is_client_resources_query(self, msg: str) -> bool:
127         """Проверить, относится ли запрос к ресурсам программ
    клиента или оценке Gemini."""
128         has_prog = any(w in msg for w in ["программ", "приложе
    н", "софт", "клиент", "пользовател", "запущенн"])
129         has_res = any(w in msg for w in ["мест", "ресурс", "за
    нимает", "жрет", "памят", "диск", "объем", "тяжел", "неизвестн
    ", "gemini", "нормальн"])
130         return (has_prog and has_res) or ("сколько места" in m
    sg) or ("неизвестная программа" in msg)
131
132     def _is_cpu_timeline_query(self, msg: str) -> bool:
133         """Проверить, относится ли запрос к графику ЦП по врем
    ени."""
134         keywords = ["график", "таймлайн", "timeline", "динамик
    ", "истори", "по времени"]
135         targets = ["цпу", "cpu", "процессор", "загрузк", "нагр
    узк", "частот"]
136         has_kw = any(k in msg for k in keywords)
137         has_target = any(t in msg for t in targets)
138         return (has_kw and has_target) or ("загрузк" in msg an

```

```

139 | d ("цпу" in msg or "cpu" in msg or "процессор" in msg))
140 |     def _is_power_query(self, msg: str) -> bool:
141 |         """Проверить, относится ли запрос к энергопотреблению
142 |         или круговой диаграмме мощности."""
143 |         keywords = [
144 |             "энерг",
145 |             "электроэнерг",
146 |             "потреблен",
147 |             "ватт",
148 |             "квт",
149 |             "мощност",
150 |             "кругов",
151 |             "диаграмм",
152 |             "pie",
153 |             "doughnut",
154 |             "power",
155 |         ]
156 |         return any(k in msg for k in keywords) and any(
157 |             t in msg for t in ["компьют", "пк", "систем", "потреблен", "суточн", "кругов", "ватт", "энерг", "power"]
158 |         )
159 |     def _is_top_processes_query(self, msg: str) -> bool:
160 |         """Проверить, относится ли запрос к ресурсоемким процессам."""
161 |         return "процесс" in msg and any(k in msg for k in ["топ", "нагруж", "жрет", "груз", "памят", "cpu", "цпу", "ресурс"])
162 |
163 |     async def _handle_cpu_timeline_query(
164 |         self, user_message: str, source_path: Optional[str] = None
165 |     ) -> Dict[str, Any]:
166 |         """Сформировать ответ для графика загрузки ЦП."""
167 |         timeline_data = self.query_engine.get_cpu_timeline(limit=120, db_path=source_path)
168 |         if timeline_data.get("status") != "ok":

```

```

169         return {
170             "status": "error",
171             "reply": f"␣ Не удалось извлечь данные по загр
узке ЦП: {timeline_data.get('message')}",
172             "chart": None,
173             "sql_queries": [timeline_data.get("sql", "")],
174             "metrics_summary": {},
175             "quick_followups": [
176                 "Посчитай суточное потребление энергии (кр
уговая диаграмма)",
177                 "Какие процессы больше всего нагружают ЦП?",
178             ],
179         }
180
181     stats = timeline_data.get("statistics", {})
182     avg_cpu = stats.get("avg_cpu_percent", 0.0)
183     max_cpu = stats.get("max_cpu_percent", 0.0)
184     points_count = stats.get("points_count", 0)
185
186     # Формирование базового отчета
187     explanation = (
188         f"### ␣ Анализ динамики загрузки и частоты процес
сора (CPU)\n\n"
189         f"Отрисован временной ряд по последним **{points_c
ount} точкам замера** телеметрии:\n\n"
190         f"- **Средняя загрузка ЦП:** `{avg_cpu}%`\n"
191         f"- **Пиковая нагрузка ЦП:** `{max_cpu}%`\n"
192         f"- **Минимальная загрузка:** `{stats.get('min_cpu
_percent', 0.0)}%\n"
193         f"- **Временной интервал:** с `{stats.get('start_t
ime', '')[:19]}` по `{stats.get('end_time', '')[:19]}`\n\n"
194     )
195
196     if max_cpu > 85.0:
197         explanation += "␣ **Внимание:** Зафиксированы пико
вые всплески нагрузки свыше 85%. Рекомендуется проверить фонов
ые задачи и процессы в топе.\n"

```

```

198         else:
199             explanation += "\n **Статус:** Нагрузка на процессо
р находится в пределах нормы, троттлинг не выражен.\n"
200
201         # Дополнение от LLM
202         if self.chat_model and hasattr(self.chat_model, "ask")
:
203             try:
204                 llm_prompt = (
205                     f"Пользователь спросил: '{user_message}'.\n
n"
206                     f"Данные выборки ЦП: Средняя={avg_cpu}%, М
аксимум={max_cpu}%, Всего точек={points_count}.\n"
207                     f"Дай краткий аналитический комментарий (2
-3 предложения) о профиле нагрузки и энергоэффективности проце
ссора."
208                 )
209                 ai_comment = await asyncio.wait_for(self.chat_
model.ask(llm_prompt), timeout=3.5)
210                 if ai_comment and len(ai_comment.strip()) > 10
:
211                     explanation += f"\n **AI-анализ:** {ai_c
omment.strip()}\n"
212             except Exception as ex:
213                 logger.debug(f"LLM ask fallback: {ex}")
214
215         return {
216             "status": "ok",
217             "reply": explanation,
218             "chart": timeline_data.get("chart"),
219             "sql_queries": [timeline_data.get("sql", "")],
220             "metrics_summary": {
221                 "avg_cpu_percent": f"{avg_cpu}%",
222                 "max_cpu_percent": f"{max_cpu}%",
223                 "points_analyzed": points_count,
224             },
225             "quick_followups": [
226                 "Посчитай суточное потребление энергии (кругов

```

```

    ая диаграмма)",
227         "Топ процессов по нагрузке на ЦП",
228         "Были ли критические перезагрузки системы?",
229     ],
230 }
231
232 async def _handle_power_consumption_query(
233     self, user_message: str, source_path: Optional[str] =
None
234 ) -> Dict[str, Any]:
235     """Сформировать ответ для расчета энергопотребления и
    суточной круговой диаграммы."""
236     power_data = self.query_engine.calculate_power_consump
    tion_24h(db_path=source_path, hours=24)
237     if power_data.get("status") != "ok":
238         return {
239             "status": "error",
240             "reply": f"❌ Ошибка расчета энергопотребления:
    {power_data.get('message')}",
241             "chart": None,
242             "sql_queries": [],
243             "metrics_summary": {},
244             "quick_followups": ["Покажи график загрузки ЦП
    У по времени"],
245         }
246
247     total_kwh = power_data.get("total_kwh", 0.0)
248     total_wh = power_data.get("total_watt_hours", 0.0)
249     avg_watts = power_data.get("avg_system_power_watts", 0
    .0)
250     cpu_w = power_data.get("cpu_package_power_watts", 0.0)
251     gpu_w = power_data.get("gpu_power_watts", 0.0)
252     breakdown = power_data.get("breakdown", [])
253
254     table_rows = []
255     for item in breakdown:
256         table_rows.append(
257             f"| {item['component']} | {item['avg_power_wat

```

```

    ts']] Вт | {item['watt_hours']] Вт·ч | {item['kwh']] кВт·ч | *
    *{item['percent']]%** | "
258         )
259         table_md = "\n".join(table_rows)
260
261         explanation = (
262             f"### □ Расчет суточного энергопотребления компьют
ера и круговая диаграмма\n\n"
263             f"На основе телеметрии аппаратных датчиков мощност
и (`sensor_polls`) и загрузки компонентов за **24 часа**:\n\n"
264             f"- □ **Общее суточное потребление:** **`{total_k
wh}` кВт·ч`** (`{total_wh}` Вт·ч`)\n"
265             f"- □ **Средняя мощность всей системы:** **`{avg_
watts}` Вт`**\n"
266             f"- □ **Мощность процессора (CPU Package):** `{cp
u_w}` Вт`\n"
267             f"- □ **Мощность видеокарты (GPU):** `{gpu_w}` Вт`
\n\n"
268             f"#### □ Детализация энергопотребления по узлам с
истемы:\n\n"
269             f"| Компонент / Подсистема | Средняя мощность | За
24ч (Вт·ч) | Энергия (кВт·ч) | Доля (%) | \n"
270             f"| :--- | :--- | :--- | :--- | :--- | \n"
271             f"{table_md}\n\n"
272         )
273
274         est_cost_rub = round(total_kwh * 5.80, 2)
275         est_cost_usd = round(total_kwh * 0.15, 2)
276         explanation += (
277             f"□ *Примерная стоимость электроэнергии за сутки:
* ~**{est_cost_rub} ₽** (или ~**${est_cost_usd}** при среднем
тарифе).\n"
278         )
279
280         if self.chat_model and hasattr(self.chat_model, "ask")
:
281             try:
282                 llm_prompt = (

```

```

283         f"Пользователь спросил: '{user_message}'.\n"
284         f"Суточное энергопотребление ПК: Всего={total_kwh} кВт*ч, Средняя мощность={avg_watts} Вт (CPU={cpu_w}W, GPU={gpu_w}W).\n"
285         f"Дай краткий совет (2 предложения) по оптимизации энергопотребления и профилю электропитания Windows."
286     )
287     ai_comment = await asyncio.wait_for(self.chat_model.ask(llm_prompt), timeout=3.5)
288     if ai_comment and len(ai_comment.strip()) > 10:
289         explanation += f"\n    **AI-рекомендация:**\n    {ai_comment.strip()}\n"
290     except Exception as ex:
291         logger.debug(f"LLM ask fallback: {ex}")
292
293     return {
294         "status": "ok",
295         "reply": explanation,
296         "chart": power_data.get("chart"),
297         "sql_queries": [power_data.get("sql", "")],
298         "metrics_summary": {
299             "total_kwh_24h": f"{total_kwh} кВт·ч",
300             "avg_power_watts": f"{avg_watts} Вт",
301             "cpu_power_watts": f"{cpu_w} Вт",
302             "gpu_power_watts": f"{gpu_w} Вт",
303         },
304         "quick_followups": [
305             "Покажи график загрузки ЦПУ по времени",
306             "Топ процессов по нагрузке на ЦП",
307             "Какая средняя температура видеокарты и процессора?",
308         ],
309     }
310
311     async def _handle_top_processes_query(
312         self, user_message: str, source_path: Optional[str] =

```

```

None
313     ) -> Dict[str, Any]:
314         """Сформировать ответ по топ-процессам."""
315         proc_data = self.query_engine.get_top_processes_summary(limit=10, db_path=source_path)
316         processes = proc_data.get("processes", [])
317
318         if not processes:
319             return {
320                 "status": "ok",
321                 "reply": "В исследуемой выборке не обнаружено записей о процессах.",
322                 "chart": None,
323                 "sql_queries": [proc_data.get("sql", "")],
324                 "metrics_summary": {},
325                 "quick_followups": ["Покажи график загрузки ЦП
У по времени"],
326             }
327
328         rows_md = []
329         for idx, p in enumerate(processes, 1):
330             rows_md.append(f"| {idx} | **{p.get('name')}** | {
p.get('avg_cpu', 0)}% | {p.get('max_cpu', 0)}% | {p.get('avg_r
am_mb', 0)} MB |")
331         table_md = "\n".join(rows_md)
332
333         explanation = (
334             f"### □ Топ ресурсоемких процессов по загрузке ЦП
и памяти\n\n"
335             f"Рейтинг сформирован на основе снимков процессов
(`process_snapshots` / `process_rollups`):\n\n"
336             f"| № | Процесс | Средний CPU | Пиковый CPU | Памя
ть RAM | \n"
337             f"| :- | :--- | :--- | :--- | :--- | \n"
338             f"{table_md}\n\n"
339         )
340
341         return {

```



```

342         "status": "ok",
343         "reply": explanation,
344         "chart": proc_data.get("chart"),
345         "sql_queries": [proc_data.get("sql", "")],
346         "metrics_summary": {
347             "top_process_name": processes[0].get("name") i
f processes else "-",
348             "top_process_cpu": f"{processes[0].get('avg_cp
u', 0)}%" if processes else "0%",
349         },
350         "quick_followups": [
351             "Покажи график загрузки ЦПУ по времени",
352             "Посчитай суточное потребление энергии (кругов
ая диаграмма)",
353         ],
354     }
355
356     async def _handle_client_resources_query(
357         self, user_message: str, source_path: Optional[str] =
None
358     ) -> Dict[str, Any]:
359         """Сформировать подробный отчет по ресурсам программ к
лиента и оценке неизвестных процессов в Gemini."""
360         summary = await self.client_analyzer.analyze_client_re
sources(
361             db_path=source_path, ask_gemini_for_unknown=True,
limit=20
362         )
363         progs = summary.programs
364
365         if not progs:
366             return {
367                 "status": "ok",
368                 "reply": "В текущей телеметрии не обнаружен
о активных программ, запущенных клиентом.",
369                 "chart": None,
370                 "sql_queries": [],
371                 "metrics_summary": {"total_programs": 0},

```

```

372 |         "quick_followups": ["Покажи график загрузки ЦП
    | У по времени"],
373 |     }
374 |
375 |     # Таблица распределения ресурсов
376 |     rows_md = []
377 |     for idx, p in enumerate(progs[:12], 1):
378 |         disk_str = f"{p.disk_size_mb} MB" if p.disk_size_m
    | b else "N/A"
379 |         status_badge = "🟢 Известная" if p.is_known_softwar
    | e else ("🟡 Неизвестная (Heavy)" if p.is_resource_heavy else "i
    | Неизвестная")
380 |         rows_md.append(
381 |             f"| {idx} | **{p.display_name or p.name}** | `
    | {p.name}` | {p.cpu_percent}% | {p.memory_mb} MB | {disk_str} |
    | {status_badge} |"
382 |         )
383 |     table_md = "\n".join(rows_md)
384 |
385 |     explanation = (
386 |         f"### 📊 Ресурсы программ, запущенных клиентом\n\n
    | "
387 |         f"Проведен детальный аудит запущенного клиентского
    | ПО и занимаемых ресурсов (память RAM, нагрузка CPU, файлы на
    | диске):\n\n"
388 |         f"- 📊 **Всего клиентских программ:** `{summary.to
    | tal_client_programs}`\n"
389 |         f"- 📊 **Суммарно занято памяти RAM:** **`${summary
    | .total_ram_mb} MB`** (~`${round(summary.total_ram_mb / 1024, 2)
    | } GB`)\n"
390 |         f"- 📊 **Суммарная нагрузка на ЦП:** **`${summary.to
    | tal_cpu_percent}%`**\n"
391 |         f"- 📊 **Общий объем файлов на диске:** `{summary.
    | total_disk_size_mb} MB`\n"
392 |         f"- 📊 **Неизвестных ресурсоемких программ:** **`${
    | summary.unknown_heavy_programs_count}`**\n\n"
393 |         f"#### 📋 Детализация по программам:\n\n"
394 |         f"| № | Программа | Процесс | CPU (%) | RAM (MB) |

```

```

Диск (файл) | Статус |\n"
395         f"| :- | :--- | :--- | :--- | :--- | :--- | :--- |
\n"
396         f"{table_md}\n\n"
397     )
398
399     # Вывод оценок от Gemini для неизвестных тяжелых прогр
амм
400     if summary.evaluations:
401         explanation += "### □ Экспертное заключение модел
и Gemini по неизвестным ресурсоемким программам:\n\n"
402         for ev in summary.evaluations:
403             norm_icon = "□ Штатно" if ev.is_normal else "□
□ Аномалия / Риск"
404             explanation += (
405                 f"#### □ Программа: `{ev.program_name}` (
PID: {ev.pid})\n"
406                 f"- **Вердикт Gemini:** `{ev.verdict}` (
{norm_icon}, Риск: `{ev.risk_level.upper()}`)\n"
407                 f"- **Потребление ресурсов:** CPU `{ev.cpu
_percent}%`, RAM `{ev.memory_mb} MB`, Диск `{ev.disk_io_mb_s}
MB/s`\n"
408                 f"- **Путь запуска:** `{ev.executable_path
or 'N/A'}`\n"
409                 f"- **Анализ:** {ev.analysis}\n"
410             )
411             if ev.recommendations:
412                 recs_str = "\n".join([f" * {r}" for r in
ev.recommendations])
413                 explanation += f"- **Рекомендации:**\n{rec
s_str}\n\n"
414             else:
415                 explanation += "□ **Все ресурсоемкие программы кла
сифицированы как известные и доверенные.** Аномальных неизвес
тных процессов не обнаружено.\n"
416
417     # Chart configuration for top client programs RAM & CP
U

```

```

418 |         chart_labels = [p.display_name or p.name for p in prog
| s[:8]]
419 |         chart_ram = [p.memory_mb for p in progs[:8]]
420 |         chart = {
421 |             "type": "bar",
422 |             "title": "Потребление оперативной памяти RAM клиен
| тскими программами (МБ)",
423 |             "labels": chart_labels,
424 |             "datasets": [
425 |                 {
426 |                     "label": "Память RAM (МБ)",
427 |                     "data": chart_ram,
428 |                     "backgroundColor": "rgba(99, 102, 241, 0.7
| )",
429 |                     "borderColor": "#6366f1",
430 |                     "borderWidth": 1,
431 |                 }
432 |             ],
433 |         }
434 |
435 |         return {
436 |             "status": "ok",
437 |             "reply": explanation,
438 |             "chart": chart,
439 |             "sql_queries": [],
440 |             "metrics_summary": {
441 |                 "total_client_programs": summary.total_client_
| programs,
442 |                 "total_ram_mb": f"{summary.total_ram_mb} MB",
443 |                 "total_cpu_percent": f"{summary.total_cpu_perc
| ent}%",
444 |                 "unknown_heavy_count": summary.unknown_heavy_p
| rograms_count,
445 |             },
446 |             "quick_followups": [
447 |                 "Покажи график загрузки ЦПУ по времени",
448 |                 "Посчитай суточное потребление энергии (кругов
| ая диаграмма)",

```

```

449         "Топ процессов по нагрузке на ЦП",
450     ],
451 }
452
453 def _handle_direct_sql_query(
454     self, sql_query: str, source_path: Optional[str] = Non
455 e
456 ) -> Dict[str, Any]:
457     """Выполнить прямой SELECT SQL-запрос."""
458     res = self.query_engine.execute_safe_sql(sql_query, db
459 _path=source_path)
460     if res.get("status") != "ok":
461         return {
462             "status": "error",
463             "reply": f"❌ Ошибка выполнения SQL-запроса:\n`
464 ``\n{res.get('message')}\n``",
465             "chart": None,
466             "sql_queries": [res.get("sql", sql_query)],
467             "metrics_summary": {},
468             "quick_followups": ["Покажи график загрузки ЦП
469 У по времени"],
470         }
471
472     cols = res.get("columns", [])
473     rows = res.get("rows", [])
474     count = res.get("count", 0)
475
476     header_md = "| " + " | ".join(cols) + " |"
477     sep_md = "| " + " | ".join([":---" * len(cols)] + " |"
478 "
479
480     data_md = []
481     for r in rows[:15]:
482         vals = [str(r.get(c, "")) for c in cols]
483         data_md.append("| " + " | ".join(vals) + " |")
484
485     table_str = f"{header_md}\n{sep_md}\n" + "\n".join(dat
486 a_md)

```

```

481         explanation = (
482             f"### □ Результаты SQL-запроса к telemetry.db\n\n
483             f"Возвращено строк: **{count}**\n\n"
484             f"{table_str}\n\n"
485         )
486         if count > 15:
487             explanation += f"*Отображены первые 15 строк из {c
488 out}\n"
489         return {
490             "status": "ok",
491             "reply": explanation,
492             "chart": None,
493             "sql_queries": [res.get("sql", "")],
494             "metrics_summary": {"rows_count": count},
495             "quick_followups": [
496                 "Покажи график загрузки ЦПУ по времени",
497                 "Посчитай суточное потребление энергии (кругов
498 ая диаграмма)",
499             ],
500         }
501     async def _handle_general_ai_query(
502         self,
503         user_message: str,
504         source_path: Optional[str] = None,
505         history: Optional[List[Dict[str, Any]]] = None,
506     ) -> Dict[str, Any]:
507         """Обработать общий запрос через LLM с контекстом схем
508 ы базы данных."""
509         schema_info = self.query_engine.get_schema_summary(db_
510 path=source_path)
511         context_prompt = (
512             f"Пользователь задал вопрос о телеметрии Windows:
513             '{user_message}'.\n"
514             f"Доступные таблицы в telemetry.db: {list(schema_i

```

```

nfo.get('tables', {}).keys())}.\\n"
513         f"Ответь информативно и структурированно на русско
м языке. "
514         f"Если пользователю нужны графики, предложи ему го
товые фразы ('покажи график загрузки цпу по времени' или 'посч
итай потребление энергии')."
515     )
516
517     reply_text = ""
518     if self.chat_model and hasattr(self.chat_model, "ask")
:
519         try:
520             reply_text = await asyncio.wait_for(self.chat_
model.ask(context_prompt), timeout=4.0)
521         except Exception as ex:
522             logger.warning(f"Ошибка вызова LLM: {ex}")
523
524     if not reply_text:
525         reply_text = (
526             f"Я готов помочь вам с исследованием системной
телеметрии Windows!\\n\\n"
527             f"**Доступные команды и возможности:**\\n"
528             f"- □ **График ЦП:** введите *«покажи график
загрузки цпу по времени»*\\n"
529             f"- □ **Энергопотребление:** введите *«Посчита
й примерное общее потребление электроэнергии компьютером и пок
ажи суточную круговую диаграмму»*\\n"
530             f"- □ **Топ процессов:** введите *«топ процес
сов по сру»*\\n"
531             f"- □ **SQL-запросы:** введите любой запрос,
начинающийся с `SELECT ...` к базе `telemetry.db`"
532         )
533
534     return {
535         "status": "ok",
536         "reply": reply_text,
537         "chart": None,
538         "sql_queries": [],

```

```
539 |         "metrics_summary": {},
540 |         "quick_followups": [
541 |             "Покажи график загрузки цпу по времени",
542 |             "Посчитай суточное потребление энергии (кругов
    | ая диаграмма)",
543 |             "Топ процессов по нагрузке на ЦП",
544 |         ],
545 |     }
546 |
```


audit_startup_checker.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Audit Startup Checker
5 # Description:
6 #     Проверка критических аудитов при запуске приложения.
7 #
8 # Usage Examples:
9 #     Python API:
10 #         from apps.windows.telemetry_research.audit_startup_checker import StartupAuditResult
11 #
12 #         service = StartupAuditResult()
13 #
14 # File: audit_startup_checker.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Проверка критических аудитов при запуске приложения."""
24
25 import time
26 from dataclasses import dataclass, field
27 from typing import Any, Dict, List, Optional
28
29 try:
30     from logger import logger
31 except ImportError:
```

```

32 | import logging
33 | logger = logging.getLogger(__name__)
34 |
35 |
36 | @dataclass
37 | class StartupAuditResult:
38 |     """Результат проверки аудитов при запуске."""
39 |     is_healthy: bool
40 |     critical_count: int = 0
41 |     warning_count: int = 0
42 |     findings: List[Dict[str, Any]] = field(default_factory=list)
43 |
44 |     checks_passed: List[str] = field(default_factory=list)
45 |     duration_ms: float = 0.0
46 |
47 |     def to_dict(self) -> Dict[str, Any]:
48 |         return {
49 |             'is_healthy': self.is_healthy,
50 |             'critical_count': self.critical_count,
51 |             'warning_count': self.warning_count,
52 |             'findings': self.findings,
53 |             'checks_passed': self.checks_passed,
54 |             'duration_ms': self.duration_ms
55 |         }
56 |
57 | class AuditStartupChecker:
58 |     """Проверяет критические состояния системы при запуске."""
59 |
60 |     def __init__(self) -> None:
61 |         """Инициализирует чекер с ленивой загрузкой коллекторов. """
62 |         self._integrity_collector = None
63 |         self._performance_collector = None
64 |         self._driver_collector = None
65 |         self._eventlog_collector = None
66 |
67 |     def _get_integrity_collector(self):

```

```
68 |         """Ленивая загрузка IntegrityCollector."""
69 |         if self._integrity_collector is None:
70 |             from apps.windows.core.audits.integrity_collector
import IntegrityCollector
71 |             self._integrity_collector = IntegrityCollector()
72 |             return self._integrity_collector
73 |
74 |     def _get_performance_collector(self):
75 |         """Ленивая загрузка PerformanceCollector."""
76 |         if self._performance_collector is None:
77 |             from apps.windows.core.audits.performance_collecto
r import PerformanceCollector
78 |             self._performance_collector = PerformanceCollector
79 |             ()
80 |             return self._performance_collector
81 |
82 |     def _get_driver_collector(self):
83 |         """Ленивая загрузка DriverCollector."""
84 |         if self._driver_collector is None:
85 |             from apps.windows.core.audits.driver_collector imp
ort DriverCollector
86 |             self._driver_collector = DriverCollector()
87 |             return self._driver_collector
88 |
89 |     def _get_eventlog_collector(self):
90 |         """Ленивая загрузка EventLogCollector."""
91 |         if self._eventlog_collector is None:
92 |             from apps.windows.core.audits.eventlog_collector i
mport EventLogCollector
93 |             self._eventlog_collector = EventLogCollector()
94 |             return self._eventlog_collector
95 |
96 |     def check_startup_health(
97 |         self,
98 |         check_integrity: bool = True,
99 |         check_performance: bool = True,
100 |         check_drivers: bool = False,
        check_eventlog: bool = False,
```

```

101         eventlog_hours: int = 1
102     ) -> StartupAuditResult:
103         """Выполняет быструю проверку критических состояний при запуске.
104
105         Args:
106             check_integrity: Проверять pending reboot и целостность системы.
107             check_performance: Проверять критическую загрузку CPU/RAM.
108             check_drivers: Проверять проблемные драйверы (медленнее).
109             check_eventlog: Проверять критические события в логах (медленнее).
110             eventlog_hours: Сколько часов логов анализировать.
111
112         Returns:
113             StartupAuditResult с результатами проверки.
114         """
115         start_t = time.perf_counter()
116         result = StartupAuditResult(is_healthy=True)
117
118         # 1. Проверка целостности (pending reboot) - быстро
119         if check_integrity:
120             try:
121                 integrity = self._get_integrity_collector().collect()
122                 pending_reboot = integrity.metrics.get('pending_reboot', False)
123
124                 if pending_reboot:
125                     result.findings.append({
126                         'domain': 'integrity',
127                         'severity': 'warning',
128                         'title': 'Требуется перезагрузка системы',
129                         'description': 'Обнаружены отложенные операции, требующие перезагрузки'

```

```

130         })
131         result.warning_count += 1
132     else:
133         result.checks_passed.append('integrity: no
pending reboot')
134
135     except Exception as e:
136         logger.warning(f'Ошибка при проверке целостнос
ти: {e}')
137         result.checks_passed.append('integrity: check
skipped')
138
139     # 2. Проверка производительности - быстро
140     if check_performance:
141         try:
142             perf = self._get_performance_collector().colle
ct()
143             cpu_pct = perf.metrics.get('cpu_percent', 0.0)
144             mem_pct = perf.metrics.get('memory_percent', 0
.0)
145
146             if cpu_pct >= 90.0:
147                 result.findings.append({
148                     'domain': 'performance',
149                     'severity': 'critical',
150                     'title': f'Критическая загрузка CPU: {
cpu_pct:.1f}%',
151                     'description': 'Процессор загружен бол
ее 90% при запуске'
152                 })
153                 result.critical_count += 1
154                 result.is_healthy = False
155             elif cpu_pct >= 75.0:
156                 result.findings.append({
157                     'domain': 'performance',
158                     'severity': 'warning',
159                     'title': f'Повышенная загрузка CPU: {c
pu_pct:.1f}%',

```

```

160         'description': 'Процессор загружен бол
ее 75% при запуске'
161     })
162     result.warning_count += 1
163     else:
164         result.checks_passed.append(f'performance:
CPU {cpu_pct:.1f}%')
165
166         if mem_pct >= 90.0:
167             result.findings.append({
168                 'domain': 'performance',
169                 'severity': 'critical',
170                 'title': f'Критическая загрузка RAM: {
mem_pct:.1f}%',
171                 'description': 'Оперативная память зап
олнена более 90% при запуске'
172             })
173             result.critical_count += 1
174             result.is_healthy = False
175         elif mem_pct >= 80.0:
176             result.findings.append({
177                 'domain': 'performance',
178                 'severity': 'warning',
179                 'title': f'Высокая загрузка RAM: {mem_
pct:.1f}%',
180                 'description': 'Оперативная память зап
олнена более 80%'
181             })
182             result.warning_count += 1
183         else:
184             result.checks_passed.append(f'performance:
RAM {mem_pct:.1f}%')
185
186     except Exception as e:
187         logger.warning(f'Ошибка при проверке производи
тельности: {e}')
188         result.checks_passed.append('performance: chec
k skipped')

```

```

189
190     # 3. Проверка драйверов - опционально, медленнее
191     if check_drivers:
192         try:
193             drivers = self._get_driver_collector().collect
194             (
195                 problem_count = sum(
196                     1 for f in drivers.findings
197                     if f.severity.value in ('critical', 'cauti
198 on'))
199                 )
200             if problem_count > 0:
201                 result.findings.append({
202                     'domain': 'drivers',
203                     'severity': 'warning',
204                     'title': f'Обнаружено проблем с драйве
205 рами: {problem_count}',
206                     'description': 'Проверьте раздел драйв
207 еров для деталей'
208                 })
209                 result.warning_count += 1
210             else:
211                 result.checks_passed.append('drivers: no p
212 roblems')
213
214         except Exception as e:
215             logger.warning(f'Ошибка при проверке драйверов
216 : {e}')
217             result.checks_passed.append('drivers: check sk
218 ipped')
219
220     # 4. Проверка критических событий в логах - опциональн
221 о, медленнее
222     if check_eventlog:
223         try:
224             events = self._get_eventlog_collector().collec
225 t(hours=eventlog_hours)

```

```

218 |         critical_events = [
219 |             f for f in events.findings
220 |             if f.severity.value == 'critical'
221 |         ]
222 |
223 |         if critical_events:
224 |             result.findings.append({
225 |                 'domain': 'eventlog',
226 |                 'severity': 'warning',
227 |                 'title': f'Критические события в логах
: {len(critical_events)}',
228 |                 'description': 'Обнаружены критические
    события за последние часы'
229 |             })
230 |             result.warning_count += 1
231 |         else:
232 |             result.checks_passed.append('eventlog: no
critical events')
233 |
234 |         except Exception as e:
235 |             logger.warning(f'Ошибка при проверке логов: {e
}')
236 |             result.checks_passed.append('eventlog: check s
kipped')
237 |
238 |         result.duration_ms = round((time.perf_counter() - star
t_t) * 1000, 2)
239 |
240 |         # Логируем результат
241 |         if result.critical_count > 0:
242 |             logger.error(
243 |                 f'Startup audit: CRITICAL issues found: {resul
t.critical_count}, '
244 |                 f'warnings: {result.warning_count}'
245 |             )
246 |         elif result.warning_count > 0:
247 |             logger.warning(
248 |                 f'Startup audit: warnings found: {result.warni

```



```

    ng_count}, '
249         f'checks passed: {len(result.checks_passed)}'
250     )
251     else:
252         logger.info(
253             f'Startup audit: all checks passed ({result.du
ration_ms}ms)'
254         )
255
256     return result
257
258     def check_pending_reboot_only(self) -> bool:
259         """Быстрая проверка только pending reboot.
260
261         Returns:
262             True если требуется перезагрузка, False иначе.
263         """
264         try:
265             integrity = self._get_integrity_collector().collec
t()
266             return integrity.metrics.get('pending_reboot', Fal
se)
267         except Exception as e:
268             logger.warning(f'Ошибка при проверке pending reboo
t: {e}')
269             return False
270
271
272     def run_startup_audit(
273         check_integrity: bool = True,
274         check_performance: bool = True,
275         check_drivers: bool = False,
276         check_eventlog: bool = False
277     ) -> StartupAuditResult:
278         """Утилитарная функция для запуска проверки из точки входа
279         .
280
281         Args:

```

```
281 |         check_integrity: Проверять pending reboot.
282 |         check_performance: Проверять CPU/RAM.
283 |         check_drivers: Проверять драйверы.
284 |         check_eventlog: Проверять логи событий.
285 |
286 |     Returns:
287 |         StartupAuditResult с результатами.
288 |     """
289 |     checker = AuditStartupChecker()
290 |     return checker.check_startup_health(
291 |         check_integrity=check_integrity,
292 |         check_performance=check_performance,
293 |         check_drivers=check_drivers,
294 |         check_eventlog=check_eventlog
295 |     )
296 |
```

charts.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Charts
5 # Description:
6 #   Генератор графиков и HTML/SVG визуализаций телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.charts import Telem
11 #       etryChartGenerator
12 #
13 #       service = TelemetryChartGenerator()
14 #
15 # File: charts.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry_research
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-01 13:28:28
21 # =====
22
23 from __future__ import annotations
24
25 """Генератор графиков и HTML/SVG визуализаций телеметрии."""
26
27 import json
28
29 from datetime import datetime
30 from typing import Any, Dict, List, Optional, Tuple
31
32 from .models import ChartConfig, DeviceEventSummary, MetricSta
33 ts, TelemetryResearchReport, TimeSeriesDataset
```

```

31
32 class TelemetryChartGenerator:
33     """Генератор графиков и визуальных отчетов телеметрии."""
34
35     def generate_chart_configs(self, time_series: Dict[str, TimeSeriesDataset], report: TelemetryResearchReport) -> List[ChartConfig]:
36         """Сформировать стандартный набор конфигураций графиков для отчета.
37
38         Args:
39             time_series: Словарь извлеченных временных рядов.
40             report: Отчет исследования.
41
42         Returns:
43             List[ChartConfig]: Список спецификаций графиков.
44         """
45         charts: List[ChartConfig] = []
46         res_keys = [k for k in ("cpu_load", "ram_used_percent", "gpu_load") if k in time_series]
47         if res_keys:
48             labels, datasets = self._merge_series_for_chart(time_series, res_keys)
49             charts.append(
50                 ChartConfig(
51                     id="chart_resource_utilization",
52                     title="Утилизация ресурсов (CPU, RAM, GPU)",
53                     chart_type="line",
54                     labels=labels,
55                     datasets=datasets,
56                     y_axis_label="Загрузка (%)",
57                     description="Динамика потребления вычислительных ресурсов системы во времени.",
58                 )
59             )
60
61         temp_keys = [k for k in ("cpu_temp", "gpu_temp") if k

```

```

    in time_series]
62         if temp_keys:
63             labels, datasets = self._merge_series_for_chart(ti
me_series, temp_keys)
64             charts.append(
65                 ChartConfig(
66                     id="chart_temperatures",
67                     title="Температурный профиль оборудования"
68                     ,
69                     chart_type="line",
70                     labels=labels,
71                     datasets=datasets,
72                     y_axis_label="Температура (°C)",
73                     description="Мониторинг нагрева процессора
и графического ускорителя.",
74                 )
75             )
76         disk_keys = [k for k in ("disk_read_mb_s", "disk_write
_mb_s") if k in time_series]
77         if disk_keys:
78             labels, datasets = self._merge_series_for_chart(ti
me_series, disk_keys)
79             charts.append(
80                 ChartConfig(
81                     id="chart_disk_io",
82                     title="Пропускная способность дисковой под
системы",
83                     chart_type="line",
84                     labels=labels,
85                     datasets=datasets,
86                     y_axis_label="Скорость (MB/s)",
87                     description="Потоки чтения и записи на нак
опителях.",
88                 )
89             )
90
91         net_keys = [k for k in ("net_recv_mb_s", "net_sent_mb_

```



```

123         description="Структура активности и сбоев
| подключенного оборудования.",
124     )
125 )
126
127     if report.statistics:
128         stat_labels = list(report.statistics.keys())
129         avg_vals = [report.statistics[k].avg_val for k in
| stat_labels]
130         p95_vals = [report.statistics[k].p95_val for k in
| stat_labels]
131         max_vals = [report.statistics[k].max_val for k in
| stat_labels]
132         charts.append(
133             ChartConfig(
134                 id="chart_eda_stats",
135                 title="Сравнительный профиль метрик (Avg,
| p95, Max)",
136                 chart_type="bar",
137                 labels=stat_labels,
138                 datasets=[
139                     {"label": "Среднее (Avg)", "data": avg
| _vals, "backgroundColor": "#3B82F6"},
140                     {"label": "95-й перцентиль (p95)", "da
| ta": p95_vals, "backgroundColor": "#F59E0B"},
141                     {"label": "Максимум (Max)", "data": ma
| x_vals, "backgroundColor": "#EF4444"},
142                 ],
143                 y_axis_label="Значение",
144                 description="Сравнение типичных и пиковых
| нагрузок по ключевым показателям.",
145             )
146         )
147
148     return charts
149
150 def _merge_series_for_chart(
151     self, time_series: Dict[str, TimeSeriesDataset], keys:

```

```

    List[str], max_points: int = 150
152     ) -> Tuple[List[str], List[Dict[str, Any]]]:
153         """Свести несколько временных рядов в синхронизированн
    ые списки для графика."""
154         all_timestamps = set()
155         for k in keys:
156             for pt in time_series[k].points:
157                 all_timestamps.add(pt.timestamp)
158
159         sorted_ts = sorted(list(all_timestamps))
160         if len(sorted_ts) > max_points:
161             step = max(1, len(sorted_ts) // max_points)
162             sample_ts = sorted_ts[::step]
163         else:
164             sample_ts = sorted_ts
165
166         labels = [self._format_ts_label(t) for t in sample_ts]
167         datasets = []
168         for k in keys:
169             ds = time_series[k]
170             pt_dict = {p.timestamp: p.value for p in ds.points
    }
171             data_points = [pt_dict.get(t, None) for t in sampl
    e_ts]
172             datasets.append(
173                 {
174                     "label": f"{ds.name} ({ds.unit})",
175                     "data": data_points,
176                     "borderColor": ds.color or "#3B82F6",
177                     "backgroundColor": (ds.color or "#3B82F6")
    + "22",
178                     "fill": False,
179                     "tension": 0.3,
180                 }
181             )
182         return (labels, datasets)
183
184     @staticmethod

```



```

185     def _format_ts_label(iso_ts: str) -> str:
186         """Сформировать короткую человекочитаемую подпись врем
ени."""
187         try:
188             dt = datetime.fromisoformat(iso_ts.replace("Z", "+
00:00"))
189             return dt.strftime("%H:%M:%S")
190         except Exception:
191             return iso_ts.split("T")[-1][:8] if "T" in iso_ts
else iso_ts[-8:]
192
193     def render_svg_chart(self, chart: ChartConfig, width: int
= 800, height: int = 400) -> str:
194         """Сгенерировать автономный векторный SVG график.
195
196         Args:
197             chart: Конфигурация графика.
198             width: Ширина SVG холста в пикселях.
199             height: Высота SVG холста в пикселях.
200
201         Returns:
202             str: XML-строка с SVG графиком.
203         """
204         padding = 50
205         graph_w = width - 2 * padding
206         graph_h = height - 2 * padding
207
208         lines_svg = [
209             f'<rect width="{width}" height="{height}" fill="#1
E293B" rx="8" />',
210             f'<text x="{width // 2}" y="30" fill="#F8FAFC" fon
t-size="16" font-family="sans-serif" font-weight="bold" text-a
nchor="middle">{chart.title}</text>',
211         ]
212
213         all_vals = []
214         for ds in chart.datasets:
215             for v in ds.get("data", []):

```

```

216         if v is not None and isinstance(v, (int, float
    )):
217             all_vals.append(v)
218
219         max_v = max(all_vals) if all_vals else 100.0
220         min_v = 0.0
221         val_range = max(1.0, max_v - min_v)
222
223         for i in range(5):
224             y_pos = padding + graph_h - int(i / 4.0 * graph_h)
225             grid_val = round(min_v + i / 4.0 * val_range, 1)
226             lines_svg.append(f'<line x1="{padding}" y1="{y_pos
    }" x2="{width - padding}" y2="{y_pos}" stroke="#334155" stroke
    -dasharray="4" />')
227             lines_svg.append(f'<text x="{padding - 10}" y="{y_
    pos + 4}" fill="#94A3B8" font-size="11" font-family="sans-seri
    f" text-anchor="end">{grid_val}</text>')
228
229         n_points = len(chart.labels)
230         if n_points > 1:
231             x_step = graph_w / (n_points - 1)
232             for ds in chart.datasets:
233                 color = ds.get("borderColor", "#38BDF8")
234                 points_str = []
235                 for idx, v in enumerate(ds.get("data", [])):
236                     if v is not None:
237                         x = padding + idx * x_step
238                         norm_y = (v - min_v) / val_range
239                         y = padding + graph_h - norm_y * graph
    _h
240                 points_str.append(f"{x:.1f},{y:.1f}")
241                 lines_svg.append(f'<circle cx="{x:.1f}
    " cy="{y:.1f}" r="3" fill="{color}" />')
242                 if points_str:
243                     polyline = " ".join(points_str)
244                     lines_svg.append(f'<polyline fill="none" s
    troke="{color}" stroke-width="2.5" points="{polyline}" />')
245

```

```

246         content = "\n".join(lines_svg)
247         return f'<svg xmlns="http://www.w3.org/2000/svg" viewB
ox="0 0 {width} {height}" width="{width}" height="{height}">\n
{content}\n</svg>'
248
249     def render_html_dashboard(self, report: TelemetryResearchR
eport) -> str:
250         """Сгенерировать автономный интерактивный HTML-дашборд
с графиками."""
251         charts_json = json.dumps([c.model_dump() for c in repo
rt.charts], ensure_ascii=False)
252         score_color = "#10B981" if report.health_score >= 85 e
lse "#F59E0B" if report.health_score >= 60 else "#EF4444"
253
254         anomalies_rows = ""
255         for a in report.anomalies:
256             badge_cls = "badge-danger" if a.severity == "criti
cal" else "badge-warning"
257             anomalies_rows += f"""
258                 <tr>
259                     <td><span class="badge {badge_cls}">{a.severit
y.upper()}</span></td>
260                     <td>{a.timestamp}</td>
261                     <td><strong>{a.metric}</strong></td>
262                     <td>{a.value} (норм: {a.threshold})</td>
263                     <td>{a.description}</td>
264                 </tr>
265             """
266
267         conclusions_list = "".join(f"<li>{c}</li>" for c in re
port.summary_conclusions)
268
269         return f"""<!DOCTYPE html>
270 <html lang="ru">
271 <head>
272     <meta charset="UTF-8">
273     <meta name="viewport" content="width=device-width, initial
-scale=1.0">

```

```
274 |     <title>AI-Breadboard: Исследование телеметрии #{report.rep
    | ort_id}</title>
275 |     <script src="https://cdn.jsdelivr.net/npm/chart.js"></scri
    | pt>
276 |     <style>
277 |         :root {{
278 |             --bg: #0F172A;
279 |             --surface: #1E293B;
280 |             --surface-hover: #334155;
281 |             --border: #475569;
282 |             --text-main: #F8F AFC;
283 |             --text-muted: #94A3B8;
284 |             --primary: #38BDF8;
285 |             --success: #10B981;
286 |             --warning: #F59E0B;
287 |             --danger: #EF4444;
288 |         }}
289 |         * {{ box-sizing: border-box; margin: 0; padding: 0; }}
290 |         body {{
291 |             font-family: -apple-system, BlinkMacSystemFont, 'S
    | egoe UI', Roboto, Helvetica, Arial, sans-serif;
292 |             background-color: var(--bg);
293 |             color: var(--text-main);
294 |             padding: 24px;
295 |             line-height: 1.5;
296 |         }}
297 |         .header {{
298 |             display: flex;
299 |             justify-content: space-between;
300 |             align-items: center;
301 |             border-bottom: 1px solid var(--border);
302 |             padding-bottom: 20px;
303 |             margin-bottom: 24px;
304 |         }}
305 |         .health-card {{
306 |             background: var(--surface);
307 |             padding: 16px 24px;
308 |             border-radius: 12px;
```

```
309 |         border-left: 6px solid {score_color};
310 |         text-align: center;
311 |     }}
312 |     .health-value {{
313 |         font-size: 36px;
314 |         font-weight: 800;
315 |         color: {score_color};
316 |     }}
317 |     .grid-cards {{
318 |         display: grid;
319 |         grid-template-columns: repeat(auto-fit, minmax(280
px, 1fr));
320 |         gap: 16px;
321 |         margin-bottom: 24px;
322 |     }}
323 |     .card {{
324 |         background: var(--surface);
325 |         border-radius: 12px;
326 |         padding: 20px;
327 |         border: 1px solid var(--border);
328 |     }}
329 |     .charts-grid {{
330 |         display: grid;
331 |         grid-template-columns: repeat(auto-fit, minmax(500
px, 1fr));
332 |         gap: 24px;
333 |         margin-bottom: 24px;
334 |     }}
335 |     .chart-container {{
336 |         background: var(--surface);
337 |         border-radius: 12px;
338 |         padding: 20px;
339 |         border: 1px solid var(--border);
340 |         position: relative;
341 |     }}
342 |     table {{
343 |         width: 100%;
344 |         border-collapse: collapse;
```

```
345         margin-top: 12px;
346     }}
347     th, td {{
348         padding: 10px 14px;
349         text-align: left;
350         border-bottom: 1px solid var(--border);
351         font-size: 14px;
352     }}
353     th {{ color: var(--text-muted); font-weight: 600; }}
354     .badge {{
355         display: inline-block;
356         padding: 4px 8px;
357         border-radius: 6px;
358         font-size: 12px;
359         font-weight: bold;
360     }}
361     .badge-danger {{ background: rgba(239, 68, 68, 0.2); color: #F87171; }}
362     .badge-warning {{ background: rgba(245, 158, 11, 0.2); color: #FBBF24; }}
363     ul {{ padding-left: 20px; }}
364     li {{ margin-bottom: 8px; }}
365 </style>
366 </head>
367 <body>
368     <div class="header">
369         <div>
370             <h1>Исследование телеметрии Windows</h1>
371             <p style="color: var(--text-muted);">Отчет: {report.report_id} | Создан: {report.generated_at}</p>
372         </div>
373         <div class="health-card">
374             <div style="font-size: 13px; color: var(--text-muted); text-transform: uppercase;">Индекс здоровья</div>
375             <div class="health-value">{report.health_score}%</div>
376         </div>
377     </div>
```

```

378
379     <div class="grid-cards">
380         <div class="card">
381             <h3 style="color: var(--primary); margin-bottom: 8
382 px;">□ Объем выборки</h3>
383             <p><strong>Записей в логах:</strong> {report.recor
384 ds_analyzed}</p>
385             <p><strong>Период:</strong> {report.time_window_st
386 art or 'N/A'} – {report.time_window_end or 'N/A'}</p>
387         </div>
388         <div class="card">
389             <h3 style="color: var(--warning); margin-bottom: 8
390 px;">□ Аномалии и сбои</h3>
391             <p><strong>Обнаружено аномалий:</strong> {len(repo
392 rt.anomalies)}</p>
393             <p><strong>Ошибок устройств:</strong> {report.devi
394 ce_summary.error_count}</p>
395         </div>
396         <div class="card">
397             <h3 style="color: var(--success); margin-bottom: 8
398 px;">□ Выводы исследования</h3>
399             <ul>{conclusions_list}</ul>
400         </div>
401     </div>
402
403     <h2 style="margin-bottom: 16px;">□ Графики телеметрии</h2>
404 >
405     <div class="charts-grid" id="chartsContainer"></div>
406
407     <div class="card" style="margin-top: 24px;">
408         <h2 style="margin-bottom: 12px;">□ Зафиксированные аномалии и выбросы</h2>
409         {f'<table><thead><tr><th>Уровень</th><th>Время</th><th>Метрика</th><th>Значение</th><th>Описание</th></tr></thead><tbody>{anomalies_rows}</tbody></table>' if report.anomalies else "<p style='color: var(--text-muted);'>Аномалий за исследуемый период не выявлено.</p>"}
410     </div>

```

```

403
404     <script>
405         const chartConfigs = {charts_json};
406         const container = document.getElementById('chartsConta
407 iner');
408         chartConfigs.forEach((cfg) => {{
409             const wrapper = document.createElement('div');
410             wrapper.className = 'chart-container';
411             wrapper.innerHTML = `
412                 <h3 style="margin-bottom: 4px;">${{cfg.title}}
413 </h3>
414                 <p style="color: var(--text-muted); font-size:
415 13px; margin-bottom: 16px;">${{cfg.description || ''}}</p>
416                 <canvas id="${{cfg.id}}"></canvas>
417             `;
418             container.appendChild(wrapper);
419             const ctx = document.getElementById(cfg.id).getCon
420 text('2d');
421             new Chart(ctx, {{
422                 type: cfg.chart_type,
423                 data: {{
424                     labels: cfg.labels,
425                     datasets: cfg.datasets
426                 }},
427                 options: {{
428                     responsive: true,
429                     maintainAspectRatio: true,
430                     plugins: {{
431                         legend: {{ labels: {{ color: '#F8FAFC'
432 }} }}
433                     }},
434                     scales: cfg.chart_type === 'doughnut' ? {{
435 }} : {{
436                     x: {{ ticks: {{ color: '#94A3B8' }} }, g
437 rid: {{ color: '#334155' }} }},
438                     y: {{ ticks: {{ color: '#94A3B8' }} }, g

```



```
rid: {{ color: '#334155' }}, title: {{ display: true, text: cf
g.y_axis_label, color: '#94A3B8' }} }}
434 |         }}
435 |         }}
436 |     }));
437 |     });
438 | </script>
439 </body>
440 </html>
441 ""
442
```

cli.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Cli
5 # Description:
6 #   CLI интерфейс для исследования логов телеметрии и построения графиков.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry_research.cli
11 #   Python API:
12 #       from apps.windows.telemetry_research.cli import parse_args
13 #
14 #       res = parse_args()
15 #
16 # File: cli.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry_research
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-01 13:28:28
22 # =====
23
24 from __future__ import annotations
25 """CLI интерфейс для исследования логов телеметрии и построения графиков."""
26
27 import argparse
28 import json
29 import sys
```

```
30 from pathlib import Path
31
32 _root_dir = Path(__file__).resolve().parents[3]
33 if str(_root_dir) not in sys.path:
34     sys.path.insert(0, str(_root_dir))
35
36 try:
37     from logger import logger
38 except ImportError:
39     from logger import logger
40
41 from .analyzer import TelemetryResearcher
42 from .charts import TelemetryChartGenerator
43 from .models import ResearchScenarioRequest
44
45
46 def parse_args() -> argparse.Namespace:
47     """Парсинг аргументов командной строки."""
48     parser = argparse.ArgumentParser(
49         description="AI-Breadboard: Исследование логов телемет
50     рии и генерация графиков",
51         formatter_class=argparse.RawDescriptionHelpFormatter,
52     )
53     parser.add_argument(
54         "--source",
55         "-s",
56         type=str,
57         default=None,
58         help="Путь к файлу логов (.json, .jsonl, .db) или дире
59     ктории с логами телеметрии",
60     )
61     parser.add_argument(
62         "--output",
63         "-o",
64         type=str,
65         default=None,
66         help="Путь для сохранения результата (по умолчанию: te
67    lemetry_research_report.html)",
```

```

65 |     )
66 |     parser.add_argument(
67 |         "--format",
68 |         "-f",
69 |         choices=["html", "json", "svg", "summary"],
70 |         default="html",
71 |         help="Формат вывода: html, json, svg, summary",
72 |     )
73 |     return parser.parse_args()
74 |
75 |
76 | def main() -> int:
77 |     """Точка входа CLI."""
78 |     args = parse_args()
79 |     researcher = TelemetryResearcher()
80 |     chart_gen = TelemetryChartGenerator()
81 |
82 |     logger.info(
83 |         f"Запуск исследования логов телеметрии (источник: {arg
84 | s.source or 'системные директории по умолчанию'})..."
85 |     )
86 |
87 |     if args.format == "summary":
88 |         req = ResearchScenarioRequest(source_path=args.source)
89 |         deep_report = researcher.run_deep_research(scenario=req, chart_generator=chart_gen)
90 |         print("\n" + "=" * 70)
91 |         print(f"  Отчет исследования телеметрии: {deep_report
92 | .report_id}")
93 |         print("=" * 70)
94 |         print(f"Дата формирования: {deep_report.generated_at}"
95 | )
96 |         print(f"Проанализировано записей: {deep_report.base_re
97 | port.records_analyzed}")
98 |         print(f"Индекс здоровья системы: {deep_report.base_rep
99 | ort.health_score}/100")
100 |         print(f"Обнаружено аномалий: {len(deep_report.base_rep
101 | ort.anomalies)}")

```

```

96         print("\n--- Проверенные гипотезы ---")
97         for h in deep_report.hypotheses:
98             status_icon = "✅ [ПОДТВЕРЖДЕНА]" if h.confirmed else "❌ [В ПОРЯДКЕ]"
99             print(f"{status_icon} {h.title} (уверенность {h.confidence * 100:.0f}%)")
100             for ev in h.evidence:
101                 print(f"    • {ev}")
102         print("\n--- Рекомендации ---")
103         for rec in deep_report.actionable_recommendations:
104             print(f"    📌 {rec}")
105         print("=" * 70 + "\n")
106         return 0
107
108     report = researcher.analyze(args.source)
109     records = researcher.extractor.load_all_records(args.source)
110     ts_map = researcher._extract_time_series(records)
111     report.charts = chart_gen.generate_chart_configs(ts_map, report)
112
113     out_path_str = args.output or ("telemetry_research_report.html" if args.format == "html" else "report.json")
114     out_path = Path(out_path_str)
115     out_path.parent.mkdir(parents=True, exist_ok=True)
116
117     if args.format == "html":
118         html_content = chart_gen.render_html_dashboard(report)
119         out_path.write_text(html_content, encoding="utf-8")
120         print(f"✅ HTML дашборд с графиками сохранен в: {out_path.resolve()}")
121     elif args.format == "json":
122         json_content = json.dumps(report.model_dump(), ensure_ascii=False, indent=2)
123         out_path.write_text(json_content, encoding="utf-8")
124         print(f"✅ JSON отчет сохранен в: {out_path.resolve()}")
125     elif args.format == "svg":

```

```
126         for chart in report.charts:
127             svg_file = out_path.parent / f"{chart.id}.svg"
128             svg_content = chart_gen.render_svg_chart(chart)
129             svg_file.write_text(svg_content, encoding="utf-8")
130             print(f"    SVG график сохранен в: {svg_file.resolve  
    (())}")
131
132     print("\n--- Результаты исследования ---")
133     print(f"Проанализировано записей: {report.records_analyzed  
    }")
134     print(f"Индекс здоровья системы (Health Score): {report.he  
    alth_score}%")
135     print(f"Обнаружено аномалий: {len(report.anomalies)}")
136     print(f"Сгенерировано графиков: {len(report.charts)}")
137     for conclusion in report.summary_conclusions:
138         print(f"    • {conclusion}")
139
140     return 0
141
142
143 if __name__ == "__main__":
144     sys.exit(main())
145
```

client_resource_analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Client Resource Analyzer
5 # =====
6 # Description:
7 #   Анализ ресурсов программ, запущенных клиентом (память, процессор, диск),
8 #   и экспертная оценка аномальных/неизвестных ресурсоемких процессов через модель Gemini.
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry_research.client_resource_analyzer import ClientResourceAnalyzer
13 #       analyzer = ClientResourceAnalyzer()
14 #       summary = await analyzer.analyze_client_resources()
15 #
16 # File: client_resource_analyzer.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry_research
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-04 07:47:00
22 # =====
23
24 from __future__ import annotations
25 """Анализ ресурсов программ, запущенных клиентом, и оценка неизвестных процессов через Gemini."""
26
27 import asyncio
28 import json
```

```
29 import os
30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Set, Tuple, Union
33
34 try:
35     from logger import logger
36 except ImportError:
37     import logging
38     logger = logging.getLogger(__name__)
39
40 from .models import ClientProgramResource, ClientResourceSummary, UnknownProgramEvaluation
41 from .query_engine import TelemetryQueryEngine
42
43 try:
44     from src.ai.orchestration.unified_chat import UnifiedChatModel
45 except Exception as ex:
46     logger.warning(f"Не удалось импортировать UnifiedChatModel: {ex}")
47     UnifiedChatModel = None
48
49 # Базовый список системных процессов ядра Windows (не являются программами клиента)
50 SYSTEM_PROCESS_NAMES: Set[str] = {
51     "system", "system idle process", "idle", "registry", "smss.exe", "csrss.exe",
52     "wininit.exe", "services.exe", "lsass.exe", "svchost.exe", "fontdrvhost.exe",
53     "dwm.exe", "spoolsv.exe", "conhost.exe", "sihost.exe", "taskhostw.exe",
54     "winlogon.exe", "memory compression", "werfault.exe", "wmiprvse.exe",
55     "wmiprvse.exe", "searchindexer.exe", "audiodg.exe", "dashost.exe",
56     "securityhealthservice.exe", "msmpeng.exe", "nissrv.exe",
```



```
"smartscreen.exe",
57     "lsiso.exe", "rundll32.exe", "dllhost.exe"
58 }
59
60 # Каталог известных доверенных клиентских программ
61 KNOWN_CLIENT_APPS: Dict[str, Dict[str, str]] = {
62     "chrome.exe": {"display_name": "Google Chrome", "publisher": "Google LLC", "category": "Браузер"},
63     "msedge.exe": {"display_name": "Microsoft Edge", "publisher": "Microsoft Corporation", "category": "Браузер"},
64     "firefox.exe": {"display_name": "Mozilla Firefox", "publisher": "Mozilla Foundation", "category": "Браузер"},
65     "brave.exe": {"display_name": "Brave Browser", "publisher": "Brave Software", "category": "Браузер"},
66     "opera.exe": {"display_name": "Opera Browser", "publisher": "Opera Norway", "category": "Браузер"},
67     "code.exe": {"display_name": "Visual Studio Code", "publisher": "Microsoft Corporation", "category": "Разработка"},
68     "devenv.exe": {"display_name": "Visual Studio", "publisher": "Microsoft Corporation", "category": "Разработка"},
69     "pycharm64.exe": {"display_name": "PyCharm", "publisher": "JetBrains s.r.o.", "category": "Разработка"},
70     "idea64.exe": {"display_name": "IntelliJ IDEA", "publisher": "JetBrains s.r.o.", "category": "Разработка"},
71     "telegram.exe": {"display_name": "Telegram Desktop", "publisher": "Telegram FZ-LLC", "category": "Мессенджер"},
72     "discord.exe": {"display_name": "Discord", "publisher": "Discord Inc.", "category": "Мессенджер"},
73     "slack.exe": {"display_name": "Slack", "publisher": "Slack Technologies", "category": "Мессенджер"},
74     "teams.exe": {"display_name": "Microsoft Teams", "publisher": "Microsoft Corporation", "category": "Мессенджер"},
75     "steam.exe": {"display_name": "Steam Client", "publisher": "Valve Corporation", "category": "Игры"},
76     "epicgameslauncher.exe": {"display_name": "Epic Games Launcher", "publisher": "Epic Games", "category": "Игры"},
77     "spotify.exe": {"display_name": "Spotify", "publisher": "Spotify AB", "category": "Медиа"},

```

```
78     "vlc.exe": {"display_name": "VLC Media Player", "publisher": "VideoLAN", "category": "Медиа"},
79     "explorer.exe": {"display_name": "Проводник Windows", "publisher": "Microsoft Corporation", "category": "Система"},
80     "powershell.exe": {"display_name": "PowerShell", "publisher": "Microsoft Corporation", "category": "Инструменты"},
81     "pwsh.exe": {"display_name": "PowerShell 7", "publisher": "Microsoft Corporation", "category": "Инструменты"},
82     "cmd.exe": {"display_name": "Командная строка", "publisher": "Microsoft Corporation", "category": "Инструменты"},
83     "git.exe": {"display_name": "Git", "publisher": "Software Freedom Conservancy", "category": "Разработка"},
84     "python.exe": {"display_name": "Python Interpreter", "publisher": "Python Software Foundation", "category": "Разработка"},
85     "py.exe": {"display_name": "Python Launcher", "publisher": "Python Software Foundation", "category": "Разработка"},
86     "node.exe": {"display_name": "Node.js Runtime", "publisher": "OpenJS Foundation", "category": "Разработка"},
87     "docker.exe": {"display_name": "Docker Desktop", "publisher": "Docker Inc.", "category": "Разработка"},
88     "postman.exe": {"display_name": "Postman", "publisher": "Postman Inc.", "category": "Разработка"},
89     "winrar.exe": {"display_name": "WinRAR", "publisher": "win.rar GmbH", "category": "Утилиты"},
90     "7z.exe": {"display_name": "7-Zip", "publisher": "Igor Pavlov", "category": "Утилиты"},
91     "7zfm.exe": {"display_name": "7-Zip File Manager", "publisher": "Igor Pavlov", "category": "Утилиты"},
92     "notepad.exe": {"display_name": "Блокнот", "publisher": "Microsoft Corporation", "category": "Утилиты"},
93     "notepad++.exe": {"display_name": "Notepad++", "publisher": "Don HO", "category": "Разработка"},
94 }
95
96
97 class ClientResourceAnalyzer:
98     """Анализатор потребления ресурсов клиентскими программами
```

```

    с диагностикой через Gemini."""
99
100     def __init__(
101         self,
102         query_engine: Optional[TelemetryQueryEngine] = None,
103         chat_model: Optional[Any] = None,
104         cpu_heavy_threshold: float = 15.0,
105         memory_heavy_mb_threshold: float = 500.0,
106         disk_io_heavy_mb_threshold: float = 10.0,
107     ) -> None:
108         """Инициализация анализатора клиентских ресурсов.
109
110         Args:
111             query_engine: Движок запросов к telemetry.db.
112             chat_model: Экземпляр модели AI (по умолчанию UnifiedChatModel с провайдером Gemini).
113             cpu_heavy_threshold: Порог загрузки процессора (%) для классификации как ресурсоемкий.
114             memory_heavy_mb_threshold: Порог памяти (МБ) для классификации как ресурсоемкий.
115             disk_io_heavy_mb_threshold: Порог дискового ввода-вывода (МБ/с).
116         """
117         self.query_engine = query_engine or TelemetryQueryEngine()
118         self._chat_model = chat_model
119         self.cpu_heavy_threshold = cpu_heavy_threshold
120         self.memory_heavy_mb_threshold = memory_heavy_mb_threshold
121         self.disk_io_heavy_mb_threshold = disk_io_heavy_mb_threshold
122         self._evaluations_cache: Dict[str, UnknownProgramEvaluation] = {}
123
124     @property
125     def chat_model(self) -> Any:
126         """Получить активный экземпляр языковой модели."""
127         if self._chat_model is None and UnifiedChatModel is no

```

```

t None:
128         try:
129             self._chat_model = UnifiedChatModel()
130         except Exception as ex:
131             logger.warning(f"Не удалось инициализировать U
UnifiedChatModel: {ex}")
132             self._chat_model = None
133         return self._chat_model
134
135     def is_client_program(self, proc_name: str, username: Opti
onal[str] = None, path: Optional[str] = None) -> bool:
136         """Определить, запущена ли программа клиентом (пользо
вателем), а не ядром системы.
137
138         Args:
139             proc_name: Имя исполняемого файла процесса.
140             username: Учетная запись владельца процесса.
141             path: Путь к исполняемому файлу.
142
143         Returns:
144             bool: True если процесс относится к пользовательск
им приложениям.
145         """
146         name_lower = (proc_name or "").strip().lower()
147         if not name_lower or name_lower in SYSTEM_PROCESS_NAME
S:
148             return False
149
150         # Если имя пользователя явно указывает на системные ак
каунты NT AUTHORITY
151         user_lower = (username or "").strip().lower()
152         if user_lower in {"nt authority\\system", "nt authorit
y\\local service", "nt authority\\network service", "system"}:
153             # Некоторые пользовательские программы могут запус
каться под SYSTEM как сервисы, но по умолчанию фильтруем
154             if name_lower not in KNOWN_CLIENT_APPS:
155                 return False
156

```

```

157         return True
158
159     def is_known_software(self, proc_name: str, path: Optional
[    [str] = None) -> bool:
160         """Проверить, является ли программа известным доверенн
ым ПО.
161
162         Args:
163             proc_name: Имя процесса.
164             path: Путь к файлу.
165
166         Returns:
167             bool: True если программа находится в базе известн
ых приложений или системных директорий.
168         """
169         name_lower = (proc_name or "").strip().lower()
170         if name_lower in KNOWN_CLIENT_APPS or name_lower in SY
STEM_PROCESS_NAMES:
171             return True
172
173         if path:
174             path_lower = path.lower()
175             # Программы из Program Files с известными вендорам
и
176             if ("program files" in path_lower or "windowsapps"
in path_lower) and not ("temp" in path_lower):
177                 return True
178
179         return False
180
181     def get_file_size_mb(self, path: Optional[str]) -> Optiona
l[float]:
182         """Получить размер исполняемого файла программы на дис
ке в МБ.
183
184         Args:
185             path: Путь к исполняемому файлу.
186

```

```

187 |         Returns:
188 |             Optional[float]: Размер файла в МБ или None если ф
айл недоступен.
189 |             """
190 |         if not path:
191 |             return None
192 |         try:
193 |             p = Path(path)
194 |             if p.exists() and p.is_file():
195 |                 return round(p.stat().st_size / (1024.0 * 1024
.0), 2)
196 |         except Exception:
197 |             pass
198 |         return None
199 |
200 |     def calculate_resource_score(self, cpu_pct: float, ram_mb:
float, disk_io_mbs: float) -> float:
201 |         """Рассчитать совокупный скор ресурсоемкости процесса
(0..100+).
202 |
203 |         Args:
204 |             cpu_pct: Загрузка ЦП в %.
205 |             ram_mb: Память в МБ.
206 |             disk_io_mbs: Дисковая активность в МБ/с.
207 |
208 |         Returns:
209 |             float: Индекс ресурсоемкости.
210 |             """
211 |         # Веса: 1% CPU = 1.0 балл, 100 МБ RAM = 1.0 балл, 5 МБ
/c Disk = 1.0 балл
212 |         score = (cpu_pct * 1.0) + (ram_mb / 100.0) + (disk_io_
mbs / 5.0)
213 |         return round(score, 2)
214 |
215 |     def is_heavy_resource_consumer(self, cpu_pct: float, ram_m
b: float, disk_io_mbs: float) -> bool:
216 |         """Определить, потребляет ли программа аномально много
системных ресурсов.

```

```

217 |
218 |     Args:
219 |         cpu_pct: Загрузка ЦП в %.
220 |         ram_mb: Память в МБ.
221 |         disk_io_mbs: Скорость диска в МБ/с.
222 |
223 |     Returns:
224 |         bool: True если процесс превышает установленные по
    | роговые значения.
225 |     """
226 |     if cpu_pct >= self.cpu_heavy_threshold:
227 |         return True
228 |     if ram_mb >= self.memory_heavy_mb_threshold:
229 |         return True
230 |     if disk_io_mbs >= self.disk_io_heavy_mb_threshold:
231 |         return True
232 |     return False
233 |
234 |     def load_raw_client_processes(self, db_path: Optional[Union[
    | str, Path]] = None, limit: int = 150) -> List[Dict[str, Any]]:
235 |         """Загрузить сырые данные о процессах из telemetry.db
    | и живого снимка.
236 |
237 |     Args:
238 |         db_path: Путь к базе данных SQLite.
239 |         limit: Лимит выборки записей.
240 |
241 |     Returns:
242 |         List[Dict[str, Any]]: Список словарей процессов.
243 |     """
244 |     target_db = self.query_engine.get_db_path(db_path)
245 |     raw_list: List[Dict[str, Any]] = []
246 |
247 |     if target_db and target_db.exists():
248 |         import sqlite3
249 |         try:
250 |             conn = sqlite3.connect(str(target_db), timeout

```

```

=5.0)
251         conn.row_factory = sqlite3.Row
252         cursor = conn.cursor()
253
254         # 1. Извлекаем последние снимки процессов из p
rocess_snapshots
255         cursor.execute("""
256             SELECT pid, name, cpu_percent, memory_mb,
memory_percent,
257             num_threads, num_handles, username,
read_bytes_sec, write_bytes_sec,
258             executable as executable_path, cmdl
ine as command_line
259             FROM process_snapshots
260             WHERE id IN (
261                 SELECT MAX(id) FROM process_snapshots
GROUP BY pid
262             )
263             ORDER BY cpu_percent DESC, memory_mb DESC
264             LIMIT ?
265             """, (limit,))
266         rows = cursor.fetchall()
267         for r in rows:
268             raw_list.append(dict(r))
269
270         # 2. Если таблица пуста, пробуем process_rollu
ps_daily / process_outliers
271         if not raw_list:
272             cursor.execute("""
273                 SELECT 0 as pid, name, avg_cpu_percent
as cpu_percent, avg_memory_mb as memory_mb,
274                 0.0 as memory_percent, 1 as num
_threads, 0 as num_handles,
275                 'Client' as username, 0.0 as re
ad_bytes_sec, 0.0 as write_bytes_sec,
276                 '' as executable_path, '' as co
mmand_line
277                 FROM process_rollups_daily

```



```

278 |             ORDER BY avg_cpu_percent DESC, avg_mem
    | ory_mb DESC
279 |             LIMIT ?
280 |             """, (limit,))
281 |         for r in cursor.fetchall():
282 |             raw_list.append(dict(r))
283 |
284 |         conn.close()
285 |     except Exception as ex:
286 |         logger.warning(f"Ошибка чтения процессов из SQ
    | Lite базы {target_db}: {ex}")
287 |
288 |     # 3. Дополняем активными процессами через psutil при н
    | алиции
289 |     if not raw_list:
290 |         try:
291 |             import psutil
292 |             for p in psutil.process_iter(['pid', 'name', '
    | cpu_percent', 'memory_info', 'memory_percent', 'num_threads',
    | 'num_handles', 'username', 'exe', 'cmdline']):
293 |                 try:
294 |                     info = p.info
295 |                     mem_mb = (info.get('memory_info').rss
    | / (1024 * 1024)) if info.get('memory_info') else 0.0
296 |                     raw_list.append({
297 |                         'pid': info.get('pid') or 0,
298 |                         'name': info.get('name') or '',
299 |                         'cpu_percent': info.get('cpu_perce
    | nt') or 0.0,
300 |                         'memory_mb': round(mem_mb, 2),
301 |                         'memory_percent': round(info.get('
    | memory_percent') or 0.0, 2),
302 |                         'num_threads': info.get('num_threa
    | ds') or 0,
303 |                         'num_handles': info.get('num_handl
    | es') or 0,
304 |                         'username': info.get('username') o
    | r '',

```

```

305         'read_bytes_sec': 0.0,
306         'write_bytes_sec': 0.0,
307         'executable_path': info.get('exe')
    or '',
308         'command_line': ' '.join(info.get(
    'cmdline') or []),
309     })
310     except (psutil.NoSuchProcess, psutil.Access
    Denied):
311         continue
312     except Exception as ex:
313         logger.debug(f"psutil fallback недоступен: {ex
    }")
314
315     return raw_list
316
317     async def evaluate_program_with_gemini(self, program: Clie
    ntProgramResource) -> UnknownProgramEvaluation:
318         """Спросить у модели Gemini, насколько нормально такое
    потребление ресурсов для программы.
319
320         Args:
321             program: Объект с данными ресурсоемкости программы
    .
322
323         Returns:
324             UnknownProgramEvaluation: Структурированное заключ
    ение модели Gemini.
325         """
326         cache_key = f"{program.name}:{program.pid}:{round(prog
    ram.cpu_percent, -1)}:{round(program.memory_mb, -2)}"
327         if cache_key in self._evaluations_cache:
328             return self._evaluations_cache[cache_key]
329
330         disk_io = round((program.disk_read_bytes_sec + program
    .disk_write_bytes_sec) / (1024.0 * 1024.0), 2)
331         path_str = program.executable_path or "Путь не определ
    ен (запуск из памяти/скрипта)"

```

```

332 |         cmd_str = program.command_line or "Командная строка не
      | доступна"
333 |
334 |         prompt = (
335 |             f"Ты – ведущий эксперт по информационной безопасно
      | сти и оптимизации производительности Windows.\n"
336 |             f"Проанализируй запущенную на компьютере пользоват
      | еля программу и ответь, насколько нормально такое потребление
      | ресурсов:\n\n"
337 |             f"□  **Параметры программы:**\n"
338 |             f"- Имя процесса: `{program.name}` (PID: {program.
      | pid})\n"
339 |             f"- Путь к файлу: `{path_str}`\n"
340 |             f"- Аргументы запуска: `{cmd_str}`\n"
341 |             f"- Пользователь: `{program.username or 'Текущий к
      | лиент'}`\n"
342 |             f"- Загрузка ЦП (CPU): **{program.cpu_percent}%**\n
      | n"
343 |             f"- Использование RAM: **{program.memory_mb} МБ**
      | ({program.memory_percent}% от общей памяти)\n"
344 |             f"- Дисковый ввод-вывод (I/O): **{disk_io} МБ/с**\n
      | n"
345 |             f"- Размер исполняемого файла: **{program.disk_siz
      | e_mb or 'Неизвестен'}` МБ**\n"
346 |             f"- Потокoв: {program.num_threads}, Дескрипторов:
      | {program.num_handles}\n\n"
347 |             f"Ответь строго в формате JSON со следующими полям
      | и:\n"
348 |             f"{{\n"
349 |             f'  "is_normal": true|false,\n'
350 |             f'  "confidence": 0.0-1.0,\n'
351 |             f'  "risk_level": "low"|"medium"|"high"|"critical"
      | ,\n'
352 |             f'  "verdict": "Краткий вывод на русском языке (до
      | 10 слов)",\n'
353 |             f'  "analysis": "Подробное объяснение на русском я
      | зыке: что это за программа, нормально ли для нее такое потребл
      | ение памяти/CPU/диска, может ли это быть майнер, утечка памяти

```

```

или вредоносная активность.",\n'
354         f'  "recommendations": ["Рекомендация 1", "Рекомен
дация 2"]\n'
355         f"}}"
356     )
357
358     model_name = "gemini-3.1-flash"
359     eval_result: Optional[UnknownProgramEvaluation] = None
360
361     if self.chat_model and hasattr(self.chat_model, "ask")
:
362         try:
363             raw_response = await asyncio.wait_for(self.cha
t_model.ask(prompt), timeout=6.0)
364             if raw_response:
365                 cleaned_json = raw_response.strip()
366                 if "```json" in cleaned_json:
367                     cleaned_json = cleaned_json.split("```
json")[1].split("```")[0].strip()
368                 elif "```" in cleaned_json:
369                     cleaned_json = cleaned_json.split("```
")[1].split("```")[0].strip()
370
371                 parsed = json.loads(cleaned_json)
372                 eval_result = UnknownProgramEvaluation(
373                     program_name=program.name,
374                     pid=program.pid,
375                     executable_path=program.executable_pat
h,
376                     command_line=program.command_line,
377                     cpu_percent=program.cpu_percent,
378                     memory_mb=program.memory_mb,
379                     disk_io_mb_s=disk_io,
380                     disk_size_mb=program.disk_size_mb,
381                     is_normal=bool(parsed.get("is_normal",
True)),
382                     confidence=float(parsed.get("confidenc
e", 0.85)),

```

```

383         risk_level=str(parsed.get("risk_level"
    , "medium"))),
384         verdict=str(parsed.get("verdict", "Ана
    лиз завершен")),
385         analysis=str(parsed.get("analysis", "Э
    кспертный анализ модели Gemini.")),
386         recommendations=list(parsed.get("recom
    mendations", [])),
387         evaluated_at=datetime.now(timezone.utc
    ).isoformat(),
388         model_used=model_name,
389     )
390     except Exception as ex:
391         logger.warning(f"Ошибка вызова Gemini для оцен
    ки программы {program.name}: {ex}")
392
393     # Детерминированный fallback если LLM недоступна
394     if eval_result is None:
395         is_suspicious_location = any(
396             loc in (program.executable_path or "").lower()
    for loc in ("\\temp\\", "\\appdata\\local\\temp", "\\users\\p
    ublic\\"))
397     )
398     is_high_risk = is_suspicious_location or (program.
    cpu_percent > 70.0 and not program.is_known_software)
399     risk = "high" if is_high_risk else ("medium" if no
    t program.is_known_software else "low")
400     verdict = "Подозрительная активность" if is_high_r
    isk else ("Штатная активность" if program.is_known_software el
    se "Неизвестное приложение")
401     analysis = (
402         f"Программа {program.name} потребляет {program
    .cpu_percent}% ЦП и {program.memory_mb} МБ RAM. "
403         + ("Запуск из временного каталога Temp требует
    повышенного внимания." if is_suspicious_location else "Рекоме
    ндуется проверить цифровую подпись исполняемого файла.")
404     )
405     recs = [

```

```

406 |         "Проверить контрольную сумму и происхождение ф
айла",
407 |         "При непрекращающейся высокой нагрузке заверши
ть процесс через диспетчер задач"
408 |     ]
409 |     eval_result = UnknownProgramEvaluation(
410 |         program_name=program.name,
411 |         pid=program.pid,
412 |         executable_path=program.executable_path,
413 |         command_line=program.command_line,
414 |         cpu_percent=program.cpu_percent,
415 |         memory_mb=program.memory_mb,
416 |         disk_io_mb_s=disk_io,
417 |         disk_size_mb=program.disk_size_mb,
418 |         is_normal=not is_high_risk,
419 |         confidence=0.75,
420 |         risk_level=risk,
421 |         verdict=verdict,
422 |         analysis=analysis,
423 |         recommendations=recs,
424 |         evaluated_at=datetime.now(timezone.utc).isofor
mat(),
425 |         model_used="deterministic-heuristics",
426 |     )
427 |
428 |     self._evaluations_cache[cache_key] = eval_result
429 |     return eval_result
430 |
431 |     async def analyze_client_resources(
432 |         self,
433 |         db_path: Optional[Union[str, Path]] = None,
434 |         ask_gemini_for_unknown: bool = True,
435 |         limit: int = 50,
436 |     ) -> ClientResourceSummary:
437 |         """Провести комплексный аудит ресурсов клиентских прог
рамм.
438 |
439 |         Args:

```

```
440         db_path: Путь к базе данных телеметрии.
441         ask_gemini_for_unknown: Опрашивать ли Gemini для н
известных ресурсоемких программ.
442         limit: Максимальное количество анализируемых прог
амм.
443
444     Returns:
445         ClientResourceSummary: Сводка по ресурсам всех про
грамм клиента и вердикты Gemini.
446         """
447         raw_procs = self.load_raw_client_processes(db_path=db_
path, limit=limit * 2)
448         client_programs: List[ClientProgramResource] = []
449         evaluations: List[UnknownProgramEvaluation] = []
450
451         seen_keys: Set[Tuple[str, int]] = set()
452
453         for raw in raw_procs:
454             p_name = raw.get("name") or "unknown.exe"
455             p_pid = int(raw.get("pid") or 0)
456             key = (p_name.lower(), p_pid)
457             if key in seen_keys:
458                 continue
459             seen_keys.add(key)
460
461             path = raw.get("executable_path") or raw.get("path
")
462             username = raw.get("username")
463
464             # Проверяем, что это клиентская программа, а не си
стемная служба
465             if not self.is_client_program(p_name, username=use
rname, path=path):
466                 continue
467
468             cpu_pct = float(raw.get("cpu_percent") or 0.0)
469             mem_mb = float(raw.get("memory_mb") or 0.0)
470             mem_pct = float(raw.get("memory_percent") or 0.0)
```

```
471         read_bytes = float(raw.get("read_bytes_sec") or 0.  
472         0)  
473         write_bytes = float(raw.get("write_bytes_sec") or  
474         0.0)  
475         disk_io_mbs = round((read_bytes + write_bytes) / (  
476         1024.0 * 1024.0), 2)  
477         disk_size = self.get_file_size_mb(path)  
478         is_known = self.is_known_software(p_name, path=pat  
479         h)  
480         is_heavy = self.is_heavy_resource_consumer(cpu_pct  
481         , mem_mb, disk_io_mbs)  
482         score = self.calculate_resource_score(cpu_pct, mem  
483         _mb, disk_io_mbs)  
484         known_meta = KNOWN_CLIENT_APPS.get(p_name.lower(),  
485         {})  
486         disp_name = known_meta.get("display_name") or p_na  
487         me  
488         publisher = known_meta.get("publisher")  
489         category = known_meta.get("category", "Клиентское  
490         приложение")  
491         prog = ClientProgramResource(  
492             pid=p_pid,  
493             name=p_name,  
494             display_name=disp_name,  
495             executable_path=path,  
496             command_line=raw.get("command_line"),  
497             username=username,  
498             cpu_percent=round(cpu_pct, 1),  
499             memory_mb=round(mem_mb, 1),  
500             memory_percent=round(mem_pct, 1),  
501             disk_read_bytes_sec=read_bytes,  
502             disk_write_bytes_sec=write_bytes,  
503             disk_size_mb=disk_size,  
504             num_threads=int(raw.get("num_threads") or 0),  
505             num_handles=int(raw.get("num_handles") or 0),
```



```

500         is_known_software=is_known,
501         is_client_program=True,
502         is_resource_heavy=is_heavy,
503         resource_score=score,
504         category=category,
505         publisher=publisher,
506     )
507
508     # Если программа неизвестная и ресурсоемкая – запра
509     # шиваем оценку Gemini
510     if is_heavy and not is_known and ask_gemini_for_un
511     known:
512         eval_item = await self.evaluate_program_with_g
513         emini(prog)
514         prog.gemini_evaluation = eval_item
515         evaluations.append(eval_item)
516
517         client_programs.append(prog)
518         if len(client_programs) >= limit:
519             break
520
521     # Сортировка по скорости ресурсоемкости (самые тяжелые в в
522     ерху)
523     client_programs.sort(key=lambda p: p.resource_score, r
524     everse=True)
525
526     total_ram = round(sum(p.memory_mb for p in client_prog
527     rams), 1)
528     total_cpu = round(sum(p.cpu_percent for p in client_pr
529     ograms), 1)
530     total_disk = round(sum((p.disk_size_mb or 0.0) for p i
531     n client_programs), 1)
532     heavy_count = sum(1 for p in client_programs if p.is_r
533     esource_heavy)
534     unknown_heavy_count = sum(1 for p in client_programs i
535     f p.is_resource_heavy and not p.is_known_software)
536
537     return ClientResourceSummary(

```

```
528 |         total_client_programs=len(client_programs),
529 |         total_ram_mb=total_ram,
530 |         total_cpu_percent=total_cpu,
531 |         total_disk_size_mb=total_disk,
532 |         heavy_programs_count=heavy_count,
533 |         unknown_heavy_programs_count=unknown_heavy_count,
534 |         programs=client_programs,
535 |         evaluations=evaluations,
536 |         scanned_at=datetime.now(timezone.utc).isoformat(),
537 |     )
538 |
```

compactor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Compactor
5 # Description:
6 #   Модуль логарифмического и многоуровневого сжатия (rollup)
7 #   временных рядов
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.compactor import TelemetryCompactor
11 #
12 #   service = TelemetryCompactor()
13 #
14 # File: compactor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль логарифмического и многоуровневого сжатия (rollup) в
24   временных рядов"""
25
26 import math
27 from datetime import datetime, timezone
28 from typing import Any, Dict, List, Optional, Sequence, Tuple
29 from apps.windows.telemetry.models import ProcessMetricRollup,
```

SystemMetricRollup

```
30
31
32 def compute_percentile(values: Sequence[float], p: float = 95.
    0) -> float:
33     """Вычисляет перцентиль выборки методом линейной интерполя
        ции.
34
35     Args:
36         values: Последовательность числовых значений.
37         p: Желаемый перцентиль (0..100), по умолчанию 95.0.
38
39     Returns:
40         float: Значение перцентиля (0.0 при пустой выборке).
41     """
42     if not values:
43         return 0.0
44     sorted_vals = sorted(values)
45     n = len(sorted_vals)
46     if n == 1:
47         return float(sorted_vals[0])
48
49     p_clamped = max(0.0, min(100.0, p))
50     rank = (p_clamped / 100.0) * (n - 1)
51     lower = int(math.floor(rank))
52     upper = int(math.ceil(rank))
53     weight = rank - lower
54
55     return float(sorted_vals[lower] + weight * (sorted_vals[up
        per] - sorted_vals[lower]))
56
57
58 class TelemetryCompactor:
59     """Компаратор и агрегатор временных рядов телеметрии."""
60
61     @staticmethod
62     def get_tier_for_age(age_seconds: float) -> str:
63         """Определяет рекомендуемую гранулярность (tier) в зав
```

исимости от возраста данных.

```
64 |
65 |     Args:
66 |         age_seconds: Возраст данных в секундах.
67 |
68 |     Returns:
69 |         str: Наименование уровня агрегации ('1s', '2s', '5
70 | s', '10s', '30s', '1m', '5m', '15m', '1h', '6h').
71 |         """
72 |     if age_seconds <= 60.0:
73 |         return "1s"
74 |     elif age_seconds <= 300.0: # 5 мин
75 |         return "2s"
76 |     elif age_seconds <= 900.0: # 15 мин
77 |         return "5s"
78 |     elif age_seconds <= 1800.0: # 30 мин
79 |         return "10s"
80 |     elif age_seconds <= 3600.0: # 1 час
81 |         return "30s"
82 |     elif age_seconds <= 10800.0: # 3 часа
83 |         return "1m"
84 |     elif age_seconds <= 43200.0: # 12 часов
85 |         return "5m"
86 |     elif age_seconds <= 86400.0: # 24 часа
87 |         return "15m"
88 |     elif age_seconds <= 604800.0: # 7 дней
89 |         return "1h"
90 |     else:
91 |         return "6h"
92 |
93 | @staticmethod
94 | def compact_system_metrics(
95 |     samples: List[Dict[str, Any]],
96 |     period_start: str,
97 |     period_end: str,
98 |     tier: str = "1m",
99 | ) -> SystemMetricRollup:
    """Агрегирует список сырых замеров хоста в один System
```

```

MetricRollup.
100
101     Args:
102         samples: Список словарей замеров (SystemSnapshot и
ли сжатых метрик).
103         period_start: ISO timestamp начала периода.
104         period_end: ISO timestamp окончания периода.
105         tier: Уровень гранулярности бакета.
106
107     Returns:
108         SystemMetricRollup: Сжатый бакет с расчетом avg, m
in, max, p95 и total.
109     """
110     count = len(samples)
111     if count == 0:
112         return SystemMetricRollup(
113             period_start=period_start,
114             period_end=period_end,
115             duration_seconds=0.0,
116             sample_count=0,
117             tier=tier,
118         )
119
120     # Вычисление длительности периода
121     duration_sec = 60.0
122     try:
123         t0 = datetime.fromisoformat(period_start)
124         t1 = datetime.fromisoformat(period_end)
125         duration_sec = max(1.0, (t1 - t0).total_seconds())
126     except Exception:
127         duration_sec = float(count)
128
129     cpu_vals: List[float] = []
130     ram_used_vals: List[float] = []
131     ram_pct_vals: List[float] = []
132     disk_read_rates: List[float] = []
133     disk_write_rates: List[float] = []
134     net_rx_rates: List[float] = []

```

```
135 |         net_tx_rates: List[float] = []
136 |
137 |         for s in samples:
138 |             # CPU
139 |             cpu_val = float(s.get("cpu_total_percent") or s.ge
140 | t("cpu_percent") or 0.0)
141 |             cpu_vals.append(cpu_val)
142 |
143 |             # RAM
144 |             ram_used = float(s.get("memory_used_gb") or 0.0)
145 |             ram_pct = float(s.get("memory_percent") or 0.0)
146 |             ram_used_vals.append(ram_used)
147 |             ram_pct_vals.append(ram_pct)
148 |
149 |             # Disk (байты/сек переводим в МБ/с)
150 |             disk_r = float(s.get("disk_read_bytes_sec") or 0.0
151 | ) / (1024.0 * 1024.0)
152 |             disk_w = float(s.get("disk_write_bytes_sec") or 0.
153 | 0) / (1024.0 * 1024.0)
154 |             disk_read_rates.append(disk_r)
155 |             disk_write_rates.append(disk_w)
156 |
157 |             # Net (байты/сек переводим в МБ/с)
158 |             net_rx = float(s.get("network_recv_bytes_sec") or
159 | 0.0) / (1024.0 * 1024.0)
160 |             net_tx = float(s.get("network_sent_bytes_sec") or
161 | 0.0) / (1024.0 * 1024.0)
162 |             net_rx_rates.append(net_rx)
163 |             net_tx_rates.append(net_tx)
164 |
165 |             # Расчет средних и тоталов (rate * dt per sample)
166 |             sample_dt = duration_sec / max(1, count)
167 |
168 |             cpu_avg = sum(cpu_vals) / count
169 |             cpu_min = min(cpu_vals)
170 |             cpu_max = max(cpu_vals)
171 |             cpu_p95 = compute_percentile(cpu_vals, 95.0)
```

```
168 |         ram_avg = sum(ram_used_vals) / count
169 |         ram_min = min(ram_used_vals)
170 |         ram_max = max(ram_used_vals)
171 |         ram_pct_avg = sum(ram_pct_vals) / count
172 |         ram_pct_max = max(ram_pct_vals)
173 |
174 |         disk_r_avg = sum(disk_read_rates) / count
175 |         disk_r_max = max(disk_read_rates)
176 |         disk_r_total = sum(disk_read_rates) * sample_dt
177 |
178 |         disk_w_avg = sum(disk_write_rates) / count
179 |         disk_w_max = max(disk_write_rates)
180 |         disk_w_total = sum(disk_write_rates) * sample_dt
181 |
182 |         net_rx_avg = sum(net_rx_rates) / count
183 |         net_rx_max = max(net_rx_rates)
184 |         net_rx_total = sum(net_rx_rates) * sample_dt
185 |
186 |         net_tx_avg = sum(net_tx_rates) / count
187 |         net_tx_max = max(net_tx_rates)
188 |         net_tx_total = sum(net_tx_rates) * sample_dt
189 |
190 |     return SystemMetricRollup(
191 |         period_start=period_start,
192 |         period_end=period_end,
193 |         duration_seconds=round(duration_sec, 2),
194 |         sample_count=count,
195 |         tier=tier,
196 |         cpu_avg=round(cpu_avg, 2),
197 |         cpu_min=round(cpu_min, 2),
198 |         cpu_max=round(cpu_max, 2),
199 |         cpu_p95=round(cpu_p95, 2),
200 |         ram_avg_gb=round(ram_avg, 3),
201 |         ram_min_gb=round(ram_min, 3),
202 |         ram_max_gb=round(ram_max, 3),
203 |         ram_percent_avg=round(ram_pct_avg, 2),
204 |         ram_percent_max=round(ram_pct_max, 2),
205 |         disk_read_avg_mbs=round(disk_r_avg, 3),
```



```

206         disk_read_max_mbs=round(disk_r_max, 3),
207         disk_read_total_mb=round(disk_r_total, 3),
208         disk_write_avg_mbs=round(disk_w_avg, 3),
209         disk_write_max_mbs=round(disk_w_max, 3),
210         disk_write_total_mb=round(disk_w_total, 3),
211         network_rx_avg_mbs=round(net_rx_avg, 3),
212         network_rx_max_mbs=round(net_rx_max, 3),
213         network_rx_total_mb=round(net_rx_total, 3),
214         network_tx_avg_mbs=round(net_tx_avg, 3),
215         network_tx_max_mbs=round(net_tx_max, 3),
216         network_tx_total_mb=round(net_tx_total, 3),
217     )
218
219     @staticmethod
220     def compact_process_metrics(
221         process_samples: List[Dict[str, Any]],
222         period_start: str,
223         period_end: str,
224     ) -> List[ProcessMetricRollup]:
225         """Группирует и агрегирует замеры процессов по имени/P
ID.
226
227         Args:
228             process_samples: Список замеров процессов.
229             period_start: Начало периода.
230             period_end: Конец периода.
231
232         Returns:
233             List[ProcessMetricRollup]: Агрегированные показате
ли каждого процесса.
234         """
235         duration_sec = 60.0
236         try:
237             t0 = datetime.fromisoformat(period_start)
238             t1 = datetime.fromisoformat(period_end)
239             duration_sec = max(1.0, (t1 - t0).total_seconds())
240         except Exception:
241             duration_sec = 60.0

```

```

242 |
243 |     # Группировка по (name, pid)
244 |     grouped: Dict[Tuple[str, Optional[int]], List[Dict[str
    | , Any]]] = {}
245 |     for p in process_samples:
246 |         key = (str(p.get("name", "unknown")), p.get("pid")
    | )
247 |         if key not in grouped:
248 |             grouped[key] = []
249 |             grouped[key].append(p)
250 |
251 |     results: List[ProcessMetricRollup] = []
252 |     for (name, pid), p_list in grouped.items():
253 |         count = len(p_list)
254 |         cpu_vals = [float(item.get("cpu_percent") or 0.0)
    | for item in p_list]
255 |         ram_vals = [float(item.get("memory_mb") or 0.0) fo
    | r item in p_list]
256 |         read_bytes_sec = [float(item.get("read_bytes_sec")
    | or 0.0) for item in p_list]
257 |         write_bytes_sec = [float(item.get("write_bytes_sec
    | ") or 0.0) for item in p_list]
258 |         rx_bytes_sec = [float(item.get("rx_bytes_sec") or
    | 0.0) for item in p_list]
259 |         tx_bytes_sec = [float(item.get("tx_bytes_sec") or
    | 0.0) for item in p_list]
260 |
261 |         sample_dt = duration_sec / max(1, count)
262 |         path = p_list[0].get("path") or p_list[0].get("exe
    | ")
263 |
264 |         cpu_avg = sum(cpu_vals) / count
265 |         cpu_max = max(cpu_vals)
266 |         cpu_p95 = compute_percentile(cpu_vals, 95.0)
267 |
268 |         ram_avg = sum(ram_vals) / count
269 |         ram_max = max(ram_vals)
270 |

```

```
271 |         disk_r_tot = (sum(read_bytes_sec) * sample_dt) / (  
    | 1024.0 * 1024.0)  
272 |         disk_w_tot = (sum(write_bytes_sec) * sample_dt) /  
    | (1024.0 * 1024.0)  
273 |         net_rx_tot = (sum(rx_bytes_sec) * sample_dt) / (10  
    | 24.0 * 1024.0)  
274 |         net_tx_tot = (sum(tx_bytes_sec) * sample_dt) / (10  
    | 24.0 * 1024.0)  
275 |  
276 |         results.append(  
277 |             ProcessMetricRollup(  
278 |                 name=name,  
279 |                 pid=pid,  
280 |                 period_start=period_start,  
281 |                 period_end=period_end,  
282 |                 duration_seconds=round(duration_sec, 2),  
283 |                 sample_count=count,  
284 |                 path=path,  
285 |                 cpu_avg=round(cpu_avg, 2),  
286 |                 cpu_max=round(cpu_max, 2),  
287 |                 cpu_p95=round(cpu_p95, 2),  
288 |                 ram_avg_mb=round(ram_avg, 2),  
289 |                 ram_max_mb=round(ram_max, 2),  
290 |                 disk_read_total_mb=round(disk_r_tot, 3),  
291 |                 disk_write_total_mb=round(disk_w_tot, 3),  
292 |                 network_rx_total_mb=round(net_rx_tot, 3),  
293 |                 network_tx_total_mb=round(net_tx_tot, 3),  
294 |             )  
295 |         )  
296 |  
297 |     return results  
298 |
```

deep_diagnostics.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Deep Diagnostics
5 # Description:
6 #   Модуль глубокой аппаратной и системной диагностики, поведе
7 #   нческой форензики и анализа утечек.
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.deep_diagnostics im
11 #   port LASTINPUTINFO
12 #   service = LASTINPUTINFO()
13 #
14 # File: deep_diagnostics.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 04:11:30
20 # =====
21
22 from __future__ import annotations
23 """Модуль глубокой аппаратной и системной диагностики, поведе
24   нческой форензики и анализа утечек."""
25
26 import codecs
27 import ctypes
28 import ctypes.wintypes as wintypes
29 import os
30 import re
```

```
30 import struct
31 import subprocess
32 import sys
33 import time
34 from datetime import datetime, timezone, timedelta
35 from pathlib import Path
36 from typing import Any, Dict, List, Optional, Tuple
37
38 try:
39     import psutil
40     PSUTIL_AVAILABLE = True
41 except ImportError:
42     psutil = None
43     PSUTIL_AVAILABLE = False
44
45 try:
46     import winreg
47     WINREG_AVAILABLE = True
48 except ImportError:
49     winreg = None
50     WINREG_AVAILABLE = False
51
52 try:
53     from logger import logger
54 except ImportError:
55     from logger import logger
56
57 from apps.windows.telemetry.models import (
58     DefenderTelemetrySummary,
59     ExtendedSystemAuditReport,
60     ForensicsActivityReport,
61     KernelThrottlingReport,
62     PeripheralsNetworkReport,
63     ProcessLeakDiagnosticsReport,
64     ProcessLeakItem,
65     StartupTelemetrySummary,
66     StorageBatteryWearReport,
67     UserAccountsTelemetrySummary,
68     VssTelemetrySummary,
69 )
```

```

68
69 GR_GDIOBJECTS = 0
70 GR_USEROBJECTS = 1
71 PROCESS_QUERY_INFORMATION = 1024
72 PROCESS_VM_READ = 16
73
74
75 class LASTINPUTINFO(ctypes.Structure):
76     """Структура Win32 для GetLastInputInfo."""
77     _fields_ = [('cbSize', wintypes.UINT), ('dwTime', wintypes
78         .DWORD)]
79
80 class PROCESS_MEMORY_COUNTERS_EX(ctypes.Structure):
81     """Структура PSAPI для GetProcessMemoryInfo."""
82     _fields_ = [('cb', wintypes.DWORD), ('PageFaultCount', win
83         types.DWORD), ('PeakWorkingSetSize', ctypes.c_size_t), ('Worki
84         ngSetSize', ctypes.c_size_t), ('QuotaPeakPagedPoolUsage', ctyp
85         es.c_size_t), ('QuotaPagedPoolUsage', ctypes.c_size_t), ('Quot
86         aPeakNonPagedPoolUsage', ctypes.c_size_t), ('QuotaNonPagedPool
87         Usage', ctypes.c_size_t), ('PagefileUsage', ctypes.c_size_t),
88         ('PeakPagefileUsage', ctypes.c_size_t), ('PrivateUsage', ctype
89         s.c_size_t)]
90
91
92 class DeepDiagnosticsEngine:
93     """Движок глубокой системной диагностики и форензики."""
94
95     def __init__(self) -> None:
96         """Инициализация Win32 API интерфейсов."""
97         self._user32 = getattr(ctypes.windll, 'user32', None)
98         if os.name == 'nt' else None
99         self._kernel32 = getattr(ctypes.windll, 'kernel32', No
100         ne) if os.name == 'nt' else None
101         self._psapi = getattr(ctypes.windll, 'psapi', None) if
102         os.name == 'nt' else None
103
104     def collect_process_leaks(self, limit: int=100) -> Process

```

```

LeakDiagnosticsReport:
95     """Собрать метрики скрытых утечек процессов (Handles,
GDI, USER, Page Faults).
96
97     Args:
98         limit: Максимальное количество процессов в общем с
писке.
99
100     Returns:
101         ProcessLeakDiagnosticsReport: Отчет с ранжирование
м подозрительных процессов.
102     """
103     if not PSUTIL_AVAILABLE:
104         return ProcessLeakDiagnosticsReport()
105     items: List[ProcessLeakItem] = []
106     suspicious_count = 0
107     for proc in psutil.process_iter(['pid', 'name', 'statu
s', 'num_threads', 'cpu_percent', 'memory_info']):
108         try:
109             info = proc.info
110             pid = info.get('pid')
111             if not pid or pid <= 4:
112                 continue
113             name = info.get('name') or 'unknown'
114             status = info.get('status') or 'running'
115             threads_count = info.get('num_threads') or 1
116             cpu_percent = info.get('cpu_percent') or 0.0
117             mem_info = info.get('memory_info')
118             memory_mb = round(mem_info.rss / (1024 * 1024)
, 2) if mem_info else 0.0
119             handles_count = 0
120             gdi_objects = 0
121             user_objects = 0
122             page_faults = 0
123             peak_ws_mb = memory_mb
124             if self._kernel32 and self._user32:
125                 h_process = self._kernel32.OpenProcess(PRO
CESS_QUERY_INFORMATION | PROCESS_VM_READ, False, pid)

```

```

126         if h_process:
127             try:
128                 dw_handles = wintypes.DWORD()
129                 if self._kernel32.GetProcessHandle
Count(h_process, ctypes.byref(dw_handles)):
130                     handles_count = int(dw_handles
.value)
131                     gdi_objects = int(self._user32.Get
GuiResources(h_process, GR_GDIOBJECTS))
132                     user_objects = int(self._user32.Ge
tGuiResources(h_process, GR_USEROBJECTS))
133                     if self._psapi:
134                         counters = PROCESS_MEMORY_COUN
TERS_EX()
135                         counters.cb = ctypes.sizeof(PR
OCESS_MEMORY_COUNTERS_EX)
136                         if self._psapi.GetProcessMemor
yInfo(h_process, ctypes.byref(counters), counters.cb):
137                             page_faults = int(counters
.PageFaultCount)
138                             peak_ws_mb = round(counter
s.PeakWorkingSetSize / (1024 * 1024), 2)
139                             finally:
140                                 self._kernel32.CloseHandle(h_proce
ss)
141                     if handles_count == 0:
142                         try:
143                             handles_count = proc.num_handles() if
hasattr(proc, 'num_handles') else 0
144                         except Exception:
145                             handles_count = 0
146                     risk_reasons: List[str] = []
147                     risk_score = 'normal'
148                     if handles_count >= 4000:
149                         risk_score = 'critical'
150                         risk_reasons.append(f'Критическое число де
скрипторов: {handles_count}')
151                     elif handles_count >= 1500:

```



```
152         if risk_score != 'critical':
153             risk_score = 'warning'
154             risk_reasons.append(f'Высокое число дескрипторов: {handles_count}')
155         if gdi_objects >= 2000:
156             risk_score = 'critical'
157             risk_reasons.append(f'Угроза истощения GDI пула: {gdi_objects} (лимит 10 000)')
158         elif gdi_objects >= 800:
159             if risk_score != 'critical':
160                 risk_score = 'warning'
161                 risk_reasons.append(f'Повышенное использование GDI: {gdi_objects}')
162         if user_objects >= 500:
163             if risk_score != 'critical':
164                 risk_score = 'warning'
165                 risk_reasons.append(f'Много USER объектов: {user_objects}')
166         if page_faults >= 2000000:
167             risk_reasons.append(f'Высокая нагрузка на подкачку ({page_faults:,} Hard Faults)')
168         if memory_mb >= 2048:
169             risk_reasons.append(f'Высокое потребление памяти: {memory_mb:.1f} MB')
170         if risk_score != 'normal':
171             suspicious_count += 1
172         item = ProcessLeakItem(pid=pid, name=name, status=status, handles_count=handles_count, gdi_objects=gdi_objects, user_objects=user_objects, page_faults_total=page_faults, peak_working_set_mb=peak_ws_mb, memory_mb=memory_mb, threads_count=threads_count, cpu_percent=cpu_percent, leak_risk_score=risk_score, leak_risk_reasons=risk_reasons)
173         items.append(item)
174     except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
175         continue
176     except Exception as ex:
177         logger.debug(f'Ошибка сбора дескрипторов проце
```

```

cca: {ex}')
178         top_handles = sorted(items, key=lambda x: x.handles_co
unt, reverse=True)[:10]
179         top_gdi = sorted(items, key=lambda x: x.gdi_objects, r
everse=True)[:10]
180         top_page_faults = sorted(items, key=lambda x: x.page_f
aults_total, reverse=True)[:10]
181         risk_priority = {'critical': 0, 'warning': 1, 'normal'
: 2}
182         sorted_all = sorted(items, key=lambda x: (risk_priorit
y.get(x.leak_risk_score, 3), -x.handles_count))[:limit]
183         return ProcessLeakDiagnosticsReport(timestamp=datetime
.now(timezone.utc).isoformat(), total_processes=len(items), su
spicious_count=suspicious_count, top_handle_hogs=top_handles,
top_gdi_hogs=top_gdi, top_page_fault_hogs=top_page_faults, all
_processes=sorted_all)
184
185     def collect_forensics_activity(self) -> ForensicsActivityR
eport:
186         """Собрать данные активности пользователя, активного о
кна, доступа к сенсорам и UserAssist."""
187         fg_window: Dict[str, Any] = {'title': 'Рабочий стол /
Системный интерфейс', 'process_name': 'explorer.exe', 'pid': 0
}
188         idle_seconds = 0.0
189         if self._user32:
190             try:
191                 hwnd = self._user32.GetForegroundWindow()
192                 if hwnd:
193                     length = self._user32.GetWindowTextLengthW
(hwnd)
194                     buff = ctypes.create_unicode_buffer(length
+ 1)
195                     self._user32.GetWindowTextW(hwnd, buff, le
ngth + 1)
196                     title = buff.value.strip() or 'Без заголов
ка'
197                     lp_pid = wintypes.DWORD()

```

```

198         self._user32.GetWindowThreadProcessId(hwnd
    , ctypes.byref(lp_pid))
199         pid = int(lp_pid.value)
200         proc_name = 'unknown'
201         if PSUTIL_AVAILABLE and pid > 0:
202             try:
203                 proc_name = psutil.Process(pid).name()
204             except Exception:
205                 proc_name = f'PID {pid}'
206                 fg_window = {'title': title, 'process_name': proc_name, 'pid': pid}
207             except Exception as ex:
208                 logger.debug(f'Ошибка получения ForegroundWindow: {ex}')
209         if self._user32 and self._kernel32:
210             try:
211                 lii = LASTINPUTINFO()
212                 lii.cbSize = ctypes.sizeof(LASTINPUTINFO)
213                 if self._user32.GetLastInputInfo(ctypes.byref(lii)):
214                     tick_count = self._kernel32.GetTickCount()
215                     idle_ms = max(0, tick_count - lii.dwTime)
216                     idle_seconds = round(idle_ms / 1000.0, 1)
217             except Exception as ex:
218                 logger.debug(f'Ошибка расчета UserIdleTime: {ex}')
219         cam_apps, mic_apps = self._get_camera_mic_access()
220         userassist_apps = self._get_userassist_apps(limit=15)
221         return ForensicsActivityReport(timestamp=datetime.now(
    timezone.utc).isoformat(), foreground_window=fg_window, user_idle_seconds=idle_seconds, camera_active_apps=cam_apps, microphone_active_apps=mic_apps, userassist_top_apps=userassist_apps)
222
223     def _get_camera_mic_access(self) -> Tuple[List[Dict[str, Any]], List[Dict[str, Any]]]:
224         """Определить приложения, использующие камеру и микрофон через реестр Windows."""

```

```

225 |         cam_list: List[Dict[str, Any]] = []
226 |         mic_list: List[Dict[str, Any]] = []
227 |         if not WINREG_AVAILABLE:
228 |             return (cam_list, mic_list)
229 |         capabilities = [('webcam', cam_list), ('microphone', mic_list)]
230 |         base_path = 'Software\\Microsoft\\Windows\\CurrentVersion\\CapabilityAccessManager\\ConsentStore'
231 |         for cap_name, target_list in capabilities:
232 |             try:
233 |                 cap_key_path = f'{base_path}\\{cap_name}'
234 |                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER,
235 |                                     cap_key_path, 0, winreg.KEY_READ) as key:
236 |                     num_subkeys, _, _ = winreg.QueryInfoKey(key)
237 |                     for i in range(num_subkeys):
238 |                         try:
239 |                             sub_name = winreg.EnumKey(key, i)
240 |                             if sub_name == 'NonPackaged':
241 |                                 non_pkg_path = f'{cap_key_path}\\NonPackaged'
242 |                                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER, non_pkg_path, 0, winreg.KEY_READ) as np_key:
243 |                                     np_count, _, _ = winreg.QueryInfoKey(np_key)
244 |                                     for j in range(np_count):
245 |                                         try:
246 |                                             app_sub = winreg.EnumKey(np_key, j)
247 |                                             with winreg.OpenKey(np_key, app_sub, 0, winreg.KEY_READ) as app_key:
248 |                                                 stop_val, _ = winreg.QueryValueEx(app_key, 'LastUsedTimeStop')
249 |                                                 start_val, _ = winreg.QueryValueEx(app_key, 'LastUsedTimeStart')
250 |                                                 is_active = stop_val == 0 and start_val > 0

```

```

    = app_sub.replace('#', '\\').split('\\')[-1]
251 |                                     target_list.append({'app_name': clean_app_name, 'full_path': app_sub.replace
    ('#', '\\'), 'is_active_now': is_active, 'type': 'Win32 Executable'})
252 |                                     except Exception:
253 |                                         continue
254 |                                     else:
255 |                                         with winreg.OpenKey(key, sub_name, 0, winreg.KEY_READ) as uwp_key:
256 |                                             try:
257 |                                                 stop_val, _ = winreg.QueryValueEx(uwp_key, 'LastUsedTimeStop')
258 |                                                 start_val, _ = winreg.QueryValueEx(uwp_key, 'LastUsedTimeStart')
259 |                                                 is_active = stop_val == 0 and start_val > 0
260 |                                     target_list.append({'app_name': sub_name, 'full_path': sub_name, 'is_active_now': is_active, 'type': 'UWP / Windows App'})
261 |                                     except Exception:
262 |                                         continue
263 |                                     except Exception:
264 |                                         continue
265 |                                     except Exception as ex:
266 |                                         logger.debug(f'Ошибка проверки доступа к {cam_name}: {ex}')
267 |                                     return (cam_list, mic_list)
268 |
269 |     def _get_userassist_apps(self, limit: int=15) -> List[Dict[str, Any]]:
270 |         """Извлечь историю запусков и фокусное время из ключей UserAssist реестра (ROT13)."""
271 |         apps: List[Dict[str, Any]] = []
272 |         if not WINREG_AVAILABLE:
273 |             return apps
274 |         ua_path = 'Software\\Microsoft\\Windows\\CurrentVersion\\Explorer\\UserAssist'

```

```

275         try:
276             with winreg.OpenKey(winreg.HKEY_CURRENT_USER, ua_p
ath, 0, winreg.KEY_READ) as ua_key:
277                 num_guids, _, _ = winreg.QueryInfoKey(ua_key)
278                 for i in range(num_guids):
279                     guid_name = winreg.EnumKey(ua_key, i)
280                     count_path = f'{ua_path}\\{guid_name}\\Cou
nt'
281                 try:
282                     with winreg.OpenKey(winreg.HKEY_CURREN
T_USER, count_path, 0, winreg.KEY_READ) as count_key:
283                         num_vals, _, _ = winreg.QueryInfoK
ey(count_key)
284                         for v_idx in range(num_vals):
285                             try:
286                                 val_name, val_data, val_ty
pe = winreg.EnumValue(count_key, v_idx)
287                                 decoded_name = codecs.deco
de(val_name, 'rot_13')
288                                 if decoded_name.startswith
('{') or not decoded_name.strip():
289                                     continue
290                                 run_count = 0
291                                 focus_seconds = 0
292                                 if val_type == winreg.REG_
BINARY and len(val_data) >= 72:
293                                     run_count = struct.unp
ack_from('<I', val_data, 4)[0]
294                                     focus_ms = struct.unpa
ck_from('<I', val_data, 12)[0]
295                                     focus_seconds = round(
focus_ms / 1000.0)
296                                     if run_count > 0:
297                                         app_display = Path(dec
oded_name).name if '\\ ' in decoded_name or '/' in decoded_name
else decoded_name
298                                         apps.append({'name': a
pp_display, 'path': decoded_name, 'run_count': run_count, 'foc

```

```

        us_seconds': focus_seconds, 'focus_formatted': f'{focus_seconds
s // 60} мин {focus_seconds % 60} с'})
299         except Exception:
300             continue
301         except Exception:
302             continue
303     except Exception as ex:
304         logger.debug(f'Ошибка UserAssist: {ex}')
305     apps.sort(key=lambda x: (x['run_count'], x['focus_seconds']), reverse=True)
306     return apps[:limit]
307
308     def collect_kernel_throttling(self) -> KernelThrottlingReport:
309         """Собрать данные по прерываниям DPC/ISR, троттлингу,
Uptime и сбоям."""
310         dpc_pct = 0.0
311         interrupt_pct = 0.0
312         uptime_sec = 0.0
313         uptime_str = '0 дн 0 ч'
314         thermal_throttling = False
315         power_throttling = False
316         bsod_list: List[Dict[str, Any]] = []
317         pcie_info: Dict[str, Any] = {'gpu_name': 'PCIe Device'
, 'current_link_speed': 'Gen 3/4', 'current_link_width': 'x16'
, 'status': 'Optimal'}
318         if PSUTIL_AVAILABLE:
319             try:
320                 boot_time = psutil.boot_time()
321                 uptime_sec = max(0.0, time.time() - boot_time)
322                 days = int(uptime_sec // 86400)
323                 hours = int(uptime_sec % 86400 // 3600)
324                 mins = int(uptime_sec % 3600 // 60)
325                 uptime_str = f'{days} дн {hours} ч {mins} мин'
326                 cpu_times = psutil.cpu_times_percent(interval=
0.1)
327                 dpc_pct = round(getattr(cpu_times, 'dpc', 0.0)
or 0.0, 2)

```

```

328         interrupt_pct = round(getattr(cpu_times, 'interrupt', 0.0) or 0.0, 2)
329         except Exception as ex:
330             logger.debug(f'Ошибка сбора cpu_times: {ex}')
331         dpc_status = 'optimal'
332         if dpc_pct > 5.0 or interrupt_pct > 3.0:
333             dpc_status = 'severe'
334         elif dpc_pct > 1.5 or interrupt_pct > 1.0:
335             dpc_status = 'elevated'
336         minidump_dir = Path(os.environ.get('SystemRoot', 'C:\\Windows')) / 'Minidump'
337         if minidump_dir.exists() and minidump_dir.is_dir():
338             try:
339                 dumps = sorted(minidump_dir.glob('*.dmp'), key
= lambda p: p.stat().st_mtime, reverse=True)[:5]
340                 for d in dumps:
341                     mtime = datetime.fromtimestamp(d.stat().st
_mtime, tz=timezone.utc).isoformat()
342                     bsod_list.append({'file_name': d.name, 'ti
mestamp': mtime, 'size_kb': round(d.stat().st_size / 1024, 1),
'type': 'Crash Dump (BugCheck)'})
343             except Exception as ex:
344                 logger.debug(f'Ошибка проверки Minidump: {ex}'
)
345         try:
346             from apps.windows.telemetry.sensors import get_har
dware_sensors
347             sensors = get_hardware_sensors()
348             for s in sensors:
349                 name_l = s.name.lower()
350                 if 'prochot' in name_l or 'thermal throttling'
in name_l:
351                     if s.value > 0:
352                         thermal_throttling = True
353                 if 'power limit' in name_l or 'pl1' in name_l
or 'pl2' in name_l:
354                     if s.value > 0:
355                         power_throttling = True

```



```

356         except Exception:
357             pass
358         return KernelThrottlingReport(timestamp=datetime.now(t
imezone.utc).isoformat(), dpc_latency_pct=dpc_pct, interrupt_l
atency_pct=interrupt_pct, dpc_status=dpc_status, thermal_throt
tling_detected=thermal_throttling, power_limit_throttling_dete
cted=power_throttling, system_uptime_seconds=uptime_sec, uptim
e_formatted=uptime_str, last_bsod_crashes=bsod_list, gpu_pcie_
link=pcie_info)
359
360     def collect_storage_battery_wear(self) -> StorageBatteryWe
arReport:
361         """Собрать данные износа SSD/NVMe и батареи питания."""
362         disks: List[Dict[str, Any]] = []
363         battery_data: Dict[str, Any] = {'has_battery': False,
'design_capacity_mwh': 0.0, 'full_charge_capacity_mwh': 0.0, '
wear_level_pct': 0.0, 'cycle_count': 0, 'charge_rate_mw': 0.0,
'is_charging': False, 'percent': 0.0, 'power_source': 'AC Mai
ns (Стационарное питание)'}
364         try:
365             from apps.windows.modules.storage_manager.core.win
dows_storage_sensor import WindowsStorageSensor
366             sensor = WindowsStorageSensor()
367             snapshot = sensor.collect_snapshot(sync=True)
368             raw_partitions = snapshot.get('sources', {}).get('
partitions', [])
369             raw_logical = snapshot.get('sources', {}).get('log
ical_disks', [])
370
371             # Извлечем сведения о логических дисках (буква том
а -> ФС и свободное место)
372             logical_info: Dict[str, Dict[str, Any]] = {}
373             for log_d in raw_logical:
374                 if isinstance(log_d, dict):
375                     ld_id = str(log_d.get('DeviceID') or '').u
pper().rstrip('\\')
376                     if ld_id:

```

```

377 |             logical_info[ld_id] = {
378 |                 'fs': log_d.get('FileSystem') or '
| NTFS',
379 |                 'free_bytes': int(log_d.get('FreeS
| pace') or 0),
380 |             }
381 |
382 |     part_map: Dict[str, List[str]] = {}
383 |     part_free_map: Dict[str, float] = {}
384 |     for p in raw_partitions:
385 |         if isinstance(p, dict):
386 |             dn = p.get('DiskNumber')
387 |             dl = p.get('DriveLetter')
388 |             if dn is not None and dl:
389 |                 dn_str = str(dn)
390 |                 letter = f'{dl}:' .upper()
391 |                 fs_name = logical_info.get(letter, {}).
| .get('fs', 'NTFS')
392 |                 part_map.setdefault(dn_str, []).append
| (f'{letter} [{fs_name}]')
393 |                 free_b = logical_info.get(letter, {}).
| get('free_bytes', 0)
394 |                 if free_b > 0:
395 |                     part_free_map[dn_str] = part_free_
| map.get(dn_str, 0.0) + (free_b / 1024 ** 3)
396 |
397 |     io_counters = psutil.disk_io_counters(perdisk=True
| ) if PSUTIL_AVAILABLE and hasattr(psutil, 'disk_io_counters')
| else {}
398 |     now_dt = datetime.now(timezone.utc)
399 |     physical_disks = sensor.get_physical_disks(sync=Tr
| ue)
400 |     if physical_disks:
401 |         for d in physical_disks:
402 |             dev_num_str = ''.join(filter(str.isdigit,
| d.device_id))
403 |             mounted = part_map.get(dev_num_str, [])
404 |             has_free = dev_num_str in part_free_map

```

```

405         free_total = part_free_map.get(dev_num_str
    , 0.0)
406         if not has_free and PSUTIL_AVAILABLE and m
ounted:
407             for part_label in mounted:
408                 m_letter = part_label.split()[0] i
f part_label else ''
409                 if m_letter and ':' in m_letter:
410                     try:
411                         u = psutil.disk_usage(f'{m
_letter}\\')
412                         free_total += (u.free / 10
24 ** 3)
413                         has_free = True
414                     except Exception:
415                         pass
416                 io_key = f'PhysicalDrive{dev_num_str}'
417                 io = io_counters.get(io_key)
418                 bytes_read = io.read_bytes if io else (d.l
ifetime_read_bytes or 0)
419                 bytes_written = io.write_bytes if io else
(d.lifetime_write_bytes or 0)
420                 read_count = io.read_count if io else (d.r
ead_errors_total or 0)
421                 write_count = io.write_count if io else (d
.write_errors_total or 0)
422                 poh = d.power_on_hours
423                 first_on_str = (now_dt - timedelta(hours=i
nt(poh))).strftime('%Y-%m-%d') if poh and poh > 0 else None
424                 health_pct = 100.0
425                 if d.wear_percentage is not None and d.wea
r_percentage > 0:
426                     health_pct = max(0.0, min(100.0, 100.0
- float(d.wear_percentage)))
427                     elif d.health_status and d.health_status.l
ower() in ('warning', 'caution'):
428                         health_pct = 70.0
429                     elif d.health_status and d.health_status.l

```

```

ower() in ('unhealthy', 'critical', 'failed'):
430         health_pct = 20.0
431         status_str = 'Healthy (SMART OK)'
432         if d.health_status and d.health_status not
in ('Healthy', 'OK', '0', 'PASSED'):
433             status_str = d.health_status
434             disks.append({
435                 'device_id': f'Disk {dev_num_str}' if
dev_num_str else d.device_id,
436                 'name': d.model or d.friendly_name or
'Физический накопитель',
437                 'model': d.model or d.friendly_name,
438                 'serial_number': d.serial_number if d.
serial_number and d.serial_number != 'N/A' else None,
439                 'bus_type': d.bus_type or 'Unknown',
440                 'media_type': d.media_type or 'SSD',
441                 'partitions': ', '.join(mounted) if mo
unted else '-',
442                 'total_gb': round(d.size_gb, 1),
443                 'free_gb': round(free_total, 1) if has
_free else None,
444                 'health_pct': round(health_pct, 1),
445                 'wear_level_pct': d.wear_percentage or
0.0,
446                 'power_on_hours': poh,
447                 'first_power_on': first_on_str,
448                 'bytes_written': bytes_written,
449                 'bytes_read': bytes_read,
450                 'read_count': read_count,
451                 'write_count': write_count,
452                 'temperature_c': d.temperature_c,
453                 'status': status_str,
454             })
455         disks.sort(key=lambda x: int(''.join(filter(st
r.isdigit, str(x.get('device_id', '0'))))) or 0))
456     except Exception as ex:
457         logger.debug(f'Ошибка сбора физических дисков чере
з WindowsStorageSensor: {ex}')

```

```

458         if not disks and PSUTIL_AVAILABLE:
459             try:
460                 for part in psutil.disk_partitions(all=False):
461                     try:
462                         usage = psutil.disk_usage(part.mountpo
int)
463                         disks.append({'device_id': part.device
, 'name': f'Tom {part.device}', 'model': part.device, 'serial_
number': None, 'bus_type': part.fstype, 'media_type': 'Drive',
'partitions': part.mountpoint, 'total_gb': round(usage.total
/ 1024 ** 3, 1), 'free_gb': round(usage.free / 1024 ** 3, 1),
'health_pct': 100.0, 'wear_level_pct': 0.0, 'temperature_c': N
one, 'power_on_hours': None, 'status': 'Healthy (SMART OK)'})
464                     except Exception:
465                         continue
466             except Exception as ex:
467                 logger.debug(f'Ошибка сбора дисков (psutil fal
lback): {ex}')
468         if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_batte
ry'):
469             try:
470                 bat = psutil.sensors_battery()
471                 if bat is not None:
472                     battery_data['has_battery'] = True
473                     battery_data['percent'] = round(bat.percen
t, 1)
474                     battery_data['is_charging'] = bat.power_pl
ugged
475                     battery_data['power_source'] = 'AC Adapter
' if bat.power_plugged else 'Battery Discharge'
476                     battery_data['secsleft'] = bat.secsleft if
bat.secsleft != psutil.POWER_TIME_UNLIMITED else -1
477             except Exception as ex:
478                 logger.debug(f'Ошибка опроса батареи: {ex}')
479             if battery_data['has_battery']:
480                 try:
481                     ps_cmd = 'Get-CimInstance -Namespace root/wmi
-ClassName BatteryStaticData -ErrorAction SilentlyContinue | S

```

```

    elect-Object DesignedCapacity, FullChargeCapacity | ConvertTo-
    Json'
482         res = subprocess.run(f'powershell.exe -NoProfi
    le -NonInteractive -Command "{ps_cmd}"', capture_output=True,
    text=True, timeout=5, shell=True)
483         if res.returncode == 0 and res.stdout.strip():
484             import json
485             b_json = json.loads(res.stdout.strip())
486             if isinstance(b_json, dict):
487                 des = float(b_json.get('DesignedCapaci
    ty') or 0.0)
488                 full = float(b_json.get('FullChargeCap
    acity') or 0.0)
489                 if des > 0 and full > 0:
490                     battery_data['design_capacity_mwh'
    ] = des
491                     battery_data['full_charge_capacity
    _mwh'] = full
492                     wear = max(0.0, min(100.0, (1.0 -
    full / des) * 100.0))
493                     battery_data['wear_level_pct'] = r
    ound(wear, 1)
494             except Exception:
495                 pass
496             return StorageBatteryWearReport(timestamp=datetime.now
    (timezone.utc).isoformat(), disks_wear=disks, battery_wear=bat
    tery_data)
497
498     def collect_peripherals_network(self) -> PeripheralsNetwor
    kReport:
499         """Собрать данные по USB устройствам, беспроводной сет
    и Wi-Fi и аудиоустройствам."""
500         usb_devices: List[Dict[str, Any]] = []
501         wifi_data: Dict[str, Any] = {'is_connected': False, 's
    id': 'Не подключено', 'bssid': '--', 'signal_pct': 0, 'rssi_d
    bm': -100, 'channel': 0, 'radio_type': '--', 'auth_cipher': '-
    -'}
502         audio_endpoints: List[Dict[str, Any]] = []

```

```

503         try:
504             from apps.windows.telemetry.win32_ffi.setupapi import SetupAPI
505             pnp = SetupAPI()
506             devs = pnp.get_all_devices()
507             for d in devs:
508                 hw_id = d.hardware_id or ''
509                 if 'USB' in hw_id.upper() or 'USB' in (d.device_class or '').upper():
510                     vid_match = re.search('VID_([0-9A-Fa-f]{4})', hw_id)
511                     pid_match = re.search('PID_([0-9A-Fa-f]{4})', hw_id)
512                     vid = vid_match.group(1) if vid_match else '----'
513                     pid = pid_match.group(1) if pid_match else '----'
514                     usb_devices.append({'name': d.friendly_name or 'USB Устройство', 'device_id': d.device_instance_id, 'vendor_id': vid, 'product_id': pid, 'device_class': d.device_class, 'has_problem': d.has_problem, 'status': 'Error' if d.has_problem else 'Connected & Active'})
515             except Exception as ex:
516                 logger.debug(f'Ошибка сбора USB через SetupAPI: {ex}')
517         try:
518             res = subprocess.run('netsh wlan show interfaces', capture_output=True, text=True, timeout=4, shell=True)
519             if res.returncode == 0 and res.stdout:
520                 out = res.stdout
521                 state_match = re.search('State\s*:\s*connected|Состояние\s*:\s*подключено', out, re.IGNORECASE)
522                 if state_match:
523                     wifi_data['is_connected'] = True
524                     ssid_m = re.search('SSID\s*:\s*(.+)', out)
525                     bssid_m = re.search('BSSID\s*:\s*(.+)', out)

```

```

526         sig_m = re.search('Signal|Сигнал\\s*:\\s*(
    \\d+)%', out)
527         chan_m = re.search('Channel|Канал\\s*:\\s*(
    (\\d+)', out)
528         radio_m = re.search('Radio type|Тип радиом
    одуля\\s*:\\s*(.+)', out)
529         auth_m = re.search('Authentication|Проверк
    а подлинности\\s*:\\s*(.+)', out)
530         if ssid_m:
531             wifi_data['ssid'] = ssid_m.group(1).st
    rip()
532         if bssid_m:
533             wifi_data['bssid'] = bssid_m.group(1).
    strip()
534         if sig_m:
535             sig_pct = int(sig_m.group(1))
536             wifi_data['signal_pct'] = sig_pct
537             wifi_data['rssi_dbm'] = int(sig_pct /
    2.0 - 100)
538         if chan_m:
539             wifi_data['channel'] = int(chan_m.grou
    p(1))
540         if radio_m:
541             wifi_data['radio_type'] = radio_m.grou
    p(1).strip()
542         if auth_m:
543             wifi_data['auth_cipher'] = auth_m.grou
    p(1).strip()
544         except Exception as ex:
545             logger.debug(f'Ошибка Wi-Fi netsh: {ex}')
546         try:
547             audio_endpoints.append({'name': 'Основное устройст
    во воспроизведения (Default Playback)', 'type': 'Speakers / He
    adphones', 'is_default': True, 'status': 'Active'})
548             audio_endpoints.append({'name': 'Основное устройст
    во записи (Default Recording)', 'type': 'Microphone', 'is_defa
    ult': True, 'status': 'Active'})
549         except Exception:

```



```

550         pass
551         return PeripheralsNetworkReport(timestamp=datetime.now
(timezone.utc).isoformat(), usb_devices=usb_devices, wifi_tele
metry=wifi_data, audio_endpoints=audio_endpoints)
552
553     def collect_defender_telemetry(self) -> DefenderTelemetryS
ummary:
554         """Собрать сводку телеметрии Windows Defender (CFA, AS
R, исключения, угрозы)."""
555         summary = DefenderTelemetrySummary()
556         try:
557             from apps.windows.modules.defender.core.cfa_manage
r import ControlledFolderAccessManager
558             cfa_mgr = ControlledFolderAccessManager()
559             cfa_info = cfa_mgr.get_cfa_status()
560             summary.cfa_enabled = bool(cfa_info.enabled)
561             summary.cfa_mode = str(cfa_info.mode.value if hasa
ttr(cfa_info.mode, 'value') else cfa_info.mode)
562             summary.cfa_protected_folders_count = len(cfa_info
.protected_folders)
563         except Exception as ex:
564             logger.debug(f'Ошибка сбора статуса CFA Defender:
{ex}')
565
566         try:
567             from apps.windows.modules.defender.core.asr_manage
r import ASRManager
568             asr_mgr = ASRManager()
569             rules = asr_mgr.get_asr_rules()
570             summary.asr_rules_count = len(rules)
571             summary.asr_enabled_count = len([r for r in rules
if getattr(r, 'state', None) and str(r.state).lower() in ('ena
bled', 'block', 'audit', 'warn')])
572         except Exception as ex:
573             logger.debug(f'Ошибка сбора правил ASR Defender: {
ex}')
574
575         try:

```

```

576         from apps.windows.modules.defender.core.exclusions
|_auditor import ExclusionsAuditor
577         excl_auditor = ExclusionsAuditor()
578         excl = excl_auditor.audit_exclusions()
579         paths = getattr(excl, 'path_exclusions', []) or []
580         processes = getattr(excl, 'process_exclusions', []
|) or []
581         summary.path_exclusions = [str(p.value if hasattr(
|p, 'value') else p) for p in paths]
582         summary.process_exclusions = [str(p.value if hasat
|tr(p, 'value') else p) for p in processes]
583         summary.exclusions_count = len(summary.path_exclus
|ions) + len(summary.process_exclusions)
584         except Exception as ex:
585             logger.debug(f'Ошибка сбора исключений Defender: {
|ex}')
586
587         try:
588             from apps.windows.modules.defender.core.threat_man
|ager import ThreatManager
589             threat_mgr = ThreatManager()
590             threat_history = threat_mgr.get_threats_history(li
|mit=50)
591             active_threats = [t for t in threat_history if str
|(getattr(t, 'status', '')).lower() == 'active']
592             summary.active_threats_count = len(active_threats)
593             summary.threat_history_count = len(threat_history)
594         except Exception as ex:
595             logger.debug(f'Ошибка сбора угроз Defender: {ex}')
596
597         return summary
598
599     def collect_startup_telemetry(self) -> StartupTelemetrySum
|mary:
600         """Собрать срез точек автозапуска и персистентности пр
|ограмм."""
601         summary = StartupTelemetrySummary()
602         try:

```

```

603         from apps.windows.startup.core.scanner import Star
        tupScanner
604         scanner = StartupScanner()
605         entries = scanner.scan_all()
606         summary.total_entries = len(entries)
607         serialized_entries: List[Dict[str, Any]] = []
608         for e in entries:
609             loc_type = str(getattr(e, 'location_type', ''))
        )
610             if 'REGISTRY' in loc_type:
611                 summary.registry_run_count += 1
612             elif 'STARTUP_FOLDER' in loc_type:
613                 summary.startup_folders_count += 1
614             elif 'SCHEDULED_TASK' in loc_type:
615                 summary.scheduled_tasks_count += 1
616             elif 'SERVICE' in loc_type:
617                 summary.startup_services_count += 1
618
619             serialized_entries.append({
620                 'name': getattr(e, 'name', 'Unknown'),
621                 'command': getattr(e, 'command_line', ''),
622                 'location_type': loc_type,
623                 'is_enabled': getattr(e, 'is_enabled', True)
        e),
624                 'publisher': getattr(e, 'publisher', ''),
625             })
626         summary.entries = serialized_entries
627     except Exception as ex:
628         logger.debug(f'Ошибка сбора автозагрузки: {ex}')
629     return summary
630
631     def collect_vss_telemetry(self) -> VssTelemetrySummary:
632         """Собрать состояние теневых копий томов VSS."""
633         summary = VssTelemetrySummary()
634         try:
635             from apps.windows.modules.backup_manager.core.vss_
manager import VssManager
636             vss = VssManager()

```

```

637         snapshots = vss.list_snapshots()
638         summary.total_snapshots_count = len(snapshots)
639         vols: List[str] = []
640         serialized_snaps: List[Dict[str, Any]] = []
641         for s in snapshots:
642             vol = getattr(s, 'original_volume', '')
643             if vol and vol not in vols:
644                 vols.append(vol)
645                 serialized_snaps.append({
646                     'snapshot_id': getattr(s, 'snapshot_id', '
647                     '),
648                     'original_volume': vol,
649                     'creation_time': getattr(s, 'creation_time
650                     ', None),
651                     'shadow_volume_path': getattr(s, 'shadow_v
652                     olume_path', None),
653                     })
654                 summary.protected_volumes = vols
655                 summary.snapshots = serialized_snaps
656                 if snapshots and hasattr(snapshots[-1], 'creation_
657                 time'):
658                     summary.latest_snapshot_time = snapshots[-1].c
659                     reation_time
660                 except Exception as ex:
661                     logger.debug(f'Ошибка сбора теневых копий VSS: {ex
662                     }')
663                 return summary
664
665     def collect_users_telemetry(self) -> UserAccountsTelemetry
666     Summary:
667         """Собрать аудит локальных пользователей и привилегий.
668         """
669         summary = UserAccountsTelemetrySummary()
670         try:
671             from apps.windows.sysadmin.src.user_collector impo
672             rt WindowsUserCollector
673             collector = WindowsUserCollector()
674             users = collector.get_all_users()

```

```

666         summary.total_users_count = len(users)
667         admins: List[str] = []
668         logged_in: List[str] = []
669         locked_count = 0
670         serialized_users: List[Dict[str, Any]] = []
671         for u in users:
672             uname = getattr(u, 'name', '')
673             is_admin = getattr(u, 'is_admin', False)
674             is_logged = getattr(u, 'is_logged_in', False)
675             is_locked = getattr(u, 'lockout_status', False
) or not getattr(u, 'enabled', True)
676             if is_admin:
677                 admins.append(uname)
678             if is_logged:
679                 logged_in.append(uname)
680             if is_locked:
681                 locked_count += 1
682             serialized_users.append({
683                 'name': uname,
684                 'full_name': getattr(u, 'full_name', ''),
685                 'is_admin': is_admin,
686                 'is_logged_in': is_logged,
687                 'enabled': getattr(u, 'enabled', True),
688                 'account_type': getattr(u, 'account_type',
'Local'),
689                 'last_logon': getattr(u, 'last_logon', Non
e),
690             })
691         summary.admin_users_count = len(admins)
692         summary.admin_usernames = admins
693         summary.logged_in_users = logged_in
694         summary.locked_users_count = locked_count
695         summary.users = serialized_users
696     except Exception as ex:
697         logger.debug(f'Ошибка сбора локальных пользователе
й: {ex}')
698     return summary
699

```

```
700 |     def collect_extended_system_audit(self) -> ExtendedSystemA
      | uditReport:
701 |         """Собрать комплексный расширенный аудит безопасности,
      | автозагрузки, VSS и пользователей."""
702 |         import socket
703 |         return ExtendedSystemAuditReport(
704 |             timestamp=datetime.now(timezone.utc).isoformat(),
705 |             hostname=socket.gethostname(),
706 |             defender=self.collect_defender_telemetry(),
707 |             startup=self.collect_startup_telemetry(),
708 |             vss=self.collect_vss_telemetry(),
709 |             users=self.collect_users_telemetry(),
710 |         )
711 |
```

diagnostic_engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Diagnostic Engine
5 # Description:
6 #    Универсальный диагностический движок и системный анализатор
7 #    телеметрии Windows.
8 # Usage Examples:
9 #    Python API:
10 #    from apps.windows.telemetry_research.diagnostic_engine i
11 #    mport DiagnosticEngine
12 #    service = DiagnosticEngine()
13 #
14 # File: diagnostic_engine.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Универсальный диагностический движок и системный анализатор
24 телеметрии Windows."""
25
26 import json
27 from abc import ABC, abstractmethod
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional, Tuple
```

```

30 try:
31     from logger import logger
32 except ImportError:
33     from logger import logger
34
35 from apps.windows.telemetry.models import AnomalyItem, SystemD
    iagnosticReport, SystemSnapshot, TelemetryIncident
36 from apps.windows.telemetry_research.grouped_telemetry import
    (
37     GroupDiagnosticResult,
38     GroupedTelemetryBuilder,
39     SynthesisDiagnosticResult,
40     TelemetryGroupInfo,
41 )
42 from apps.windows.telemetry_research.incident_detector import
    IncidentDetector
43
44
45 class DiagnosticEngine(ABC):
46     """Абстрактный базовый класс для диагностических движков т
    елеметрии."""
47
48     def __init__(self, chat_model: Optional[Any] = None) -> No
    ne:
49         self.chat_model = chat_model
50
51     @abstractmethod
52     def evaluate_heuristics(self, data: Any) -> Tuple[int, Lis
    t[AnomalyItem], List[str]]:
53         """Выполнение детерминированных эвристических проверок
    ."""
54         pass
55
56     def build_prompt(self, data: Any, score: int, anomalies: L
    ist[AnomalyItem], summary_text: str) -> str:
57         """Построение контекстного промпта для языковой модели
    ."""
58         return (

```



```

59         "Проанализируйте обнаруженные системные аномалии и
    метрики хоста:\n\n"
60         f"Аномалии: {[a.model_dump() for a in anomalies]}\n"
    n"
61         f"Текущий статус: {summary_text}\n\n"
62         "ТРЕБОВАНИЯ К ОТВЕТУ:\n"
63         "1. Немедленно предоставьте краткую диагностическую
    оценку (2-3 предложения) и 2-3 ключевые меры по устранению у
    зких мест.\n"
64         "2. Выведите готовый аналитический ответ сразу на
    русском языке без шаблонных вводных фраз."
65     )
66
67     async def diagnose(self, data: Any) -> SystemDiagnosticRep
    ort:
68         """Запуск эвристического анализа с последующим опциона
    льным LLM-синтезом."""
69         stages: List[Dict[str, Any]] = [
70             {
71                 "stage": "init",
72                 "title": "Сбор телеметрии",
73                 "message": "□ Получение актуального среза тел
    еметрии и сенсоров хоста...",
74                 "details": "WMI, LibreHardwareMonitor, системн
    ые процессы и ресурсы хранилища",
75             }
76         ]
77         system_instruction = (
78             "Вы – ведущий инженер по системной диагностике и а
    нализу производительности хоста. "
79             "Ваша задача: на основе переданных сенсоров и теле
    метрии немедленно предоставить краткую экспертную оценку и пра
    ктические рекомендации по оптимизации ресурсов. "
80             "Сразу выведите готовый аналитический ответ на рус
    ском языке."
81         )
82         model_name = "Heuristic Analyzer"
83         generated_prompt: Optional[str] = None

```

```

84 |         raw_response: Optional[str] = None
85 |         error_msg: Optional[str] = None
86 |
87 |         # 1. Эвристический анализ
88 |         try:
89 |             score, anomalies, recommendations = self.evaluate_
| heuristics(data)
90 |             summary_text = (
91 |                 f"Интегральный индекс здоровья: {score}/100. "
92 |                 f"Обнаружено аномалий: {len(anomalies)}. "
93 |                 + (f"Критические узкие места: {'', ' '.join([a.ti
| tle for a in anomalies if a.severity == 'critical'])}." if ano
| malies else "Система в стабильном состоянии.")
94 |             )
95 |         except Exception as ex:
96 |             logger.error(f"Ошибка эвристического анализа: {ex}
| ")
97 |             score = 50
98 |             anomalies = [AnomalyItem(subsystem="Diagnostics",
| severity="warning", title="Evaluation Error", description=str(
| ex))]
99 |             recommendations = ["Проверьте доступность сенсоров
| и службы сбора телеметрии."]
100 |             summary_text = f"Ошибка оценки: {ex}"
101 |
102 |         # 2. Нейросетевой анализ при наличии активной модели
103 |         executor: Optional[Any] = self.chat_model
104 |         if executor is not None:
105 |             if hasattr(executor, "active_provider"):
106 |                 model_name = getattr(executor, "active_provide
| r", "Unified AI")
107 |             elif hasattr(executor, "model_id"):
108 |                 model_name = f"Gemini CLI ({getattr(executor,
| 'model_id', 'gemini-3.1-flash-lite'})}"
109 |             else:
110 |                 model_name = "AI Model"
111 |
112 |             prompt = self.build_prompt(data, score, anomalies,

```

```

summary_text)
113         generated_prompt = prompt
114         stages.append({
115             "stage": "prompt",
116             "title": "Формирование контекста сенсоров",
117             "message": "□ Передача состояния всех сенсоро
в и параметров в AI-модель...",
118             "details": "Телеметрия, датчики, накопители, п
роцессы и память",
119             "generated_prompt": prompt,
120             "system_instruction": system_instruction,
121         })
122
123         try:
124             if hasattr(executor, "ask"):
125                 response = await executor.ask(prompt, syst
em_instruction=system_instruction)
126             elif hasattr(executor, "chat"):
127                 response = await executor.chat(prompt)
128             elif callable(executor):
129                 response = await executor(prompt)
130             else:
131                 response = str(executor)
132
133             raw_response = str(response).strip()
134             if raw_response:
135                 summary_text = raw_response
136         except Exception as ai_err:
137             logger.warning(f"Ошибка вызова языковой модели
: {ai_err}")
138             error_msg = str(ai_err)
139
140         return SystemDiagnosticReport(
141             health_score=score,
142             summary=summary_text,
143             anomalies=anomalies,
144             recommendations=recommendations,
145             ai_model_used=model_name,

```

```

146         system_instruction=system_instruction,
147         generated_prompt=generated_prompt,
148         raw_response=raw_response,
149         error=error_msg,
150         stages=stages,
151     )
152
153
154 class SystemDiagnosticEngine(DiagnosticEngine):
155     """Диагностический движок для комплексного анализа телемет
156     рии хоста и сенсоров."""
157
158     def evaluate_heuristics(self, snapshot: SystemSnapshot) ->
159     Tuple[int, List[AnomalyItem], List[str]]:
160         """Выполнение детерминированного эвристического анализ
161         а сенсоров и телеметрии хоста.
162
163         Args:
164             snapshot: Актуальный срез телеметрии системы (Syst
165             emSnapshot).
166
167         Returns:
168             Tuple[int, List[AnomalyItem], List[str]]: Кортеж (
169             индекс_здоровья, список_аномалий, рекомендации).
170
171         """
172         score = 100
173         anomalies: List[AnomalyItem] = []
174         recommendations: List[str] = []
175
176         # ЦП
177         cpu = snapshot.cpu
178         if cpu and cpu.total_percent is not None:
179             if cpu.total_percent >= 90.0:
180                 score -= 25
181                 anomalies.append(AnomalyItem(subsystem="CPU",
182                 severity="critical", title="Critical CPU Saturation", descript
183                 ion=f"Overall CPU usage is at {cpu.total_percent}%, causing sy
184                 stem lag.))

```

```

176         recommendations.append("Inspect top CPU proces
ses and consider terminating unresponsive tasks.")
177         elif cpu.total_percent >= 75.0:
178             score -= 10
179             anomalies.append(AnomalyItem(subsystem="CPU",
severity="warning", title="Elevated CPU Load", description=f"C
PU load is elevated ({cpu.total_percent}%)."))
180
181         # Оперативная память
182         mem = snapshot.memory
183         if mem and mem.percent is not None:
184             if mem.percent >= 92.0:
185                 score -= 30
186                 anomalies.append(AnomalyItem(subsystem="RAM",
severity="critical", title="RAM Exhaustion Danger", descriptio
n=f"System RAM is {mem.percent}% full ({mem.used_gb}/{mem.tota
l_gb} GB)."))
187                 recommendations.append("Close memory-heavy app
lications or increase virtual memory paging size.")
188             elif mem.percent >= 80.0:
189                 score -= 10
190                 anomalies.append(AnomalyItem(subsystem="RAM",
severity="warning", title="High Memory Pressure", description=
f"RAM utilization is at {mem.percent}%."))
191
192         # Дисковые разделы
193         for disk in snapshot.disks or []:
194             d_pct = getattr(disk, "percent", getattr(disk, "lo
ad_percent", 0.0))
195             if d_pct >= 95.0:
196                 score -= 15
197                 anomalies.append(AnomalyItem(subsystem="Disk",
severity="critical", title=f"Drive {disk.device} Almost Full"
, description=f"Volume {disk.mountpoint} is {d_pct}% full with
only {disk.free_gb} GB remaining."))
198                 recommendations.append(f"Free up disk space on
drive {disk.device} to prevent write failures.")
199             elif d_pct >= 85.0:

```

```

200         score -= 5
201         anomalies.append(AnomalyItem(subsystem="Disk",
    severity="warning", title=f"Low Space on {disk.device}", desc
    ription=f"Volume {disk.mountpoint} is at {d_pct}% capacity."))
202
203         # Физические накопители и SMART
204         for p_disk in snapshot.physical_disks or []:
205             if p_disk.health_status and p_disk.health_status.l
    ower() in ["warning", "unhealthy", "error", "bad"]:
206                 score -= 25
207                 anomalies.append(AnomalyItem(subsystem="Storag
    e", severity="critical", title=f"SMART Warning on {p_disk.mode
    l}", description=f"Physical drive {getattr(p_disk, 'device_id'
    , '')} ({p_disk.model}) reports health status: {p_disk.health_
    status}."))
208                 recommendations.append(f"Perform disk backup a
    nd replace failing drive {p_disk.model}.")
209                 if p_disk.temperature_celsius and p_disk.temperatu
    re_celsius >= 60.0:
210                     score -= 10
211                     anomalies.append(AnomalyItem(subsystem="Storag
    e", severity="warning", title=f"Overheating Storage: {p_disk.m
    odel}", description=f"Drive temperature {p_disk.temperature_ce
    lsius}°C exceeds recommended operating limits."))
212
213         # Температурные сенсоры LHM
214         for sensor in snapshot.sensors or []:
215             if getattr(sensor, "category", "") == "temperature
    " or getattr(sensor, "sensor_category", "") == "Temperatures":
216                 val = getattr(sensor, "value", 0.0)
217                 s_name = getattr(sensor, "name", getattr(senso
    r, "sensor_name", "Sensor"))
218                 if val >= 85.0:
219                     score -= 20
220                     anomalies.append(AnomalyItem(subsystem="Th
    ermals", severity="critical", title=f"High Temperature on {s_n
    ame}", description=f"Sensor {s_name} reading {val}°C exceeds s
    afe operating threshold."))

```

```

221         recommendations.append("Check chassis airf
low, fan speeds, and clean heatsink dust filters.")
222         elif val >= 75.0:
223             score -= 5
224             anomalies.append(AnomalyItem(subsystem="Th
ermals", severity="warning", title=f"Elevated Temperature on {
s_name}", description=f"Sensor {s_name} reading {val}°C is ele
vated."))
225
226         # Топ-процессы
227         ignored_processes = {"system idle process", "system",
"idle"}
228         for proc in (snapshot.top_processes or [])[:10]:
229             p_name_lower = (proc.name or "").strip().lower()
230             if proc.pid in [0, 4] or p_name_lower in ignored_p
rocesses:
231                 continue
232                 if proc.cpu_percent and proc.cpu_percent >= 60.0:
233                     anomalies.append(AnomalyItem(subsystem="Proces
s", severity="warning", title=f"High CPU Consumer: {proc.name}
(PID {proc.pid})", description=f"Process {proc.name} is consu
ming {proc.cpu_percent:.1f}% CPU single-handedly."))
234                     if proc.memory_mb and proc.memory_mb >= 6144.0:
235                         anomalies.append(AnomalyItem(subsystem="Proces
s", severity="warning", title=f"High Memory Consumer: {proc.na
me} (PID {proc.pid})", description=f"Process {proc.name} is us
ing {proc.memory_mb:.0f} MB RAM."))
236
237         # Стабильность ОС и обновления
238         if snapshot.alerts and getattr(snapshot.alerts, "reboo
t_pending", False):
239             anomalies.append(AnomalyItem(subsystem="Updates",
severity="warning", title="System Reboot Pending", description
="Windows Updates or system components require a restart."))
240             recommendations.append("Schedule a restart to fina
lize pending Windows system updates.")
241
242         score = max(0, min(100, score))

```

```

243         if not recommendations:
244             recommendations.append("System telemetry is within
healthy operating parameters.")
245         return (score, anomalies, recommendations)
246
247     def build_prompt(self, snapshot: Any, score: int, anomalie
s: List[AnomalyItem], summary_text: str) -> str:
248         """Построение обогащенного диагностического контекста
в формате JSON для языковой модели."""
249         if not isinstance(snapshot, SystemSnapshot):
250             return super().build_prompt(snapshot, score, anoma
lies, summary_text)
251
252         builder = GroupedTelemetryBuilder()
253         groups = builder.build_all_groups(snapshot)
254         groups_payload = {g.group_id: g.payload for g in group
s}
255
256         return (
257             "Вы – ведущий инженер по анализу системной телемет
рии и надежности Windows.\n"
258             "Ниже приведен структурированный срез телеметрии х
оста по 4 ключевым доменам:\n\n"
259             f"```\njson\n{json.dumps(groups_payload, ensure_asci
i=False, indent=2)}\n```\n\n"
260             f"Обнаруженные детерминированные аномалии: {[a.mod
el_dump() for a in anomalies]}\n"
261             f"Расчетный индекс здоровья (Heuristic Score): {sc
ore}/100\n\n"
262             "ИНСТРУКЦИИ:\n"
263             "1. Немедленно выдайте краткое аналитическое резюм
е состояния хоста (2-3 предложения).\n"
264             "2. Перечислите 2-4 конкретных практических шага п
о оптимизации и устранению найденных узких мест.\n"
265             "3. Отвечайте строго на русском языке."
266         )
267
268     async def diagnose_group(

```



```

269         self,
270         group_id: str = "",
271         payload: Optional[Dict[str, Any]] = None,
272         title: str = "",
273         group_info: Optional[TelemetryGroupInfo] = None,
274     ) -> GroupDiagnosticResult:
275         """Диагностика одной изолированной группы телеметрии с
помощью эвристики и LLM."""
276         gid = group_id or (group_info.group_id if group_info e
lse "general")
277         gtitle = title or (group_info.title if group_info else
gid)
278         gpayload = payload if payload is not None else (group_
info.payload if group_info else {})
279
280         status = "ok"
281         key_metrics: Dict[str, Any] = {}
282         anomalies: List[str] = []
283         recommendations: List[str] = []
284
285         if gid == "compute_thermals":
286             cpu = gpayload.get("cpu", {})
287             cpu_load = cpu.get("load_percent", 0.0)
288             pkg_temp = cpu.get("package_temperature_celsius")
or 0.0
289             key_metrics["cpu_load"] = f"{cpu_load}%"
290             key_metrics["package_temp"] = f"{pkg_temp}°C" if p
kg_temp else "N/A"
291             if cpu_load >= 85.0 or pkg_temp >= 85.0:
292                 status = "critical"
293                 anomalies.append(f"Критическая нагрузка/нагрев
CPU: {cpu_load}%, {pkg_temp}°C")
294                 recommendations.append("Проверьте систему охла
ждения и фоновые процессы.")
295             elif cpu_load >= 70.0 or pkg_temp >= 75.0:
296                 status = "warning"
297                 anomalies.append(f"Повышенная нагрузка CPU: {c
pu_load}%")

```

```

298         elif gid == "memory_processes":
299             ram = gpayload.get("ram", {})
300             ram_used = ram.get("used_percent", 0.0)
301             key_metrics["ram_used_percent"] = f"{ram_used}%"
302             if ram_used >= 90.0:
303                 status = "critical"
304                 anomalies.append(f"Память исчерпана: {ram_used}
305 | %")
306                 recommendations.append("Закройте неиспользуемы
307 | е программы.")
308             elif ram_used >= 75.0:
309                 status = "warning"
310                 anomalies.append(f"Высокое потребление памяти:
311 | {ram_used}%")
312             elif gid == "storage_smart":
313                 parts = gpayload.get("storage_partitions", [])
314                 max_d = max([p.get("used_percent", 0.0) for p in p
315 | arts] or [0.0])
316                 key_metrics["max_disk_used"] = f"{max_d}%"
317                 if max_d >= 95.0:
318                     status = "critical"
319                     anomalies.append(f"Том заполнен на {max_d}%")
320                     recommendations.append("Очистите дисковое прос
321 | транство.")
322                 elif max_d >= 85.0:
323                     status = "warning"
324                     anomalies.append(f"Низкий объем свободного мес
325 | та на диске ({max_d}%")
326             elif gid == "system_network":
327                 reboot = gpayload.get("system_updates", {}).get("r
328 | eboot_pending", False)
329                 key_metrics["reboot_pending"] = reboot
330                 if reboot:
331                     status = "warning"
332                     anomalies.append("Ожидается перезагрузка для з
333 | авершения обновлений ОС.")
334                 recommendations.append("Запланируйте перезагру
335 | зку компьютера.")

```

```

327
328         summary = f"Подсистема '{gtitle}' находится в статусе:
        {status.upper()}."
329         model_name = "Heuristic Analyzer"
330         raw_response = None
331
332         executor = self.chat_model
333         if executor is not None:
334             if hasattr(executor, "active_provider"):
335                 model_name = getattr(executor, "active_provide
r", "Unified AI")
336             elif hasattr(executor, "model_id"):
337                 model_name = f"Gemini CLI ({getattr(executor,
'model_id', 'gemini-3.1-flash-lite')})"
338             else:
339                 model_name = "AI Model"
340
341         prompt = (
342             f"Домен: {gtitle}\n"
343             f"Данные телеметрии:\n```json\n{json.dumps(gpa
yload, ensure_ascii=False, indent=2)}\n```\n\n"
344             "Сформулируйте краткий вердикт и 1-2 рекоменда
ции."
345         )
346         try:
347             if hasattr(executor, "ask"):
348                 res = await executor.ask(prompt)
349             elif hasattr(executor, "chat"):
350                 res = await executor.chat(prompt)
351             elif callable(executor):
352                 res = await executor(prompt)
353             else:
354                 res = str(executor)
355             raw_response = str(res).strip()
356             summary = raw_response
357         except Exception as ex:
358             logger.warning(f"Ошибка LLM анализа группы {gi
d}: {ex}")

```

```
359 |
360 |         return GroupDiagnosticResult(
361 |             group_id=gid,
362 |             title=gtitle,
363 |             icon="bi-cpu",
364 |             status=status,
365 |             summary=summary,
366 |             anomalies=anomalies,
367 |             recommendations=recommendations,
368 |             key_metrics=key_metrics,
369 |             raw_response=raw_response,
370 |             ai_model_used=model_name,
371 |         )
372 |
373 |     async def synthesize_final_report(self, groups: List[Group
DiagnosticResult]) -> SynthesisDiagnosticResult:
374 |         """Синтез общего вердикта по результатам всех групп."""
375 |         has_critical = any(g.status == "critical" for g in gro
ups)
376 |         has_warning = any(g.status == "warning" for g in group
s)
377 |         health_score = 60 if has_critical else (78 if has_warn
ing else 100)
378 |         status_label = "Критическое" if has_critical else ("Вн
имание" if has_warning else "Отличное")
379 |
380 |         actions = []
381 |         for g in groups:
382 |             actions.extend(g.recommendations)
383 |
384 |         summary = f"Общий вердикт: синтез завершен по {len(gro
ups)} доменам. Индекс здоровья: {health_score}/100 ({status_la
bel})."
385 |         model_name = "Heuristic Analyzer"
386 |
387 |         executor = self.chat_model
388 |         if executor is not None:
```

```

389         if hasattr(executor, "active_provider"):
390             model_name = getattr(executor, "active_provider", "Unified AI")
391             prompt = f"Синтезируйте общий вердикт по группам: {[g.model_dump() for g in groups]}"
392             try:
393                 if hasattr(executor, "ask"):
394                     res = await executor.ask(prompt)
395                     summary = str(res).strip()
396             except Exception:
397                 pass
398
399         return SynthesisDiagnosticResult(
400             health_score=health_score,
401             status_label=status_label,
402             executive_summary=summary,
403             critical_actions=actions[:4] if actions else ["Система функционирует штатно."],
404             ai_model_used=model_name,
405             groups_evaluated=len(groups),
406         )
407
408     async def synthesize_all_groups(self, groups: List[GroupDiagnosticResult]) -> SynthesisDiagnosticResult:
409         """Алиас для synthesize_final_report."""
410         return await self.synthesize_final_report(groups)
411
412     async def diagnose_incident(self, incident: TelemetryIncident) -> SystemDiagnosticReport:
413         """Формирует экспертный диагностический отчет по инциденту аномалии с привлечением LLM.
414
415         Args:
416             incident: Зафиксированный объект TelemetryIncident.
417
418         Returns:
419             SystemDiagnosticReport: Диагностический отчет с ре

```

```

    комендациями.
420         """
421         score = 40 if incident.severity == 'critical' else 65
422         anomalies = [
423             AnomalyItem(
424                 subsystem=f'Incident [{incident.trigger_type}]
',
425                 severity=incident.severity,
426                 title=incident.title,
427                 description=incident.description,
428             )
429         ]
430         recommendations = [
431             f"Проверьте процессы, вызвавшие триггер ({', '.join([p.get('name', '') for p in incident.suspect_processes]) or
'системная активность'}).",
432             "Изучите сопутствующие системные события и сетевые
соединения в окне инцидента.",
433         ]
434
435         summary_text = f"Инцидент {incident.incident_id} [{incident.severity.upper()}]: {incident.title}. {incident.description}"
436         model_name = "Incident Heuristic Analyzer"
437         prompt = IncidentDetector.format_incident_markdown_for_
_llm(incident)
438         raw_response = None
439         error_msg = None
440
441         executor = self.chat_model
442         if executor is not None:
443             if hasattr(executor, "active_provider"):
444                 model_name = getattr(executor, "active_provider", "Unified AI")
445             elif hasattr(executor, "model_id"):
446                 model_name = f"Gemini CLI ({getattr(executor, 'model_id', 'gemini-3.1-flash-lite'))"
447             else:

```

```
448         model_name = "AI Model"
449
450     try:
451         if hasattr(executor, "ask"):
452             res = await executor.ask(prompt)
453         elif hasattr(executor, "chat"):
454             res = await executor.chat(prompt)
455         elif callable(executor):
456             res = await executor(prompt)
457         else:
458             res = str(executor)
459         raw_response = str(res).strip()
460         if raw_response:
461             summary_text = raw_response
462     except Exception as ex:
463         logger.warning(f"Ошибка вызова LLM для инциден
та {incident.incident_id}: {ex}")
464         error_msg = str(ex)
465
466     return SystemDiagnosticReport(
467         health_score=score,
468         summary=summary_text,
469         anomalies=anomalies,
470         recommendations=recommendations,
471         ai_model_used=model_name,
472         generated_prompt=prompt,
473         raw_response=raw_response,
474         error=error_msg,
475     )
476
```

extractor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Extractor
5 # =====
6 # Description:
7 #   Загрузчик и парсер логов телеметрии из файлов и директорий
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry_research.extractor import TelemetryDataExtractor
12 #       service = TelemetryDataExtractor()
13 #
14 # File: extractor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Загрузчик и парсер логов телеметрии из файлов и директорий.
24 """
25
26 import json
27 import os
28 from datetime import datetime, timezone
29 from pathlib import Path
30 from typing import Any, Dict, Generator, List, Optional, Union
```



```

30
31 try:
32     from logger import logger
33 except ImportError:
34     from logger import logger
35
36 try:
37     from apps.windows.telemetry.sqlite import TelemetryStorage
38 except ImportError:
39     TelemetryStorage = None
40
41
42 class TelemetryDataExtractor:
43     """Извлекатель и нормализатор данных из базы данных и файло
44     в логов телеметрии."""
45
46     def __init__(
47         self,
48         default_log_dirs: Optional[List[Union[str, Path]]] = None,
49         storage: Optional[Any] = None,
50     ) -> None:
51         """Инициализация извлекателя логов.
52
53         Args:
54             default_log_dirs: Список базовых директорий для по
55             иска логов по умолчанию.
56             storage: Экземпляр хранилища SQLite базы данных.
57         """
58         self._storage = storage
59         base_proj = Path(__file__).resolve().parents[3]
60         appdata = os.environ.get("APPDATA", str(Path.home() /
61         "AppData" / "Roaming"))
62         self.default_log_dirs = (
63             [Path(p) for p in default_log_dirs]
64             if default_log_dirs
65             else [
66                 base_proj / "logs" / "telemetry",

```

```

64 |             Path(appdata) / "AI-Breadboard" / "apps" / "lo
| gs",
65 |             Path(appdata) / "AI-Breadboard" / "apps" / "wi
| ndows" / "telemetry" / "logs",
66 |             base_proj / "data" / "telemetry" / "hardware_a
| rchives",
67 |         ]
68 |     )
69 |
70 |     @property
71 |     def storage(self) -> Optional[Any]:
72 |         """Получить экземпляр SQLite хранилища телеметрии."""
73 |         if self._storage is None and TelemetryStorage is not N
| one:
74 |             try:
75 |                 self._storage = TelemetryStorage.get_instance(
| )
76 |             except Exception:
77 |                 self._storage = None
78 |         return self._storage
79 |
80 |     def discover_log_files(self, search_dir: Optional[Union[str, Path]] = None) -> List[Path]:
81 |         """Обнаружить все доступные файлы логов и базы данных
| телеметрии.
82 |
83 |         Args:
84 |             search_dir: Опциональная конкретная директория для
| поиска.
85 |
86 |         Returns:
87 |             List[Path]: Список обнаруженных путей к файлам лог
| ов.
88 |         """
89 |         dirs_to_scan = [Path(search_dir)] if search_dir else s
| elf.default_log_dirs
90 |         found_files: List[Path] = []
91 |         for d in dirs_to_scan:

```

```

92         if not d.exists() or not d.is_dir():
93             continue
94         for pattern in ("*.db", "*.json", "*.jsonl", "*.log",
95                         "*.csv"):
96             try:
97                 for file_path in d.glob(pattern):
98                     if file_path.is_file() and file_path.stat().st_size > 0:
99                         found_files.append(file_path)
100             except Exception as ex:
101                 logger.warning(f"Ошибка сканирования директории {d} по шаблону {pattern}: {ex}")
102
103     seen = set()
104     unique_files = []
105     for f in found_files:
106         resolved = str(f.resolve())
107         if resolved not in seen:
108             seen.add(resolved)
109             unique_files.append(f)
110     return unique_files
111
112     def parse_file(self, file_path: Union[str, Path]) -> List[Dict[str, Any]]:
113         """Распарсить конкретный файл логов или базу данных телеметрии.
114
115         Args:
116             file_path: Путь к файлу лога (.db, .json, .jsonl, .csv).
117
118         Returns:
119             List[Dict[str, Any]]: Список нормализованных записей с метками времени.
120
121         """
122         path = Path(file_path)
123         if not path.exists() or not path.is_file():
124             logger.warning(f"Файл логов не найден: {path}")

```

[illegible]

```

155 |                 entries.append(self._n
| normalize_entry(item, source_file=path.name))
156 |                 elif "records" in content and isin
| stance(content["records"], list):
157 |                     for item in content["records"]
| :
158 |                         if isinstance(item, dict):
159 |                             entries.append(self._n
| ormalize_entry(item, source_file=path.name))
160 |                     else:
161 |                         entries.append(self._normalize
| _entry(content, source_file=path.name))
162 |                 except json.JSONDecodeError:
163 |                     f.seek(0)
164 |                     for line in f:
165 |                         line_str = line.strip()
166 |                         if not line_str:
167 |                             continue
168 |                         try:
169 |                             data = json.loads(line_str)
170 |                             if isinstance(data, dict):
171 |                                 entries.append(self._norma
| lize_entry(data, source_file=path.name))
172 |                     except Exception:
173 |                         continue
174 |             except Exception as err:
175 |                 logger.error(f"Ошибка чтения файла логов {path}: {
| err}")
176 |             return entries
177 |
178 |     def get_default_db_path(self) -> Optional[Path]:
179 |         """Получить путь к активной базе данных telemetry.db."
| ""
180 |         if self.storage and hasattr(self.storage, "db_path") a
| nd self.storage.db_path and Path(self.storage.db_path).exists(
| ):
181 |             return Path(self.storage.db_path)
182 |         appdata = os.environ.get("APPDATA") or os.environ.get(

```

```

"LOCALAPPDATA") or str(Path.home() / "AppData" / "Roaming")
183     candidates = [
184         Path(appdata) / "AI-Breadboard" / "apps" / "window
s" / "telemetry" / "logs" / "telemetry.db",
185         Path(__file__).resolve().parents[3] / "logs" / "te
lemetry" / "telemetry.db",
186         Path(appdata) / "AI-Breadboard" / "apps" / "logs"
/ "telemetry.db",
187     ]
188     for p in candidates:
189         if p.exists() and p.is_file():
190             return p
191     return None
192
193     def get_current_system_state(
194         self, db_path: Optional[Union[str, Path]] = None, limi
t: int = 100
195     ) -> Dict[str, Any]:
196         """Получить актуальное состояние системы и временные р
яды последних замеров из SQLite БД.
197
198         Args:
199             db_path: Путь к файлу telemetry.db (если не указан
, используется база по умолчанию).
200             limit: Количество последних точек временного ряда
(по умолчанию 100).
201
202         Returns:
203             Dict[str, Any]: Словарь с текущим срезом метрик, в
ременными рядами, сенсорами и процессами.
204         """
205         target_db = Path(db_path) if db_path else self.get_def
ault_db_path()
206         if not target_db or not target_db.exists():
207             return {
208                 "status": "not_found",
209                 "message": "База данных telemetry.db не найден
a",

```

```

210         "db_path": str(target_db) if target_db else No
ne,
211         "latest_snapshot": {},
212         "time_series": {},
213         "sensors": [],
214         "top_processes": [],
215         "recent_events": [],
216     }
217
218     try:
219         import sqlite3
220         conn = sqlite3.connect(str(target_db), timeout=5.0
221 )
222         conn.row_factory = sqlite3.Row
223         cursor = conn.cursor()
224
225         cursor.execute("SELECT name FROM sqlite_master WHE
RE type='table'")
226         tables = {row[0] for row in cursor.fetchall()}
227
228         # 1. Последние системные снелшоты (хронологический
порядок для графиков)
229         snapshots: List[Dict[str, Any]] = []
230         total_snapshots = 0
231         if "system_snapshots" in tables:
232             cursor.execute("SELECT count(*) FROM system_sn
apshots")
233             total_snapshots = cursor.fetchone()[0]
234
235             cursor.execute(
236                 "SELECT * FROM system_snapshots ORDER BY i
d DESC LIMIT ?", (max(10, min(limit, 2000)),)
237             )
238             raw_snaps = [dict(r) for r in cursor.fetchall(
239 )]
240
241             # Переворачиваем, чтобы временной ряд шел от с
тарых к новым
242             snapshots = list(reversed(raw_snaps))

```

```

240
241         latest = snapshots[-1] if snapshots else {}
242
243         # 2. Формирование временных рядов
244         time_series = {
245             "labels": [],
246             "cpu_load": [],
247             "cpu_freq_ghz": [],
248             "ram_percent": [],
249             "ram_used_gb": [],
250             "gpu_load": [],
251             "gpu_temp": [],
252             "disk_read_mb": [],
253             "disk_write_mb": [],
254             "net_recv_kb": [],
255             "net_sent_kb": [],
256         }
257
258         for s in snapshots:
259             ts_str = str(s.get("timestamp", ""))
260             # Короткая метка времени HH:MM:SS
261             short_time = ts_str[11:19] if len(ts_str) >= 1
262             9 else ts_str
263             time_series["labels"].append(short_time)
264             time_series["cpu_load"].append(round(float(s.g
265             et("cpu_total_percent") or 0.0), 1))
266             freq_mhz = float(s.get("cpu_frequency_mhz") or
267             0.0)
268             time_series["cpu_freq_ghz"].append(round(freq_
269             mhz / 1000.0, 2) if freq_mhz > 0 else 0.0)
270             time_series["ram_percent"].append(round(float(
271             s.get("memory_percent") or 0.0), 1))
272             time_series["ram_used_gb"].append(round(float(
273             s.get("memory_used_gb") or 0.0), 2))
274             time_series["gpu_load"].append(round(float(s.g
275             et("gpu_load_percent") or 0.0), 1))
276             time_series["gpu_temp"].append(round(float(s.g
277             et("gpu_temp_c") or 0.0), 1))

```



```

270 |
271 |         # Диск: байты/сек -> MB/s
272 |         rb = float(s.get("disk_read_bytes_sec") or 0.0
273 |         )
274 |         wb = float(s.get("disk_write_bytes_sec") or 0.
275 |         0)
276 |         time_series["disk_read_mb"].append(round(rb /
277 |         (1024 * 1024), 2))
278 |         time_series["disk_write_mb"].append(round(wb /
279 |         (1024 * 1024), 2))
280 |
281 |         # Сеть: байты/сек -> KB/s
282 |         nr = float(s.get("network_recv_bytes_sec") or
283 |         0.0)
284 |         ns = float(s.get("network_sent_bytes_sec") or
285 |         0.0)
286 |         time_series["net_recv_kb"].append(round(nr / 1
287 |         024, 1))
288 |         time_series["net_sent_kb"].append(round(ns / 1
289 |         024, 1))
290 |
291 |     # 3. Актуальные сенсоры (LHM / Hardware)
292 |     sensors: List[Dict[str, Any]] = []
293 |     if "sensor_polls" in tables:
294 |         cursor.execute(
295 |             """
296 |             SELECT sensor_name, sensor_category, unit,
297 |             value, hardware_name, timestamp
298 |             FROM sensor_polls
299 |             ORDER BY id DESC LIMIT 50
300 |             """
301 |         )
302 |         seen_sensors = set()
303 |         for r in cursor.fetchall():
304 |             s_dict = dict(r)
305 |             key = f"{s_dict.get('hardware_name')}_{s_d
306 |             ict.get('sensor_name')}"
307 |             if key not in seen_sensors:

```

```

298 |             seen_sensors.add(key)
299 |             sensors.append(s_dict)
300 |
301 |         # 4. Топ активных процессов
302 |         top_processes: List[Dict[str, Any]] = []
303 |         if "process_snapshots" in tables and latest.get("i
    | d"):
304 |             cursor.execute(
305 |                 """
306 |                 SELECT pid, name, cpu_percent, memory_mb,
    | memory_percent, num_threads, username, status
307 |                 FROM process_snapshots
308 |                 WHERE snapshot_id = ?
309 |                 ORDER BY cpu_percent DESC LIMIT 15
310 |                 """,
311 |                 (latest["id"],),
312 |             )
313 |             top_processes = [dict(r) for r in cursor.fetch
    | all()]
314 |
315 |         if not top_processes and "process_outliers" in tab
    | les:
316 |             cursor.execute(
317 |                 """
318 |                 SELECT pid, name, cpu_percent, memory_mb,
    | memory_percent, num_threads, username, status
319 |                 FROM process_outliers
320 |                 ORDER BY id DESC LIMIT 15
321 |                 """,
322 |             )
323 |             top_processes = [dict(r) for r in cursor.fetch
    | all()]
324 |
325 |         # 5. Последние события
326 |         recent_events: List[Dict[str, Any]] = []
327 |         if "device_events" in tables:
328 |             cursor.execute(
329 |                 """

```

```

330         SELECT timestamp, event_type, friendly_name, category, has_problem, problem_code
331         FROM device_events
332         ORDER BY id DESC LIMIT 8
333         """
334     )
335     for r in cursor.fetchall():
336         d = dict(r)
337         d["source"] = "device"
338         recent_events.append(d)
339
340     if "w64_events" in tables:
341         cursor.execute(
342             """
343             SELECT timestamp, event_type, path, name,
pid, provider
344             FROM w64_events
345             ORDER BY id DESC LIMIT 8
346             """
347         )
348         for r in cursor.fetchall():
349             d = dict(r)
350             d["source"] = "system_activity"
351             recent_events.append(d)
352
353     recent_events.sort(key=lambda x: str(x.get("timestamp", "")), reverse=True)
354     recent_events = recent_events[:15]
355
356     stat_info = target_db.stat()
357     conn.close()
358
359     return {
360         "status": "ok",
361         "db_path": str(target_db.resolve()),
362         "db_name": target_db.name,
363         "db_size_bytes": stat_info.st_size,
364         "db_size_mb": round(stat_info.st_size / (1024

```

```

* 1024), 2),
365         "total_snapshots": total_snapshots,
366         "returned_points": len(snapshots),
367         "latest_snapshot": latest,
368         "time_series": time_series,
369         "sensors": sensors,
370         "top_processes": top_processes,
371         "recent_events": recent_events,
372     }
373     except Exception as ex:
374         logger.error(f"Ошибка получения текущего состояния
из telemetry.db: {ex}")
375         return {
376             "status": "error",
377             "message": str(ex),
378             "db_path": str(target_db) if target_db else No
ne,
379             "latest_snapshot": {},
380             "time_series": {},
381             "sensors": [],
382             "top_processes": [],
383             "recent_events": [],
384         }
385
386     def load_from_sqlite_db(self, db_path: Union[str, Path], l
imit: int = 5000) -> List[Dict[str, Any]]:
387         """Загрузить исторические записи телеметрии напрямую и
з произвольного SQLite файла.
388
389         Args:
390             db_path: Путь к файлу базы данных SQLite.
391             limit: Максимальное число извлекаемых записей.
392
393         Returns:
394             List[Dict[str, Any]]: Список нормализованных запис
ей из БД.
395         """
396         records: List[Dict[str, Any]] = []

```

```
397 |         try:
398 |             import sqlite3
399 |             conn = sqlite3.connect(str(db_path))
400 |             conn.row_factory = sqlite3.Row
401 |             cursor = conn.cursor()
402 |
403 |             # Проверка наличия таблиц
404 |             cursor.execute("SELECT name FROM sqlite_master WHE
RE type='table'")
405 |             tables = {row[0] for row in cursor.fetchall()}
406 |
407 |             if "system_snapshots" in tables:
408 |                 cursor.execute("SELECT * FROM system_snapshots
ORDER BY id DESC LIMIT ?", (limit,))
409 |                 for row in cursor.fetchall():
410 |                     row_dict = dict(row)
411 |                     row_dict["source_file"] = f"{Path(db_path)
.name}:system_snapshots"
412 |                     records.append(self._normalize_entry(row_d
ict, source_file=f"{Path(db_path).name}:system_snapshots"))
413 |
414 |             if "sensor_polls" in tables:
415 |                 cursor.execute("SELECT * FROM sensor_polls ORD
ER BY id DESC LIMIT ?", (limit,))
416 |                 for row in cursor.fetchall():
417 |                     row_dict = dict(row)
418 |                     row_dict["source_file"] = f"{Path(db_path)
.name}:sensor_polls"
419 |                     records.append(self._normalize_entry(row_d
ict, source_file=f"{Path(db_path).name}:sensor_polls"))
420 |
421 |             if "telemetry_events" in tables:
422 |                 cursor.execute("SELECT * FROM telemetry_events
ORDER BY id DESC LIMIT ?", (limit,))
423 |                 for row in cursor.fetchall():
424 |                     row_dict = dict(row)
425 |                     row_dict["source_file"] = f"{Path(db_path)
.name}:telemetry_events"
```

```

426         records.append(self._normalize_entry(row_d
|         ict, source_file=f"{Path(db_path).name}:telemetry_events"))
427
428         conn.close()
429     except Exception as ex:
430         logger.warning(f"Ошибка загрузки телеметрии из БД
| SQLite {db_path}: {ex}")
431     return records
432
433     def load_from_database(self, limit: int = 2000) -> List[Di
| ct[str, Any]]:
434         """Загрузить исторические записи телеметрии из стандар
| тного хранилища.
435
436         Args:
437             limit: Максимальное число извлекаемых записей.
438
439         Returns:
440             List[Dict[str, Any]]: Список нормализованных запис
| ей из БД.
441         """
442         if self.storage is None:
443             return []
444         records: List[Dict[str, Any]] = []
445         try:
446             snapshots = self.storage.get_snapshots(limit=limit
| )
447             for snap in snapshots:
448                 rec = dict(snap) if not isinstance(snap, dict)
| and hasattr(snap, "__dict__") else (snap if isinstance(snap,
| dict) else {})
449                 rec["source_file"] = "sqlite:system_snapshots"
450                 records.append(self._normalize_entry(rec, sour
| ce_file="sqlite:system_snapshots"))
451             sensor_polls = self.storage.get_sensor_history(lim
| it=limit)
452             for s in sensor_polls:
453                 rec = dict(s) if not isinstance(s, dict) and h

```

```

    asattr(s, "__dict__") else (s if isinstance(s, dict) else {}))
454         rec["source_file"] = "sqlite:sensor_polls"
455         records.append(self._normalize_entry(rec, source_file="sqlite:sensor_polls"))
456         events = self.storage.get_events(limit=limit)
457         for ev in events:
458             rec = dict(ev) if not isinstance(ev, dict) and
hasattr(ev, "__dict__") else (ev if isinstance(ev, dict) else
                {})
459             rec["source_file"] = "sqlite:telemetry_events"
460             records.append(self._normalize_entry(rec, source_file="sqlite:telemetry_events"))
461         except Exception as ex:
462             logger.warning(f"Ошибка загрузки телеметрии из базы данных: {ex}")
463         return records
464
465     def load_all_records(
466         self,
467         source: Optional[Union[str, Path, List[Dict[str, Any]]]] = None,
468         include_database: bool = True,
469     ) -> List[Dict[str, Any]]:
470         """Загрузить все записи телеметрии из источника, базы данных или найденных файлов.
471
472         Args:
473             source: Путь к директории, файлу или готовый список записей.
474             include_database: Загружать ли записи из базы данных SQLite.
475
476         Returns:
477             List[Dict[str, Any]]: Отсортированный по времени список записей телеметрии.
478
479         """
479         if isinstance(source, list):
480             return [self._normalize_entry(x, source_file="memo

```

```

    ry") for x in source if isinstance(x, dict)]
481
482     all_records: List[Dict[str, Any]] = []
483     if source:
484         source_path = Path(source)
485         if source_path.is_file():
486             all_records.extend(self.parse_file(source_path
487         ))
488         elif source_path.is_dir():
489             for f in self.discover_log_files(source_path):
490                 all_records.extend(self.parse_file(f))
491     else:
492         if include_database:
493             all_records.extend(self.load_from_database())
494             for f in self.discover_log_files():
495                 all_records.extend(self.parse_file(f))
496     all_records.sort(key=lambda item: item.get("timestamp"
497     , ""))
498     return all_records
499
500     def _normalize_entry(self, raw: Dict[str, Any], source_file: str = "") -> Dict[str, Any]:
501         """Привести сырую запись телеметрии к единому стандарту.
502
503         Args:
504             raw: Исходный словарь данных.
505             source_file: Имя исходного файла.
506
507         Returns:
508             Dict[str, Any]: Нормализованная запись.
509
510         """
511         normalized = dict(raw)
512         normalized.setdefault("source_file", source_file)
513         ts = raw.get("timestamp") or raw.get("time") or raw.get("created_at") or raw.get("recorded_at")
514         if not ts:

```



```
513 |         ts = datetime.now(timezone.utc).isoformat()
514 |     elif isinstance(ts, (int, float)):
515 |         try:
516 |             ts = datetime.fromtimestamp(ts, tz=timezone.ut
c).isoformat()
517 |         except Exception:
518 |             ts = str(ts)
519 |     else:
520 |         ts = str(ts)
521 |     normalized["timestamp"] = ts
522 |     return normalized
523 |
```

grouped_telemetry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Grouped Telemetry
5 # Description:
6 #   Разбиение телеметрии хоста на специализированные доменные
7 #   группы для LLM-анализа.
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.grouped_telemetry i
11 #   mport TelemetryGroupInfo
12 #   service = TelemetryGroupInfo()
13 #
14 # File: grouped_telemetry.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Разбиение телеметрии хоста на специализированные доменные г
24   руппы для LLM-анализа."""
25
26 from typing import Any, Dict, List, Optional
27 from pydantic import BaseModel, Field
28 from apps.windows.telemetry.models import SystemSnapshot
29
```

```

30 class TelemetryGroupInfo(BaseModel):
31     """Описание и структурированные данные одной диагностическ
ой группы."""
32     group_id: str = Field(..., description="Уникальный идентиф
икатор группы")
33     title: str = Field(..., description="Человекочитаемый заго
ловок домена")
34     icon: str = Field(default="bi-cpu", description="Bootstrap
-иконка")
35     description: str = Field(..., description="Краткое описани
е охватываемых подсистем")
36     order: int = Field(default=1, description="Порядковый номе
р выполнения в цепочке")
37     payload: Dict[str, Any] = Field(default_factory=dict, desc
ription="Срез данных телеметрии и сенсоров")
38
39
40 class GroupedTelemetryBuilder:
41     """Генератор изолированных пакетов телеметрии по функциона
льным группам."""
42
43     @staticmethod
44     def build_compute_thermals_group(snapshot: SystemSnapshot)
-> TelemetryGroupInfo:
45         """Формирует группу 1: Вычислительные ресурсы (CPU/GPU
) и тепловой режим."""
46         try:
47             sensors = snapshot.sensors or []
48             temps = {s.name: s.value for s in sensors if s.name
and (s.category == "temperature" or "temp" in (s.sensor_id or
"").lower())}
49             fans = {s.name: f"{s.value} {s.unit}" for s in sen
sors if s.name and (s.category == "fan" or "fan" in (s.sensor_
id or "").lower())}
50             voltages = {s.name: f"{s.value} {s.unit}" for s in
sensors if s.name and (s.category == "voltage" or "voltage" i
n (s.sensor_id or "").lower())}
51             powers = {s.name: f"{s.value} {s.unit}" for s in s

```

```

    ensors if s.name and (s.category == "power" or "power" in (s.s
    ensor_id or "").lower()))}
52         gpus_data = [
53             {
54                 "name": g.name or "GPU",
55                 "load_percent": g.load_percent,
56                 "memory_total_gb": g.memory_total_gb,
57                 "memory_used_gb": g.memory_used_gb,
58                 "memory_free_gb": g.memory_free_gb,
59                 "temperature_celsius": g.temperature_celsi
    us,
60             }
61             for g in snapshot.gpus or []
62         ]
63         cpu = snapshot.cpu
64         cpu_model = cpu.model if cpu else ""
65         cpu_arch = cpu.architecture if cpu else "x86_64"
66         cpu_load = round(cpu.total_percent, 1) if cpu and
    cpu.total_percent is not None else 0.0
67         p_cores = cpu.physical_cores if cpu else 1
68         l_cores = cpu.logical_cores if cpu else 1
69         freq = cpu.frequency_mhz if cpu else 0.0
70         pkg_temp = cpu.temperature_celsius if cpu else Non
    e
71         per_core = [round(c, 1) for c in cpu.per_core_perc
    ent or [] if c is not None] if cpu else []
72
73         payload: Dict[str, Any] = {
74             "cpu": {
75                 "model": cpu_model,
76                 "architecture": cpu_arch,
77                 "load_percent": cpu_load,
78                 "physical_cores": p_cores,
79                 "logical_cores": l_cores,
80                 "frequency_mhz": freq,
81                 "package_temperature_celsius": pkg_temp,
82                 "per_core_percent": per_core,
83             },

```

```

84         "gpu": gpu_data,
85         "sensors": {
86             "temperatures_celsius": temps,
87             "fans_speed": fans,
88             "voltages": voltages,
89             "powers_watt": powers,
90         },
91     }
92     except Exception:
93         payload = {
94             "cpu": {"model": "CPU", "load_percent": 0.0, "
physical_cores": 1, "logical_cores": 1},
95             "gpu": [],
96             "sensors": {"temperatures_celsius": {}, "fans_
speed": {}, "voltages": {}, "powers_watt": {}},
97         }
98     return TelemetryGroupInfo(
99         group_id="compute_thermals",
100         title="CPU, GPU и Охлаждение",
101         icon="bi-fire",
102         description="Анализ загрузки процессора, ядер, вид
еокарт, температурных датчиков и кулеров",
103         order=1,
104         payload=payload,
105     )
106
107     @staticmethod
108     def build_memory_processes_group(snapshot: SystemSnapshot)
-> TelemetryGroupInfo:
109         """Формирует группу 2: Оперативная память, Swap и акти
вные процессы."""
110         try:
111             ignored_processes = {"system idle process", "system", "idle"}
112             procs = snapshot.top_processes or []
113             active_processes = [
114                 {
115                     "pid": p.pid,

```

```

116         "name": p.name or "process",
117         "cpu_percent": round(p.cpu_percent, 1) if
p.cpu_percent is not None else 0.0,
118         "memory_mb": round(p.memory_mb, 1) if p.me
mory_mb is not None else 0.0,
119         "memory_percent": round(p.memory_percent,
1) if p.memory_percent is not None else 0.0,
120     }
121     for p in procs
122         if p.pid not in [0, 4] and (p.name or "").lowe
r() not in ignored_processes
123         ][:12]
124         mem = snapshot.memory
125         ram_total = mem.total_gb if mem else 0.0
126         ram_used = mem.used_gb if mem else 0.0
127         ram_avail = mem.available_gb if mem else 0.0
128         ram_pct = mem.percent if mem else 0.0
129         swap_total = mem.swap_total_gb if mem else 0.0
130         swap_used = mem.swap_used_gb if mem else 0.0
131         swap_pct = mem.swap_percent if mem else 0.0
132         ram_modules = [
133             {
134                 "bank_label": r.bank_label,
135                 "capacity_gb": r.capacity_gb,
136                 "speed_mhz": r.speed_mhz,
137                 "manufacturer": r.manufacturer,
138             }
139             for r in snapshot.ram_sticks or []
140         ]
141         payload: Dict[str, Any] = {
142             "ram": {"total_gb": ram_total, "used_gb": ram_
used, "available_gb": ram_avail, "used_percent": ram_pct},
143             "swap_paging_file": {"total_gb": swap_total, "
used_gb": swap_used, "used_percent": swap_pct},
144             "ram_modules": ram_modules,
145             "top_active_processes": active_processes,
146         }
147     except Exception:

```

```

148         payload = {
149             "ram": {"total_gb": 0.0, "used_gb": 0.0, "available_gb": 0.0, "used_percent": 0.0},
150             "swap_paging_file": {"total_gb": 0.0, "used_gb": 0.0, "used_percent": 0.0},
151             "ram_modules": [],
152             "top_active_processes": [],
153         }
154     return TelemetryGroupInfo(
155         group_id="memory_processes",
156         title="Память и Процессы",
157         icon="bi-memory",
158         description="Анализ объема и утилизации RAM, Swap-файла и топ-процессов потребителей ресурсов",
159         order=2,
160         payload=payload,
161     )
162
163     @staticmethod
164     def build_storage_smart_group(snapshot: SystemSnapshot) -> TelemetryGroupInfo:
165         """Формирует группу 3: Дисковые разделы, SMART-здоровье и Disk I/O."""
166         try:
167             partitions = [
168                 {
169                     "device": d.device or "",
170                     "mountpoint": d.mountpoint or "",
171                     "fstype": d.fstype or "",
172                     "total_gb": round(d.total_gb, 2) if d.total_gb is not None else 0.0,
173                     "used_gb": round(d.used_gb, 2) if d.used_gb is not None else 0.0,
174                     "free_gb": round(d.free_gb, 2) if d.free_gb is not None else 0.0,
175                     "used_percent": round(d.percent, 1) if d.percent is not None else 0.0,
176                 }

```

```

177         for d in snapshot.disks or []
178     ]
179     physical_disks = [
180         {
181             "model": p.model or "Disk",
182             "media_type": p.media_type or "SSD",
183             "size_gb": p.size_gb or 0.0,
184             "smart_health": p.health_status or "Health
185 y",
186             "temperature_celsius": p.temperature_celsi
187 us,
188             "interface_type": getattr(p, "interface_ty
189 pe", None) or "Unknown",
190         }
191     ]
192     for p in snapshot.physical_disks or []
193 ]
194     dio = snapshot.disk_io
195     r_mb = round((dio.read_bytes_per_sec or 0.0) / (10
196 24 * 1024), 2) if dio else 0.0
197     w_mb = round((dio.write_bytes_per_sec or 0.0) / (1
198 024 * 1024), 2) if dio else 0.0
199     r_iops = round(dio.read_count_per_sec or 0.0, 1) i
200 f dio else 0.0
201     w_iops = round(dio.write_count_per_sec or 0.0, 1)
202 if dio else 0.0
203     payload: Dict[str, Any] = {
204         "storage_partitions": partitions,
205         "physical_drives_smart": physical_disks,
206         "disk_io_rates": {"read_mb_per_sec": r_mb, "wr
207 ite_mb_per_sec": w_mb, "read_iops": r_iops, "write_iops": w_iops},
208     },
209     }
210 except Exception:
211     payload = {
212         "storage_partitions": [],
213         "physical_drives_smart": [],
214         "disk_io_rates": {"read_mb_per_sec": 0.0, "wri
215 te_mb_per_sec": 0.0, "read_iops": 0.0, "write_iops": 0.0},

```



```

205         }
206         return TelemetryGroupInfo(
207             group_id="storage_smart",
208             title="Диски и Хранилище",
209             icon="bi-hdd",
210             description="Оценка свободного места на томах, SMA
RT-состояния накопителей и дискового I/O",
211             order=3,
212             payload=payload,
213         )
214
215     @staticmethod
216     def build_system_network_group(snapshot: SystemSnapshot) -
> TelemetryGroupInfo:
217         """Формирует группу 4: Сеть, обновления Windows и точк
и восстановления."""
218         try:
219             backup_info: Dict[str, Any] = {"system_protection"
: "Enabled", "restore_points_count": 0, "latest_restore_point"
: "None"}
220             try:
221                 from apps.windows.core.system_restore import W
indowsSystemRestoreManager
222                 restore_mgr = WindowsSystemRestoreManager(time
out_seconds=3)
223                 prot = restore_mgr.check_protection_status()
224                 backup_info["system_protection"] = "Enabled" i
f prot.get("system_protection_enabled") else "Disabled"
225                 points = restore_mgr.list_restore_points()
226                 backup_info["restore_points_count"] = len(poin
ts)
227                 if points:
228                     backup_info["latest_restore_point"] = poin
ts[0].get("creation_time") or points[0].get("description", "")
229             except Exception:
230                 pass
231
232         networks = [

```

```

233 |         {
234 |             "name": n.name or "Adapter",
235 |             "is_up": n.is_up,
236 |             "speed_mbps": n.speed_mbps or 0,
237 |             "recv_kb_per_sec": round((n.bytes_recv_per
|_sec or 0.0) / 1024, 1),
238 |             "sent_kb_per_sec": round((n.bytes_sent_per
|_sec or 0.0) / 1024, 1),
239 |         }
240 |         for n in snapshot.network or []
241 |     ]
242 |     ports = [
243 |         {"port": p.port, "process_name": p.process_name or "", "protocol": p.protocol or "TCP", "pid": p.pid}
244 |         for p in (snapshot.listening_ports or [])[:15]
245 |     ]
246 |     payload: Dict[str, Any] = {
247 |         "host_info": {
248 |             "hostname": snapshot.hostname or "localhost",
249 |             "os": f"{snapshot.os_name or 'Windows'} {s
|napshot.os_build or ''}".strip(),
250 |             "uptime_hours": round((snapshot.uptime_sec
|onds or 0.0) / 3600, 1),
251 |         },
252 |         "system_updates": {
253 |             "reboot_pending": bool(snapshot.alerts and
|snapshot.alerts.reboot_pending),
254 |             "status": snapshot.updates.status if snaps
|hot.updates else "Up to date",
255 |             "installed_kb_count": snapshot.updates.ins
|talled_kb_count if snapshot.updates else 0,
256 |         },
257 |         "backup_and_restore": backup_info,
258 |         "network_adapters": networks,
259 |         "listening_ports_sample": ports,
260 |     }
261 | except Exception:

```

```

262         payload = {
263             "host_info": {"hostname": "localhost", "os": "
Windows", "uptime_hours": 0.0},
264             "system_updates": {"reboot_pending": False, "s
tatus": "Up to date", "installed_kb_count": 0},
265             "backup_and_restore": {"system_protection": "E
nabled", "restore_points_count": 0, "latest_restore_point": "N
one"},
266             "network_adapters": [],
267             "listening_ports_sample": [],
268         }
269         return TelemetryGroupInfo(
270             group_id="system_network",
271             title="Сеть и Стабильность ОС",
272             icon="bi-shield-check",
273             description="Проверка сети, обновлений ОС, отложен
ных перезагрузок и точек восстановления",
274             order=4,
275             payload=payload,
276         )
277
278     def build_all_groups(self, snapshot: SystemSnapshot) -> Li
st[TelemetryGroupInfo]:
279         """Формирует полный упорядоченный список диагностическ
их групп."""
280         return [
281             self.build_compute_thermals_group(snapshot),
282             self.build_memory_processes_group(snapshot),
283             self.build_storage_smart_group(snapshot),
284             self.build_system_network_group(snapshot),
285         ]
286
287
288 class GroupDiagnoseRequest(BaseModel):
289     """Запрос на AI-диагностику одной конкретной группы телеме
трии."""
290     group_id: str = Field(..., description="Идентификатор груп
пы (compute_thermals, memory_processes, storage_smart, system_

```

```

network)")
291     title: Optional[str] = Field(default="", description="Опци
ональный заголовок группы")
292     payload: Dict[str, Any] = Field(..., description="Данные т
елеметрии группы")
293
294
295 class GroupDiagnosticResult(BaseModel):
296     """Результат AI-анализа одной специализированной группы."""
297     group_id: str = Field(..., description="Идентификатор груп
пы")
298     title: str = Field(..., description="Заголовок группы")
299     icon: str = Field(default="bi-cpu", description="Bootstrap
-иконка")
300     status: str = Field(default="ok", description="Статус: ok,
warning, critical")
301     summary: str = Field(default="", description="Экспертное з
аключение AI-модели по данной группе")
302     anomalies: List[str] = Field(default_factory=list, descrip
tion="Выявленные отклонения и узкие места")
303     recommendations: List[str] = Field(default_factory=list, d
escription="Рекомендации по оптимизации домена")
304     key_metrics: Dict[str, Any] = Field(default_factory=dict,
description="Ключевые метрики для карточки UI")
305     raw_response: Optional[str] = Field(default=None, descript
ion="Сырой ответ языковой модели")
306     ai_model_used: str = Field(default="AI Model", description
="Использованный AI-провайдер/модель")
307
308
309 class SynthesisRequest(BaseModel):
310     """Запрос на синтез итогового вердикта по всем завершенным
группам."""
311     groups: List[GroupDiagnosticResult] = Field(..., descripti
on="Результаты анализа всех 4 групп")
312
313

```

```
314 class SynthesisDiagnosticResult(BaseModel):
315     """Итоговый синтез состояния всей системы с интегральным H
    ealth Score."""
316     health_score: int = Field(default=100, ge=0, le=100, descr
    iption="Интегральный индекс здоровья (0-100)")
317     status_label: str = Field(default="Отличное", description=
    "Метка статуса (Отличное, Внимание, Критическое)")
318     executive_summary: str = Field(..., description="Итоговое
    резюме инженера по всей системе")
319     critical_actions: List[str] = Field(default_factory=list,
    description="Приоритетные шаги для администратора")
320     ai_model_used: str = Field(default="AI Model", description=
    "Использованный провайдер для синтеза")
321     groups_evaluated: int = Field(default=4, description="Коли
    чество оцененных доменов")
322
```

hardware_auditor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Hardware Auditor
5 # Description:
6 #     Движок аудита оборудования, драйверов и привязки датчиков.
7 #
8 # Usage Examples:
9 #     Python API:
10 #         from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
11 #
12 #         service = HardwareAuditor()
13 #
14 # File: hardware_auditor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 03:10:00
20 # =====
21
22 from __future__ import annotations
23 """Движок аудита оборудования, драйверов и привязки датчиков."""
24
25 import os
26 import platform
27 import re
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional
30 from logger import logger
```

```

31 from apps.windows.telemetry.win32_fffi.setupapi import SetupAPI
32 from apps.windows.telemetry.models import DriverInfo, Hardware
   AuditReport, HardwareDeviceAudit, HardwareSensor
33 from apps.windows.telemetry.sensors import get_hardware_sensors
34
35
36 class HardwareAuditor:
37     """Аудитор аппаратного обеспечения, драйверов и параметров
   устройств."""
38
39     def __init__(self, setupapi_client: Optional[SetupAPI] = None, storage: Optional[Any] = None) -> None:
40         """Инициализация аудитора оборудования.
41
42         Args:
43             setupapi_client: Опциональный экземпляр нативного
   клиента SetupAPI.
44             storage: Опциональный экземпляр хранилища TelemetryStorage.
45         """
46         self._setupapi = setupapi_client or (SetupAPI() if os.name == 'nt' else None)
47         self._storage = storage
48
49     def _parse_wmi_date(self, date_str: Optional[str], context: Optional[str] = None) -> Optional[datetime]:
50         """Парсинг даты из формата WMI (YYYYMMDDHHMMSS.mmmmmmm+UUU) или ISO.
51
52         Args:
53             date_str: Строка с датой.
54             context: Опциональное описание контекста (имя устройства/драйвера).
55
56         Returns:
57             Optional[datetime]: Объект datetime или None.
58         """

```

```

59         if not date_str:
60             return None
61         raw = str(date_str).strip()
62         if not raw or raw.lower() == 'none':
63             return None
64         if len(raw) >= 8 and raw[:8].isdigit():
65             try:
66                 year = int(raw[0:4])
67                 month = int(raw[4:6])
68                 day = int(raw[6:8])
69                 return datetime(year, month, day, tzinfo=timezone.
one.utc)
70             except Exception:
71                 pass
72         iso_match = re.match('^(\\d{4})-(\\d{2})-(\\d{2})', ra
w)
73         if iso_match:
74             try:
75                 return datetime(int(iso_match.group(1)), int(i
so_match.group(2)), int(iso_match.group(3)), tzinfo=timezone.u
tc)
76             except Exception:
77                 pass
78         ru_match = re.match('^(\\d{2})\\.(\\d{2})\\.(\\d{4})',
raw)
79         if ru_match:
80             try:
81                 return datetime(int(ru_match.group(3)), int(ru
_match.group(2)), int(ru_match.group(1)), tzinfo=timezone.utc)
82             except Exception:
83                 pass
84         target_info = f' для {context}' if context else ''
85         logger.debug(f'Не удалось распарсить дату WMI{target_i
nfo}: \\'{date_str}\\'')
86         return None
87
88     def _evaluate_driver_currency(self, driver_date_dt: Option
al[datetime], provider: str, device_class: str) -> tuple[str,

```



```

Optional[int]]:
89     """Расчет актуальности драйвера на основе даты выпуска
    и поставщика.
90
91     Args:
92         driver_date_dt: Дата выпуска драйвера.
93         provider: Поставщик драйвера (NVIDIA, Intel, Micro
    soft и др.).
94         device_class: Класс оборудования (Display, Net, Pr
    ocessor и др.).
95
96     Returns:
97         tuple[str, Optional[int]]: (Статус актуальности, в
    озраст в днях).
98     """
99     now = datetime.now(timezone.utc)
100     age_days: Optional[int] = None
101     if driver_date_dt:
102         age_days = max(0, (now - driver_date_dt).days)
103     is_ms_inbox = 'microsoft' in provider.lower()
104     if age_days is not None:
105         if age_days > 1825:
106             if is_ms_inbox and device_class in ('System',
    'Volume', 'Processor'):
107                 return ('Стандартный системный драйвер', a
    ge_days)
108             return ('Критически устарел (> 5 лет)', age_da
    ys)
109         elif age_days > 730:
110             if is_ms_inbox and device_class in ('System',
    'Volume', 'Processor'):
111                 return ('Стандартный системный драйвер', a
    ge_days)
112             return ('Устарел (> 2 лет)', age_days)
113         else:
114             return ('Актуален', age_days)
115     if is_ms_inbox:
116         return ('Базовый драйвер ОС', age_days)

```

```

117         return ('Не определена', age_days)
118
119     def _get_drivers_from_wmi(self) -> Dict[str, Dict[str, Any
120 ]]:
121         """Получение карты драйверов из WMI Win32_PnPSignedDri
122 ver.
123
124         Returns:
125             Dict[str, Dict[str, Any]]: Словарь с метаданными д
126 райверов по DeviceID.
127
128         """
129         drivers_map: Dict[str, Dict[str, Any]] = {}
130         if os.name != 'nt':
131             return drivers_map
132         try:
133             import pythoncom
134             pythoncom.CoInitialize()
135             import wmi
136             w = wmi.WMI()
137             for d in w.Win32_PnPSignedDriver():
138                 dev_id = getattr(d, 'DeviceID', None)
139                 if not dev_id:
140                     continue
141                 norm_id = str(dev_id).strip().upper()
142                 drivers_map[norm_id] = {'name': getattr(d, 'De
143 viceName', '') or getattr(d, 'Description', '') or '', 'driver
144 _version': getattr(d, 'DriverVersion', '') or '', 'driver_date
145 _raw': getattr(d, 'DriverDate', None), 'provider': getattr(d,
146 'DriverProviderName', 'Unknown') or 'Unknown', 'inf_name': get
147 attr(d, 'InfName', None), 'is_signed': bool(getattr(d, 'IsSign
148 ed', True))}
149         except Exception:
150             pass
151         return drivers_map
152
153     def _get_device_install_date_registry(
154         self,
155         device_instance_id: str,

```

```

146         device_name: Optional[str] = None,
147         device_class: Optional[str] = None
148     ) -> Optional[str]:
149         """Чтение даты установки устройства из системного реес
150 тра или базы данных.
151
152         Если дата в системном реестре не найдена, выполняется
153 поиск в базе данных.
154
155         Если устройство уже есть в базе данных с датой – возвр
156 ащается существующая дата (без перезаписи).
157
158         Если устройства в базе данных нет – записывается текущ
159 ая дата.
160
161     Args:
162         device_instance_id: Идентификатор PnP устройства.
163         device_name: Человекочитаемое имя устройства.
164         device_class: Класс оборудования.
165
166     Returns:
167         Optional[str]: Строка даты установки в формате DD.
168 MM.YYYY HH:MM или None.
169
170     """
171     if not device_instance_id:
172         return None
173
174     reg_date: Optional[str] = None
175     if os.name == 'nt':
176         try:
177             import winreg
178             key_path = f'SYSTEM\\CurrentControlSet\\Enum\\
179 {device_instance_id}'
180             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
181 key_path) as key:
182                 for val_name in ['InstallDate', 'FirstInst
183 allDate']:
184                     try:
185                         val, reg_type = winreg.QueryValueE
186 x(key, val_name)

```

```

175 |             if isinstance(val, int) and val >
176 | 0:
177 |                 reg_date = datetime.fromtimestamp
178 | amp(val, tz=timezone.utc).strftime('%d.%m.%Y %H:%M')
179 |                 break
180 |             except OSError:
181 |                 pass
182 |
183 |         except Exception:
184 |             pass
185 |
186 |         storage = self._storage
187 |         if storage is None:
188 |             try:
189 |                 from apps.windows.telemetry.sqlite import Tele
190 | metryStorage
191 |                 storage = TelemetryStorage.get_instance()
192 |             except Exception:
193 |                 storage = None
194 |
195 |         if reg_date:
196 |             if storage:
197 |                 try:
198 |                     storage.get_or_create_device_install_date(
199 |                         device_instance_id=device_instance_id,
200 |                         friendly_name=device_name,
201 |                         device_class=device_class,
202 |                         default_date=reg_date
203 |                     )
204 |                 except Exception as ex:
205 |                     logger.debug(f'Ошибка сохранения даты устр
206 | ойства в БД: {ex}')
207 |             return reg_date
208 |
209 |         dev_label = f"'{device_name}' [{device_instance_id}]"
210 |         if device_name else f"[{device_instance_id}]"
211 |         logger.debug(f'Дата установки для устройства {dev_labe
212 | l} не найдена в реестре')
213 |

```

```

207         if storage:
208             try:
209                 now_str = datetime.now(timezone.utc).strftime(
210                     '%d.%m.%Y %H:%M')
211                 return storage.get_or_create_device_install_da
212 te(
213             device_instance_id=device_instance_id,
214             friendly_name=device_name,
215             device_class=device_class,
216             default_date=now_str
217         )
218     except Exception as ex:
219         logger.debug(f'Ошибка работы с БД для даты уст
220 роЙства: {ex}')
221
222     return None
223
224     def _bind_sensors_to_device(self, device: HardwareDeviceAu
225 dit, all_sensors: List[HardwareSensor]) -> List[HardwareSensor
226 ]:
227         """Привязка релевантных датчиков телеметрии к аппаратн
228 ому устройству.
229
230         Args:
231             device: Аудируемое устройство.
232             all_sensors: Полный список активных аппаратных сен
233 соров.
234
235         Returns:
236             List[HardwareSensor]: Список датчиков, относящихся
237 к данному устройству.
238         """
239         matched: List[HardwareSensor] = []
240         dev_class = device.device_class.lower()
241         dev_name = device.name.lower()
242         for s in all_sensors:
243             s_name = s.name.lower()
244             s_id = s.sensor_id.lower()

```

```

237         if dev_class in ('display', 'gpu') or 'nvidia' in
dev_name or 'geforce' in dev_name or ('radeon' in dev_name):
238             if 'gpu' in s_id or 'gpu' in s_name or 'nvidia
' in s_name:
239                 matched.append(s)
240                 continue
241             if dev_class in ('processor', 'cpu') or 'intel' in
dev_name or 'amd' in dev_name or ('ryzen' in dev_name):
242                 if 'cpu' in s_id or 'cpu' in s_name or 'therma
l' in s_id or ('acpi' in s_id):
243                     matched.append(s)
244                     continue
245                 if dev_class in ('diskdrive', 'storage', 'nvme') o
r 'ssd' in dev_name or 'hdd' in dev_name:
246                     if 'disk' in s_id or 'drive' in s_id or 'smart
' in s_id or ('storage' in s_id):
247                         matched.append(s)
248                         continue
249                 if dev_class in ('net', 'network') or 'ethernet' i
n dev_name or 'wi-fi' in dev_name or ('wireless' in dev_name):
250                     if 'net_' in s_id or 'internet_' in s_id:
251                         matched.append(s)
252                         continue
253             return matched
254
255     def audit_hardware(self, sensors: Optional[List[HardwareSe
nsor]]=None) -> HardwareAuditReport:
256         """Проведение полного аудита оборудования, драйверов и
привязки датчиков.
257
258         Args:
259             sensors: Опциональный список предварительно собран
ных сенсоров.
260
261         Returns:
262             HardwareAuditReport: Полный отчет по оборудованию.
263         """
264         all_sensors = sensors if sensors is not None else get_

```

```

hardware_sensors()
265     wmi_drivers = self._get_drivers_from_wmi()
266     device_audits: List[HardwareDeviceAudit] = []
267     problem_count = 0
268     outdated_count = 0
269     if self._setupapi:
270         try:
271             raw_devices = self._setupapi.get_all_devices(o
nly_present=True)
272             for dev in raw_devices:
273                 norm_id = dev.device_instance_id.strip().u
pper()
274                 driver_data = wmi_drivers.get(norm_id)
275                 driver_info: Optional[DriverInfo] = None
276                 if driver_data:
277                     date_dt = self._parse_wmi_date(
278                         driver_data.get('driver_date_raw')
,
279                         context=f"'{dev.friendly_name}' [{
norm_id}]"
280                     )
281                     date_str = date_dt.strftime('%Y-%m-%d'
) if date_dt else None
282                     provider = driver_data.get('provider',
'Unknown')
283                     currency_status, age_days = self._eval
uate_driver_currency(driver_date_dt=date_dt, provider=provider
, device_class=dev.device_class or 'Device')
284                     if 'устарел' in currency_status.lower(
):
285                         outdated_count += 1
286                         driver_info = DriverInfo(name=driver_d
ata.get('name') or dev.friendly_name, driver_version=driver_da
ta.get('driver_version', ''), driver_date=date_str, provider=p
rovider, inf_name=driver_data.get('inf_name'), is_signed=drive
r_data.get('is_signed', True), is_inbox='microsoft' in provide
r.lower(), age_days=age_days, currency_status=currency_status)
287                         install_date = self._get_device_install_da

```

```

te_registry(dev.device_instance_id, dev.friendly_name, dev.device_class)
288         if not install_date and driver_info and driver_info.driver_date:
289             install_date = driver_info.driver_date
290             status_str = 'Problem' if dev.has_problem
else 'OK'
291         if dev.has_problem:
292             problem_count += 1
293             audit_item = HardwareDeviceAudit(device_id=dev.device_instance_id, name=dev.friendly_name, device_class=dev.device_class or 'Device', manufacturer=dev.manufacturer or 'Unknown', install_date=install_date, status=status_str, problem_code=dev.problem_code, driver=driver_info, properties={'hardware_id': dev.hardware_id, 'status_code': dev.status_code})
294             audit_item.sensors = self._bind_sensors_to_device(audit_item, all_sensors)
295             device_audits.append(audit_item)
296         except Exception as ex:
297             logger.debug(f'Ошибка при аудите через SetupAPI: {ex}')
298         if not device_audits:
299             dummy_cpu = HardwareDeviceAudit(device_id='ROOT/CP
U/0000', name=platform.processor() or 'System Processor', device_class='Processor', manufacturer='Generic', install_date=datetime.now(timezone.utc).strftime('%d.%m.%Y %H:%M'), status='OK', problem_code=0, driver=DriverInfo(name='Processor Driver', driver_version='10.0.26100.1', driver_date='2024-05-01', provider='Microsoft', is_signed=True, is_inbox=True, age_days=100, currency_status='Актуален'))
300             dummy_cpu.sensors = self._bind_sensors_to_device(dummy_cpu, all_sensors)
301             device_audits.append(dummy_cpu)
302             summary_msg = f'Аудит завершен: обнаружено устройств: {len(device_audits)}, проблемных PnP: {problem_count}, с устаревшими драйверами: {outdated_count}'
303             return HardwareAuditReport(devices_count=len(device_audits), problem_devices_count=problem_count, outdated_drivers_c

```



```
ount=outdated_count, devices=device_audits, summary=summary_ms  
g)
```

304

hardware_history_manager.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Hardware History Manager
5 # =====
6 # Description:
7 # Менеджер истории, архивации и детектора изменений аппаратного обеспечения.
8 #
9 # Usage Examples:
10 # Python API:
11 # from apps.windows.telemetry_research.hardware_history_manager import HardwareHistoryManager
12 # service = HardwareHistoryManager()
13 #
14 # File: hardware_history_manager.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Менеджер истории, архивации и детектора изменений аппаратного обеспечения."""
24
25 import json
26 import os
27 import uuid
28 from datetime import datetime, timezone
29 from pathlib import Path
```

```

30 from typing import Any, Dict, List, Optional
31 from logger import logger
32 from apps.windows.telemetry.models import HardwareArchiveEntry
   , HardwareAuditReport, HardwareChangeItem, HardwareDeviceAudit
33
34
35 class HardwareHistoryManager:
36     """Менеджер архивов и отслеживания аппаратных изменений."""
37     "
38     def __init__(self, archive_dir: Optional[Path] = None, max
   _archives: int = 100, use_file_index: bool = True) -> None:
39         """Инициализация менеджера истории оборудования.
40
41         Args:
42             archive_dir: Каталог для хранения архивных снимков
   (по умолчанию data/telemetry/hardware_archives).
43             max_archives: Максимальное количество хранимых арх
   ивов для ротации.
44             use_file_index: Использовать ли индексный файл арх
   ивов.
45         """
46         if archive_dir is None:
47             base_dir = Path(__file__).resolve().parent.parent.
   parent.parent
48             self.archive_dir = base_dir / 'data' / 'telemetry'
   / 'hardware_archives'
49         else:
50             self.archive_dir = Path(archive_dir)
51             self.max_archives = max(10, max_archives)
52             self.archive_dir.mkdir(parents=True, exist_ok=True)
53             self.index_file = self.archive_dir / 'archives_index.j
   sonl'
54             self._cached_latest_report: Optional[HardwareAuditRepo
   rt] = None
55             self.use_file_index = use_file_index
56
57     def detect_changes(self, current_report: HardwareAuditRepo

```

```

    rt, baseline_report: Optional[HardwareAuditReport]=None) -> Li
    st[HardwareChangeItem]:
58         """Сравнение двух снимков оборудования и выявление все
    х изменений (Diff Engine).
59
60         Args:
61             current_report: Текущий срез аудита оборудования.
62             baseline_report: Базовый срез для сравнения (если
    None, берется последний сохраненный).
63
64         Returns:
65             List[HardwareChangeItem]: Список зафиксированных и
    зменений.
66         """
67         base = baseline_report
68         if base is None:
69             latest_entry = self.get_latest_archive()
70             if latest_entry:
71                 base = latest_entry.report
72         if base is None:
73             return []
74         changes: List[HardwareChangeItem] = []
75         now_ts = datetime.now(timezone.utc).isoformat()
76         base_devs: Dict[str, HardwareDeviceAudit] = {d.device_
    id.upper(): d for d in base.devices}
77         curr_devs: Dict[str, HardwareDeviceAudit] = {d.device_
    id.upper(): d for d in current_report.devices}
78         for dev_id, dev in curr_devs.items():
79             if dev_id not in base_devs:
80                 changes.append(HardwareChangeItem(change_type=
    'added', device_id=dev.device_id, device_name=dev.name, descri
    ption=f'Подключено новое устройство: {dev.name} (Класс: {dev.d
    evice_class}, Производитель: {dev.manufacturer})', previous_va
    lue=None, current_value=dev.model_dump(), timestamp=now_ts))
81         for dev_id, dev in base_devs.items():
82             if dev_id not in curr_devs:
83                 changes.append(HardwareChangeItem(change_type=
    'removed', device_id=dev.device_id, device_name=dev.name, desc

```

```

ription=f'Отключено или удалено устройство: {dev.name} (Класс:
{dev.device_class})', previous_value=dev.model_dump(), current
t_value=None, timestamp=now_ts))
84         for dev_id, dev in curr_devs.items():
85             if dev_id in base_devs:
86                 old_dev = base_devs[dev_id]
87                 old_drv = old_dev.driver
88                 new_drv = dev.driver
89                 if old_drv and new_drv:
90                     if old_drv.driver_version != new_drv.driver_
r_version or old_drv.driver_date != new_drv.driver_date:
91                         desc = f"Обновлен драйвер для '{dev.name}': версия с '{old_drv.driver_version}' (от {old_drv.driver_
date or 'N/A'}) на '{new_drv.driver_version}' (от {new_drv.dri
ver_date or 'N/A'})"
92                         changes.append(HardwareChangeItem(change_type='driver_updated', device_id=dev.device_id, device_name
=dev.name, description=desc, previous_value=old_drv.model_dump
(), current_value=new_drv.model_dump(), timestamp=now_ts))
93                     elif not old_drv and new_drv:
94                         changes.append(HardwareChangeItem(change_t
ype='driver_updated', device_id=dev.device_id, device_name=dev
.name, description=f"Установлен драйвер для '{dev.name}': {new
_drv.driver_version} ({new_drv.provider})", previous_value=Non
e, current_value=new_drv.model_dump(), timestamp=now_ts))
95                     if old_dev.status != dev.status or old_dev.pro
blem_code != dev.problem_code:
96                         desc = f"Изменен статус устройства '{dev.n
ame}': было '{old_dev.status}' (код {old_dev.problem_code}) ->
стало '{dev.status}' (код {dev.problem_code})"
97                         changes.append(HardwareChangeItem(change_t
ype='status_changed', device_id=dev.device_id, device_name=dev
.name, description=desc, previous_value={'status': old_dev.sta
tus, 'problem_code': old_dev.problem_code}, current_value={'st
atus': dev.status, 'problem_code': dev.problem_code}, timestam
p=now_ts))
98         return changes
99

```

```

100     def archive_report(self, report: HardwareAuditReport, auto
|_diff: bool=True) -> HardwareArchiveEntry:
101         """Сохранение снимка аудита оборудования в архив с фик
|сацией изменений.
102
103         Args:
104             report: Отчет аудита оборудования.
105             auto_diff: Выполнять ли автоматическое вычисление
|изменений.
106
107         Returns:
108             HardwareArchiveEntry: Созданная запись архива.
109         """
110         if auto_diff:
111             changes = self.detect_changes(report)
112             report.changes_since_last_archive = changes
113             if changes:
114                 logger.info(f'Зафиксировано {len(changes)} изм
|енений в аппаратной конфигурации Windows!')
115                 for c in changes:
116                     logger.info(f' [Железо] {c.change_type.upp
|er(): {c.description}')
117                 archive_id = f"hw_{datetime.now(timezone.utc).strftime
|('%Y%m%d_%H%M%S')}-{uuid.uuid4().hex[:6]}"
118                 timestamp = datetime.now(timezone.utc).isoformat()
119                 entry = HardwareArchiveEntry(archive_id=archive_id, ti
|mestamp=timestamp, devices_count=report.devices_count, changes
|_count=len(report.changes_since_last_archive), report=report)
120                 archive_file = self.archive_dir / f'{archive_id}.json'
121                 try:
122                     with open(archive_file, 'w', encoding='utf-8') as
|f:
123                         json.dump(entry.model_dump(mode='json'), f, en
|sure_ascii=False, indent=2)
124                 except Exception as ex:
125                     logger.error(f'Ошибка сохранения архива оборудован
|ия в файл: {ex}')
126                 index_entry = {'archive_id': archive_id, 'timestamp':

```

```

timestamp, 'devices_count': report.devices_count, 'problem_dev
ices_count': report.problem_devices_count, 'outdated_drivers_c
ount': report.outdated_drivers_count, 'changes_count': len(rep
ort.changes_since_last_archive), 'file_name': f'{archive_id}.j
son'}
127         if self.use_file_index:
128             try:
129                 with open(self.index_file, 'a', encoding='utf-
130                     8') as f:
131                     f.write(json.dumps(index_entry, ensure_asc
132                         ii=False) + '\n')
133             except Exception as ex:
134                 logger.error(f'Ошибка записи индекса архивов о
135                     борудования: {ex}')
136             else:
137                 logger.warning('Запись в файл-индекс отключена кон
138                     фигурацией.')
139             self._cached_latest_report = report
140             self._rotate_archives()
141             return entry
142
143     def _rotate_archives(self) -> None:
144         """Ротация старых архивных файлов при превышении лимит
145             а."""
146         try:
147             json_files = sorted(self.archive_dir.glob('hw_*.js
148                 on'), key=os.path.getmtime)
149             if len(json_files) > self.max_archives:
150                 to_delete = json_files[:len(json_files) - self
151                     .max_archives]
152                 for p in to_delete:
153                     try:
154                         p.unlink()
155                     except OSError:
156                         pass
157             except Exception as ex:
158                 logger.debug(f'Ошибка при ротации архивов оборудо
159                     вания: {ex}')

```

```

152
153     def get_latest_archive(self) -> Optional[HardwareArchiveEn
try]:
154         """Получение самого свежего архива оборудования из хра
нилица.
155
156         Returns:
157             Optional[HardwareArchiveEntry]: Последний архив ил
и None.
158         """
159         try:
160             json_files = sorted(self.archive_dir.glob('hw_*.js
on'), key=os.path.getmtime, reverse=True)
161             if json_files:
162                 latest_path = json_files[0]
163                 with open(latest_path, 'r', encoding='utf-8')
as f:
164                     data = json.load(f)
165                     return HardwareArchiveEntry.model_validate
(data)
166             except Exception as ex:
167                 logger.error(f'Ошибка чтения последнего архива обо
рудования: {ex}', exc_info=True)
168                 logger.error('Функция get_latest_archive вернула пусто
й результат')
169                 return None
170
171     def get_history(self, limit: int=50) -> List[Dict[str, Any
]]:
172         """Получение сводного списка сохраненных архивов обо
рудования.
173
174         Args:
175             limit: Максимальное количество записей архива.
176
177         Returns:
178             List[Dict[str, Any]]: Список метаданных архивов.
179         """

```



```

180         records: List[Dict[str, Any]] = []
181         if not self.index_file.exists():
182             return records
183         try:
184             with open(self.index_file, 'r', encoding='utf-8')
as f:
185                 for line in f:
186                     line = line.strip()
187                     if line:
188                         try:
189                             records.append(json.loads(line))
190                         except json.JSONDecodeError:
191                             continue
192         except Exception as ex:
193             logger.debug(f'Ошибка чтения индекса архивов: {ex}
')
194         return list(reversed(records))[:limit]
195
196     def get_archive_by_id(self, archive_id: str) -> Optional[HardwareArchiveEntry]:
197         """Получение конкретного полного архивного снимка по ID.
198
199         Args:
200             archive_id: Идентификатор архива.
201
202         Returns:
203             Optional[HardwareArchiveEntry]: Найденный архив или None.
204         """
205         safe_id = Path(archive_id).name
206         target = self.archive_dir / f'{safe_id}.json'
207         if not target.exists():
208             target = self.archive_dir / safe_id
209         if target.exists():
210             try:
211                 with open(target, 'r', encoding='utf-8') as f:
212                     data = json.load(f)

```

```

213         return HardwareArchiveEntry.model_validate
    (data)
214         except Exception as ex:
215             logger.debug(f'Ошибка загрузки архива {archive
_id}: {ex}')
216             logger.error('Функция get_archive_by_id вернула пустой
результат')
217             return None
218
219     def get_change_timeline(self, limit: int=100) -> List[Hard
wareChangeItem]:
220         """Получение сводного таймлайна всех изменений оборудо
вания из архивов.
221
222         Args:
223             limit: Лимит изменений для возврата.
224
225         Returns:
226             List[HardwareChangeItem]: Хронологический список з
афиксированных изменений.
227         """
228         all_changes: List[HardwareChangeItem] = []
229         json_files = sorted(self.archive_dir.glob('hw_*.json')
, key=os.path.getmtime, reverse=True)
230         for p in json_files:
231             try:
232                 with open(p, 'r', encoding='utf-8') as f:
233                     data = json.load(f)
234                     entry = HardwareArchiveEntry.model_validat
e(data)
235                     if entry.report and entry.report.changes_s
ince_last_archive:
236                         all_changes.extend(entry.report.change
s_since_last_archive)
237                     if len(all_changes) >= limit:
238                         break
239             except Exception:
240                 continue

```

```
241 |         return all_changes[:limit]
242 |
```

incident_detector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Incident Detector
5 # =====
6 # Description:
7 #   Модуль непрерывного анализа аномалий и автоматического выяв
8 #   ления сбоев системы.
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry_research.incident_detector i
13 #       mport IncidentDetector
14 #
15 #       service = IncidentDetector()
16 #
17 # File: incident_detector.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry_research
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-01 13:28:28
23 # =====
24
25 from __future__ import annotations
26 """Модуль непрерывного анализа аномалий и автоматического выяв
27 ления сбоев системы."""
28
29 import time
30 import uuid
31 from datetime import datetime, timezone
32 from typing import Any, Dict, List, Optional, Tuple
```

```

30 try:
31     from logger import logger
32 except ImportError:
33     from logger import logger
34
35 from apps.windows.telemetry.models import TelemetryIncident
36 from apps.windows.telemetry.ring_buffer import TelemetryRingBu
37 ffer
38
39 class IncidentDetector:
40     """Детектор аномалий в потоках метрик и событий с фиксац
41     ие инцидентов."""
42
43     def __init__(
44         self,
45         cpu_threshold: float = 90.0,
46         disk_write_burst_mb: float = 100.0,
47         net_burst_mb: float = 50.0,
48         ring_buffer: Optional[TelemetryRingBuffer] = None,
49     ) -> None:
50         """Инициализирует детектор пороговыми значениями и ссы
51         лкой на кольцевой буфер.
52
53         Args:
54             cpu_threshold: Порог срабатывания по CPU % (по умо
55             лчанию 90.0).
56             disk_write_burst_mb: Порог дисковой записи в МБ/с
57             (по умолчанию 100.0).
58             net_burst_mb: Порог сетевого трафика в МБ/с (по ум
59             олчанию 50.0).
60             ring_buffer: Ссылка на экземпляра кольцевого буфера
61             памяти.
62         """
63         self.cpu_threshold = cpu_threshold
64         self.disk_write_burst_mb = disk_write_burst_mb
65         self.net_burst_mb = net_burst_mb
66         self.ring_buffer = ring_buffer or TelemetryRingBuffer(

```

```

)
61         self._last_incident_epoch: float = 0.0
62         self._cooldown_seconds: float = 60.0 # Не спамить оди
наковыми инцидентами чаще раза в минуту
63
64     def check_anomaly(
65         self,
66         system_snapshot: Dict[str, Any],
67         processes: Optional[List[Dict[str, Any]]] = None,
68         recent_events: Optional[List[Dict[str, Any]]] = None,
69     ) -> Optional[TelemetryIncident]:
70         """Анализирует срез метрик и возвращает TelemetryIncident при обнаружении аномалии.
71
72         Args:
73             system_snapshot: Текущий снимок системы (SystemSnapshot dict).
74             processes: Список процессов текущего среза.
75             recent_events: Последние дискретные события (W64, ETW, Defender).
76
77         Returns:
78             Optional[TelemetryIncident]: Объект инцидента или None при штатной работе.
79         """
80         now = time.time()
81         cpu_pct = float(system_snapshot.get("cpu_total_percent") or system_snapshot.get("cpu_percent") or 0.0)
82         disk_w_mbs = float(system_snapshot.get("disk_write_bytes_sec") or 0.0) / (1024.0 * 1024.0)
83         net_tx_mbs = float(system_snapshot.get("network_sent_bytes_sec") or 0.0) / (1024.0 * 1024.0)
84         net_rx_mbs = float(system_snapshot.get("network_recv_bytes_sec") or 0.0) / (1024.0 * 1024.0)
85
86         trigger_type: Optional[str] = None
87         severity = "warning"
88         title = ""

```

```

89         description = ""
90         suspects: List[Dict[str, Any]] = []
91
92         # 1. Проверка CPU Spike
93         if cpu_pct >= self.cpu_threshold:
94             trigger_type = "cpu_spike"
95             severity = "critical" if cpu_pct >= 95.0 else "war
ning"
96             title = f"Критическая нагрузка на процессор: {cpu_
pct:.1f}%"
97             description = f"Общая загрузка CPU превысила порог
{self.cpu_threshold}%, достигнув {cpu_pct:.1f}%."
98
99         # 2. Проверка Disk Write Burst
100        elif disk_w_mbs >= self.disk_write_burst_mb:
101            trigger_type = "disk_burst"
102            severity = "warning"
103            title = f"Аномальная скорость записи на диск: {dis
k_w_mbs:.1f} МБ/с"
104            description = f"Скорость дисковой записи превысила
порог {self.disk_write_burst_mb} МБ/с."
105
106        # 3. Проверка Network Burst
107        elif (net_tx_mbs + net_rx_mbs) >= self.net_burst_mb:
108            trigger_type = "network_burst"
109            severity = "warning"
110            title = f"Всплеск сетевой активности: {(net_tx_mbs
+ net_rx_mbs):.1f} МБ/с"
111            description = f"Суммарный сетевой трафик достиг {(
net_tx_mbs + net_rx_mbs):.1f} МБ/с (TX: {net_tx_mbs:.1f}, RX:
{net_rx_mbs:.1f})."
112
113        # 4. Проверка подозрительных процессов (AppData, Temp,
Public)
114        if processes:
115            for p in processes:
116                exe_path = str(p.get("path") or p.get("exe") o
r "").lower()

```

```

117 |         p_cpu = float(p.get("cpu_percent") or 0.0)
118 |         is_suspicious_location = any(
119 |             loc in exe_path for loc in ("\\temp\\", "\\
| \\appdata\\local\\temp", "\\users\\public\\")
120 |         )
121 |         if is_suspicious_location and (p_cpu > 15.0 or
| trigger_type is not None):
122 |             suspects.append({
123 |                 "pid": p.get("pid"),
124 |                 "name": p.get("name"),
125 |                 "path": exe_path,
126 |                 "cpu_percent": p_cpu,
127 |                 "memory_mb": p.get("memory_mb"),
128 |                 "reason": "Запуск из временного катало
| га с повышенной активностью",
129 |             })
130 |             if trigger_type is None and p_cpu > 30.0:
131 |                 trigger_type = "suspicious_process"
132 |                 severity = "critical"
133 |                 title = f"Подозрительный процесс из Te
| mp: {p.get('name')} (PID {p.get('pid')})"
134 |                 description = f"Процесс {p.get('name')}
| } выполняется из {exe_path} и потребляет {p_cpu:.1f}% CPU."
135 |
136 |             elif p_cpu >= 50.0:
137 |                 suspects.append({
138 |                     "pid": p.get("pid"),
139 |                     "name": p.get("name"),
140 |                     "path": exe_path,
141 |                     "cpu_percent": p_cpu,
142 |                     "memory_mb": p.get("memory_mb"),
143 |                     "reason": f"Высокое индивидуальное пот
| ребление CPU: {p_cpu:.1f}%",
144 |                 })
145 |
146 |     # 5. Проверка событий безопасности (Defender / ETW)
147 |     matched_events: List[Dict[str, Any]] = []
148 |     if recent_events:

```



```

149         for ev in recent_events:
150             ev_type = str(ev.get("event_type") or "").lower()
151             ev_str = str(ev).lower()
152             if "defender" in ev_type or "threat" in ev_str
or "malware" in ev_str:
153                 matched_events.append(ev)
154                 if trigger_type is None:
155                     trigger_type = "defender_alert"
156                     severity = "critical"
157                     title = "Срабатывание Windows Defender
/ Защиты от угроз"
158                     description = f"Зафиксировано событие
безопасности: {ev.get('name') or ev_type}"
159
160             if not trigger_type:
161                 return None
162
163             # Проверка кулдауна, чтобы не генерировать дубликаты е
жесекундно
164             if (now - self._last_incident_epoch) < self._cooldown_
seconds:
165                 return None
166
167             self._last_incident_epoch = now
168             inc_id = f"INC-{{datetime.now(timezone.utc).strftime('%
Y%m%d%H%M%S')}}-{{uuid.uuid4().hex[:4].upper()}}"
169
170             # Захватываем срез из кольцевого буфера (например, 5 м
инут до события)
171             raw_window = self.ring_buffer.get_window(center_timest
amp=now, before_seconds=300.0, after_seconds=0.0)
172
173             incident = TelemetryIncident(
174                 incident_id=inc_id,
175                 timestamp=datetime.now(timezone.utc).isoformat(),
176                 trigger_type=trigger_type,
177                 severity=severity,

```

```

178         title=title,
179         description=description,
180         trigger_metrics={
181             "cpu_percent": cpu_pct,
182             "disk_write_mbs": disk_w_mbs,
183             "network_tx_mbs": net_tx_mbs,
184             "network_rx_mbs": net_rx_mbs,
185         },
186         suspect_processes=suspects,
187         related_events=matched_events or (recent_events[:1
188 0] if recent_events else []),
189         metrics_summary={
190             "buffer_samples_captured": len(raw_window),
191             "peak_cpu": cpu_pct,
192             "peak_disk_write_mbs": disk_w_mbs,
193         },
194         raw_window=raw_window,
195     )
196     logger.warning(f"Зафиксирован инцидент телеметрии [{incident.incident_id}]: {incident.title}")
197     return incident
198
199     @staticmethod
200     def format_incident_markdown_for_llm(incident: TelemetryIncident) -> str:
201         """Форматирует инцидент в компактный структурированный
202         Markdown для AI / LLM анализа.
203
204         Args:
205             incident: Объект TelemetryIncident.
206
207         Returns:
208             str: Готовый markdown блок для контекста языковой
209             модели.
210         """
211         lines = [
212             f"### □ ДИАГНОСТИЧЕСКИЙ ИНЦИДЕНТ: [{incident.incident_id}]

```

```

dent_id}}]",
211         f"- **Тип триггера:** `{incident.trigger_type}` |
    **Критичность:** `{incident.severity.upper()}`",
212         f"- **Время:** {incident.timestamp}",
213         f"- **Сводка:** {incident.title}",
214         f"- **Описание:** {incident.description}",
215         "",
216         "#### □ Метрики в момент инцидента:",
217         f"- CPU: `{incident.trigger_metrics.get('cpu_perce
nt', 0.0)}%`",
218         f"- Диск (Запись): `{incident.trigger_metrics.get(
'disk_write_mbs', 0.0)} МБ/с`",
219         f"- Сеть (TX / RX): `{incident.trigger_metrics.get
('network_tx_mbs', 0.0)} МБ/с` / `{incident.trigger_metrics.ge
t('network_rx_mbs', 0.0)} МБ/с`",
220         f"- Сохранено секундных замеров в буфере: `{incide
nt.metrics_summary.get('buffer_samples_captured', 0)}`",
221         "",
222     ]
223
224     if incident.suspect_processes:
225         lines.append("#### □ Подозрительные процессы:")
226         lines.append("| PID | Имя | Путь | CPU % | Причина
|")
227         lines.append("|---|---|---|---|---|")
228         for sp in incident.suspect_processes:
229             lines.append(
230                 f"| {sp.get('pid', '-')} | `{sp.get('name'
, '-')}` | `{sp.get('path', '-')}` | {sp.get('cpu_percent', 0.
0)}% | {sp.get('reason', '-')}"
231             )
232             lines.append("")
233
234     if incident.related_events:
235         lines.append("#### □ Сопутствующие события систем
ы (Events):")
236         for ev in incident.related_events[:5]:
237             ev_name = ev.get("name") or ev.get("event_type")

```

```
) or "event"
238         ev_time = ev.get("timestamp", "")
239         lines.append(f"- `{ev_time}`: **{ev_name}** ({
ev.get('path') or ev.get('details') or ''})")
240         lines.append("")
241
242         lines.append("Инструкция: Оцените потенциальную угрозу
(вирус/майнер/зависание/утечка) и предложите конкретные меры
противодействия.")
243         return "\n".join(lines)
244
```

main.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Main
4 # =====
5 # Description:
6 #   Главная точка входа для запуска Web GUI и FastAPI сервиса
  исследования телеметрии.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry_research.main
11 #   Python API:
12 #       from apps.windows.telemetry_research.main import parse_a
  rgs
13 #
14 #       res = parse_args()
15 #
16 # File: main.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry_research
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-01 13:28:28
22 # =====
23
24 from __future__ import annotations
25 """Главная точка входа для запуска Web GUI и FastAPI сервиса и
  сследования телеметрии."""
26
27 import argparse
28 import sys
29 from pathlib import Path
```

```
30 |
31 | _project_root = str(Path(__file__).resolve().parents[3])
32 | if _project_root not in sys.path:
33 |     sys.path.insert(0, _project_root)
34 |
35 | try:
36 |     from logger import logger
37 | except ImportError:
38 |     from logger import logger
39 |
40 |
41 | def parse_args() -> argparse.Namespace:
42 |     """Парсинг аргументов запуска веб-сервера."""
43 |     parser = argparse.ArgumentParser(
44 |         description="Запуск Web GUI и FastAPI сервера исследов
45 | ания телеметрии",
46 |         formatter_class=argparse.RawDescriptionHelpFormatter,
47 |     )
48 |     parser.add_argument("--host", type=str, default="127.0.0.1",
49 | help="Хост для биндинга (по умолчанию: 127.0.0.1)")
50 |     parser.add_argument("--port", type=int, default=8090, help=
51 | "Порт для веб-сервера (по умолчанию: 8090)")
52 |     parser.add_argument("--reload", action="store_true", help=
53 | "Включить автоперезагрузку при изменении кода")
54 |     return parser.parse_args()
55 |
56 |
57 | def main() -> int:
58 |     """Запуск FastAPI сервера через Uvicorn."""
59 |     args = parse_args()
60 |     try:
61 |         import uvicorn
62 |     except ImportError:
63 |         logger.error("Для запуска Web GUI необходим пакет 'uvi
64 | corn'. Установите его: pip install uvicorn")
65 |         print("❌ Ошибка: пакет 'uvicorn' не установлен. Устано
66 | вите: pip install uvicorn")
67 |     return 1
```

```
62 |
63 |     logger.info("=" * 65)
64 |     logger.info(f"    Запуск Web GUI исследования телеметрии на
    | http://{args.host}:{args.port}")
65 |     logger.info("=" * 65)
66 |     print("\n" + "=" * 65)
67 |     print(f"    Web GUI исследования телеметрии доступен по адр
    | есу:")
68 |     print(f"    http://{args.host}:{args.port}")
69 |     print("=" * 65 + "\n")
70 |
71 |     uvicorn.run(
72 |         "apps.windows.telemetry_research.server:app",
73 |         host=args.host,
74 |         port=args.port,
75 |         reload=args.reload,
76 |         log_level="info",
77 |     )
78 |     return 0
79 |
80 |
81 | if __name__ == "__main__":
82 |     sys.exit(main())
83 |
```

models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Models
4 # =====
5 # Description:
6 #   Модели данных для исследования телеметрии и построения гра
  фиков.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.models import Metri
    cPoint
11 #
12 #       service = MetricPoint()
13 #
14 # File: models.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:47:00
20 # =====
21
22 from __future__ import annotations
23 """Модели данных для исследования телеметрии, ресурсов клиентс
    ких программ и построения графиков."""
24
25 from datetime import datetime
26 from typing import Any, Dict, List, Optional
27 from pydantic import BaseModel, Field
28
29
```



```

30 class MetricPoint(BaseModel):
31     """Единичная точка временного ряда для метрики."""
32     timestamp: str = Field(..., description="Временная метка т
очки (ISO-8601 или formatted)")
33     value: float = Field(..., description="Числовое значение м
етрики")
34     label: Optional[str] = Field(default=None, description="Оп
циональная подпись или тег")
35
36
37 class TimeSeriesDataset(BaseModel):
38     """Набор данных временного ряда для графика."""
39     name: str = Field(..., description="Имя метрики или серии
данных")
40     unit: str = Field(default="%", description="Единица измере
ния (% , MB , °C , MB/s , ops/s)")
41     points: List[MetricPoint] = Field(default_factory=list, de
scription="Список точек временного ряда")
42     color: Optional[str] = Field(default=None, description="HE
X-код цвета серии для визуализации")
43
44
45 class AnomalyEvent(BaseModel):
46     """Зафиксированная аномалия или всплеск нагрузки в логах."
""
47     timestamp: str = Field(..., description="Время фиксации а
номалии")
48     metric: str = Field(..., description="Название метрики с а
номалией")
49     value: float = Field(..., description="Фактическое значени
е в момент всплеска")
50     threshold: float = Field(..., description="Пороговое или н
ормальное значение")
51     severity: str = Field(default="warning", description="Уров
ень критичности (info, warning, critical)")
52     description: str = Field(..., description="Понятное описан
ие выявленной проблемы")
53

```

```
54
55 class MetricStats(BaseModel):
56     """Сводная описательная статистика по метрике (EDA)."""
57     count: int = Field(default=0, description="Количество заме
ров")
58     min_val: float = Field(default=0.0, description="Минимальн
ое значение")
59     max_val: float = Field(default=0.0, description="Максималь
ное значение")
60     avg_val: float = Field(default=0.0, description="Среднее а
рифметическое значение")
61     median_val: float = Field(default=0.0, description="Медиан
ное значение")
62     p95_val: float = Field(default=0.0, description="95-й перц
ентиль")
63     std_dev: float = Field(default=0.0, description="Стандартн
ое отклонение")
64     unit: str = Field(default="%", description="Единица измере
ния")
65
66
67 class DeviceEventSummary(BaseModel):
68     """Сводная информация по событиям устройств (флаппинг, отк
лючения, сбои)."""
69     total_events: int = Field(default=0, description="Всего со
бытий устройств")
70     error_count: int = Field(default=0, description="Количеств
о событий со сбоями")
71     by_category: Dict[str, int] = Field(default_factory=dict,
description="Распределение по категориям устройств")
72     by_event_type: Dict[str, int] = Field(default_factory=dict
, description="Распределение по типам событий")
73     flapping_devices: List[str] = Field(default_factory=list,
description="Список устройств с частыми переподключениями")
74
75
76 class ChartConfig(BaseModel):
77     """Конфигурация графика для фронтенда / Chart.js."""
```

```

78     id: str = Field(..., description="Уникальный идентификатор
    графика")
79     title: str = Field(..., description="Заголовок графика")
80     chart_type: str = Field(default="line", description="Тип г
    рафика (line, bar, doughnut, radar, heatmap)")
81     labels: List[str] = Field(default_factory=list, descriptio
    n="Подписи оси X (временные метки)")
82     datasets: List[Dict[str, Any]] = Field(default_factory=lis
    t, description="Массив датасетов с данными и стилями")
83     y_axis_label: str = Field(default="", description="Подпись
    оси Y")
84     description: Optional[str] = Field(default=None, descripti
    on="Описание или аналитические выводы по графику")
85
86
87 class CorrelationMatrixItem(BaseModel):
88     """Элемент матрицы корреляции между парой метрик."""
89     metric_a: str = Field(description="Идентификатор первой ме
    трики")
90     metric_b: str = Field(description="Идентификатор второй ме
    трики")
91     coefficient: float = Field(description="Коэффициент коррел
    яции Пирсона (-1.0 ... 1.0)")
92     sample_size: int = Field(description="Число общих точек вр
    еменного ряда")
93     interpretation: str = Field(description="Текстовая интерпр
    етация взаимосвязи")
94
95
96 class HypothesisResult(BaseModel):
97     """Результат проверки аналитической гипотезы о поведении с
    истемы."""
98     hypothesis_id: str = Field(description="Уникальный идентиф
    икатор гипотезы")
99     title: str = Field(description="Краткое название гипотезы"
    )
100     description: str = Field(description="Подробная формулиров
    ка гипотезы")

```

```

101     confirmed: bool = Field(description="Подтверждена ли гипот
еза на основе собранных данных")
102     confidence: float = Field(description="Уровень уверенности
(0.0 - 1.0)")
103     evidence: List[str] = Field(default_factory=list, descript
ion="Список подтверждающих или опровергающих фактов")
104     recommendation: Optional[str] = Field(default=None, descri
ption="Практическая рекомендация по устранению проблемы")
105
106
107 class ResearchScenarioRequest(BaseModel):
108     """Параметры запроса для проведения исследовательского сце
нария."""
109     source_path: Optional[str] = Field(default=None, descripti
on="Путь к файлу логов, директории или шаблону поиска")
110     records: Optional[List[Dict[str, Any]]] = Field(default=No
ne, description="Массив сырых записей телеметрии, переданный н
апрямую")
111     start_time: Optional[str] = Field(default=None, descriptio
n="Начало временного интервала фильтрации (ISO 8601)")
112     end_time: Optional[str] = Field(default=None, description=
"Конец временного интервала фильтрации (ISO 8601)")
113     subsystems: Optional[List[str]] = Field(default=None, desc
ription="Список исследуемых подсистем (cpu, ram, gpu, disk, ne
twork, devices, sensors)")
114     cpu_anomaly_threshold: float = Field(default=90.0, descrip
tion="Порог загрузки CPU (%) для детекции аномалий")
115     temp_anomaly_threshold: float = Field(default=85.0, descri
ption="Порог температуры (°C) для детекции перегрева")
116     enable_hypotheses_check: bool = Field(default=True, descri
ption="Флаг активации автоматической проверки системных гипоте
з")
117
118
119 class TelemetryResearchReport(BaseModel):
120     """Итоговый отчет исследования логов телеметрии."""
121     report_id: str = Field(..., description="Идентификатор отч
ета исследования")

```

```
122     generated_at: str = Field(..., description="Время формирова  
ания отчета")  
123     records_analyzed: int = Field(default=0, description="Коли  
чество проанализированных записей логов")  
124     time_window_start: Optional[str] = Field(default=None, des  
cription="Начало исследуемого временного окна")  
125     time_window_end: Optional[str] = Field(default=None, descr  
iption="Окончание исследуемого временного окна")  
126     statistics: Dict[str, MetricStats] = Field(default_factory  
=dict, description="Статистические профили метрик")  
127     anomalies: List[AnomalyEvent] = Field(default_factory=list  
, description="Список выявленных аномалий")  
128     device_summary: DeviceEventSummary = Field(default_factory  
=DeviceEventSummary, description="Сводка событий устройств")  
129     charts: List[ChartConfig] = Field(default_factory=list, de  
scription="Список сгенерированных графиков")  
130     health_score: float = Field(default=100.0, description="Ит  
оговый индекс здоровья системы (0-100)")  
131     summary_conclusions: List[str] = Field(default_factory=lis  
t, description="Ключевые аналитические выводы исследования")  
132  
133  
134 class DeepResearchReport(BaseModel):  
135     """Расширенный отчет глубокого исследования телеметрии."""  
136     report_id: str = Field(..., description="Уникальный иденти  
фикатор отчета исследования")  
137     generated_at: str = Field(..., description="Время формирова  
ния отчета (ISO 8601)")  
138     base_report: TelemetryResearchReport = Field(..., descript  
ion="Базовый отчет статистического анализа телеметрии")  
139     correlations: List[CorrelationMatrixItem] = Field(default_  
factory=list, description="Матрица корреляций между ключевыми  
системными метриками")  
140     hypotheses: List[HypothesisResult] = Field(default_factory  
=list, description="Результаты автоматической проверки гипотез  
производительности")  
141     investigation_summary: str = Field(..., description="Сводн  
ое экспертное заключение исследования")
```

```
142     actionable_recommendations: List[str] = Field(default_factory=list, description="Список практических рекомендаций по оптимизации и обслуживанию системы")
143
144
145 class UnknownProgramEvaluation(BaseModel):
146     """Оценка неизвестной или ресурсоемкой программы языковой моделью Gemini."""
147     program_name: str = Field(..., description="Имя исполняемого файла или процесса")
148     pid: Optional[int] = Field(default=None, description="Идентификатор процесса (PID)")
149     executable_path: Optional[str] = Field(default=None, description="Полный путь к исполняемому файлу")
150     command_line: Optional[str] = Field(default=None, description="Командная строка запуска")
151     cpu_percent: float = Field(default=0.0, description="Потребление ЦП (%)")
152     memory_mb: float = Field(default=0.0, description="Использование оперативной памяти (МБ)")
153     disk_io_mb_s: float = Field(default=0.0, description="Интенсивность дискового ввода-вывода (МБ/с)")
154     disk_size_mb: Optional[float] = Field(default=None, description="Размер файла на диске (МБ)")
155     is_normal: bool = Field(default=True, description="Является ли такое поведение штатным и нормальным")
156     confidence: float = Field(default=0.85, description="Уверенность модели в оценке (0.0 - 1.0)")
157     risk_level: str = Field(default="low", description="Уровень риска: low, medium, high, critical")
158     verdict: str = Field(..., description="Краткий вердикт модели (например: 'Штатная нагрузка', 'Подозрительный процесс')")
159     analysis: str = Field(..., description="Развернутое экспертное заключение модели о потреблении ресурсов")
160     recommendations: List[str] = Field(default_factory=list, description="Список практических рекомендаций для пользователя")
161     evaluated_at: str = Field(default_factory=lambda: datetime
```

```

    .now().isoformat(), description="Временная метка проведения оц
енки")
162     model_used: str = Field(default="gemini-3.1-flash", descri
ption="Использованная модель ИИ")
163
164
165 class ClientProgramResource(BaseModel):
166     """Снимок ресурсоемкости отдельной клиентской/пользователь
ской программы."""
167     pid: int = Field(..., description="PID процесса")
168     name: str = Field(..., description="Имя процесса (например
, chrome.exe, code.exe)")
169     display_name: Optional[str] = Field(default=None, descript
ion="Понятное отображаемое имя программы")
170     executable_path: Optional[str] = Field(default=None, descr
iption="Путь к исполняемому файлу")
171     command_line: Optional[str] = Field(default=None, descript
ion="Командная строка запуска")
172     username: Optional[str] = Field(default=None, description=
"Учетная запись пользователя, запустившего программу")
173     cpu_percent: float = Field(default=0.0, description="Текущ
ая / средняя загрузка ЦП (%)")
174     memory_mb: float = Field(default=0.0, description="Использ
ование оперативной памяти RAM (МБ)")
175     memory_percent: float = Field(default=0.0, description="До
ля от общей оперативной памяти (%)")
176     disk_read_bytes_sec: float = Field(default=0.0, descriptio
n="Скорость чтения с диска (байт/с)")
177     disk_write_bytes_sec: float = Field(default=0.0, descripti
on="Скорость записи на диск (байт/с)")
178     disk_size_mb: Optional[float] = Field(default=None, descri
ption="Размер исполняемого файла на диске (МБ)")
179     num_threads: int = Field(default=0, description="Количеств
о потоков процесса")
180     num_handles: int = Field(default=0, description="Количеств
о дескрипторов процесса")
181     is_known_software: bool = Field(default=True, description=
"Флаг: известное доверенное приложение")

```

```
182     is_client_program: bool = Field(default=True, description=
    "Флаг: программа запущена клиентом/пользователем")
183     is_resource_heavy: bool = Field(default=False, description
    ="Флаг: повышенное потребление ресурсов (CPU/RAM/Disk)")
184     resource_score: float = Field(default=0.0, description="Ин
    тегральный скор ресурсоемкости")
185     category: str = Field(default="Пользовательское приложение
    ", description="Категория ПО")
186     publisher: Optional[str] = Field(default=None, description
    ="Издатель / разработчик программы")
187     gemini_evaluation: Optional[UnknownProgramEvaluation] = Fi
    eld(default=None, description="Оценка от Gemini при аномалиях"
    )
188
189
190 class ClientResourceSummary(BaseModel):
191     """Сводный отчет по ресурсам программ, запущенных клиентом
    ."""
192     total_client_programs: int = Field(default=0, description=
    "Общее число клиентских процессов")
193     total_ram_mb: float = Field(default=0.0, description="Сумм
    арно занятая память RAM клиентскими программами (МБ)")
194     total_cpu_percent: float = Field(default=0.0, description=
    "Суммарная загрузка ЦП клиентскими программами (%)")
195     total_disk_size_mb: float = Field(default=0.0, description
    ="Суммарный объем файлов на диске (МБ)")
196     heavy_programs_count: int = Field(default=0, description="
    Число ресурсоемких программ")
197     unknown_heavy_programs_count: int = Field(default=0, descr
    iption="Число неизвестных ресурсоемких программ")
198     programs: List[ClientProgramResource] = Field(default_fact
    ory=list, description="Список программ с детальными метриками"
    )
199     evaluations: List[UnknownProgramEvaluation] = Field(defaul
    t_factory=list, description="Список полученных оценок от Gemin
    i")
200     scanned_at: str = Field(default_factory=lambda: datetime.n
    ow().isoformat(), description="Время проведения аудита")
```


201 |

202 |

query_engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Query Engine
5 # Description:
6 #   Движок SQL-запросов и аналитических расчетов по базе данны
7 #   x telemetry.db.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.query_engine import
11 #         TelemetryQueryEngine
12 #       service = TelemetryQueryEngine()
13 #
14 # File: query_engine.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:47:00
20 # =====
21
22 from __future__ import annotations
23 """Движок SQL-запросов и аналитических расчетов по базе данных
24   telemetry.db."""
25
26 import sqlite3
27 from datetime import datetime, timezone
28 from pathlib import Path
29 from typing import Any, Dict, List, Optional, Union
```

```

30 try:
31     from logger import logger
32 except ImportError:
33     import logging
34     logger = logging.getLogger(__name__)
35
36 from .extractor import TelemetryDataExtractor
37
38
39 class TelemetryQueryEngine:
40     """Аналитический движок для выборки данных и выполнения SQL-запросов к telemetry.db."""
41
42     def __init__(self, extractor: Optional[TelemetryDataExtractor] = None) -> None:
43         """Инициализация движка запросов.
44
45         Args:
46             extractor: Экземпляр извлекателя данных телеметрии
47             .
48             """
49         self.extractor = extractor or TelemetryDataExtractor()
50
51     def get_db_path(self, custom_path: Optional[Union[str, Path]] = None) -> Optional[Path]:
52         """Получить актуальный путь к файлу SQLite базы данных
53         .
54
55         Args:
56             custom_path: Пользовательский путь к БД.
57
58         Returns:
59             Optional[Path]: Валидный путь к файлу базы данных.
60             """
61         if custom_path:
62             p = Path(custom_path)
63             if p.exists() and p.is_file():
64                 return p

```

```

63 |         return self.extractor.get_default_db_path()
64 |
65 |     def execute_safe_sql(
66 |         self, sql_query: str, db_path: Optional[Union[str, Path]] = None, limit: int = 500
67 |     ) -> Dict[str, Any]:
68 |         """Безопасное выполнение произвольного SELECT SQL-запроса к базе телеметрии.
69 |
70 |         Args:
71 |             sql_query: Строка SQL-запроса (только SELECT).
72 |             db_path: Опциональный путь к базе данных.
73 |             limit: Максимальное число возвращаемых строк.
74 |
75 |         Returns:
76 |             Dict[str, Any]: Результат запроса со столбцами, строками и метаданными.
77 |         """
78 |         target_db = self.get_db_path(db_path)
79 |         if not target_db:
80 |             return {
81 |                 "status": "error",
82 |                 "message": "База данных telemetry.db не найдена",
83 |                 "columns": [],
84 |                 "rows": [],
85 |                 "count": 0,
86 |             }
87 |
88 |         cleaned_sql = sql_query.strip().rstrip(";")
89 |         lower_sql = cleaned_sql.lower()
90 |
91 |         # Разрешаем только SELECT запросы
92 |         if not lower_sql.startswith("select") and not lower_sql.startswith("with"):
93 |             return {
94 |                 "status": "error",
95 |                 "message": "Разрешены только запросы на чтение

```

```

    данных (SELECT)",
96         "columns": [],
97         "rows": [],
98         "count": 0,
99     }
100
101     # Ограничиваем количество записей
102     if "limit" not in lower_sql:
103         cleaned_sql = f"{cleaned_sql} LIMIT {min(limit, 10
104         00)}"
105
106     try:
107         conn = sqlite3.connect(str(target_db), timeout=5.0
108         )
109         conn.row_factory = sqlite3.Row
110         cursor = conn.cursor()
111         cursor.execute(cleaned_sql)
112
113         rows = cursor.fetchall()
114         columns = [col[0] for col in cursor.description] i
115         f cursor.description else []
116         data = [dict(r) for r in rows]
117         conn.close()
118
119         return {
120             "status": "ok",
121             "sql": cleaned_sql,
122             "columns": columns,
123             "rows": data,
124             "count": len(data),
125         }
126     except Exception as ex:
127         logger.warning(f"Ошибка выполнения SQL-запроса '{c
128         leaned_sql}': {ex}")
129         return {
130             "status": "error",
131             "sql": cleaned_sql,
132             "message": str(ex),

```

```

129         "columns": [],
130         "rows": [],
131         "count": 0,
132     }
133
134     def get_cpu_timeline(
135         self, limit: int = 120, db_path: Optional[Union[str, Path]] = None
136     ) -> Dict[str, Any]:
137         """Получить временной ряд загрузки процессора и частоты для построения графика.
138
139         Args:
140             limit: Количество точек замера (по умолчанию 120).
141             db_path: Путь к базе данных.
142
143         Returns:
144             Dict[str, Any]: Структурированные временные ряды, метки времени и статистика.
145         """
146         target_db = self.get_db_path(db_path)
147         if not target_db:
148             return {"status": "error", "message": "telemetry.db не найдена"}
149
150         sql = """
151             SELECT id, timestamp, cpu_total_percent, cpu_frequency_mhz, uptime_seconds
152             FROM system_snapshots
153             ORDER BY id DESC
154             LIMIT ?
155         """
156         try:
157             conn = sqlite3.connect(str(target_db), timeout=5.0)
158
159             conn.row_factory = sqlite3.Row
160             cursor = conn.cursor()
161             cursor.execute(sql, (max(10, min(limit, 2000)),))

```

```

161         raw_rows = [dict(r) for r in cursor.fetchall()]
162         conn.close()
163
164         if not raw_rows:
165             return {
166                 "status": "empty",
167                 "message": "В таблице system_snapshots нет
записей",
168                 "labels": [],
169                 "cpu_load": [],
170                 "cpu_freq_ghz": [],
171                 "sql": sql.strip(),
172             }
173
174         # Хронологический порядок: от старых к новым
175         chronological = list(reversed(raw_rows))
176         labels: List[str] = []
177         cpu_load: List[float] = []
178         cpu_freq_ghz: List[float] = []
179
180         for row in chronological:
181             ts_str = str(row.get("timestamp") or "")
182             time_label = ts_str[11:19] if len(ts_str) >= 1
9 else ts_str
183             labels.append(time_label)
184
185             load = round(float(row.get("cpu_total_percent"
) or 0.0), 1)
186             cpu_load.append(load)
187
188             mhz = float(row.get("cpu_frequency_mhz") or 0.
0)
189             freq_ghz = round(mhz / 1000.0, 2) if mhz > 0 e
lse 0.0
190             cpu_freq_ghz.append(freq_ghz)
191
192             avg_cpu = round(sum(cpu_load) / len(cpu_load), 1)
if cpu_load else 0.0

```

```

193         max_cpu = max(cpu_load) if cpu_load else 0.0
194         min_cpu = min(cpu_load) if cpu_load else 0.0
195
196         return {
197             "status": "ok",
198             "sql": sql.strip(),
199             "labels": labels,
200             "cpu_load": cpu_load,
201             "cpu_freq_ghz": cpu_freq_ghz,
202             "chart": {
203                 "type": "line",
204                 "title": f"График загрузки и частоты ЦП по
времени (Ср: {avg_cpu}%, Макс: {max_cpu}%)",
205                 "labels": labels,
206                 "datasets": [
207                     {
208                         "label": "Загрузка ЦП (%)",
209                         "data": cpu_load,
210                         "borderColor": "#38bdf8",
211                         "backgroundColor": "rgba(56, 189,
248, 0.15)",
212                         "borderWidth": 2,
213                         "fill": True,
214                         "tension": 0.3,
215                         "yAxisID": "y",
216                     },
217                     {
218                         "label": "Частота ЦП (GHz)",
219                         "data": cpu_freq_ghz,
220                         "borderColor": "#f59e0b",
221                         "backgroundColor": "transparent",
222                         "borderWidth": 2,
223                         "borderDash": [4, 4],
224                         "tension": 0.2,
225                         "yAxisID": "y1",
226                     },
227                 ],
228             },

```



```

229         "statistics": {
230             "avg_cpu_percent": avg_cpu,
231             "max_cpu_percent": max_cpu,
232             "min_cpu_percent": min_cpu,
233             "points_count": len(labels),
234             "start_time": chronological[0].get("timest
amp"),
235             "end_time": chronological[-1].get("timesta
mp"),
236         },
237     }
238     except Exception as ex:
239         logger.error(f"Ошибка извлечения таймлайна CPU: {e
x}")
240         return {"status": "error", "message": str(ex), "sq
l": sql.strip()}
241
242     def calculate_power_consumption_24h(
243         self, db_path: Optional[Union[str, Path]] = None, hour
s: int = 24
244     ) -> Dict[str, Any]:
245         """Рассчитать общее потребление электроэнергии и подго
товить суточную круговую диаграмму.
246
247         Интегрирует показания сенсоров мощности (Вт) из sensor
_polls во времени
248         и рассчитывает общее количество ватт-часов (Вт·ч) и кВт
т·ч, а также долю каждого компонента.
249
250         Args:
251             db_path: Путь к базе данных.
252             hours: Период анализа в часах (по умолчанию 24).
253
254         Returns:
255             Dict[str, Any]: Аналитический отчет, кВт·ч, разбив
ка по компонентам для круговой диаграммы.
256         """
257         target_db = self.get_db_path(db_path)

```



```

290         AVG(gpu_load_percent) as avg_gpu_load
291     FROM (
292         SELECT * FROM system_snapshots
293         ORDER BY id DESC LIMIT 1000
294     )
295     """
296     cursor.execute(sql_snapshots)
297     snap_stat = dict(cursor.fetchone() or {})
298     conn.close()
299
300     # 3. Анализ и распределение мощности
301     cpu_cores_w = 0.0
302     cpu_package_w = 0.0
303     cpu_memory_w = 0.0
304     gpu_w = 0.0
305     other_sensors_w = 0.0
306
307     for s in sensor_power_rows:
308         s_name = (s.get("sensor_name") or "").lower()
309         avg_w = float(s.get("avg_watts") or 0.0)
310         if "cpu cores" in s_name or "core power" in s_
name:
311             cpu_cores_w = max(cpu_cores_w, avg_w)
312             elif "cpu package" in s_name or "package power
" in s_name:
313                 cpu_package_w = max(cpu_package_w, avg_w)
314                 elif "cpu memory" in s_name or "dram" in s_nam
e:
315                     cpu_memory_w = max(cpu_memory_w, avg_w)
316                     elif "gpu" in s_name or "graphics" in s_name:
317                         gpu_w = max(gpu_w, avg_w)
318                     else:
319                         other_sensors_w += avg_w
320
321     avg_cpu_load = float(snap_stat.get("avg_cpu") or 1
5.0)
322     if cpu_package_w <= 0.0:
323         cpu_total_power = round(10.0 + 55.0 * (avg_cpu

```

```

    _load / 100.0), 2)
324         cpu_cores_w = round(cpu_total_power * 0.75, 2)
325         cpu_uncore_w = round(cpu_total_power * 0.25, 2
    )
326     else:
327         cpu_total_power = round(cpu_package_w, 2)
328         if cpu_cores_w <= 0.0:
329             cpu_cores_w = round(cpu_package_w * 0.75,
    2)
330             cpu_uncore_w = max(0.0, round(cpu_package_w -
    cpu_cores_w, 2))
331
332         if gpu_w <= 0.0:
333             avg_gpu_load = float(snap_stat.get("avg_gpu_lo
    ad") or 5.0)
334             gpu_w = round(5.0 + 35.0 * (avg_gpu_load / 100
    .0), 2)
335         else:
336             gpu_w = round(gpu_w, 2)
337
338         avg_ram_gb = float(snap_stat.get("avg_ram_gb") or
    8.0)
339         ram_power_w = round(max(2.0, avg_ram_gb * 0.4), 2)
    # ~0.4W на 1 GB RAM
340         motherboard_chipset_w = 12.0 # Чипсет материнской
    платы и VRM
341         disks_and_fans_w = 8.0          # Накопители и кулер
    ы
342
343         system_idle_base_w = round(ram_power_w + motherboa
    rd_chipset_w + disks_and_fans_w, 2)
344
345         # Итоговая средняя мощность системы (Вт)
346         total_avg_watts = round(cpu_total_power + gpu_w +
    system_idle_base_w, 2)
347
348         # Энергия за период (hours)
349         total_watt_hours = round(total_avg_watts * hours,

```

2)

```
350     total_kwh = round(total_watt_hours / 1000.0, 3)
351
352     # Доли компонентов для круговой диаграммы
353     pie_labels = [
354         "Ядра процессора (CPU Cores)",
355         "Контроллеры и Uncore ЦП",
356         "Видеокарта (GPU)",
357         "Оперативная память (RAM)",
358         "Материнская плата и чипсет",
359         "Накопители (SSD/HDD) и кулеры",
360     ]
361     pie_values = [
362         round(cpu_cores_w * hours, 2),
363         round(cpu_uncore_w * hours, 2),
364         round(gpu_w * hours, 2),
365         round(ram_power_w * hours, 2),
366         round(motherboard_chipset_w * hours, 2),
367         round(disks_and_fans_w * hours, 2),
368     ]
369     pie_colors = [
370         "#38bdf8", # Ядра CPU (голубой)
371         "#0284c7", # Uncore (синий)
372         "#a855f7", # GPU (фиолетовый)
373         "#10b981", # RAM (зеленый)
374         "#f59e0b", # Материнская плата (янтарный)
375         "#ec4899", # Накопители и кулеры (розовый)
376     ]
377
378     total_wh_sum = sum(pie_values) or 1.0
379     pie_percentages = [round((v / total_wh_sum) * 100.
380 0, 1) for v in pie_values]
381
382     breakdown_table = []
383     for lbl, val_wh, pct in zip(pie_labels, pie_values
384 , pie_percentages):
385         breakdown_table.append({
386             "component": lbl,
```

```

385         "watt_hours": val_wh,
386         "kwh": round(val_wh / 1000.0, 3),
387         "percent": pct,
388         "avg_power_watts": round(val_wh / hours, 2
    ),
389     })
390
391     return {
392         "status": "ok",
393         "period_hours": hours,
394         "total_kwh": total_kwh,
395         "total_watt_hours": total_watt_hours,
396         "avg_system_power_watts": total_avg_watts,
397         "cpu_package_power_watts": cpu_total_power,
398         "gpu_power_watts": gpu_w,
399         "system_board_power_watts": system_idle_base_w
    ,
400         "chart": {
401             "type": "doughnut",
402             "title": f"Суточная круговая диаграмма эне
    ргопотребления (Всего: {total_kwh} кВт·ч, {total_avg_watts} Вт
    )",
403             "labels": pie_labels,
404             "datasets": [
405                 {
406                     "data": pie_values,
407                     "backgroundColor": pie_colors,
408                     "borderWidth": 2,
409                     "borderColor": "#131b2e",
410                 }
411             ],
412         },
413         "breakdown": breakdown_table,
414         "sensor_readings": sensor_power_rows,
415         "sql": f"{sql_sensors.strip()}\n-- и статистик
    а нагрузки:\n{sql_snapshots.strip()}",
416     }
417 except Exception as ex:

```

```

418         logger.error(f"Ошибка расчета энергопотребления: {
ex}")
419         return {"status": "error", "message": str(ex)}
420
421     def get_top_processes_summary(
422         self, limit: int = 10, db_path: Optional[Union[str, Pa
th]] = None
423     ) -> Dict[str, Any]:
424         """Получить сводку самых ресурсоемких процессов по CPU
и памяти.
425
426         Args:
427             limit: Количество процессов.
428             db_path: Путь к базе данных.
429
430         Returns:
431             Dict[str, Any]: Список топ-процессов и SQL-запрос.
432         """
433         target_db = self.get_db_path(db_path)
434         if not target_db:
435             return {"status": "error", "message": "telemetry.d
b не найдена"}
436
437         sql = """
438             SELECT name,
439                    ROUND(AVG(cpu_percent), 1) as avg_cpu,
440                    ROUND(MAX(cpu_percent), 1) as max_cpu,
441                    ROUND(AVG(memory_mb), 1) as avg_ram_mb,
442                    COUNT(*) as samples_count
443             FROM (
444                 SELECT name, cpu_percent, memory_mb FROM proce
ss_snapshots
445                 ORDER BY id DESC LIMIT 500
446             )
447             GROUP BY name
448             ORDER BY avg_cpu DESC
449             LIMIT ?
450         """

```

```

451         try:
452             conn = sqlite3.connect(str(target_db), timeout=5.0
453         )
454             conn.row_factory = sqlite3.Row
455             cursor = conn.cursor()
456             cursor.execute(sql, (limit,))
457             rows = [dict(r) for r in cursor.fetchall()]
458             conn.close()
459
460             # Если snapshots пусты, попробуем process_rollups_da
ily
461             if not rows:
462                 fallback_sql = """
463                     SELECT name, avg_cpu_percent as avg_cpu, m
ax_cpu_percent as max_cpu, avg_memory_mb as avg_ram_mb, sample
_count as samples_count
464                     FROM process_rollups_daily
465                     ORDER BY avg_cpu_percent DESC LIMIT ?
466                 """
467                 conn = sqlite3.connect(str(target_db), timeout
=5.0)
468                 conn.row_factory = sqlite3.Row
469                 cursor = conn.cursor()
470                 cursor.execute(fallback_sql, (limit,))
471                 rows = [dict(r) for r in cursor.fetchall()]
472                 conn.close()
473                 sql = fallback_sql
474
475                 chart_labels = [r["name"] for r in rows]
476                 chart_cpu = [r["avg_cpu"] for r in rows]
477
478             return {
479                 "status": "ok",
480                 "sql": sql.strip(),
481                 "processes": rows,
482                 "count": len(rows),
483                 "chart": {
484                     "type": "bar",

```



```

484         "title": "Топ процессов по средней нагрузк
е на ЦП (%)",
485         "labels": chart_labels,
486         "datasets": [
487             {
488                 "label": "Средняя загрузка ЦП (%)"
489             ,
490                 "data": chart_cpu,
491                 "backgroundColor": "rgba(56, 189,
248, 0.7)",
492                 "borderColor": "#38bdf8",
493                 "borderWidth": 1,
494             }
495         ],
496     },
497     except Exception as ex:
498         logger.error(f"Ошибка выборки топ-процессов: {ex}"
499 )
500         return {"status": "error", "message": str(ex), "sq
l": sql.strip()}
501
502     def get_schema_summary(self, db_path: Optional[Union[str,
Path]] = None) -> Dict[str, Any]:
503         """Получить краткую схему таблиц telemetry.db для конт
екста AI-модели.
504
505         Args:
506             db_path: Путь к базе данных.
507
508         Returns:
509             Dict[str, Any]: Словарь таблиц с их полями и колич
еством строк.
510
511         """
512         target_db = self.get_db_path(db_path)
513         if not target_db:
514             return {"status": "error", "message": "telemetry.d
b не найдена"}

```

```

513 |
514 |         try:
515 |             conn = sqlite3.connect(str(target_db), timeout=5.0
516 | )
517 |             cursor = conn.cursor()
518 |             cursor.execute("SELECT name FROM sqlite_master WHE
519 | RE type='table'")
520 |             tables = [r[0] for r in cursor.fetchall() if r[0]
521 | != "sqlite_sequence"]
522 |
523 |             schema: Dict[str, Any] = {}
524 |             for t in tables:
525 |                 cursor.execute(f"PRAGMA table_info({t})")
526 |                 cols = [col[1] for col in cursor.fetchall()]
527 |                 cursor.execute(f"SELECT count(*) FROM {t}")
528 |                 cnt = cursor.fetchone()[0]
529 |                 schema[t] = {"columns": cols, "rows_count": cn
530 | t}
531 |
532 |             conn.close()
533 |             return {"status": "ok", "tables": schema}
534 |         except Exception as ex:
535 |             return {"status": "error", "message": str(ex)}
536 |
537 |     def get_client_processes_data(
538 |         self, db_path: Optional[Union[str, Path]] = None, limi
539 | t: int = 50
540 |     ) -> Dict[str, Any]:
541 |         """Получить данные о ресурсах клиентских процессов из
542 | SQLite.
543 |
544 |     Args:
545 |         db_path: Путь к файлу базы данных telemetry.db.
546 |         limit: Максимальное число процессов.
547 |
548 |     Returns:
549 |         Dict[str, Any]: Словарь с процессами, SQL-запросом
550 | и статусом.

```

```

544         """
545         target_db = self.get_db_path(db_path)
546         if not target_db:
547             return {"status": "error", "message": "telemetry.d
b не найдена", "processes": []}
548
549         sql = """
550             SELECT pid, name, cpu_percent, memory_mb, memory_p
ercent,
551                 num_threads, num_handles, username, read_by
tes_sec, write_bytes_sec,
552                 executable as executable_path, cmdline as c
ommand_line
553             FROM process_snapshots
554             WHERE id IN (
555                 SELECT MAX(id) FROM process_snapshots GROUP BY
pid
556             )
557             ORDER BY cpu_percent DESC, memory_mb DESC
558             LIMIT ?
559         """
560         try:
561             conn = sqlite3.connect(str(target_db), timeout=5.0
)
562             conn.row_factory = sqlite3.Row
563             cursor = conn.cursor()
564             cursor.execute(sql, (limit,))
565             rows = [dict(r) for r in cursor.fetchall()]
566             conn.close()
567             return {"status": "ok", "processes": rows, "count"
: len(rows), "sql": sql.strip()}
568         except Exception as ex:
569             logger.error(f"Ошибка выборки клиентских процессов
: {ex}")
570             return {"status": "error", "message": str(ex), "pr
ocesses": [], "sql": sql.strip()}
571
572

```

reboot_analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Reboot Analyzer
5 # Description:
6 #   Модуль глубокого анализа и корреляции причин перезагрузок
7 #   Windows.
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.reboot_analyzer imp
11 #   ort WindowsRebootAnalyzer
12 #   service = WindowsRebootAnalyzer()
13 #
14 # File: reboot_analyzer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль глубокого анализа и корреляции причин перезагрузок W
24 indows."""
25
26 import os
27 import re
28 import socket
29 import subprocess
30 import time
```

```
30 from datetime import datetime, timezone, timedelta
31 from typing import Any, Dict, List, Optional, Tuple
32
33 from logger import logger
34 from apps.windows.telemetry.win32_fffi.wevtapi import WevtAPI
35 from apps.windows.telemetry.win32_fffi.error_decoder import decoder_bugcheck_code, format_system_error
36 from apps.windows.telemetry.models import (
37     RebootAnalysisReport,
38     RebootCorrelatedEvent,
39     RebootSession,
40     ShutdownType,
41 )
42 from apps.windows.telemetry.sqlite import TelemetryStorage
43
44 # Целевые Event ID для корреляции жизненного цикла загрузки и
45 # выключения
46 TARGET_EVENT_IDS = {
47     1074: ('User32', 'Плановый перезапуск или выключение иници-
48     ировано процессом/пользователем'),
49     41: ('Kernel-Power', 'Система перезагрузилась без корректн-
50     ого предварительного выключения'),
51     6008: ('EventLog', 'Предыдущее завершение работы системы б-
52     ыло неожиданным'),
53     1001: ('WER-SystemErrorReporting', 'Критический системный
54     сбой ОС (BSOD / BugCheck)'),
55     19: ('WindowsUpdateClient', 'Успешная установка обновления
56     Windows Update'),
57     20: ('WindowsUpdateClient', 'Ошибка установки обновления W-
58     indows Update'),
59     21: ('WindowsUpdateClient', 'Завершена установка обновлени-
60     й, требуется перезапуск'),
61     12: ('Kernel-General', 'Запуск ядра операционной системы'),
62     ,
63     13: ('Kernel-General', 'Завершение работы операционной сис-
64     темы'),
65     6005: ('EventLog', 'Запуск службы журнала событий Windows
66     (система загружена)'),
```

```

56     6006: ('EventLog', 'Остановка службы журнала событий Windo
ws (штатное исключение)'),
57     7045: ('Service Control Manager', 'В системе была установл
ена новая служба'),
58     109: ('Kernel-Power', 'Переход ядра в режим гибернации / F
ast Startup'),
59 }
60
61
62 class WindowsRebootAnalyzer:
63     """Анализатор жизненного цикла перезагрузок и исключений о
перационной системы."""
64
65     def __init__(
66         self,
67         storage: Optional[TelemetryStorage] = None,
68         wevtapi: Optional[WevtAPI] = None,
69     ) -> None:
70         """Инициализация анализатора перезагрузок.
71
72         Args:
73             storage: Хранилище телеметрии TelemetryStorage.
74             wevtapi: Экземпляр низкоуровневого API WevtAPI.
75         """
76         self.storage = storage or TelemetryStorage.get_instanc
e()
77         self.wevtapi = wevtapi or WevtAPI()
78
79     def get_current_boot_info(self) -> Tuple[datetime, float]:
80         """Получить время текущей загрузки ОС и текущий аптайм
.
81
82         Returns:
83             Tuple[datetime, float]: (время старта, аптайм в се
кундах).
84         """
85         try:
86             import psutil

```

```

87         b_epoch = psutil.boot_time()
88         boot_dt = datetime.fromtimestamp(b_epoch, tz=timezone.
one.utc)
89         uptime_sec = max(0.0, time.time() - b_epoch)
90         return boot_dt, uptime_sec
91     except Exception:
92         # Fallback
93         boot_dt = datetime.now(timezone.utc)
94         return boot_dt, 0.0
95
96     def collect_reboot_history(
97         self,
98         limit: int = 20,
99         hours: int = 720,
100         persist_to_storage: bool = True,
101     ) -> RebootAnalysisReport:
102         """Сбор и глубокая корреляция истории перезагрузок за
103         указанный период.
104
105         Args:
106             limit: Максимальное количество анализируемых сессии
107             й перезагрузки.
108             hours: Глубина выборки событий в часах (по умолчанию 30
109             дней = 720 ч).
110             persist_to_storage: Сохранять ли выявленные новые
111             сессии в БД SQLite.
112
113         Returns:
114             RebootAnalysisReport: Полный аналитический отчет с
115             сессиями и оценкой стабильности.
116         """
117         cur_boot_dt, cur_uptime = self.get_current_boot_info()
118         cur_boot_str = cur_boot_dt.strftime('%Y-%m-%d %H:%M:%S
119         ')
120
121         hostname = socket.gethostname()
122
123         # 1. Извлечение событий из EventLog
124         raw_events = self._query_system_events(hours=hours)

```

```
118 |
119 |         # 2. Корреляция событий в хронологические сессии
120 |         sessions = self._correlate_events_into_sessions(raw_events, limit=limit)
121 |
122 |         # 3. Сохранение в постоянное хранилище при необходимости
123 |         if persist_to_storage and self.storage:
124 |             for session in sessions:
125 |                 try:
126 |                     self.storage.save_reboot_session(session)
127 |                 except Exception as ex:
128 |                     logger.debug(f'Ошибка сохранения сессии при перезагрузки {session.boot_id}: {ex}')
129 |
130 |         # 4. Подсчет статистики стабильности
131 |         planned_cnt = sum(1 for s in sessions if s.shutdown_type in (
132 |             ShutdownType.PLANNED, ShutdownType.UPDATE_RESTART,
133 |             ShutdownType.USER_INITIATED, ShutdownType.HYBRID_SHUTDOWN
134 |         ))
135 |         unexpected_cnt = sum(1 for s in sessions if s.shutdown_type in (
136 |             ShutdownType.UNEXPECTED, ShutdownType.DIRTY_POWER_LOSS, ShutdownType.HARDWARE_POWER_BUTTON
137 |         ))
138 |         bsod_cnt = sum(1 for s in sessions if s.shutdown_type == ShutdownType.CRASH_BSOD)
139 |         update_cnt = sum(1 for s in sessions if s.shutdown_type == ShutdownType.UPDATE_RESTART)
140 |         total_cnt = len(sessions)
141 |
142 |         if total_cnt > 0:
143 |             stability_score = round(max(0.0, 100.0 - (unexpected_cnt * 25.0 + bsod_cnt * 35.0) / total_cnt), 1)
144 |         else:
145 |             stability_score = 100.0
```



```

146 |         # Формирование текстового резюме
147 |         summary_parts = [
148 |             f'Проанализировано {total_cnt} перезапусков Window
| s.',
149 |             f'Плановых: {planned_cnt}, Обновлений: {update_cnt
| }, Аварийных/внезапных: {unexpected_cnt}, BSOD: {bsod_cnt}.',
150 |             f'Индекс стабильности завершений работы: {stabilit
| y_score}%/100%.',
151 |         ]
152 |         if bsod_cnt > 0:
153 |             summary_parts.append(f'ВНИМАНИЕ: Зафиксировано {bs
| od_cnt} критических сбоев синего экрана (BSOD).')
154 |             elif unexpected_cnt > 0:
155 |                 summary_parts.append(f'ПРЕДУПРЕЖДЕНИЕ: Зафиксирова
| но {unexpected_cnt} внезапных обесточиваний или зависаний.')
156 |             else:
157 |                 summary_parts.append('Все недавние перезагрузки си
| стемы завершены в штатном запланированном режиме.')
158 |
159 |         return RebootAnalysisReport(
160 |             hostname=hostname,
161 |             current_boot_time=cur_boot_str,
162 |             current_uptime_seconds=round(cur_uptime, 1),
163 |             current_uptime_human=self._format_duration(cur_upt
| ime),
164 |             total_reboots_analyzed=total_cnt,
165 |             planned_count=planned_cnt,
166 |             unexpected_count=unexpected_cnt,
167 |             bsod_count=bsod_cnt,
168 |             update_reboot_count=update_cnt,
169 |             stability_score=stability_score,
170 |             sessions=sessions,
171 |             summary_ru=' '.join(summary_parts),
172 |         )
173 |
174 |     def analyze_current_boot(self) -> Optional[RebootSession]:
175 |         """Быстрый анализ причины завершения предыдущей сессии
| перед текущим запуском.

```

```

176
177         Returns:
178             Optional[RebootSession]: Сессия текущего запуска с
            исследованием предшествующего выключения.
179         """
180         report = self.collect_reboot_history(limit=1, hours=72
            , persist_to_storage=True)
181         if report.sessions:
182             return report.sessions[0]
183         return None
184
185     def _query_system_events(self, hours: int = 720) -> List[Dict[str, Any]]:
186         """Запрос событий из журналов Windows через WevtAPI и
            PowerShell fallback."""
187         events: List[Dict[str, Any]] = []
188
189         # 1. Попытка нативного сбора через WevtAPI
190         if self.wevtapi and self.wevtapi.is_available():
191             try:
192                 for eid in TARGET_EVENT_IDS.keys():
193                     # Читаем System
194                     sys_evs = self.wevtapi.read_events(channel
            ='System', limit=100, event_id=eid, hours=hours)
195                     events.extend(sys_evs)
196                     # Читаем Application
197                     if eid in (1001, 1074):
198                         app_evs = self.wevtapi.read_events(channel
            ='Application', limit=50, event_id=eid, hours=hours)
199                         events.extend(app_evs)
200                     # Читаем WindowsUpdateClient
201                     if eid in (19, 20, 21):
202                         wu_evs = self.wevtapi.read_events(
203                             channel='Microsoft-Windows-Windows
            UpdateClient/Operational',
204                             limit=50,
205                             event_id=eid,
206                             hours=hours,

```

```

207         )
208         events.extend(wu_evs)
209     except Exception as ex:
210         logger.debug(f'Ошибка нативного чтения WevtAPI
для перезагрузок: {ex}')
211
212     # 2. Если событий мало или WevtAPI недоступен, выполня
ем надежный PowerShell сбор
213     if len(events) < 2:
214         ps_events = self._query_events_powershell(hours=ho
urs)
215         events.extend(ps_events)
216
217     # 3. Дедупликация и сортировка от самых новых к старым
218     unique_events: List[Dict[str, Any]] = []
219     seen_keys = set()
220     for ev in events:
221         t = ev.get('timestamp', '')
222         eid = ev.get('event_id', 0)
223         prov = ev.get('provider', '')
224         key = f'{t}_{eid}_{prov}'
225         if key not in seen_keys:
226             seen_keys.add(key)
227             unique_events.append(ev)
228
229     # Сортировка по времени (по убыванию)
230     def _parse_ts(e: Dict[str, Any]) -> str:
231         return e.get('timestamp', '')
232
233     unique_events.sort(key=_parse_ts, reverse=True)
234     return unique_events
235
236     def _query_events_powershell(self, hours: int = 720) -> Li
st[Dict[str, Any]]:
237         """Fallback сбор событий через PowerShell Get-WinEvent
. """
238         if os.name != 'nt':
239             return []

```

```

240
241     ids_str = ','.join(str(k) for k in TARGET_EVENT_IDS.keys())
242     ps_script = f"""
243     $ErrorActionPreference = 'SilentlyContinue'
244     $startTime = (Get-Date).AddHours(-{hours})
245     $filter = @{{
246         LogName = 'System', 'Application'
247         Id = @({ids_str})
248         StartTime = $startTime
249     }}
250     $events = Get-WinEvent -FilterHashtable $filter -MaxEvents 300
251     $result = @()
252     foreach ($e in $events) {{
253         $result += [PSCustomObject]@{{
254             timestamp = $e.TimeCreated.ToString("yyyy-MM-dd HH:mm:ss")
255             event_id = $e.Id
256             provider = $e.ProviderName
257             level = $e.LevelDisplayName
258             message = $e.Message
259             event_data = @{{}}
260         }}
261     }}
262     $result | ConvertTo-Json -Depth 3 -Compress
263     """
264     try:
265         res = subprocess.run(
266             ['powershell.exe', '-NoProfile', '-NonInteractive', '-Command', ps_script],
267             capture_output=True,
268             text=True,
269             timeout=12,
270         )
271         if res.returncode == 0 and res.stdout.strip():
272             import json
273             data = json.loads(res.stdout)

```

```

274         if isinstance(data, list):
275             return data
276         elif isinstance(data, dict):
277             return [data]
278     except Exception as ex:
279         logger.debug(f'Ошибка выполнения PowerShell для со
бытий перезагрузки: {ex}')
280     return []
281
282     def _correlate_events_into_sessions(
283         self,
284         events: List[Dict[str, Any]],
285         limit: int = 20,
286     ) -> List[RebootSession]:
287         """Связывает сырые события в структурированные сессии
перезагрузок."""
288         # Находим маркеры запуска ОС (Event 12, Event 6005, ил
и Event 41 как признак нового старта)
289         boot_events: List[Dict[str, Any]] = []
290         for ev in events:
291             eid = int(ev.get('event_id', 0))
292             if eid in (12, 6005, 41):
293                 boot_events.append(ev)
294
295         # Если маркеров старта нет, но есть события выключения
(1074, 6006, 13), группируем вокруг них
296         if not boot_events:
297             for ev in events:
298                 eid = int(ev.get('event_id', 0))
299                 if eid in (1074, 6006, 13, 6008):
300                     boot_events.append(ev)
301
302         # Кластеризация событий по сессиям
303         sessions: List[RebootSession] = []
304         session_idx = 1
305
306         # Отслеживаем временные границы сессий
307         processed_boots = set()

```

```

308
309     for boot_ev in boot_events:
310         boot_time_str = boot_ev.get('timestamp', '')
311         if not boot_time_str:
312             continue
313
314         # Нормализация времени для дедупликации близких ме
ток старта (в пределах 2 минут)
315         boot_dt = self._parse_iso_time(boot_time_str)
316         if not boot_dt:
317             continue
318
319         is_duplicate = False
320         for seen_dt in processed_boots:
321             if abs((boot_dt - seen_dt).total_seconds()) <
120:
322                 is_duplicate = True
323                 break
324         if is_duplicate:
325             continue
326
327         processed_boots.add(boot_dt)
328
329         # Находим события в окне [-15 мин; +3 мин] вокруг
момента старта / выключения
330         window_start = boot_dt - timedelta(minutes=15)
331         window_end = boot_dt + timedelta(minutes=3)
332
333         session_events = [
334             ev for ev in events
335             if self._is_time_in_window(ev.get('timestamp',
''), window_start, window_end)
336         ]
337
338         session = self._analyze_session_window(boot_dt, se
ssion_events, session_idx)
339         sessions.append(session)
340         session_idx += 1

```

```

341
342         if len(sessions) >= limit:
343             break
344
345         # Рассчитываем uptime между последовательными сессиями
346         for i in range(len(sessions)):
347             curr = sessions[i]
348             if i + 1 < len(sessions):
349                 prev = sessions[i + 1]
350                 curr.previous_boot_time = prev.boot_time
351                 prev_dt = self._parse_iso_time(prev.boot_time)
352                 curr_dt = self._parse_iso_time(curr.boot_time)
353                 if prev_dt and curr_dt and curr_dt > prev_dt:
354                     diff_sec = (curr_dt - prev_dt).total_seconds()
355                     curr.uptime_seconds = round(diff_sec, 1)
356                     curr.uptime_human = self._format_duration(
357                         diff_sec)
358
359         return sessions
360
361     def _analyze_session_window(
362         self,
363         boot_dt: datetime,
364         window_events: List[Dict[str, Any]],
365         session_idx: int,
366     ) -> RebootSession:
367         """Анализирует события в окне сессии и строит вывод о
368         причине перезагрузки."""
369         boot_id = f"reboot-{boot_dt.strftime('%Y%m%d-%H%M%S')}"
370
371         boot_time_str = boot_dt.strftime('%Y-%m-%d %H:%M:%S')
372
373         # Извлечение специфических событий
374         ev_1074 = self._find_event(window_events, 1074)
375         ev_41 = self._find_event(window_events, 41)
376         ev_6008 = self._find_event(window_events, 6008)
377         ev_1001 = self._find_event(window_events, 1001)

```

```

375 |         ev_19 = self._find_event(window_events, 19)
376 |         ev_7045 = self._find_event(window_events, 7045)
377 |         ev_109 = self._find_event(window_events, 109)
378 |
379 |         evidence: List[str] = []
380 |         events_chain: List[RebootCorrelatedEvent] = []
381 |
382 |         for ev in window_events:
383 |             eid = int(ev.get('event_id', 0))
384 |             prov = ev.get('provider', '')
385 |             desc = TARGET_EVENT_IDS.get(eid, (prov, 'Системное
| событие'))[1]
386 |             events_chain.append(
387 |                 RebootCorrelatedEvent(
388 |                     event_id=eid,
389 |                     provider=prov or 'System',
390 |                     timestamp=ev.get('timestamp', ''),
391 |                     level=ev.get('level') or 'Information',
392 |                     description=desc,
393 |                     details=ev.get('event_data') or {},
394 |                 )
395 |             )
396 |
397 |         # 1. Извлечение полей из Event 1074 (User32)
398 |         initiating_proc = None
399 |         initiating_user = None
400 |         shutdown_action = None
401 |         reason_text = None
402 |         reason_code = None
403 |
404 |         if ev_1074:
405 |             msg = ev_1074.get('message', '')
406 |             data = ev_1074.get('event_data', {})
407 |             initiating_proc = data.get('param1') or self._extr
| act_regex(msg, r'process\s+([^\s]+)\s+has', r'процесс\s+([^\s]
| +)\s+инициировал')
408 |             initiating_user = data.get('param7') or data.get('
| param2') or self._extract_regex(msg, r'on behalf of user\s+([^\s]

```



```

\s]+)', r'от имени пользователя\s+([\s]+)')
409 shutdown_action = data.get('param5') or self._extract_regex(msg, r'Shutdown Type:\s*([\r\n]+)', r'Тип выключения:\s*([\r\n]+)')
410 reason_text = data.get('param3') or self._extract_regex(msg, r'following reason:\s*([\r\n]+)', r'следующей причине:\s*([\r\n]+)')
411 reason_code = data.get('param4') or self._extract_regex(msg, r'Reason Code:\s*([0-9a-fx]+)', r'Код причины:\s*([0-9a-fx]+)')
412 evidence.append(f'Event 1074 (User32): Инициатор={initiating_proc or "N/A"}, Пользователь={initiating_user or "N/A"}, Действие={shutdown_action or "restart"}')
413 if reason_code:
414     decoded_reason = format_system_error(reason_code)
415     if decoded_reason:
416         evidence.append(f'Код причины {reason_code}: {decoded_reason}')
417
418 # 2. Извлечение полей из Event 41 (Kernel-Power)
419 bugcheck_code = None
420 bugcheck_params: List[str] = []
421 power_button_ts = None
422
423 if ev_41:
424     data = ev_41.get('event_data', {})
425     bc_raw = data.get('BugcheckCode', '0')
426     try:
427         bc_int = int(bc_raw)
428         if bc_int != 0:
429             bugcheck_code = f'0x{bc_int:08X}'
430     except ValueError:
431         pass
432     p_btn = data.get('PowerButtonTimestamp', '0')
433     try:
434         p_btn_int = int(p_btn)
435         if p_btn_int > 0:

```

```

436         power_button_ts = p_btn_int
437     except ValueError:
438         pass
439     for i in range(1, 5):
440         p_val = data.get(f'BugcheckParameter{i}')
441         if p_val:
442             bugcheck_params.append(str(p_val))
443         evidence.append(f'Event 41 (Kernel-Power): BugChec
k={bugcheck_code or "none"}, PowerButtonTS={power_button_ts or
"none"}')
444
445     # 3. Извлечение полей из Event 6008 (EventLog)
446     if ev_6008:
447         evidence.append('Event 6008 (EventLog): Неожиданно
е завершение предыдущей сессии')
448
449     # 4. Извлечение полей из Event 1001 (BSOD / WER)
450     if ev_1001:
451         msg = ev_1001.get('message', '')
452         bc_match = self._extract_regex(msg, r'0x[0-9a-fA-F
]{8}')
453         if bc_match and not bugcheck_code:
454             bugcheck_code = bc_match
455         evidence.append(f'Event 1001 (WER/BugCheck): Зафик
сирован аварийный дамп {bugcheck_code or ""}')
456
457     # 5. Извлечение полей из Event 19 (WindowsUpdateClient
)
458     kb_number = None
459     if ev_19:
460         msg = ev_19.get('message', '')
461         kb_number = self._extract_regex(msg, r'KB\d+')
462         evidence.append(f'Event 19 (WindowsUpdateClient):
Установка обновления {kb_number or "KB"}')
463
464     # 6. Извлечение полей из Event 7045 (SCM)
465     service_name = None
466     if ev_7045:

```

```

467 |         msg = ev_7045.get('message', '')
468 |         service_name = self._extract_regex(msg, r'Service
| Name:\s*([\r\n]+)', r'Имя службы:\s*([\r\n]+)')
469 |         evidence.append(f'Event 7045 (SCM): Установка служ
| бы {service_name or "Unknown"}')
470 |
471 |         # -----
| -----
472 |         # Дерево принятия решений (Decision Gate) для классифи
| кации
473 |         # -----
| -----
474 |         shutdown_type = ShutdownType.UNKNOWN
475 |         likely_class = 'unknown'
476 |         conclusion = ''
477 |
478 |         # Сценарий А: BSOD / Синий экран
479 |         if ev_1001 or (bugcheck_code and bugcheck_code != '0x0
| 0000000'):
480 |             shutdown_type = ShutdownType.CRASH_BSOD
481 |             likely_class = 'bsod_crash'
482 |             bc_info = decode_bugcheck_code(bugcheck_code) if b
| ugcheck_code else {}
483 |             sym = bc_info.get('symbol')
484 |             desc = bc_info.get('description_ru')
485 |             if sym and sym != 'UNKNOWN_BUGCHECK':
486 |                 conclusion = f'Критический сбой операционной с
| истемы (BSOD): {sym} ({bugcheck_code}) – {desc}'
487 |                 elif desc and desc != 'Критический сбой ядра Windo
| ws.':
488 |                     conclusion = f'Критический сбой операционной с
| истемы (BSOD): {bugcheck_code} – {desc}'
489 |                 else:
490 |                     conclusion = f'Критический сбой операционной с
| истемы (BSOD). Код ошибки: {bugcheck_code or "Неизвестен"}.'
491 |
492 |         # Сценарий Б: Принудительное выключение физической кно
| пкой питания

```

```

493         elif power_button_ts and power_button_ts > 0:
494             shutdown_type = ShutdownType.HARDWARE_POWER_BUTTON
495             likely_class = 'hardware_power_button'
496             conclusion = 'Принудительное выключение долгим наж
ати́ем аппаратной кнопки питания.'
497
498         # Сценарий В: Внезапное отключение электропитания / за
висание (Event 41 / 6008 без 1074)
499         elif (ev_41 or ev_6008) and not ev_1074:
500             shutdown_type = ShutdownType.DIRTY_POWER_LOSS
501             likely_class = 'dirty_power_loss'
502             conclusion = 'Внезапная потеря электропитания или
мгновенный аппаратный сброс без сохранения состояния.'
503
504         # Сценарий Г: Плановый рестарт после обновления Window
s Update
505         elif ev_1074 and (
506             ev_19
507             or (initiating_proc and any(k in initiating_proc.l
ower() for k in ('svchost', 'trustedinstaller', 'tiworker', 'w
uaucflt'))))
508             or (reason_text and any(k in reason_text.lower() f
or k in ('update', 'обновлен', 'security', 'безопасн'))))
509             or (reason_code and '80020010' in reason_code.lowe
r())
510         ):
511             shutdown_type = ShutdownType.UPDATE_RESTART
512             likely_class = 'update_related_restart'
513             kb_str = f'({kb_number})' if kb_number else ''
514             conclusion = f'Плановый перезапуск службы обновлен
ия Windows Update{kb_str} от имени {initiating_user or "SYSTEM
"}. '
515
516         # Сценарий Д: Перезапуск после установки программы / с
лужбы
517         elif ev_1074 and (ev_7045 or (reason_text and any(k in
reason_text.lower() for k in ('install', 'установк', 'установ
')))):

```

```
518         shutdown_type = ShutdownType.PLANNED
519         likely_class = 'software_installation_restart'
520         svc_str = f' (служба: {service_name})' if service_
name else ''
521         conclusion = f'Плановый перезапуск после инсталляц
ии программного обеспечения{svc_str}.'
522
523         # Сценарий Е: Пользовательское штатное выключение / пе
резапуск
524         elif ev_1074:
525             shutdown_type = ShutdownType.USER_INITIATED if (in
itiating_proc and 'explorer' in initiating_proc.lower()) else
ShutdownType.PLANNED
526             likely_class = 'planned_clean_reboot'
527             act = shutdown_action or 'перезапуск'
528             conclusion = f'Штатный {act} инициирован процессом
{initiating_proc or "Система"} (пользователь: {initiating_use
r or "Пользователь"}).'
529
530         # Сценарий Ж: Быстрый запуск / Fast Startup (Kernel-Po
wer 109)
531         elif ev_109:
532             shutdown_type = ShutdownType.HYBRID_SHUTDOWN
533             likely_class = 'hybrid_fast_startup'
534             conclusion = 'Штатное завершение работы в режиме г
ибридного сна (Fast Startup / Hiberboot).'
535
536         # Fallback по умолчанию: Корректный перезапуск
537         else:
538             shutdown_type = ShutdownType.PLANNED
539             likely_class = 'clean_restart'
540             conclusion = 'Штатный запуск операционной системы
без зафиксированных системных ошибок.'
541
542         return RebootSession(
543             boot_id=boot_id,
544             boot_time=boot_time_str,
545             shutdown_type=shutdown_type,
```

```

546         likely_class=likely_class,
547         conclusion=conclusion,
548         initiating_process=initiating_proc,
549         initiating_user=initiating_user,
550         shutdown_action=shutdown_action,
551         reason_text=reason_text,
552         reason_code=reason_code,
553         bugcheck_code=bugcheck_code,
554         bugcheck_params=bugcheck_params,
555         power_button_timestamp=power_button_ts,
556         windows_update_kb=kb_number,
557         service_installed=service_name,
558         evidence=evidence,
559         events_chain=events_chain,
560     )
561
562     def _find_event(self, events: List[Dict[str, Any]], target
563     _id: int) -> Optional[Dict[str, Any]]:
564         """Находит первое событие с заданным ID среди списка."""
565         ""
566         for e in events:
567             if int(e.get('event_id', 0)) == target_id:
568                 return e
569         return None
570
571     def _extract_regex(self, text: str, *patterns: str) -> Opt
572     ional[str]:
573         """Извлекает подстроку по первому совпавшему регулярно
574         му выражению."""
575         if not text:
576             return None
577         for pat in patterns:
578             match = re.search(pat, text, re.IGNORECASE)
579             if match:
580                 return match.group(1).strip() if match.groups(
581                 ) else match.group(0).strip()
582         return None

```

```

579     def _parse_iso_time(self, ts_str: str) -> Optional[datetime
| e]:
580         """Парсинг строки времени в datetime UTC."""
581         if not ts_str:
582             return None
583         clean_ts = ts_str.replace('Z', '').split('.')[0]
584         for fmt in ('%Y-%m-%d %H:%M:%S', '%Y-%m-%dT%H:%M:%S',
| '%d.%m.%Y %H:%M:%S'):
585             try:
586                 dt = datetime.strptime(clean_ts, fmt)
587                 return dt.replace(tzinfo=timezone.utc)
588             except ValueError:
589                 pass
590         return None
591
592     def _is_time_in_window(self, ts_str: str, start: datetime,
| end: datetime) -> bool:
593         """Проверяет, попадает ли временная метка в заданный и
| нтервал."""
594         dt = self._parse_iso_time(ts_str)
595         if not dt:
596             return False
597         return start <= dt <= end
598
599     def _format_duration(self, seconds: Optional[float]) -> str:
| r:
600         """Форматирует секунды в читаемый вид (напр. '7h 25m')
| ."""
601         if seconds is None or seconds < 0:
602             return 'N/A'
603         mins, secs = divmod(int(seconds), 60)
604         hours, mins = divmod(mins, 60)
605         days, hours = divmod(hours, 24)
606         if days > 0:
607             return f'{days}д {hours}ч {mins}м'
608         if hours > 0:
609             return f'{hours}ч {mins}м'
610         return f'{mins}м {secs}с'

```


server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  - Server
4 # =====
5 # Description:
6 #   FastAPI сервер и веб-интерфейс для исследования телеметрии
  с поддержкой AI-чата.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.server import Telem
  etryChatRequest
11 #
12 #       service = TelemetryChatRequest()
13 #
14 # File: server.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:48:00
20 # =====
21
22 from __future__ import annotations
23 """FastAPI сервер и веб-интерфейс для исследования телеметрии
  с поддержкой AI-чата."""
24
25 import json
26 from pathlib import Path
27 from typing import Any, Dict, List, Optional
28
29 from fastapi import FastAPI, HTTPException, Query
```

```
30 from fastapi.middleware.cors import CORSMiddleware
31 from fastapi.responses import HTMLResponse, JSONResponse
32 from fastapi.staticfiles import StaticFiles
33 from pydantic import BaseModel, Field
34
35 try:
36     from logger import logger
37 except ImportError:
38     import logging
39     logger = logging.getLogger(__name__)
40
41 from .analyzer import TelemetryResearcher
42 from .assistant import TelemetryAssistant
43 from .charts import TelemetryChartGenerator
44 from .client_resource_analyzer import ClientResourceAnalyzer
45 from .extractor import TelemetryDataExtractor
46 from .models import (
47     ClientProgramResource,
48     ClientResourceSummary,
49     DeepResearchReport,
50     ResearchScenarioRequest,
51     TelemetryResearchReport,
52     UnknownProgramEvaluation,
53 )
54 from .query_engine import TelemetryQueryEngine
55
56
57 class TelemetryChatRequest(BaseModel):
58     """Схема запроса к AI-ассистенту телеметрии."""
59
60     message: str = Field(..., description="Текст запроса пользователя")
61     source_path: Optional[str] = Field(None, description="Путь к файлу базы данных или логов")
62     history: Optional[List[Dict[str, Any]]] = Field(None, description="История диалога")
63
64
```

```
65 class SqlQueryRequest(BaseModel):
66     """Схема запроса для выполнения SELECT SQL."""
67
68     sql: str = Field(..., description="Текст SQL-запроса (SELECT)")
69     db_path: Optional[str] = Field(None, description="Опциональный путь к telemetry.db")
70     limit: int = Field(500, ge=1, le=1000, description="Максимум возвращаемых строк")
71
72
73 app = FastAPI(
74     title="Telemetry Research API & Web GUI",
75     description="Веб-интерфейс, API и AI-ассистент для аналитического исследования телеметрии Windows",
76     version="1.1.0",
77 )
78
79 # CORS
80 app.add_middleware(
81     CORSMiddleware,
82     allow_origins=["*"],
83     allow_credentials=True,
84     allow_methods=["*"],
85     allow_headers=["*"],
86 )
87
88 _extractor = TelemetryDataExtractor()
89 _query_engine = TelemetryQueryEngine(extractor=_extractor)
90 _client_analyzer = ClientResourceAnalyzer(query_engine=_query_engine)
91 _assistant = TelemetryAssistant(query_engine=_query_engine, client_analyzer=_client_analyzer)
92 _researcher = TelemetryResearcher(extractor=_extractor)
93 _chart_gen = TelemetryChartGenerator()
94 _web_dir = Path(__file__).parent / "web"
95
96 if _web_dir.exists():
```

```

97     app.mount("/static", StaticFiles(directory=str(_web_dir)),
    name="static")
98
99
100 @app.get("/", response_class=HTMLResponse)
101 async def get_index() -> HTMLResponse:
102     """Отдает главную страницу Web GUI."""
103     index_file = _web_dir / "index.html"
104     if not index_file.exists():
105         raise HTTPException(status_code=404, detail="Web GUI t
    emplate not found")
106     return HTMLResponse(content=index_file.read_text(encoding=
    "utf-8"))
107
108
109 @app.get("/api/health")
110 async def health_check() -> Dict[str, str]:
111     """Проверка доступности сервиса."""
112     return {"status": "ok", "service": "telemetry_research"}
113
114
115 @app.post("/api/chat")
116 @app.post("/apps/telemetry_research/chat")
117 async def chat_endpoint(request: TelemetryChatRequest) -> Dict
    [str, Any]:
118     """Диалоговый эндпоинт AI-ассистента для аналитики и визуа
    лизации телеметрии."""
119     if not request.message or not request.message.strip():
120         raise HTTPException(status_code=400, detail="Текст зап
    роса не может быть пустым")
121
122     try:
123         response = await _assistant.handle_query(
124             user_message=request.message,
125             source_path=request.source_path,
126             history=request.history,
127         )
128         return response

```

```
129     except Exception as ex:
130         logger.error(f"Ошибка в AI-чате телеметрии: {ex}", exc
            _info=True)
131         return {
132             "status": "error",
133             "reply": f"⚠️ Произошла внутренняя ошибка при обраб
отке запроса: {str(ex)}",
134             "chart": None,
135             "sql_queries": [],
136             "metrics_summary": {},
137             "quick_followups": ["Покажи график загрузки ЦПУ по
времени"],
138         }
139
140
141 @app.get("/api/telemetry-sql/power")
142 @app.get("/apps/telemetry_research/tools/power")
143 async def get_power_data_endpoint(
144     db_path: Optional[str] = Query(None),
145     hours: int = Query(24, ge=1, le=168),
146 ) -> Dict[str, Any]:
147     """Возвращает расчет энергопотребления и конфигурацию круг
овой диаграммы."""
148     try:
149         return _query_engine.calculate_power_consumption_24h(d
b_path=db_path, hours=hours)
150     except Exception as ex:
151         logger.error(f"Ошибка вычисления энергопотребления: {e
x}")
152         raise HTTPException(status_code=500, detail=str(ex))
153
154
155 @app.get("/api/telemetry-sql/cpu-timeline")
156 @app.get("/apps/telemetry_research/tools/cpu-timeline")
157 async def get_cpu_timeline_endpoint(
158     db_path: Optional[str] = Query(None),
159     limit: int = Query(120, ge=10, le=2000),
160 ) -> Dict[str, Any]:
```

```

161     """Возвращает временной ряд загрузки и частоты процессора
    для графика."""
162     try:
163         return _query_engine.get_cpu_timeline(limit=limit, db_
path=db_path)
164     except Exception as ex:
165         logger.error(f"Ошибка вычисления таймлайна CPU: {ex}")
166         raise HTTPException(status_code=500, detail=str(ex))
167
168
169 @app.post("/api/telemetry-sql/query")
170 @app.post("/apps/telemetry_research/tools/query")
171 async def execute_sql_endpoint(request: SqlQueryRequest) -> Di
ct[str, Any]:
172     """Выполняет безопасный SELECT SQL-запрос к базе telemetry
.db."""
173     try:
174         return _query_engine.execute_safe_sql(
175             sql_query=request.sql, db_path=request.db_path, li
mit=request.limit
176         )
177     except Exception as ex:
178         logger.error(f"Ошибка выполнения SQL: {ex}")
179         raise HTTPException(status_code=500, detail=str(ex))
180
181
182 @app.get("/api/sources")
183 @app.get("/apps/telemetry_research/sources")
184 async def get_sources() -> List[Dict[str, Any]]:
185     """Возвращает список обнаруженных файлов и баз данных теле
метрии."""
186     files = _researcher.extractor.discover_log_files()
187     result = []
188     for f in files:
189         try:
190             stat = f.stat()
191             result.append(
192                 {

```

```

193         "name": f.name,
194         "path": str(f.resolve()),
195         "size_bytes": stat.st_size,
196         "modified_at": stat.st_mtime,
197     }
198 )
199 except Exception:
200     continue
201 result.sort(key=lambda x: x.get("modified_at", 0), reverse
= True)
202 return result
203
204
205 @app.get("/api/current-state")
206 @app.get("/apps/telemetry_research/current-state")
207 async def get_current_state_endpoint(
208     db_path: Optional[str] = Query(None),
209     limit: int = Query(100, ge=10, le=2000),
210 ) -> Dict[str, Any]:
211     """Возвращает актуальное состояние системы и последние зам
еры из telemetry.db."""
212     try:
213         data = _researcher.extractor.get_current_system_state(
db_path=db_path, limit=limit)
214         return data
215     except Exception as ex:
216         logger.error(f"Ошибка получения текущего состояния: {e
x}")
217         raise HTTPException(status_code=500, detail=str(ex))
218
219
220 @app.post("/api/run-research", response_model=DeepResearchRepo
rt)
221 @app.post("/apps/telemetry_research/run-research", response_mo
del=DeepResearchReport)
222 async def run_research_endpoint(request: Optional[ResearchScen
arioRequest] = None) -> DeepResearchReport:
223     """Запускает глубокое аналитическое исследование телеметри

```

```

и и проверку гипотез."""
224     try:
225         report = _researcher.run_deep_research(scenario=request, chart_generator=_chart_gen)
226         return report
227     except Exception as ex:
228         logger.error(f"Ошибка аналитического исследования телеметрии: {ex}")
229         raise HTTPException(status_code=500, detail=str(ex))
230
231
232 @app.get("/api/records")
233 @app.get("/apps/telemetry_research/records")
234 async def get_records_endpoint(
235     source_path: Optional[str] = Query(None),
236     page: int = Query(1, ge=1),
237     page_size: int = Query(20, ge=1, le=100),
238     query: Optional[str] = Query(None),
239 ) -> Dict[str, Any]:
240     """Возвращает пагинированный список сырых записей телеметрии."""
241     try:
242         all_records = _researcher.extractor.load_all_records(source_path)
243         if query:
244             q_lower = query.lower()
245             all_records = [r for r in all_records if q_lower in json.dumps(r, ensure_ascii=False).lower()]
246
247         total = len(all_records)
248         start_idx = (page - 1) * page_size
249         end_idx = start_idx + page_size
250         items = all_records[start_idx:end_idx]
251
252         return {
253             "total": total,
254             "page": page,
255             "page_size": page_size,

```



```

256         "items": items,
257     }
258 except Exception as ex:
259     logger.error(f"Ошибка чтения записей: {ex}")
260     raise HTTPException(status_code=500, detail=str(ex))
261
262
263 @app.get("/api/dashboard", response_class=HTMLResponse)
264 @app.get("/apps/telemetry_research/dashboard", response_class=
HTMLResponse)
265 async def get_dashboard_html(source_path: Optional[str] = Quer
y(None)) -> HTMLResponse:
266     """Генерирует и возвращает автономный HTML-отчет с графика
ми."""
267     try:
268         report = _researcher.analyze(source_path)
269         records = _researcher.extractor.load_all_records(sourc
e_path)
270         ts_map = _researcher._extract_time_series(records)
271         report.charts = _chart_gen.generate_chart_configs(ts_m
ap, report)
272         html = _chart_gen.render_html_dashboard(report)
273         return HTMLResponse(content=html)
274     except Exception as ex:
275         logger.error(f"Ошибка генерации HTML дашборда: {ex}")
276         raise HTTPException(status_code=500, detail=str(ex))
277
278
279 @app.get("/api/client-resources", response_model=ClientResourc
eSummary)
280 @app.get("/apps/telemetry_research/client-resources", response
_model=ClientResourceSummary)
281 async def get_client_resources_endpoint(
282     db_path: Optional[str] = Query(None),
283     ask_gemini: bool = Query(True),
284     limit: int = Query(50, ge=1, le=200),
285 ) -> ClientResourceSummary:
286     """Возвращает сводный аудит ресурсов клиентских программ с

```

```

оценками от Gemini."""
287     try:
288         summary = await _client_analyzer.analyze_client_resour
ces(
289             db_path=db_path, ask_gemini_for_unknown=ask_gemini
, limit=limit
290         )
291         return summary
292     except Exception as ex:
293         logger.error(f"Ошибка аудита ресурсов клиентских прог
амм: {ex}")
294         raise HTTPException(status_code=500, detail=str(ex))
295
296
297 @app.post("/api/client-resources/evaluate", response_model=Unk
nownProgramEvaluation)
298 @app.post("/apps/telemetry_research/client-resources/evaluate"
, response_model=UnknownProgramEvaluation)
299 async def evaluate_program_endpoint(program: ClientProgramReso
urce) -> UnknownProgramEvaluation:
300     """Запрашивает экспертную оценку неизвестной ресурсоемкой
программы у модели Gemini."""
301     try:
302         evaluation = await _client_analyzer.evaluate_program_w
ith_gemini(program)
303         return evaluation
304     except Exception as ex:
305         logger.error(f"Ошибка запроса оценки к Gemini для {pro
gram.name}: {ex}")
306         raise HTTPException(status_code=500, detail=str(ex))
307
308

```

startup_history_manager.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Startup History Manager
5 # Description:
6 # Менеджер истории, архивации и детектора изменений автозагрузки Windows.
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.telemetry_research.startup_history_manager import StartupHistoryManager
11 #
12 # service = StartupHistoryManager()
13 #
14 # File: startup_history_manager.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 13:56:00
20 # =====
21
22 from __future__ import annotations
23 """Менеджер истории, архивации и детектора изменений автозагрузки Windows."""
24
25 import json
26 import os
27 import uuid
28 from datetime import datetime, timezone
29 from pathlib import Path
```

```

30 from typing import Any, Dict, List, Optional
31 from logger import logger
32 from apps.windows.modules.startup.core.models import (
33     AuditReport,
34     StartupArchiveEntry,
35     StartupChangeItem,
36     StartupEntry,
37 )
38
39
40 class StartupHistoryManager:
41     """Менеджер архивных снимков и отслеживания изменений точек автозапуска."""
42
43     def __init__(
44         self,
45         archive_dir: Optional[Path] = None,
46         storage_dir: Optional[Path] = None,
47         max_archives: int = 100,
48         use_file_index: bool = True,
49     ) -> None:
50         """Инициализация менеджера истории автозапуска.
51
52         Args:
53             archive_dir: Каталог для хранения архивных снимков
54             (по умолчанию data/telemetry/startup_archives).
55             storage_dir: Алиас для archive_dir.
56             max_archives: Максимальное количество хранимых архивов для ротации.
57             use_file_index: Использовать ли индексный файл архивов.
58
59         """
60         target_dir = archive_dir if archive_dir is not None else storage_dir
61         if target_dir is None:
62             base_dir = Path(__file__).resolve().parent.parent.parent.parent
63             self.archive_dir = base_dir / 'data' / 'telemetry'

```

```

    / 'startup_archives'
62         else:
63             self.archive_dir = Path(target_dir)
64             self.max_archives = max(10, max_archives)
65             self.archive_dir.mkdir(parents=True, exist_ok=True)
66             self.index_file = self.archive_dir / 'startup_index.js
onl'
67             self._cached_latest_report: Optional[AuditReport] = No
ne
68             self.use_file_index = use_file_index
69
70         def detect_changes(
71             self,
72             current_report: AuditReport,
73             baseline_report: Optional[AuditReport] = None,
74         ) -> List[StartupChangeItem]:
75             """Сравнение двух снимков автозапуска и выявление всех
отличий (Diff Engine).
76
77             Args:
78                 current_report: Текущий срез аудита автозапуска.
79                 baseline_report: Базовый срез для сравнения (если
None, берется последний сохраненный).
80
81             Returns:
82                 List[StartupChangeItem]: Список зафиксированных из
менений.
83             """
84             base = baseline_report
85             if base is None:
86                 if self._cached_latest_report is not None:
87                     base = self._cached_latest_report
88                 else:
89                     latest_entry = self.get_latest_archive()
90                     if latest_entry:
91                         base = latest_entry.report
92             if base is None:
93                 return []

```

```

94 |
95 |     changes: List[StartupChangeItem] = []
96 |     now_ts = datetime.now(timezone.utc).isoformat()
97 |
98 |     if isinstance(base, dict):
99 |         raw_base_entries = base.get('entries', [])
100 |         base_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in raw_base_entries]
101 |         elif hasattr(base, 'entries'):
102 |             base_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in base.entries]
103 |         else:
104 |             return []
105 |
106 |         base_entries: Dict[str, StartupEntry] = {e.id: e for e
    | in base_entries_list}
107 |
108 |     if isinstance(current_report, dict):
109 |         raw_curr_entries = current_report.get('entries', [
    | ])
110 |         curr_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in raw_curr_entries]
111 |         elif hasattr(current_report, 'entries'):
112 |             curr_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in current_report.entrie
    | s]
113 |         else:
114 |             return []
115 |
116 |         curr_entries: Dict[str, StartupEntry] = {e.id: e for e
    | in curr_entries_list}
117 |
118 |     # 1. Новые записи в автозагрузке
119 |     for entry_id, entry in curr_entries.items():
120 |         if entry_id not in base_entries:
121 |             status_str = 'включен' if entry.is_enabled else
    | e 'отключен'
122 |             desc = (

```

```

123 |             f"Добавлен новый элемент автозапуска: «{en
| try.name}» "
124 |             f"({entry.location_type.value if hasattr(e
| ntry.location_type, 'value') else entry.location_type}, {statu
| s_str}). "
125 |             f"Команда: {entry.command or entry.executa
| ble_path}"
126 |         )
127 |         changes.append(
128 |             StartupChangeItem(
129 |                 change_type='added',
130 |                 entry_id=entry.id,
131 |                 name=entry.name,
132 |                 description=desc,
133 |                 previous_value=None,
134 |                 current_value=entry.model_dump(mode='j
| son'),
135 |                 timestamp=now_ts,
136 |             )
137 |         )
138 |
139 |     # 2. Удаленные записи из автозагрузки
140 |     for entry_id, entry in base_entries.items():
141 |         if entry_id not in curr_entries:
142 |             desc = f"Удален элемент из автозапуска: «{entr
| y.name}» (Расположение: {entry.location_type.value if hasattr(
| entry.location_type, 'value') else entry.location_type})"
143 |             changes.append(
144 |                 StartupChangeItem(
145 |                     change_type='removed',
146 |                     entry_id=entry.id,
147 |                     name=entry.name,
148 |                     description=desc,
149 |                     previous_value=entry.model_dump(mode='
| json'),
150 |                     current_value=None,
151 |                     timestamp=now_ts,
152 |                 )

```

```

153         )
154
155     # 3. Изменения параметров существующих записей
156     for entry_id, entry in curr_entries.items():
157         if entry_id in base_entries:
158             old_entry = base_entries[entry_id]
159
160             # Изменение статуса активности (ВКЛ / ОТКЛ)
161             if old_entry.is_enabled != entry.is_enabled:
162                 old_st = 'ВКЛЮЧЕН' if old_entry.is_enabled
163                 new_st = 'ВКЛЮЧЕН' if entry.is_enabled else 'ОТКЛЮЧЕН'
164                 desc = f"Изменен статус активности «{entry.name}»: было {old_st} -> стало {new_st}"
165                 changes.append(
166                     StartupChangeItem(
167                         change_type='state_changed',
168                         entry_id=entry.id,
169                         name=entry.name,
170                         description=desc,
171                         previous_value={'is_enabled': old_entry.is_enabled},
172                         current_value={'is_enabled': entry.is_enabled},
173                         timestamp=now_ts,
174                     )
175                 )
176
177             # Изменение исполняемого пути или команды
178             if (old_entry.executable_path != entry.executable_path) or (old_entry.command != entry.command):
179                 desc = f"Изменена команда запуска для «{entry.name}»: «{old_entry.command}» -> «{entry.command}»"
180                 changes.append(
181                     StartupChangeItem(
182                         change_type='path_changed',
183                         entry_id=entry.id,

```



```

184 |             name=entry.name,
185 |             description=desc,
186 |             previous_value={'command': old_ent
ry.command, 'executable_path': old_entry.executable_path},
187 |             current_value={'command': entry.co
mmand, 'executable_path': entry.executable_path},
188 |             timestamp=now_ts,
189 |         )
190 |     )
191 |
192 |     # Изменение уровня риска
193 |     old_risk = old_entry.risk_level.value if hasat
tr(old_entry.risk_level, 'value') else str(old_entry.risk_level)
194 |     new_risk = entry.risk_level.value if hasattr(e
ntry.risk_level, 'value') else str(entry.risk_level)
195 |     if old_risk != new_risk:
196 |         desc = f"Изменен уровень риска «{entry.nam
e}»: было {old_risk.upper()} -> стало {new_risk.upper()}"
197 |         changes.append(
198 |             StartupChangeItem(
199 |                 change_type='risk_changed',
200 |                 entry_id=entry.id,
201 |                 name=entry.name,
202 |                 description=desc,
203 |                 previous_value={'risk_level': old_
risk, 'reasons': old_entry.risk_reasons},
204 |                 current_value={'risk_level': new_r
isk, 'reasons': entry.risk_reasons},
205 |                 timestamp=now_ts,
206 |             )
207 |         )
208 |
209 |     return changes
210 |
211 |     def archive_report(
212 |         self,
213 |         report: AuditReport,

```

```

214         auto_diff: bool = True,
215     ) -> StartupArchiveEntry:
216         """Сохранение снимка аудита автозапуска в архив с фикс
ацией изменений.
217
218         Args:
219             report: Отчет аудита автозапуска.
220             auto_diff: Выполнять ли автоматическое вычисление
изменений.
221
222         Returns:
223             StartupArchiveEntry: Созданная запись архива.
224         """
225         changes: List[StartupChangeItem] = []
226         if auto_diff:
227             changes = self.detect_changes(report)
228             if changes:
229                 logger.info(f"Зафиксировано {len(changes)} изм
енений в автозагрузке Windows!")
230                 for c in changes:
231                     logger.info(f" [Автозапуск] {c.change_type
.upper()}: {c.description}")
232
233             archive_id = f"startup_{datetime.now(timezone.utc).str
ftime('%Y%m%d_%H%M%S')}__{uuid.uuid4().hex[:6]}"
234             timestamp = datetime.now(timezone.utc).isoformat()
235             entry = StartupArchiveEntry(
236                 archive_id=archive_id,
237                 timestamp=timestamp,
238                 total_entries=len(report.entries),
239                 health_score=report.summary.health_score if report
.summary else 100,
240                 changes_count=len(changes),
241                 changes=changes,
242                 report=report,
243             )
244
245             archive_file = self.archive_dir / f'{archive_id}.json'

```

```

246         try:
247             with open(archive_file, 'w', encoding='utf-8') as
f:
248                 json.dump(entry.model_dump(mode='json'), f, en
sure_ascii=False, indent=2)
249             except Exception as ex:
250                 logger.error(f'Ошибка сохранения архива автозапуск
а в файл: {ex}')
251
252         index_entry = {
253             'archive_id': archive_id,
254             'timestamp': timestamp,
255             'total_entries': entry.total_entries,
256             'health_score': entry.health_score,
257             'changes_count': entry.changes_count,
258             'file_name': f'{archive_id}.json',
259         }
260         if self.use_file_index:
261             try:
262                 with open(self.index_file, 'a', encoding='utf-
8') as f:
263                     f.write(json.dumps(index_entry, ensure_asc
ii=False) + '\n')
264             except Exception as ex:
265                 logger.error(f'Ошибка записи индекса архивов а
втозапуска: {ex}')
266
267         self._cached_latest_report = report
268         self._rotate_archives()
269         return entry
270
271     def record_snapshot(
272         self,
273         report: AuditReport,
274         auto_diff: bool = True,
275     ) -> tuple[StartupArchiveEntry, List[StartupChangeItem]]:
276         """Сохранение снимка автозапуска с возвратом кортежа (
архивная запись, список изменений).

```

```

277
278     Args:
279         report: Отчет аудита автозапуска.
280         auto_diff: Выполнять ли вычисление изменений.
281
282     Returns:
283         tuple[StartupArchiveEntry, List[StartupChangeItem]
284     ]: Кортеж архива и списка изменений.
285
286     """
287     archive_entry = self.archive_report(report, auto_diff=
288     auto_diff)
289     return archive_entry, archive_entry.changes
290
291     def _rotate_archives(self) -> None:
292         """Ротация старых архивных файлов при превышении лимит
293         а."""
294         try:
295             json_files = sorted(self.archive_dir.glob('startup
296             _*.json'), key=os.path.getmtime)
297             if len(json_files) > self.max_archives:
298                 to_delete = json_files[:len(json_files) - self
299                 .max_archives]
300                 for p in to_delete:
301                     try:
302                         p.unlink()
303                     except OSError:
304                         pass
305             except Exception as ex:
306                 logger.debug(f'Ошибка при ротации архивов автозапу
307                 ска: {ex}')
308
309     def get_latest_archive(self) -> Optional[StartupArchiveEnt
310     ry]:
311         """Получение самого свежего архива автозапуска из хран
312         илища.
313
314     Returns:
315         Optional[StartupArchiveEntry]: Последний архив или

```

```

None.
307     """
308     try:
309         json_files = sorted(self.archive_dir.glob('startup
_**.json'), key=os.path.getmtime, reverse=True)
310         if json_files:
311             latest_path = json_files[0]
312             with open(latest_path, 'r', encoding='utf-8')
as f:
313                 data = json.load(f)
314                 return StartupArchiveEntry.model_validate(
data)
315     except Exception as ex:
316         logger.error(f'Ошибка чтения последнего архива авт
озапуска: {ex}', exc_info=True)
317     return None
318
319     def get_history(self, limit: int = 50) -> List[Dict[str, A
ny]]:
320         """Получение сводного списка сохраненных архивов автоз
апуска.
321
322         Args:
323             limit: Максимальное количество записей архива.
324
325         Returns:
326             List[Dict[str, Any]]: Список метаданных архивов.
327         """
328         records: List[Dict[str, Any]] = []
329         if not self.index_file.exists():
330             return records
331         try:
332             with open(self.index_file, 'r', encoding='utf-8')
as f:
333                 for line in f:
334                     line = line.strip()
335                     if line:
336                         try:

```

```

337         records.append(json.loads(line))
338     except json.JSONDecodeError:
339         continue
340     except Exception as ex:
341         logger.debug(f'Ошибка чтения индекса архивов автоз
апуска: {ex}')
342     return list(reversed(records))[:limit]
343
344     def get_archive_by_id(self, archive_id: str) -> Optional[S
tartupArchiveEntry]:
345         """Получение конкретного полного архивного снимка по I
D.
346
347         Args:
348             archive_id: Идентификатор архива.
349
350         Returns:
351             Optional[StartupArchiveEntry]: Найденный архив или
None.
352         """
353         safe_id = Path(archive_id).name
354         target = self.archive_dir / f'{safe_id}.json'
355         if not target.exists():
356             target = self.archive_dir / safe_id
357         if target.exists():
358             try:
359                 with open(target, 'r', encoding='utf-8') as f:
360                     data = json.load(f)
361                 return StartupArchiveEntry.model_validate(
data)
362             except Exception as ex:
363                 logger.debug(f'Ошибка загрузки архива автозапу
ска {archive_id}: {ex}')
364             return None
365
366     def get_change_timeline(self, limit: int = 100) -> List[St
artupChangeItem]:
367         """Получение сводного списка всех изменений автозапуск

```

а из архивов.

```
368
369     Args:
370         limit: Лимит изменений для возврата.
371
372     Returns:
373         List[StartupChangeItem]: Хронологический список за
374         фиксированных изменений.
375         """
376         all_changes: List[StartupChangeItem] = []
377         json_files = sorted(self.archive_dir.glob('startup_*.j
378         son'), key=os.path.getmtime, reverse=True)
379         for p in json_files:
380             try:
381                 with open(p, 'r', encoding='utf-8') as f:
382                     data = json.load(f)
383                     entry = StartupArchiveEntry.model_validate
384                     (data)
385                     if entry.changes:
386                         all_changes.extend(entry.changes)
387                         if len(all_changes) >= limit:
388                             break
389             except Exception:
390                 continue
391         return all_changes[:limit]
392
393     # Алиас для удобства
394     get_changes = get_change_timeline
```

app.js

```
1 /**
2  * =====
3  * Process Name: Windows Web - App Script
4  * =====
5  * Description:
6  *   Клиентский скрипт управления интерфейсом модуля app.
7  *
8  * Usage Examples:
9  *   HTML Integration:
10 *     <script src="/windows/telemetry_research/web/app.js?v=2
11 *       0261001_v1" type="module"></script>
12 *
13 * File: app.js
14 * Project: ai-breadboard
15 * Package: windows/telemetry_research/web
16 * Author: hypo69
17 * Copyright: © 2026 hypo69
18 * Updated: 2026-10-04 07:49:00
19 * =====
20 */
21 /**
22 * Клиентский контроллер Web GUI Исследования и Мониторинга Те
23 * леметрии Windows.
24 * Поддерживает живой мониторинг telemetry.db, ресурсы програм
25 * м клиента, глубокий анализ и AI-ассистента с Gemini.
26 */
27
28 let currentReport = null;
29 let currentClientSummary = null;
30 let researchCharts = {};
31 let liveCharts = {};
32 let aiDynamicChart = null;
```



```

31 let currentPage = 1;
32 const pageSize = 20;
33 let totalPages = 1;
34 let searchQuery = "";
35 let liveRefreshTimer = null;
36
37 document.addEventListener("DOMContentLoaded", () => {
38     initTabs();
39     bindEvents();
40     initAiChat();
41     loadSources();
42     initLiveCharts();
43     loadCurrentState();
44     setupAutoRefresh();
45     runResearch();
46 });
47
48 /**
49  * Инициализация переключения вкладок.
50  */
51 function initTabs() {
52     const tabBtns = document.querySelectorAll(".tab-btn");
53     tabBtns.forEach((btn) => {
54         btn.addEventListener("click", () => {
55             tabBtns.forEach((b) => b.classList.remove("active"));
56             document.querySelectorAll(".tab-content").forEach((c) =>
57                 c.classList.remove("active"));
58
59             btn.classList.add("active");
60             const targetId = btn.getAttribute("data-tab");
61             const targetContent = document.getElementById(targetId);
62             if (targetContent) {
63                 targetContent.classList.add("active");
64             }
65
66             if (targetId === "tab-current") {
67                 loadCurrentState();
68             } else if (targetId === "tab-client-resources") {

```

```
68 |         loadClientResources(false);
69 |     } else if (targetId === "tab-records") {
70 |         loadRecords();
71 |     }
72 | });
73 | });
74 | }
75 |
76 | /**
77 |  * Привязка событий элементов управления.
78 |  */
79 | function bindEvents() {
80 |     const btnRun = document.getElementById("btn-run");
81 |     if (btnRun) {
82 |         btnRun.onclick = () => runResearch();
83 |     }
84 |
85 |     const selectSource = document.getElementById("select-source"
86 | );
87 |     if (selectSource) {
88 |         selectSource.onchange = () => {
89 |             runResearch();
90 |             loadCurrentState();
91 |             loadClientResources(false);
92 |         };
93 |     }
94 |
95 |     const btnDashboard = document.getElementById("btn-dashboard"
96 | );
97 |     if (btnDashboard) {
98 |         btnDashboard.onclick = () => openHtmlDashboard();
99 |     }
100 |
101 |     const btnJson = document.getElementById("btn-json");
102 |     if (btnJson) {
103 |         btnJson.onclick = () => downloadJsonReport();
104 |     }
```

```
104 |   const btnRefreshCurrent = document.getElementById("btn-refre
    | sh-current");
105 |   if (btnRefreshCurrent) {
106 |     btnRefreshCurrent.onclick = () => loadCurrentState();
107 |   }
108 |
109 |   const btnRefreshClient = document.getElementById("btn-refres
    | h-client-procs");
110 |   if (btnRefreshClient) {
111 |     btnRefreshClient.onclick = () => loadClientResources(false
    | );
112 |   }
113 |
114 |   const btnAskGeminiAll = document.getElementById("btn-ask-gem
    | ini-all-unknown");
115 |   if (btnAskGeminiAll) {
116 |     btnAskGeminiAll.onclick = () => loadClientResources(true);
117 |   }
118 |
119 |   const inputSearchClient = document.getElementById("input-sea
    | rch-client-procs");
120 |   if (inputSearchClient) {
121 |     inputSearchClient.oninput = (e) => filterClientProcsTable(
    | e.target.value);
122 |   }
123 |   if (btnRefreshCurrent) {
124 |     btnRefreshCurrent.onclick = () => loadCurrentState();
125 |   }
126 |
127 |   const rangeSelect = document.getElementById("current-range-s
    | elect");
128 |   if (rangeSelect) {
129 |     rangeSelect.onchange = () => loadCurrentState();
130 |   }
131 |
132 |   const autoRefreshToggle = document.getElementById("auto-refr
    | esh-toggle");
133 |   if (autoRefreshToggle) {
```

```
134     autoRefreshToggle.onChange = () => setupAutoRefresh();
135 }
136
137 const searchInput = document.getElementById("input-search");
138 if (searchInput) {
139     searchInput.onkeydown = (e) => {
140         if (e.key === "Enter") {
141             searchQuery = searchInput.value.trim();
142             currentPage = 1;
143             loadRecords();
144         }
145     };
146 }
147
148 const prevBtn = document.getElementById("btn-prev-page");
149 if (prevBtn) {
150     prevBtn.onclick = () => {
151         if (currentPage > 1) {
152             currentPage--;
153             loadRecords();
154         }
155     };
156 }
157
158 const nextBtn = document.getElementById("btn-next-page");
159 if (nextBtn) {
160     nextBtn.onclick = () => {
161         if (currentPage < totalPages) {
162             currentPage++;
163             loadRecords();
164         }
165     };
166 }
167 }
168
169 /**
170  * Инициализация обработчиков AI-чата и панели ввода.
171  */
```

```
172 function initAiChat() {
173   const form = document.getElementById("ai-query-form");
174   const input = document.getElementById("ai-query-input");
175   const btnClear = document.getElementById("btn-ai-clear");
176   const btnClose = document.getElementById("btn-ai-close");
177   const responseCard = document.getElementById("ai-response-card");
178
179   if (input) {
180     input.addEventListener("input", () => {
181       if (btnClear) {
182         btnClear.style.display = input.value.length > 0 ? "block" : "none";
183       }
184     });
185
186     input.addEventListener("keydown", (e) => {
187       if (e.key === "Enter" && !e.shiftKey) {
188         e.preventDefault();
189         submitAiQuery(input.value.trim());
190       }
191     });
192   }
193
194   if (btnClear && input) {
195     btnClear.addEventListener("click", () => {
196       input.value = "";
197       btnClear.style.display = "none";
198       input.focus();
199     });
200   }
201
202   if (form && input) {
203     form.addEventListener("submit", (e) => {
204       e.preventDefault();
205       submitAiQuery(input.value.trim());
206     });
207   }
```

```
208
209 if (btnClose && responseCard) {
210     btnClose.addEventListener("click", () => {
211         responseCard.style.display = "none";
212     });
213 }
214
215 // Привязка быстрых чипов
216 const chips = document.querySelectorAll(".prompt-chip");
217 chips.forEach((chip) => {
218     chip.addEventListener("click", () => {
219         const q = chip.getAttribute("data-query");
220         if (q) {
221             if (input) input.value = q;
222             submitAiQuery(q);
223         }
224     });
225 });
226 }
227
228 /**
229  * Отправка запроса к AI-ассистенту телеметрии.
230  */
231 async function submitAiQuery(queryText) {
232     if (!queryText) return;
233
234     const input = document.getElementById("ai-query-input");
235     const btnSend = document.getElementById("btn-ai-send");
236     const btnText = document.getElementById("btn-ai-send-text");
237     const spinner = document.getElementById("btn-ai-spinner");
238     const responseCard = document.getElementById("ai-response-card");
239     const selectSource = document.getElementById("select-source");
240     const sourcePath = selectSource ? selectSource.value : null;
241
242     // Установка состояния загрузки
243     if (btnSend) btnSend.disabled = true;
```

```

244 |   if (btnText) btnText.style.display = "none";
245 |   if (spinner) spinner.style.display = "inline-block";
246 |
247 |   try {
248 |     const res = await fetch("/api/chat", {
249 |       method: "POST",
250 |       headers: { "Content-Type": "application/json" },
251 |       body: JSON.stringify({
252 |         message: queryText,
253 |         source_path: sourcePath || null,
254 |       }),
255 |     });
256 |
257 |     if (!res.ok) throw new Error(`HTTP ${res.status}`);
258 |     const data = await res.json();
259 |
260 |     renderAiResponse(queryText, data);
261 |   } catch (err) {
262 |     console.error("Ошибка при выполнении AI запроса:", err);
263 |     if (responseCard) {
264 |       responseCard.style.display = "block";
265 |       const replyBody = document.getElementById("ai-reply-body
266 | ");
267 |       if (replyBody) {
268 |         replyBody.innerHTML = `<div style="color: var(--accent
269 | -red); padding: 10px;">❌ Ошибка соединения с сервером AI-ассис
270 | тента: ${err.message}</div>`;
271 |       }
272 |     }
273 |   } finally {
274 |     if (btnSend) btnSend.disabled = false;
275 |     if (btnText) btnText.style.display = "inline";
276 |     if (spinner) spinner.style.display = "none";
277 |   }
278 | }
279 |
280 | /**
281 |  * Рендеринг ответа AI-ассистента, графиков Chart.js и выполне

```

```

        нных SQL.
279     */
280     function renderAiResponse(queryText, data) {
281         const responseCard = document.getElementById("ai-response-card");
282         const titleEl = document.getElementById("ai-response-title");
283         const chartWrapper = document.getElementById("ai-chart-wrapper");
284         const chartTitle = document.getElementById("ai-chart-title");
285         const metricsRow = document.getElementById("ai-metrics-row");
286         const replyBody = document.getElementById("ai-reply-body");
287         const sqlDetails = document.getElementById("ai-sql-details");
288         const sqlCode = document.getElementById("ai-sql-code");
289         const followupRow = document.getElementById("ai-followup-row");
290         const followupChips = document.getElementById("ai-followup-chips");
291
292         if (!responseCard) return;
293         responseCard.style.display = "block";
294
295         if (titleEl) {
296             titleEl.textContent = queryText.length > 60 ? queryText.substring(0, 60) + "..." : queryText;
297         }
298
299         // 1. Форматирование текста ответа
300         if (replyBody) {
301             replyBody.innerHTML = formatMarkdownToHtml(data.reply || "");
302         }
303
304         // 2. Рендеринг динамического графика Chart.js
305         if (data.chart && data.chart.datasets && data.chart.datasets

```



```
.length > 0) {
306     chartWrapper.style.display = "block";
307     if (chartTitle) chartTitle.textContent = data.chart.title
    || "□ Визуализация данных";
308
309     renderDynamicAiChart(data.chart);
310 } else {
311     chartWrapper.style.display = "none";
312     if (aiDynamicChart) {
313         aiDynamicChart.destroy();
314         aiDynamicChart = null;
315     }
316 }
317
318 // 3. Метрики KPI
319 if (data.metrics_summary && Object.keys(data.metrics_summary
).length > 0) {
320     metricsRow.style.display = "flex";
321     metricsRow.innerHTML = "";
322     Object.entries(data.metrics_summary).forEach(([k, v]) => {
323         const pill = document.createElement("div");
324         pill.className = "ai-metric-pill";
325         const cleanKey = k.replace(/_/g, " ");
326         pill.innerHTML = `<span>${cleanKey}</span> <strong>${v}
</strong>`;
327         metricsRow.appendChild(pill);
328     });
329 } else {
330     metricsRow.style.display = "none";
331 }
332
333 // 4. SQL-запросы
334 if (data.sql_queries && data.sql_queries.length > 0 && data.
sql_queries[0]) {
335     sqlDetails.style.display = "block";
336     if (sqlCode) {
337         sqlCode.textContent = data.sql_queries.join("\n\n");
338     }
```

```
339 | } else {
340 |     sqlDetails.style.display = "none";
341 | }
342 |
343 | // 5. Быстрые уточняющие вопросы
344 | if (data.quick_followups && data.quick_followups.length > 0)
    | {
345 |     followupRow.style.display = "flex";
346 |     if (followupChips) {
347 |         followupChips.innerHTML = "";
348 |         data.quick_followups.forEach((fText) => {
349 |             const btn = document.createElement("button");
350 |             btn.className = "prompt-chip";
351 |             btn.textContent = fText;
352 |             btn.onclick = () => {
353 |                 const input = document.getElementById("ai-query-inpu
    | t");
354 |                 if (input) input.value = fText;
355 |                 submitAiQuery(fText);
356 |             };
357 |             followupChips.appendChild(btn);
358 |         });
359 |     }
360 | } else {
361 |     followupRow.style.display = "none";
362 | }
363 |
364 | // Прокрутка к ответу
365 | responseCard.scrollToView({ behavior: "smooth", block: "st
    | art" });
366 | }
367 |
368 | /**
369 |  * Динамический рендеринг любого типа графика через Chart.js.
370 |  */
371 | function renderDynamicAiChart(chartConfig) {
372 |     const canvas = document.getElementById("chart-ai-dynamic");
373 |     if (!canvas) return;
```

```
374 |
375 | if (aiDynamicChart) {
376 |     aiDynamicChart.destroy();
377 |     aiDynamicChart = null;
378 | }
379 |
380 | const ctx = canvas.getContext("2d");
381 | const chartType = chartConfig.type || "line";
382 |
383 | const options = {
384 |     responsive: true,
385 |     maintainAspectRatio: false,
386 |     animation: { duration: 500 },
387 |     plugins: {
388 |         legend: {
389 |             position: chartType === "doughnut" || chartType === "p
390 | ie" ? "right" : "top",
391 |             labels: { color: "#94a3b8", font: { size: 12 } },
392 |             tooltip: {
393 |                 backgroundColor: "#131b2e",
394 |                 titleColor: "#38bdf8",
395 |                 bodyColor: "#f1f5f9",
396 |                 borderColor: "#23304d",
397 |                 borderWidth: 1,
398 |             },
399 |         },
400 |     };
401 |
402 | if (chartType === "line" || chartType === "bar") {
403 |     options.scales = {
404 |         x: {
405 |             grid: { color: "rgba(35, 48, 77, 0.4)" },
406 |             ticks: { color: "#94a3b8", maxRotation: 0, autoSkip: t
407 | rue, maxTicksLimit: 10 },
408 |         },
409 |         y: {
410 |             grid: { color: "rgba(35, 48, 77, 0.4)" },
```

```

410         ticks: { color: "#94a3b8" },
411     },
412 };
413 }
414
415 aiDynamicChart = new Chart(ctx, {
416     type: chartType,
417     data: {
418         labels: chartConfig.labels,
419         datasets: chartConfig.datasets,
420     },
421     options: options,
422 });
423 }
424
425 /**
426  * Простой и надежный Markdown парсер для форматирования ответов
    |  AI.
427  */
428 function formatMarkdownToHtml(md) {
429     if (!md) return "";
430     let html = md
431         .replace(/&/g, "&")
432         .replace(/</g, "<")
433         .replace(/>/g, ">");
434
435     // Заголовки
436     html = html.replace(/^### (.*)/gim, "<h3>$1</h3>");
437     html = html.replace(/^#### (.*)/gim, "<h4>$1</h4>");
438     html = html.replace(/^## (.*)/gim, "<h2>$1</h2>");
439     html = html.replace(/^# (.*)/gim, "<h1>$1</h1>");
440
441     // Жирный шрифт и курсив
442     html = html.replace(/\*(.*?)\*/gim, "<strong>$1</strong>");
443     html = html.replace(/_(.*?)_/gim, "<em>$1</em>");
444     html = html.replace(/`([^\`]+)`/gim, "<code>$1</code>");
445

```

```
446 // Таблицы Markdown
447 const tableRegex = /(?:\|[\^\\n]+\|\\r?\n)((?:\|[\^\\n]+\|\\r?\n)+
|)/g;
448 html = html.replace(tableRegex, (match) => {
449     const lines = match.trim().split("\n");
450     if (lines.length < 2) return match;
451
452     let tableHtml = "<table>";
453     let isHeader = true;
454
455     for (let line of lines) {
456         if (line.includes("---")) {
457             isHeader = false;
458             continue;
459         }
460         const cells = line
461             .split("|")
462             .slice(1, -1)
463             .map((c) => c.trim());
464
465         tableHtml += "<tr>";
466         for (let cell of cells) {
467             const tag = isHeader ? "th" : "td";
468             tableHtml += `<${tag}>${cell}</${tag}>`;
469         }
470         tableHtml += "</tr>";
471         isHeader = false;
472     }
473     tableHtml += "</table>";
474     return tableHtml;
475 });
476
477 // Списки
478 html = html.replace(/^\\- (.*)/gim, "<li>$1</li>");
479 html = html.replace(/(<li>.*<\\li>)/gim, "<ul>$1</ul>");
480 html = html.replace(/<\\ul>\\s*<ul>/gim, "");
481
482 // Абзацы и переводы строк
```

```
483 |   html = html.replace(/\n\n+/g, "<br/><br/>");
484 |
485 |   return html;
486 | }
487 |
488 | /**
489 |  * Настройка периодического автообновления живого мониторинга.
490 |  */
491 | function setupAutoRefresh() {
492 |   if (liveRefreshTimer) {
493 |     clearInterval(liveRefreshTimer);
494 |     liveRefreshTimer = null;
495 |   }
496 |   const toggle = document.getElementById("auto-refresh-toggle"
497 | );
498 |   if (toggle && toggle.checked) {
499 |     liveRefreshTimer = setInterval(() => {
500 |       const currentTab = document.getElementById("tab-current"
501 | );
502 |       if (currentTab && currentTab.classList.contains("active"
503 | )) {
504 |         loadCurrentState(true);
505 |       }
506 |     }, 3000);
507 |   }
508 | }
509 |
510 | /**
511 |  * Загрузка списка доступных файлов и БД телеметрии.
512 |  */
513 | async function loadSources() {
514 |   const select = document.getElementById("select-source");
515 |   if (!select) return;
516 |   try {
517 |     const res = await fetch("/api/sources");
518 |     if (!res.ok) return;
519 |     const files = await res.json();
```

```
518     select.innerHTML = '<option value="">Все доступные источни  
ки (telemetry.db + логи)</option>';  
519     files.forEach((f) => {  
520         const opt = document.createElement("option");  
521         opt.value = f.path;  
522         const sizeMb = (f.size_bytes / (1024 * 1024)).toFixed(1)  
    ;  
523         opt.textContent = `${f.name} (${sizeMb} MB)`;  
524         select.appendChild(opt);  
525     });  
526 } catch (err) {  
527     console.warn("Ошибка загрузки источников:", err);  
528 }  
529 }  
530  
531 /**  
532  * Инициализация пустых экземпляров графиков Chart.js для теку  
щего состояния.  
533  */  
534 function initLiveCharts() {  
535     const commonOptions = {  
536         responsive: true,  
537         maintainAspectRatio: false,  
538         animation: { duration: 300 },  
539         interaction: { intersect: false, mode: "index" },  
540         plugins: {  
541             legend: { labels: { color: "#94a3b8", font: { size: 11 }  
                } },  
542             tooltip: { backgroundColor: "#131b2e", titleColor: "#38b  
df8", bodyColor: "#f1f5f9", borderColor: "#23304d", borderWidt  
h: 1 }  
543         },  
544         scales: {  
545             x: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks: {  
color: "#94a3b8", font: { size: 10 }, maxRotation: 0, autoSkip  
: true, maxTicksLimit: 8 } },  
546             y: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks: {  
color: "#94a3b8", font: { size: 10 } } }
```

```

547     }
548 };
549
550 const createChart = (id, datasets, extraY = {}) => {
551     const ctx = document.getElementById(id);
552     if (!ctx) return null;
553     const opts = JSON.parse(JSON.stringify(commonOptions));
554     if (Object.keys(extraY).length > 0) {
555         opts.scales = { ...opts.scales, ...extraY };
556     }
557     return new Chart(ctx.getContext("2d"), {
558         type: "line",
559         data: { labels: [], datasets: datasets },
560         options: opts
561     });
562 };
563
564 liveCharts.cpu = createChart("chart-live-cpu", [
565     { label: "CPU Load (%)", data: [], borderColor: "#38bdf8",
566       backgroundColor: "rgba(56, 189, 248, 0.15)", borderWidth: 2,
567       fill: true, tension: 0.3, yAxisID: "y" },
568     { label: "Частота (GHz)", data: [], borderColor: "#f59e0b"
569       , backgroundColor: "transparent", borderWidth: 2, borderDash:
570       [4, 4], tension: 0.2, yAxisID: "y1" }
571 ], {
572     y: { min: 0, max: 100, ticks: { color: "#38bdf8" } },
573     y1: { position: "right", grid: { drawOnChartArea: false },
574         ticks: { color: "#f59e0b" } }
575 });
576
577 liveCharts.ram = createChart("chart-live-ram", [
578     { label: "RAM (%)", data: [], borderColor: "#10b981", back
579       groundColor: "rgba(16, 185, 129, 0.15)", borderWidth: 2, fill:
580       true, tension: 0.3, yAxisID: "y" },
581     { label: "Занято (GB)", data: [], borderColor: "#a855f7",
582       backgroundColor: "transparent", borderWidth: 2, tension: 0.2,
583       yAxisID: "y1" }
584 ], {

```



```
576     y: { min: 0, max: 100, ticks: { color: "#10b981" } },
577     y1: { position: "right", grid: { drawOnChartArea: false },
        ticks: { color: "#a855f7" } }
578   });
579
580   liveCharts.gpu = createChart("chart-live-gpu", [
581     { label: "GPU Load (%)", data: [], borderColor: "#a855f7",
        backgroundColor: "rgba(168, 85, 247, 0.15)", borderWidth: 2,
        fill: true, tension: 0.3, yAxisID: "y" },
582     { label: "GPU Temp (°C)", data: [], borderColor: "#ef4444"
        , backgroundColor: "transparent", borderWidth: 2, tension: 0.2
        , yAxisID: "y1" }
583   ], {
584     y: { min: 0, max: 100, ticks: { color: "#a855f7" } },
585     y1: { position: "right", grid: { drawOnChartArea: false },
        ticks: { color: "#ef4444" } }
586   });
587
588   liveCharts.disk = createChart("chart-live-disk", [
589     { label: "Чтение (MB/s)", data: [], borderColor: "#06b6d4"
        , backgroundColor: "rgba(6, 182, 212, 0.15)", borderWidth: 2,
        fill: true, tension: 0.3 },
590     { label: "Запись (MB/s)", data: [], borderColor: "#ec4899"
        , backgroundColor: "rgba(236, 72, 153, 0.15)", borderWidth: 2,
        fill: true, tension: 0.3 }
591   ]);
592
593   liveCharts.net = createChart("chart-live-net", [
594     { label: "Входящий (KB/s)", data: [], borderColor: "#6366f
595     1", backgroundColor: "rgba(99, 102, 241, 0.15)", borderWidth:
        2, fill: true, tension: 0.3 },
596     { label: "Исходящий (KB/s)", data: [], borderColor: "#14b8
        a6", backgroundColor: "rgba(20, 184, 166, 0.15)", borderWidth:
        2, fill: true, tension: 0.3 }
597   ]);
598 }
599 /**
```

```

600 | * Загрузка актуального среза состояния и построение графиков
    | из telemetry.db.
601 | */
602 | async function loadCurrentState(silent = false) {
603 |   const rangeSelect = document.getElementById("current-range-s
    | elect");
604 |   const limit = rangeSelect ? rangeSelect.value : 100;
605 |   const selectSource = document.getElementById("select-source"
    | );
606 |   const dbPath = selectSource && selectSource.value && selectS
    | ource.value.endsWith(".db") ? selectSource.value : "";
607 |
608 |   try {
609 |     const url = `/api/current-state?limit=${limit}${dbPath ? `
    | &db_path=${encodeURIComponent(dbPath)}` : ""}`;
610 |     const res = await fetch(url);
611 |     if (!res.ok) throw new Error(`HTTP ${res.status}`);
612 |     const data = await res.json();
613 |
614 |     if (data.status !== "ok") {
615 |       document.getElementById("live-db-status").textContent =
    | "❏ База данных не обнаружена";
616 |       document.getElementById("live-db-sub").textContent = dat
    | a.message || "Нет данных";
617 |       return;
618 |     }
619 |
620 |     renderCurrentStatusBar(data);
621 |     renderCurrentKPI(data.latest_snapshot);
622 |     updateLiveCharts(data.time_series);
623 |     renderLiveSensors(data.sensors);
624 |     renderLiveProcesses(data.top_processes);
625 |     renderLiveActivity(data.recent_events);
626 |   } catch (err) {
627 |     if (!silent) {
628 |       console.error("Ошибка загрузки текущего состояния:", err
    | );
629 |     }

```

```

630 | }
631 | }
632 |
633 | /**
634 |  * Отображение статуса базы данных telemetry.db.
635 |  */
636 | function renderCurrentStatusBar(data) {
637 |     const statusEl = document.getElementById("live-db-status");
638 |     const subEl = document.getElementById("live-db-sub");
639 |     if (statusEl) {
640 |         statusEl.innerHTML = `❏ <strong>${data.db_name || "teleme
641 |         try.db"}</strong> (${data.db_size_mb || 0} MB)`;
642 |     }
643 |     if (subEl) {
644 |         const latestTs = data.latest_snapshot && data.latest_snap
645 |         hot.timestamp ? data.latest_snapshot.timestamp.replace("T", "
646 |         ").substring(0, 19) : "н/д";
647 |         subEl.textContent = `Всего записей: ${data.total_snapshots
648 |         .toLocaleString()} • Отображено точек: ${data.returned_points}
649 |         • Время последнего замера: ${latestTs}`;
650 |     }
651 | }
652 |
653 | /**
654 |  * Обновление мгновенных карточек текущего состояния.
655 |  */
656 | function renderCurrentKPI(snap) {
657 |     if (!snap) return;
658 |
659 |     const cpuVal = document.getElementById("curr-cpu-val");
660 |     const cpuFreq = document.getElementById("curr-cpu-freq");
661 |     const cpuMeter = document.getElementById("curr-cpu-meter");
662 |     if (cpuVal) {
663 |         const cpuPct = Number(snap.cpu_total_percent || 0).toFixed
664 |         (1);
665 |         cpuVal.textContent = `${cpuPct}%`;
666 |         if (cpuMeter) cpuMeter.style.width = `${Math.min(100, cpuP
667 |         ct)}%`;

```

```

661 | }
662 | if (cpuFreq) {
663 |     const mhz = Number(snap.cpu_frequency_mhz || 0);
664 |     cpuFreq.textContent = mhz > 0 ? `Частота: ${ (mhz / 1000).to
oFixed(2)} GHz` : "Частота: --";
665 | }
666 |
667 | const ramVal = document.getElementById("curr-ram-val");
668 | const ramGb = document.getElementById("curr-ram-gb");
669 | const ramMeter = document.getElementById("curr-ram-meter");
670 | if (ramVal) {
671 |     const ramPct = Number(snap.memory_percent || 0).toFixed(1)
;
672 |     ramVal.textContent = `${ramPct}%`;
673 |     if (ramMeter) ramMeter.style.width = `${Math.min(100, ramP
ct)}%`;
674 | }
675 | if (ramGb) {
676 |     const used = Number(snap.memory_used_gb || 0).toFixed(1);
677 |     const total = Number(snap.memory_total_gb || 0).toFixed(1)
;
678 |     ramGb.textContent = `${used} / ${total} GB (Swap: ${snap.s
wap_percent || 0}%)`;
679 | }
680 |
681 | const gpuVal = document.getElementById("curr-gpu-val");
682 | const gpuTemp = document.getElementById("curr-gpu-temp");
683 | const gpuMeter = document.getElementById("curr-gpu-meter");
684 | if (gpuVal) {
685 |     const gpuPct = Number(snap.gpu_load_percent || 0).toFixed(
1);
686 |     gpuVal.textContent = `${gpuPct}%`;
687 |     if (gpuMeter) gpuMeter.style.width = `${Math.min(100, gpuP
ct)}%`;
688 | }
689 | if (gpuTemp) {
690 |     const t = snap.gpu_temp_c;
691 |     gpuTemp.textContent = t ? `Температура: ${Number(t).toFixe

```

```

    d(1)}°C` : "Температура: --";
692     }
693
694     const diskVal = document.getElementById("curr-disk-val");
695     if (diskVal) {
696         const rb = ((snap.disk_read_bytes_sec || 0) / (1024 * 1024
        |)).toFixed(1);
697         const wb = ((snap.disk_write_bytes_sec || 0) / (1024 * 102
        |4)).toFixed(1);
698         diskVal.textContent = `R: ${rb} / W: ${wb} MB/s`;
699     }
700
701     const netVal = document.getElementById("curr-net-val");
702     const uptimeVal = document.getElementById("curr-uptime-val")
    ;
703     if (netVal) {
704         const nr = ((snap.network_recv_bytes_sec || 0) / 1024).toF
        |ixed(1);
705         const ns = ((snap.network_sent_bytes_sec || 0) / 1024).toF
        |ixed(1);
706         netVal.textContent = `↓ ${nr} / ↑ ${ns} KB/s`;
707     }
708     if (uptimeVal) {
709         const upSec = Number(snap.uptime_seconds || 0);
710         const hours = Math.floor(upSec / 3600);
711         const mins = Math.floor((upSec % 3600) / 60);
712         uptimeVal.textContent = upSec > 0 ? `Аптайм: ${hours}ч ${m
        |ins}м` : "Аптайм: --";
713     }
714 }
715
716 /**
717  * Плавное обновление временных рядов Chart.js.
718  */
719 function updateLiveCharts(ts) {
720     if (!ts || !ts.labels) return;
721     const labels = ts.labels;
722

```

```
723 | if (liveCharts.cpu) {
724 |     liveCharts.cpu.data.labels = labels;
725 |     liveCharts.cpu.data.datasets[0].data = ts.cpu_load || [];
726 |     liveCharts.cpu.data.datasets[1].data = ts.cpu_freq_ghz ||
    | [];
727 |     liveCharts.cpu.update("none");
728 | }
729 |
730 | if (liveCharts.ram) {
731 |     liveCharts.ram.data.labels = labels;
732 |     liveCharts.ram.data.datasets[0].data = ts.ram_percent || [
    | ];
733 |     liveCharts.ram.data.datasets[1].data = ts.ram_used_gb || [
    | ];
734 |     liveCharts.ram.update("none");
735 | }
736 |
737 | if (liveCharts.gpu) {
738 |     liveCharts.gpu.data.labels = labels;
739 |     liveCharts.gpu.data.datasets[0].data = ts.gpu_load || [];
740 |     liveCharts.gpu.data.datasets[1].data = ts.gpu_temp || [];
741 |     liveCharts.gpu.update("none");
742 | }
743 |
744 | if (liveCharts.disk) {
745 |     liveCharts.disk.data.labels = labels;
746 |     liveCharts.disk.data.datasets[0].data = ts.disk_read_mb ||
    | [];
747 |     liveCharts.disk.data.datasets[1].data = ts.disk_write_mb |
    | | [];
748 |     liveCharts.disk.update("none");
749 | }
750 |
751 | if (liveCharts.net) {
752 |     liveCharts.net.data.labels = labels;
753 |     liveCharts.net.data.datasets[0].data = ts.net_recv_kb || [
    | ];
754 |     liveCharts.net.data.datasets[1].data = ts.net_sent_kb || [
```

```

    ];
755     liveCharts.net.update("none");
756   }
757 }
758
759 /**
760  * Отображение плиток датчиков и сенсоров.
761  */
762 function renderLiveSensors(sensors) {
763   const container = document.getElementById("live-sensors-cont
ainer");
764   if (!container) return;
765   if (!sensors || sensors.length === 0) {
766     container.innerHTML = '<div style="color: var(--text-secon
dary); grid-column: span 2;">Сенсоры LHM не зафиксированы в БД
</div>';
767     return;
768   }
769   container.innerHTML = "";
770   sensors.slice(0, 16).forEach((s) => {
771     const tile = document.createElement("div");
772     tile.style.background = "rgba(11, 15, 25, 0.6)";
773     tile.style.border = "1px solid var(--border-color)";
774     tile.style.borderRadius = "8px";
775     tile.style.padding = "8px 10px";
776     tile.style.display = "flex";
777     tile.style.justifyContent = "space-between";
778     tile.style.alignItems = "center";
779
780     const name = `${s.hardware_name ? s.hardware_name + " - "
: ""}${s.sensor_name}`;
781     const val = `${s.value} ${s.unit || ""}`;
782     tile.innerHTML = `
783       <div style="color: var(--text-secondary); font-size: 11p
x; max-width: 140px; overflow: hidden; text-overflow: ellipsis
; white-space: nowrap;" title="${name}">${name}</div>
784       <div style="font-weight: 700; color: var(--accent-blue);
">${val}</div>

```

```

785     `;
786     container.appendChild(tile);
787 });
788 }
789
790 /**
791  * Отображение таблицы топ активных процессов.
792  */
793 function renderLiveProcesses(processes) {
794     const tbody = document.getElementById("live-processes-body")
795     ;
796     if (!tbody) return;
797     if (!processes || processes.length === 0) {
798         tbody.innerHTML = '<tr><td colspan="5" style="text-align:
799         center; color: var(--text-secondary);">Нет записей о процессах
800         в выборке</td></tr>';
801         return;
802     }
803     tbody.innerHTML = "";
804     processes.forEach((p) => {
805         const tr = document.createElement("tr");
806         const cpuPct = Number(p.cpu_percent || 0).toFixed(1);
807         const memMb = Number(p.memory_mb || 0).toFixed(1);
808         tr.innerHTML = `
809             <td><code>${p.pid}</code></td>
810             <td><strong>${p.name}</strong></td>
811             <td style="color: ${cpuPct > 20 ? 'var(--accent-amber)'
812             : 'inherit'}; font-weight: ${cpuPct > 20 ? '700' : 'normal'};"
813             >${cpuPct}%</td>
814             <td>${memMb} MB</td>
815             <td>${p.num_threads || 1}</td>
816         `;
817         tbody.appendChild(tr);
818     });
819 }
820
821 /**
822  * Отображение ленты системных событий.

```



```

818 |  */
819 | function renderLiveActivity(events) {
820 |     const list = document.getElementById("live-activity-list");
821 |     if (!list) return;
822 |     if (!events || events.length === 0) {
823 |         list.innerHTML = '<li style="color: var(--text-secondary);
      | text-align: center; padding: 10px;">События не зафиксированы<
      | /li>';
824 |         return;
825 |     }
826 |     list.innerHTML = "";
827 |     events.forEach((ev) => {
828 |         const li = document.createElement("li");
829 |         li.className = "activity-feed-item";
830 |         const ts = (ev.timestamp || "").replace("T", " ").substri
      | ng(11, 19);
831 |         const tagClass = ev.source === "device" ? "activity-tag-de
      | vice" : "activity-tag-system";
832 |         const desc = ev.friendly_name || ev.name || ev.path || ev.
      | event_type || "Событие";
833 |         li.innerHTML = `
834 |             <div class="activity-feed-left">
835 |                 <span style="font-weight: 600;">${desc}</span>
836 |                 <span style="color: var(--text-secondary); font-size:
      | 11px;">${ev.category || ev.provider || ev.event_type || ""}</s
      | pan>
837 |             </div>
838 |             <div style="text-align: right;">
839 |                 <span class="activity-tag ${tagClass}">${ev.event_type
      | || "EVT"}</span>
840 |                 <div style="font-size: 10px; color: var(--text-seconda
      | ry); margin-top: 4px;">${ts}</div>
841 |             </div>
842 |         `;
843 |         list.appendChild(li);
844 |     });
845 | }
846 |

```

```

847 /**
848  * Запуск глубокого исследования телеметрии.
849  */
850 async function runResearch() {
851     const btnRun = document.getElementById("btn-run");
852     const selectSource = document.getElementById("select-source"
853 );
854     const sourcePath = selectSource ? selectSource.value : null;
855     if (btnRun) {
856         btnRun.disabled = true;
857         btnRun.innerHTML = "⏳ Анализ...";
858     }
859
860     try {
861         const res = await fetch("/api/run-research", {
862             method: "POST",
863             headers: { "Content-Type": "application/json" },
864             body: JSON.stringify({ source_path: sourcePath || null }
865 ),
866         });
867         if (!res.ok) throw new Error(`HTTP ${res.status}`);
868         const report = await res.json();
869         currentReport = report;
870
871         renderKPI(report);
872         renderResearchCharts(report.base_report.charts);
873         renderHypotheses(report.hypotheses);
874         renderCorrelations(report.correlations);
875         renderAnomalies(report.base_report.anomalies, report.base_
876 report.summary_conclusions);
877     } catch (err) {
878         console.error("Ошибка при исследовании телеметрии:", err);
879     } finally {
880         if (btnRun) {
881             btnRun.disabled = false;
882             btnRun.innerHTML = "⏳ Анализ";

```

```
882     }
883   }
884 }
885
886 /**
887  * Рендеринг сводных KPI карточек исследования.
888  */
889 function renderKPI(report) {
890   const base = report.base_report;
891   const healthEl = document.getElementById("kpi-health");
892   const recordsEl = document.getElementById("kpi-records");
893   const cpuEl = document.getElementById("kpi-cpu-val");
894   const ramEl = document.getElementById("kpi-ram-val");
895   const anomaliesEl = document.getElementById("kpi-anomalies")
896   ;
897   const devicesEl = document.getElementById("kpi-devices");
898
899   if (healthEl) {
900     healthEl.textContent = `${base.health_score}%`;
901     healthEl.style.color = base.health_score >= 80 ? "var(--accent-green)" : base.health_score >= 50 ? "var(--accent-amber)"
902     : "var(--accent-red)";
903   }
904
905   if (recordsEl) {
906     recordsEl.textContent = base.records_analyzed.toLocaleString();
907   }
908
909   if (cpuEl) {
910     const stats = base.statistics || {};
911     const cpuStat = stats.cpu_load || stats.cpu_total_percent;
912     if (cpuStat && cpuStat.count > 0) {
913       cpuEl.textContent = `CPU: ${cpuStat.avg_val}% (Max: ${cpuStat.max_val})`;
914     } else {
915       cpuEl.textContent = "CPU: --%";
916     }
917   }
```

```
915 | }
916 |
917 | if (ramEl) {
918 |     const stats = base.statistics || {};
919 |     const ramStat = stats.ram_used_percent || stats.memory_per
cent;
920 |     if (ramStat && ramStat.count > 0) {
921 |         ramEl.textContent = `RAM: ${ramStat.avg_val}% (Max: ${ra
mStat.max_val}%)`;
922 |     } else {
923 |         ramEl.textContent = "RAM: --%";
924 |     }
925 | }
926 |
927 | if (anomaliesEl) {
928 |     const count = (base.anomalies ? base.anomalies.length : 0)
+ (base.device_summary ? base.device_summary.error_count : 0)
;
929 |     anomaliesEl.textContent = count;
930 |     anomaliesEl.style.color = count === 0 ? "var(--accent-gree
n)" : count > 5 ? "var(--accent-red)" : "var(--accent-amber)";
931 | }
932 |
933 | if (devicesEl) {
934 |     const dev = base.device_summary;
935 |     if (dev && dev.error_count > 0) {
936 |         devicesEl.textContent = `Сбоев оборудования: ${dev.error
_count}`;
937 |         devicesEl.style.color = "var(--accent-red)";
938 |     } else {
939 |         devicesEl.textContent = "Оборудование в норме";
940 |         devicesEl.style.color = "var(--text-secondary)";
941 |     }
942 | }
943 | }
944 |
945 | /**
946 |  * Рендеринг графиков вкладки Исследования.
```

```
947 | */
948 | function renderResearchCharts(charts) {
949 |     const container = document.getElementById("charts-container"
    | );
950 |     if (!container) return;
951 |
952 |     Object.values(researchCharts).forEach((chart) => chart.destr
    | oy());
953 |     researchCharts = {};
954 |     container.innerHTML = "";
955 |
956 |     if (!charts || charts.length === 0) {
957 |         container.innerHTML = '<div class="kpi-card" style="grid-c
    | olumn: 1 / -1; text-align: center; padding: 40px;">Нет данных
    | для построения графиков исследования</div>';
958 |         return;
959 |     }
960 |
961 |     charts.forEach((cfg, idx) => {
962 |         const card = document.createElement("div");
963 |         card.className = "chart-card";
964 |
965 |         const header = document.createElement("div");
966 |         header.className = "chart-header";
967 |         header.innerHTML = `<h3 class="chart-title">${cfg.title}</
    | h3>`;
968 |
969 |         const wrapper = document.createElement("div");
970 |         wrapper.className = "chart-canvas-wrapper";
971 |
972 |         const canvas = document.createElement("canvas");
973 |         canvas.id = `research-chart-${idx}`;
974 |
975 |         wrapper.appendChild(canvas);
976 |         card.appendChild(header);
977 |         card.appendChild(wrapper);
978 |         container.appendChild(card);
979 |     });
```

```

980     const ctx = canvas.getContext("2d");
981     const chartDatasets = cfg.datasets.map((ds) => ({
982         label: ds.label,
983         data: ds.data,
984         borderColor: ds.borderColor || "#38bdf8",
985         backgroundColor: ds.backgroundColor || "rgba(56, 189, 24
8, 0.1)",
986         borderWidth: 2,
987         fill: true,
988         tension: 0.3,
989     }));
990
991     researchCharts[canvas.id] = new Chart(ctx, {
992         type: cfg.chart_type === "bar" ? "bar" : "line",
993         data: { labels: cfg.labels, datasets: chartDatasets },
994         options: {
995             responsive: true,
996             maintainAspectRatio: false,
997             scales: {
998                 x: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks
: { color: "#94a3b8", maxRotation: 0, autoSkip: true, maxTicks
Limit: 8 } },
999                 y: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks
: { color: "#94a3b8" } },
1000             },
1001         },
1002     });
1003 });
1004 }
1005
1006 /**
1007  * Рендеринг гипотез.
1008  */
1009 function renderHypotheses(hypotheses) {
1010     const container = document.getElementById("hypotheses-contai
ner");
1011     if (!container) return;
1012     container.innerHTML = "";

```

```

1013
1014     if (!hypotheses || hypotheses.length === 0) {
1015         container.innerHTML = '<div class="kpi-card" style="grid-c
column: 1 / -1; text-align: center; padding: 40px;">Гипотезы не
проверялись</div>';
1016         return;
1017     }
1018
1019     hypotheses.forEach((h) => {
1020         const card = document.createElement("div");
1021         card.className = "hypothesis-card";
1022         const statusClass = h.confirmed ? "status-confirmed" : "st
atus-rejected";
1023         const statusText = h.confirmed ? "Подтверждена" : "Отвергн
ута";
1024
1025         card.innerHTML = `
1026             <div class="hyp-header">
1027                 <h4 style="font-size: 15px; font-weight: 600;">${h.nam
e}</h4>
1028                 <span class="hyp-status ${statusClass}">${statusText}<
/span>
1029             </div>
1030             <p style="font-size: 13px; color: var(--text-secondary);
margin-bottom: 8px;">${h.description}</p>
1031             <div style="font-size: 12px; color: var(--text-primary);
margin-bottom: 8px;">
1032                 <strong>Обоснование:</strong> ${h.evidence}
1033             </div>
1034             <div style="font-size: 11px; color: var(--accent-blue);
background: var(--bg-main); padding: 6px 10px; border-radius:
6px;">
1035                 □ <strong>Рекомендация:</strong> ${h.recommendation}
1036             </div>
1037         `;
1038         container.appendChild(card);
1039     });
1040 }

```

```

1041
1042 /**
1043  * Рендеринг корреляций Пирсона.
1044  */
1045 function renderCorrelations(correlations) {
1046     const container = document.getElementById("correlations-cont
ainer");
1047     if (!container) return;
1048     container.innerHTML = "";
1049
1050     if (!correlations || correlations.length === 0) {
1051         container.innerHTML = '<p style="color: var(--text-seconda
ry);">Недостаточно данных для матрицы корреляций</p>';
1052         return;
1053     }
1054
1055     correlations.forEach((c) => {
1056         const item = document.createElement("div");
1057         item.className = "corr-item";
1058         const coeff = Number(c.pearson_coeff).toFixed(2);
1059         const isStrong = Math.abs(c.pearson_coeff) >= 0.7;
1060         const coeffClass = isStrong ? "corr-strong" : "corr-med";
1061
1062         item.innerHTML = `
1063             <div>
1064                 <strong>${c.metric_a}</strong> ↔ <strong>${c.metric_b}
</strong>
1065                 <div style="font-size: 12px; color: var(--text-seconda
ry); margin-top: 2px;">${c.interpretation}</div>
1066             </div>
1067             <div class="corr-coeff ${coeffClass}">${coeff}</div>
1068         `;
1069         container.appendChild(item);
1070     });
1071 }
1072
1073 /**
1074  * Рендеринг аномалий и выводов.

```



```

1075 */
1076 function renderAnomalies(anomalies, conclusions) {
1077     const list = document.getElementById("conclusions-list");
1078     if (list) {
1079         list.innerHTML = "";
1080         if (conclusions && conclusions.length > 0) {
1081             conclusions.forEach((c) => {
1082                 const li = document.createElement("li");
1083                 li.style.marginBottom = "6px";
1084                 li.textContent = c;
1085                 list.appendChild(li);
1086             });
1087         } else {
1088             list.innerHTML = "<li>Аномалий не выявлено.</li>";
1089         }
1090     }
1091
1092     const tbody = document.getElementById("anomalies-table-body"
1093 );
1094     if (tbody) {
1095         tbody.innerHTML = "";
1096         if (anomalies && anomalies.length > 0) {
1097             anomalies.forEach((a) => {
1098                 const tr = document.createElement("tr");
1099                 const isCrit = a.severity === "critical";
1100                 const tagColor = isCrit ? "var(--accent-red)" : "var(-
1101 -accent-amber)";
1102                 tr.innerHTML = `
1103                     <td><span style="color: ${tagColor}; font-weight: 70
1104 0; text-transform: uppercase;">${a.severity}</span></td>
1105                     <td>${a.timestamp || "--"}</td>
1106                     <td><strong>${a.metric}</strong></td>
1107                     <td>${a.value}</td>
1108                     <td>${a.description}</td>
1109                 `;
1110                 tbody.appendChild(tr);
1111             });
1112         } else {

```

```
1110         tbody.innerHTML = '<tr><td colspan="5" style="text-align
: center; color: var(--accent-green);">Критических аномалий в
исследуемой выборке не обнаружено</td></tr>';
1111     }
1112 }
1113 }
1114
1115 /**
1116  * Загрузка сырых записей телеметрии с пагинацией.
1117  */
1118 async function loadRecords() {
1119     const tbody = document.getElementById("records-table-body");
1120     const selectSource = document.getElementById("select-source"
);
1121     const sourcePath = selectSource ? selectSource.value : null;
1122
1123     if (tbody) {
1124         tbody.innerHTML = '<tr><td colspan="4" style="text-align:
center; color: var(--text-secondary);">Загрузка записей...</td
></tr>';
1125     }
1126
1127     try {
1128         const params = new URLSearchParams({
1129             page: currentPage,
1130             page_size: pageSize,
1131         });
1132         if (sourcePath) params.append("source_path", sourcePath);
1133         if (searchQuery) params.append("query", searchQuery);
1134
1135         const res = await fetch(`/api/records?${params.toString()}
`);
1136         if (!res.ok) throw new Error(`HTTP ${res.status}`);
1137         const data = await res.json();
1138
1139         totalPages = Math.ceil(data.total / pageSize) || 1;
1140         const pageLabel = document.getElementById("page-label");
1141         if (pageLabel) {
```

```
1142     pageLabel.textContent = `Страница ${data.page} из ${totalPages} (Всего: ${data.total})`;
1143   }
1144
1145   if (tbody) {
1146     tbody.innerHTML = "";
1147     if (data.items.length === 0) {
1148       tbody.innerHTML = '<tr><td colspan="4" style="text-align: center; color: var(--text-secondary);">Записи не найдены</td></tr>';
1149       return;
1150     }
1151
1152     data.items.forEach((item) => {
1153       const tr = document.createElement("tr");
1154       const ts = item.timestamp || item.created_at || "--";
1155       const source = item.source_file || "sqlite";
1156       const type = item.event_type || (item.cpu_total_percent !== undefined ? "system_snapshot" : item.sensor_name ? "sensor_poll" : "custom");
1157
1158       const displayObj = { ...item };
1159       delete displayObj.source_file;
1160       delete displayObj.timestamp;
1161
1162       tr.innerHTML = `
1163         <td><code>${ts}</code></td>
1164         <td><span style="color: var(--accent-blue);">${source}</span></td>
1165         <td><span style="font-weight: 600;">${type}</span></td>
1166         <td><pre style="max-height: 80px; overflow-y: auto; font-size: 11px; background: rgba(0,0,0,0.2); padding: 4px; border-radius: 4px;">${JSON.stringify(displayObj, null, 2)}</pre></td>
1167       `;
1168       tbody.appendChild(tr);
1169     });
```

```
1170     }
1171   } catch (err) {
1172     if (tbody) {
1173       tbody.innerHTML = `|<td colspan="4" style="color: var
1174         (--accent-red); text-align: center;">Ошибка загрузки записей:
1175         ${err.message}</td></tr>`;
1176     }
1177   }
1178 }
1179 /**
1180  * Открытие автономного HTML отчета в новой вкладке.
1181  */
1182 function openHtmlDashboard() {
1183   const selectSource = document.getElementById("select-source"
1184   );
1185   const sourcePath = selectSource ? selectSource.value : "";
1186   const url = `/api/dashboard${sourcePath ? `?source_path=${en
1187     codeURIComponent(sourcePath)}` : ""}`;
1188   window.open(url, "_blank");
1189 }
1190 /**
1191  * Скачивание JSON-отчета исследования.
1192  */
1193 function downloadJsonReport() {
1194   if (!currentReport) {
1195     alert("Сначала выполните анализ для генерации отчета");
1196     return;
1197   }
1198   const blob = new Blob([JSON.stringify(currentReport, null, 2
1199   )], { type: "application/json" });
1200   const url = URL.createObjectURL(blob);
1201   const a = document.createElement("a");
1202   a.href = url;
1203   a.download = `telemetry_research_${Date.now()}.json`;
1204   a.click();
1205   URL.revokeObjectURL(url);
|  |

```

```
1203 }
1204
1205 /**
1206  * Загрузка и рендеринг ресурсов клиентских программ.
1207  */
1208 async function loadClientResources(askGemini = false) {
1209   const tbody = document.getElementById("client-procs-table-bo
1210   dy");
1211   const statusText = document.getElementById("client-status-te
1212   xt");
1213   const statusSub = document.getElementById("client-status-sub
1214   ");
1215
1216   if (statusText) statusText.textContent = askGemini ? "Запрос
1217   к Gemini AI..." : "Сканирование программ клиента...";
1218   if (statusSub) statusSub.textContent = askGemini ? "Анализи
1219   рuem неизвестные ресурсоемкие программы через LLM..." : "Сбор м
1220   етрик памяти, ЦП и диска...";
1221
1222   if (tbody) {
1223     tbody.innerHTML = `|<td colspan="8" style="text-align:
1224     center; color: var(--accent-blue); padding: 20px;">␣ ${askGemi
1225     ni ? "Оценка неизвестных процессов через Gemini AI..." : "Загр
1226     узка данных о клиентских программах..."}</td></tr>`;
1227   }
1228
1229   const selectSource = document.getElementById("select-source"
1230   );
1231   const sourcePath = selectSource ? selectSource.value : "";
1232   const queryUrl = `/api/client-resources?ask_gemini=${askGemi
1233   ni}&limit=50${sourcePath ? `&db_path=${encodeURIComponent(sour
1234   cePath)}` : ""}`;
1235
1236   try {
1237     const res = await fetch(queryUrl);
1238     if (!res.ok) throw new Error(`HTTP ${res.status}`);
1239     const data = await res.json();
1240     currentClientSummary = data;
1241   }
|  |

```

```
1229 |
1230 |     // Обновление KPI
1231 |     const kpiTotal = document.getElementById("client-kpi-total
    | ");
1232 |     const kpiRam = document.getElementById("client-kpi-ram");
1233 |     const kpiCpu = document.getElementById("client-kpi-cpu");
1234 |     const kpiUnknown = document.getElementById("client-kpi-unk
    | nown");
1235 |
1236 |     if (kpiTotal) kpiTotal.textContent = data.total_client_pro
    | grams || 0;
1237 |     if (kpiRam) kpiRam.textContent = `${data.total_ram_mb || 0
    | } MB`;
1238 |     if (kpiCpu) kpiCpu.textContent = `${data.total_cpu_percent
    | || 0}%`;
1239 |     if (kpiUnknown) kpiUnknown.textContent = data.unknown_heav
    | y_programs_count || 0;
1240 |
1241 |     renderClientProcsTable(data.programs || []);
1242 |     renderGeminiEvaluations(data.evaluations || []);
1243 |
1244 |     if (statusText) statusText.textContent = "Аудит программ к
    | лиента завершен";
1245 |     if (statusSub) statusSub.textContent = `Обнаружено ${data.
    | total_client_programs} программ (${data.unknown_heavy_programs
    | _count} неизвестных ресурсоемких)`;
1246 | } catch (err) {
1247 |     if (statusText) statusText.textContent = "Ошибка аудита";
1248 |     if (statusSub) statusSub.textContent = err.message;
1249 |     if (tbody) {
1250 |         tbody.innerHTML = `|<td colspan="8" style="color: var
    | (--accent-red); text-align: center;">Не удалось загрузить данн
    | ые: ${err.message}</td></tr>`;
1251 |     }
1252 | }
1253 | }
1254 |
1255 | /**
|  |

```

```
1256 * Отрисовка строк таблицы клиентских программ.
1257 */
1258 function renderClientProcsTable(programs) {
1259     const tbody = document.getElementById("client-procs-table-bo
1260     dy");
1261     if (!tbody) return;
1262     tbody.innerHTML = "";
1263     if (!programs || programs.length === 0) {
1264         tbody.innerHTML = `<tr><td colspan="8" style="text-align:
1265         center; color: var(--text-secondary);">Активных клиентских про
1266         грамм не обнаружено.</td></tr>`;
1267         return;
1268     }
1269     programs.forEach((p, idx) => {
1270         const tr = document.createElement("tr");
1271         const isHeavy = p.is_resource_heavy;
1272         const isKnown = p.is_known_software;
1273         let statusBadge = `<span class="badge badge-normal">□ Дове
1274         ренная</span>`;
1275         if (!isKnown && isHeavy) {
1276             statusBadge = `<span class="badge badge-critical">□ Неи
1277             звестная (Heavy)</span>`;
1278         } else if (!isKnown) {
1279             statusBadge = `<span class="badge badge-warning">i Неизв
1280             естная</span>`;
1281         } else if (isHeavy) {
1282             statusBadge = `<span class="badge badge-warning">□ Нагру
1283             зка</span>`;
1284         }
1285         const diskMb = p.disk_size_mb ? `${p.disk_size_mb} MB` : "
1286         —";
1287         const diskIo = roundVal(((p.disk_read_bytes_sec || 0) + (p
1288         .disk_write_bytes_sec || 0)) / (1024 * 1024), 2);
1289         const diskIoStr = diskIo > 0 ? ` (${diskIo} MB/s)` : "";
```

```
1285 |
1286 |     const pathOrCmd = p.executable_path || p.command_line || "
    | -";
1287 |
1288 |     let actionBtn = "";
1289 |     if (!isKnown || isHeavy) {
1290 |         actionBtn = `
```



```

OrCmd)}}">
1311     <span style="font-size: 11px; color: var(--text-second
    ary);">${escapeHtml(pathOrCmd)}</span>
1312     </td>
1313     <td>${statusBadge}</td>
1314     <td>${actionBtn}</td>
1315     `;
1316     tbody.appendChild(tr);
1317 });
1318 }
1319
1320 /**
1321  * Фильтрация таблицы клиентских программ.
1322  */
1323 function filterClientProcsTable(query) {
1324     if (!currentClientSummary || !currentClientSummary.programs)
1325         return;
1326     const q = (query || "").trim().toLowerCase();
1327     if (!q) {
1328         renderClientProcsTable(currentClientSummary.programs);
1329         return;
1330     }
1331     const filtered = currentClientSummary.programs.filter((p) =>
1332     {
1333         return (
1334             (p.name || "").toLowerCase().includes(q) ||
1335             (p.display_name || "").toLowerCase().includes(q) ||
1336             (p.executable_path || "").toLowerCase().includes(q) ||
1337             (p.category || "").toLowerCase().includes(q)
1338         );
1339     });
1340     renderClientProcsTable(filtered);
1341 }
1342
1343 /**
1344  * Отрисовка карточек с экспертным мнением Gemini.
1345  */
1346 function renderGeminiEvaluations(evaluations) {

```

```

1345 |   const container = document.getElementById("gemini-evaluation
      | s-container");
1346 |   const listEl = document.getElementById("gemini-evals-list");
1347 |   const badgeEl = document.getElementById("gemini-evals-count-
      | badge");
1348 |   if (!container || !listEl) return;
1349 |
1350 |   if (!evaluations || evaluations.length === 0) {
1351 |     container.style.display = "none";
1352 |     listEl.innerHTML = "";
1353 |     return;
1354 |   }
1355 |
1356 |   container.style.display = "block";
1357 |   if (badgeEl) badgeEl.textContent = `${evaluations.length} or
      | четов`;
1358 |   listEl.innerHTML = "";
1359 |
1360 |   evaluations.forEach((ev) => {
1361 |     const card = document.createElement("div");
1362 |     card.className = "gemini-card";
1363 |
1364 |     const isNorm = ev.is_normal;
1365 |     const badgeClass = isNorm ? "gemini-verdict-normal" : "gem
      | ini-verdict-risk";
1366 |     const badgeText = isNorm ? `✅ ${ev.verdict}` : `❌ ${ev.ve
      | rdict}`;
1367 |
1368 |     const recsHtml = (ev.recommendations || []).map((r) => `<li>
      | ${escapeHtml(r)}</li>`).join("");
1369 |
1370 |     card.innerHTML = `
1371 |       <div class="gemini-card-header">
1372 |         <div>
1373 |           <div class="gemini-card-title">${escapeHtml(ev.progr
      | am_name)} (PID ${ev.pid || 'N/A'})</div>
1374 |           <div style="font-size: 11px; color: var(--text-secon
      | dary); margin-top: 2px;">${escapeHtml(ev.executable_path || '3

```

```

    апуск из памяти/скрипта'}}</div>
1375     </div>
1376     <span class="gemini-verdict-badge ${badgeClass}">${bad
    geText}</span>
1377     </div>
1378     <div class="gemini-card-metrics">
1379         <span class="gemini-metric-tag">CPU: <b>${ev.cpu_perce
    nt}%</b></span>
1380         <span class="gemini-metric-tag">RAM: <b>${ev.memory_mb
    } MB</b></span>
1381         <span class="gemini-metric-tag">Диск IO: <b>${ev.disk_
    io_mb_s} MB/s</b></span>
1382         <span class="gemini-metric-tag">Риск: <b>${escapeHtml(
    ev.risk_level.toUpperCase())}</b></span>
1383     </div>
1384     <div class="gemini-card-analysis">
1385         ${escapeHtml(ev.analysis)}
1386     </div>
1387     ${recsHtml ? `<ul class="gemini-card-recs">${recsHtml}</
    ul>` : ""}
1388     `;
1389     listEl.appendChild(card);
1390 });
1391 }
1392
1393 /**
1394  * Оценка конкретной программы в Gemini по клику кнопки в табл
    ице.
1395  */
1396 window.evaluateSingleProgramInGemini = async function (pid, en
    codedName) {
1397     const name = decodeURIComponent(encodedName);
1398     if (!currentClientSummary || !currentClientSummary.programs)
    return;
1399
1400     const prog = currentClientSummary.programs.find((p) => p.pid
    === pid || p.name === name);
1401     if (!prog) {

```

```
1402 |     alert("Программа не найдена в текущем снимке");
1403 |     return;
1404 | }
1405 |
1406 | try {
1407 |     const res = await fetch("/api/client-resources/evaluate",
1408 | {
1409 |         method: "POST",
1410 |         headers: { "Content-Type": "application/json" },
1411 |         body: JSON.stringify(prog),
1412 |     });
1413 |     if (!res.ok) throw new Error(`HTTP ${res.status}`);
1414 |     const evaluation = await res.json();
1415 |
1416 |     prog.gemini_evaluation = evaluation;
1417 |     const existing = currentClientSummary.evaluations || [];
1418 |     const withoutSame = existing.filter((e) => e.program_name
1419 | != evaluation.program_name || e.pid != evaluation.pid);
1420 |     withoutSame.unshift(evaluation);
1421 |     currentClientSummary.evaluations = withoutSame;
1422 |
1423 |     renderGeminiEvaluations(currentClientSummary.evaluations);
1424 |     document.getElementById("gemini-evaluations-container)?.s
1425 | crollIntoView({ behavior: "smooth" });
1426 | } catch (err) {
1427 |     alert(`Ошибка оценки Gemini: ${err.message}`);
1428 | }
```

index.html

```
1 <!--
2 =====
3 |=====
4 Process Name: Windows Web Interface - Index View
5 |=====
6 Description:
7 |Шаблон клиентского веб-интерфейса index.
8 |
9 Usage Examples:
10 |    Web Route:
11 |        /windows/telemetry_research/web/index.html
12 |
13 |    Tab Frame Integration:
14 |        <iframe src="/windows/telemetry_research/web/index.html"
15 |        class="w-100 h-100 border-0"></iframe>
16 |
17 File: index.html
18 Project: ai-breadboard
19 Package: windows/telemetry_research/web
20 Author: hypo69
21 Copyright: © 2026 hypo69
22 Updated: 2026-10-01 13:04:40
23 |=====
24 |=====
25 -->
26
27 <!-- Updated: 2026-10-04 07:49:00 -->
28 <!DOCTYPE html>
29 <html lang="ru">
30 <head>
31 |    <meta charset="UTF-8">
32 |    <meta name="viewport" content="width=device-width, initial-s
33 |    cale=1.0">
34 |    <title>Исследование телеметрии Windows | Web GUI</title>
35 |    <link rel="stylesheet" href="/static/style.css?v=20261004_v1
```

```
    ">
32    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script
    >
33 </head>
34 <body>
35     <header class="app-header">
36         <div class="logo-section">
37             <div class="logo-icon">❏ </div>
38             <div class="logo-text">
39                 <h1>Исследование телеметрии Windows</h1>
40                 <p>Анализ долговременной телеметрии, EDA, ресурсы клие
    нтских программ и оценка Gemini</p>
41             </div>
42         </div>
43         <div class="header-controls">
44             <select id="select-source" class="source-select">
45                 <option value="">Загрузка источников...</option>
46             </select>
47             <button id="btn-run" class="btn btn-primary">❏ Анализ</
    button>
48             <button id="btn-dashboard" class="btn btn-secondary">❏
    HTML Дашборд</button>
49             <button id="btn-json" class="btn btn-secondary">❏ JSON<
    /button>
50         </div>
51     </header>
52
53     <main class="main-container">
54         <!-- KPI Summary Grid -->
55         <section class="kpi-grid">
56             <div class="kpi-card">
57                 <div class="kpi-title">Индекс здоровья (Health Score)<
    /div>
58                 <div class="kpi-value" id="kpi-health">--%</div>
59                 <div class="kpi-sub">Интегральный показатель стабильно
    сти</div>
60             </div>
61             <div class="kpi-card">
```

```

62 |         <div class="kpi-title">Записей в выборке</div>
63 |         <div class="kpi-value" id="kpi-records">0</div>
64 |         <div class="kpi-sub">Точек замеров телеметрии</div>
65 |     </div>
66 |     <div class="kpi-card">
67 |         <div class="kpi-title">ЦП и Память (Avg / Max)</div>
68 |         <div class="kpi-value" style="font-size: 16px; margin-
| top: 6px;" id="kpi-cpu-val">CPU: --%</div>
69 |         <div class="kpi-sub" id="kpi-ram-val">RAM: --%</div>
70 |     </div>
71 |     <div class="kpi-card">
72 |         <div class="kpi-title">Аномалии и сбои</div>
73 |         <div class="kpi-value" id="kpi-anomalies" style="color
| : var(--accent-amber);">0</div>
74 |         <div class="kpi-sub" id="kpi-devices">Оборудование в н
| орме</div>
75 |     </div>
76 | </section>
77 |
78 | <!-- Navigation Tabs -->
79 | <nav class="tabs-nav">
80 |     <button class="tab-btn active" data-tab="tab-current">□
| Текущее состояние (telemetry.db)</button>
81 |     <button class="tab-btn" data-tab="tab-client-resources">
| □ Ресурсы программ клиента</button>
82 |     <button class="tab-btn" data-tab="tab-charts">□ Графики
| исследования</button>
83 |     <button class="tab-btn" data-tab="tab-hypotheses">□ Сис
| темные гипотезы</button>
84 |     <button class="tab-btn" data-tab="tab-anomalies">□ Аном
| алии и выводы</button>
85 |     <button class="tab-btn" data-tab="tab-correlations">□ К
| орреляции</button>
86 |     <button class="tab-btn" data-tab="tab-records">□ Журнал
| записей</button>
87 | </nav>
88 |
89 | <!-- Tab: Current State (telemetry.db) -->

```

```

90 |     <div id="tab-current" class="tab-content active">
91 |         <!-- Live Status & Controls Bar -->
92 |         <div class="live-status-bar">
93 |             <div class="status-badge-container">
94 |                 <div class="pulse-dot"></div>
95 |                 <div>
96 |                     <div class="status-badge-text" id="live-db-status"
>Подключение к telemetry.db...</div>
97 |                     <div class="status-badge-sub" id="live-db-sub">Заг
рузка информации о базе данных...</div>
98 |                 </div>
99 |             </div>
100 |             <div class="live-controls">
101 |                 <label class="toggle-label">
102 |                     <input type="checkbox" id="auto-refresh-toggle" ch
ecked>
103 |                     <span>⏻ Автообновление (3с)</span>
104 |                 </label>
105 |                 <select id="current-range-select" class="source-sele
ct" style="min-width: 160px;">
106 |                     <option value="50">50 замеров</option>
107 |                     <option value="100" selected>100 замеров</option>
108 |                     <option value="300">300 замеров</option>
109 |                     <option value="500">500 замеров</option>
110 |                     <option value="1000">1000 замеров</option>
111 |                 </select>
112 |                 <button id="btn-refresh-current" class="btn btn-seco
ndary">⏻ Обновить</button>
113 |             </div>
114 |         </div>
115 |
116 |         <!-- Current Instant KPI Cards -->
117 |         <div class="current-kpi-grid">
118 |             <div class="current-kpi-card">
119 |                 <div class="current-kpi-header">
120 |                     <span class="kpi-title">Процессор (CPU)</span>
121 |                     <span class="current-kpi-icon">⏻ </span>
122 |                 </div>

```



```
123 |         <div class="kpi-value" id="curr-cpu-val">--%</div>
124 |         <div class="kpi-sub" id="curr-cpu-freq">Частота: --
    | GHz</div>
125 |         <div class="mini-meter-bg"><div class="mini-meter-fi
    | ll" id="curr-cpu-meter" style="width: 0%;"></div></div>
126 |     </div>
127 |
128 |     <div class="current-kpi-card">
129 |         <div class="current-kpi-header">
130 |             <span class="kpi-title">Память (RAM)</span>
131 |             <span class="current-kpi-icon">📊 </span>
132 |         </div>
133 |         <div class="kpi-value" id="curr-ram-val">--%</div>
134 |         <div class="kpi-sub" id="curr-ram-gb">-- / -- GB</di
    | v>
135 |         <div class="mini-meter-bg"><div class="mini-meter-fi
    | ll" id="curr-ram-meter" style="width: 0%; background: var(--ac
    | cent-green);"></div></div>
136 |     </div>
137 |
138 |     <div class="current-kpi-card">
139 |         <div class="current-kpi-header">
140 |             <span class="kpi-title">Видеокарта (GPU)</span>
141 |             <span class="current-kpi-icon">📊 </span>
142 |         </div>
143 |         <div class="kpi-value" id="curr-gpu-val">--%</div>
144 |         <div class="kpi-sub" id="curr-gpu-temp">Темп: --°C</
    | div>
145 |         <div class="mini-meter-bg"><div class="mini-meter-fi
    | ll" id="curr-gpu-meter" style="width: 0%; background: var(--ac
    | cent-purple);"></div></div>
146 |     </div>
147 |
148 |     <div class="current-kpi-card">
149 |         <div class="current-kpi-header">
150 |             <span class="kpi-title">Дисковая активность</span>
151 |             <span class="current-kpi-icon">📊 </span>
152 |         </div>
```

```
153 |         <div class="kpi-value" style="font-size: 18px;" id="
| curr-disk-val">R: 0 / W: 0 MB/s</div>
154 |         <div class="kpi-sub" id="curr-disk-sub">Чтение и зап
| ись на диск</div>
155 |     </div>
156 |
157 |     <div class="current-kpi-card">
158 |         <div class="current-kpi-header">
159 |             <span class="kpi-title">Сетевой трафик</span>
160 |             <span class="current-kpi-icon">⬇️ </span>
161 |         </div>
162 |         <div class="kpi-value" style="font-size: 18px;" id="
| curr-net-val">⬇️ 0 / ⬆️ 0 KB/s</div>
163 |         <div class="kpi-sub" id="curr-uptime-val">Аптайм: --
| </div>
164 |     </div>
165 | </div>
166 |
167 | <!-- Current Charts Grid -->
168 | <div class="charts-grid">
169 |     <div class="chart-card">
170 |         <div class="chart-header">
171 |             <h3 class="chart-title">⬇️ Нагрузка и частота ЦП (
| CPU)</h3>
172 |         </div>
173 |         <div class="chart-canvas-wrapper"><canvas id="chart-
| live-cpu"></canvas></div>
174 |     </div>
175 |
176 |     <div class="chart-card">
177 |         <div class="chart-header">
178 |             <h3 class="chart-title">⬇️ Оперативная память (RAM
| )</h3>
179 |         </div>
180 |         <div class="chart-canvas-wrapper"><canvas id="chart-
| live-ram"></canvas></div>
181 |     </div>
182 |
```

```
183 |         <div class="chart-card">
184 |             <div class="chart-header">
185 |                 <h3 class="chart-title">⬢ Нагрузка и температура
GPU</h3>
186 |             </div>
187 |             <div class="chart-canvas-wrapper"><canvas id="chart-
live-gpu"></canvas></div>
188 |         </div>
189 |
190 |         <div class="chart-card">
191 |             <div class="chart-header">
192 |                 <h3 class="chart-title">⬢ Дисковая скорость (Read
/ Write MB/s)</h3>
193 |             </div>
194 |             <div class="chart-canvas-wrapper"><canvas id="chart-
live-disk"></canvas></div>
195 |         </div>
196 |
197 |         <div class="chart-card">
198 |             <div class="chart-header">
199 |                 <h3 class="chart-title">⬢ Сетевая пропускная спос
обность (KB/s)</h3>
200 |             </div>
201 |             <div class="chart-canvas-wrapper"><canvas id="chart-
live-net"></canvas></div>
202 |         </div>
203 |
204 |         <div class="chart-card">
205 |             <div class="chart-header">
206 |                 <h3 class="chart-title">⬢ Аппаратные сенсоры (LHM
/ Hardware)</h3>
207 |             </div>
208 |             <div class="chart-canvas-wrapper" style="overflow-y:
auto; max-height: 280px;">
209 |                 <div id="live-sensors-container" style="display: g
rid; grid-template-columns: 1fr 1fr; gap: 8px; font-size: 12px
;"></div>
210 |             </div>
```

```

211         </div>
212     </div>
213
214     <!-- Split Grid: Top Processes + Activity Feed -->
215     <div class="split-grid">
216         <!-- Top Processes -->
217         <div class="table-wrapper">
218             <div class="table-header-bar">
219                 <h3 style="font-size: 15px;">☐ Топ активных проце
220                 </div>
221                 <table>
222                     <thead>
223                         <tr>
224                             <th style="width: 70px;">PID</th>
225                             <th>Процесс</th>
226                             <th style="width: 80px;">CPU %</th>
227                             <th style="width: 90px;">RAM MB</th>
228                             <th style="width: 70px;">Потоки</th>
229                         </tr>
230                     </thead>
231                     <tbody id="live-processes-body"></tbody>
232                 </table>
233             </div>
234
235             <!-- Activity Feed -->
236             <div class="table-wrapper">
237                 <div class="table-header-bar">
238                     <h3 style="font-size: 15px;">☐ Системная активност
239                     </div>
240                     <div style="padding: 12px;">
241                         <ul class="activity-feed-list" id="live-activity-l
242                         </div>
243                     </div>
244                 </div>
245             </div>

```

```
246 |
247 |     <!-- Tab: Client Program Resources & Gemini AI -->
248 |     <div id="tab-client-resources" class="tab-content">
249 |         <!-- Client KPI Strip -->
250 |         <div class="kpi-grid" style="margin-bottom: 20px;">
251 |             <div class="kpi-card">
252 |                 <div class="kpi-title">Программ клиента</div>
253 |                 <div class="kpi-value" id="client-kpi-total">0</div>
254 |                 <div class="kpi-sub">Активных пользовательских проце
255 | ссов</div>
256 |             </div>
257 |             <div class="kpi-card">
258 |                 <div class="kpi-title">Занято памяти RAM</div>
259 |                 <div class="kpi-value" id="client-kpi-ram">0 MB</div>
260 |             </div>
261 |             <div class="kpi-card">
262 |                 <div class="kpi-title">Загрузка ЦП (CPU)</div>
263 |                 <div class="kpi-value" id="client-kpi-cpu">0%</div>
264 |                 <div class="kpi-sub">Суммарная нагрузка на ядра</div>
265 |             </div>
266 |             <div class="kpi-card">
267 |                 <div class="kpi-title">Неизвестных (Heavy)</div>
268 |                 <div class="kpi-value" id="client-kpi-unknown" style
269 | ="color: var(--accent-red);">0</div>
270 |                 <div class="kpi-sub">Требуют анализа в Gemini</div>
271 |             </div>
272 |
273 |     <!-- Controls & Actions Bar -->
274 |     <div class="live-status-bar" style="margin-bottom: 16px;
275 | ">
276 |         <div class="status-badge-container">
277 |             <div class="pulse-dot"></div>
278 |             <div>
```

```
278 |         <div class="status-badge-text" id="client-status-t
| ext">Аудит программ клиента</div>
279 |         <div class="status-badge-sub" id="client-status-su
| b">Отслеживание потребления ресурсов и проверка в Gemini AI</d
| iv>
280 |     </div>
281 | </div>
282 |     <div class="live-controls">
283 |         <button id="btn-refresh-client-procs" class="btn btn
| -secondary">⏻ Обновить список</button>
284 |         <button id="btn-ask-gemini-all-unknown" class="btn b
| tn-primary">⏻ Проверить неизвестные в Gemini</button>
285 |     </div>
286 | </div>
287 |
288 |     <!-- Gemini AI Evaluations Container -->
289 |     <div id="gemini-evaluations-container" style="margin-bot
| tom: 20px; display: none;">
290 |         <div class="kpi-card" style="border-left: 4px solid va
| r(--accent-purple);">
291 |             <div style="display: flex; justify-content: space-be
| tween; align-items: center; margin-bottom: 12px;">
292 |                 <h3 style="font-size: 16px; color: var(--accent-pu
| rple);">⏻ Экспертные заключения Gemini AI по неизвестным ресу
| рсоемким программам</h3>
293 |                 <span class="badge badge-warning" id="gemini-evals
| -count-badge">0 отчетов</span>
294 |             </div>
295 |             <div id="gemini-evals-list" class="gemini-evals-grid
| "></div>
296 |         </div>
297 |     </div>
298 |
299 |     <!-- Client Programs Table -->
300 |     <div class="table-wrapper">
301 |         <div class="table-header-bar">
302 |             <h3 style="font-size: 15px;">Программы, запущенные к
| лиентом (память, процессор, диск)</h3>
```

```

303 |         <input type="text" id="input-search-client-procs" cl
      | ass="search-input" placeholder="Фильтр по названию программы..
      | .">
304 |     </div>
305 |     <table>
306 |         <thead>
307 |             <tr>
308 |                 <th style="width: 220px;">Программа / Процесс</t
      | h>
309 |                 <th style="width: 80px;">PID</th>
310 |                 <th style="width: 110px;">CPU (%)</th>
311 |                 <th style="width: 140px;">RAM (МБ / %)</th>
312 |                 <th style="width: 130px;">Диск (файл / IO)</th>
313 |                 <th>Каталог запуска / Командная строка</th>
314 |                 <th style="width: 150px;">Статус / Доверие</th>
315 |                 <th style="width: 140px;">Действие</th>
316 |             </tr>
317 |         </thead>
318 |         <tbody id="client-procs-table-body">
319 |             <tr><td colspan="8" style="text-align: center; col
      | or: var(--text-secondary);">Загрузка данных о программах клиен
      | та...</td></tr>
320 |         </tbody>
321 |     </table>
322 | </div>
323 | </div>
324 |
325 | <!-- Tab: Charts (Research) -->
326 | <div id="tab-charts" class="tab-content">
327 |     <div class="charts-grid" id="charts-container">
328 |         <!-- Rendered by Chart.js -->
329 |     </div>
330 | </div>
331 |
332 | <!-- Tab: Hypotheses -->
333 | <div id="tab-hypotheses" class="tab-content">
334 |     <div class="hypotheses-grid" id="hypotheses-container">
335 |         <!-- Rendered by app.js -->

```

```

336 |         </div>
337 |     </div>
338 |
339 |     <!-- Tab: Anomalies & Conclusions -->
340 |     <div id="tab-anomalies" class="tab-content">
341 |         <div class="kpi-card" style="margin-bottom: 20px;">
342 |             <h3 style="font-size: 16px; margin-bottom: 12px; color
: var(--accent-green);">□ Сводные выводы исследования</h3>
343 |             <ul id="conclusions-list" style="padding-left: 20px; c
olor: var(--text-primary); font-size: 14px;"></ul>
344 |         </div>
345 |
346 |         <div class="table-wrapper">
347 |             <div class="table-header-bar">
348 |                 <h3 style="font-size: 15px;">Выявленные аномалии и к
ритические всплески</h3>
349 |             </div>
350 |             <table>
351 |                 <thead>
352 |                     <tr>
353 |                         <th>Уровень</th>
354 |                         <th>Время</th>
355 |                         <th>Метрика</th>
356 |                         <th>Значение</th>
357 |                         <th>Описание</th>
358 |                     </tr>
359 |                 </thead>
360 |                 <tbody id="anomalies-table-body"></tbody>
361 |             </table>
362 |         </div>
363 |     </div>
364 |
365 |     <!-- Tab: Correlations -->
366 |     <div id="tab-correlations" class="tab-content">
367 |         <div class="kpi-card">
368 |             <h3 style="font-size: 16px; margin-bottom: 12px; color
: var(--accent-blue);">Матрица корреляций между метриками (Пир
сон)</h3>

```



```

369 |         <div class="correlation-list" id="correlations-contain
|er"></div>
370 |     </div>
371 | </div>
372 |
373 | <!-- Tab: Records -->
374 | <div id="tab-records" class="tab-content">
375 |     <div class="table-wrapper">
376 |         <div class="table-header-bar">
377 |             <input type="text" id="input-search" class="search-i
|nput" placeholder="Поиск по записям (Enter)...">
378 |             <div id="page-label" style="font-size: 13px; color:
|var(--text-secondary);">Страница 1 из 1</div>
379 |         </div>
380 |         <table>
381 |             <thead>
382 |                 <tr>
383 |                     <th style="width: 180px;">Время</th>
384 |                     <th style="width: 200px;">Источник</th>
385 |                     <th style="width: 140px;">Тип</th>
386 |                     <th>Данные (JSON)</th>
387 |                 </tr>
388 |             </thead>
389 |             <tbody id="records-table-body"></tbody>
390 |         </table>
391 |         <div class="pagination-bar">
392 |             <button id="btn-prev-page" class="btn btn-secondary"
|>◀ Назад</button>
393 |             <button id="btn-next-page" class="btn btn-secondary"
|>Вперед ▶ </button>
394 |         </div>
395 |     </div>
396 | </div>
397 |
398 | <!-- AI Response Modal / Dynamic Presentation Card -->
399 | <section id="ai-response-card" class="ai-response-card" st
|yle="display: none;">
400 |     <div class="ai-response-header">

```

```
401 |         <div class="ai-response-title-group">
402 |             <span class="ai-badge"> AI Ассистент Телеметрии</span>
403 |             <h2 id="ai-response-title" class="ai-response-title">Отчет и визуализация</h2>
404 |         </div>
405 |         <div class="ai-header-actions">
406 |             <button id="btn-ai-close" class="btn-icon" title="Закрыть панель"></button>
407 |         </div>
408 |     </div>
409 |
410 |     <div class="ai-response-content">
411 |         <!-- Dynamic Chart Area -->
412 |         <div id="ai-chart-wrapper" class="ai-chart-wrapper" style="display: none;">
413 |             <div class="ai-chart-header">
414 |                 <h3 id="ai-chart-title" style="font-size: 15px; color: var(--accent-blue);"> Визуализация</h3>
415 |             </div>
416 |             <div class="ai-canvas-container">
417 |                 <canvas id="chart-ai-dynamic"></canvas>
418 |             </div>
419 |         </div>
420 |
421 |         <!-- KPI Metrics Badges -->
422 |         <div id="ai-metrics-row" class="ai-metrics-row" style="display: none;"></div>
423 |
424 |         <!-- Markdown Reply Text -->
425 |         <div id="ai-reply-body" class="ai-reply-body"></div>
426 |
427 |         <!-- SQL Queries Collapsible Badge -->
428 |         <details id="ai-sql-details" class="ai-sql-details" style="display: none;">
429 |             <summary> Выполненные SQL-запросы к <code>telemetry.db</code></summary>
430 |             <pre id="ai-sql-code" class="ai-sql-code"></pre>
```

```
431         </details>
432
433         <!-- Follow-up Chips -->
434         <div id="ai-followup-row" class="ai-followup-row" styl
e="display: none;">
435             <span style="font-size: 12px; color: var(--text-seco
ndary); margin-right: 8px;">□ Уточнить:</span>
436             <div id="ai-followup-chips" class="ai-chips-list"></
div>
437         </div>
438     </div>
439 </section>
440
441 <!-- Bottom AI Query Bar (Sticky / Fixed) -->
442 <footer class="bottom-ai-bar">
443     <!-- Quick Prompt Chips -->
444     <div class="ai-quick-chips">
445         <button class="prompt-chip" data-query="покажи график
загрузки цпу по времени">
446             □ График загрузки ЦПУ по времени
447         </button>
448         <button class="prompt-chip" data-query="Посчитай приме
рное общее потребление электроэнергии компьютером и покажи сут
очную круговую диаграмму">
449             □ Суточное потребление энергии (круговая диаграмма)
450         </button>
451         <button class="prompt-chip" data-query="топ процессов
по cpu">
452             □ Топ процессов по CPU
453         </button>
454         <button class="prompt-chip" data-query="SELECT timesta
mp, event_type, friendly_name FROM device_events ORDER BY id D
ESC LIMIT 5">
455             □ SQL: События устройств
456         </button>
457     </div>
458
459     <!-- Input and Send Button Form -->
```

```
460 |         <form id="ai-query-form" class="ai-query-form" onsubmit=
    | "return false;">
461 |         <div class="ai-input-wrapper">
462 |             <span class="ai-input-icon">❏ </span>
463 |             <input
464 |                 type="text"
465 |                 id="ai-query-input"
466 |                 class="ai-query-input"
467 |                 placeholder="Спросите ИИ о телеметрии (например: «
    | покажи график загрузки цпу по времени», «посчитай потребление
    | электроэнергии»)... "
468 |                 autocomplete="off"
469 |             />
470 |             <button type="button" id="btn-ai-clear" class="btn-c
    | lear" title="Очистить" style="display: none;">❏ </button>
471 |         </div>
472 |         <button type="submit" id="btn-ai-send" class="btn btn-
    | primary btn-ai-send">
473 |             <span id="btn-ai-send-text">Отправить</span>
474 |             <span id="btn-ai-spinner" class="spinner" style="dis
    | play: none;"></span>
475 |         </button>
476 |     </form>
477 | </footer>
478 | </main>
479 |
480 | <script src="/static/app.js?v=20261001_v2"></script>
481 | </body>
482 | </html>
483 |
```

style.css

```
1 /**
2  * =====
3  * Process Name: Windows Web Styles - Style Stylesheet
4  * =====
5  * Description:
6  *   Таблица стилей стиля и оформления интерфейса (style).
7  *
8  * Usage Examples:
9  *   HTML Link Tag:
10 *     <link rel="stylesheet" href="/windows/telemetry_research/web/style.css?v=20261001_v1">
11 *
12 * File: style.css
13 * Project: ai-breadboard
14 * Package: windows/telemetry_research/web
15 * Author: hypo69
16 * Copyright: © 2026 hypo69
17 * Updated: 2026-10-01 13:04:40
18 * =====
19 */
20
21 /* Стили для Web GUI Исследования Телеметрии */
22 :root {
23   --bg-main: #0b0f19;
24   --bg-card: #131b2e;
25   --bg-card-hover: #1c2742;
26   --border-color: #23304d;
27   --text-primary: #f1f5f9;
28   --text-secondary: #94a3b8;
29   --accent-blue: #38bdf8;
30   --accent-green: #10b981;
31   --accent-amber: #f59e0b;
32   --accent-red: #ef4444;
```

```
33 | --accent-purple: #a855f7;
34 | --radius-md: 10px;
35 | --radius-lg: 14px;
36 | --shadow-sm: 0 2px 8px rgba(0, 0, 0, 0.3);
37 | --shadow-md: 0 4px 16px rgba(0, 0, 0, 0.4);
38 | --shadow-lg: 0 10px 30px rgba(0, 0, 0, 0.6);
39 | }
40 |
41 | * {
42 |   box-sizing: border-box;
43 |   margin: 0;
44 |   padding: 0;
45 | }
46 |
47 | body {
48 |   font-family: -apple-system, BlinkMacSystemFont, "Segoe UI",
49 |   Roboto, "Helvetica Neue", Arial, sans-serif;
50 |   background-color: var(--bg-main);
51 |   color: var(--text-primary);
52 |   min-height: 100vh;
53 |   display: flex;
54 |   flex-direction: column;
55 |   padding-bottom: 140px; /* Отступ для фиксированной нижней па
56 |   нели */
57 | }
58 |
59 | .app-header {
60 |   background: var(--bg-card);
61 |   border-bottom: 1px solid var(--border-color);
62 |   padding: 16px 28px;
63 |   display: flex;
64 |   justify-content: space-between;
65 |   align-items: center;
66 |   position: sticky;
67 |   top: 0;
68 |   z-index: 100;
69 |   box-shadow: var(--shadow-sm);
70 | }
```

```
69 |
70 | .logo-section {
71 |     display: flex;
72 |     align-items: center;
73 |     gap: 12px;
74 | }
75 |
76 | .logo-icon {
77 |     font-size: 28px;
78 |     line-height: 1;
79 | }
80 |
81 | .logo-text h1 {
82 |     font-size: 20px;
83 |     font-weight: 700;
84 |     letter-spacing: -0.5px;
85 | }
86 |
87 | .logo-text p {
88 |     font-size: 12px;
89 |     color: var(--text-secondary);
90 | }
91 |
92 | .header-controls {
93 |     display: flex;
94 |     align-items: center;
95 |     gap: 12px;
96 | }
97 |
98 | .source-select {
99 |     background: var(--bg-main);
100 |     border: 1px solid var(--border-color);
101 |     color: var(--text-primary);
102 |     padding: 8px 14px;
103 |     border-radius: var(--radius-md);
104 |     font-size: 13px;
105 |     outline: none;
106 |     min-width: 260px;
```

```
107 |   cursor: pointer;
108 | }
109 |
110 | .source-select:focus {
111 |   border-color: var(--accent-blue);
112 | }
113 |
114 | .btn {
115 |   display: inline-flex;
116 |   align-items: center;
117 |   gap: 6px;
118 |   padding: 8px 16px;
119 |   border-radius: var(--radius-md);
120 |   font-size: 13px;
121 |   font-weight: 600;
122 |   cursor: pointer;
123 |   border: none;
124 |   transition: all 0.2s ease;
125 | }
126 |
127 | .btn-primary {
128 |   background: var(--accent-blue);
129 |   color: #0b0f19;
130 | }
131 |
132 | .btn-primary:hover {
133 |   background: #7dd3fc;
134 |   box-shadow: 0 0 12px rgba(56, 189, 248, 0.4);
135 | }
136 |
137 | .btn-secondary {
138 |   background: var(--bg-card-hover);
139 |   color: var(--text-primary);
140 |   border: 1px solid var(--border-color);
141 | }
142 |
143 | .btn-secondary:hover {
144 |   background: #273659;
```



```
145 |   border-color: var(--accent-blue);
146 | }
147 |
148 | .main-container {
149 |   max-width: 1400px;
150 |   width: 100%;
151 |   margin: 0 auto;
152 |   padding: 24px 28px;
153 |   display: flex;
154 |   flex-direction: column;
155 |   gap: 24px;
156 | }
157 |
158 | /* KPI Grid */
159 | .kpi-grid {
160 |   display: grid;
161 |   grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
162 |   gap: 16px;
163 | }
164 |
165 | .kpi-card {
166 |   background: var(--bg-card);
167 |   border: 1px solid var(--border-color);
168 |   border-radius: var(--radius-lg);
169 |   padding: 18px 20px;
170 |   box-shadow: var(--shadow-sm);
171 |   transition: transform 0.2s, box-shadow 0.2s;
172 | }
173 |
174 | .kpi-card:hover {
175 |   transform: translateY(-2px);
176 |   box-shadow: var(--shadow-md);
177 | }
178 |
179 | .kpi-title {
180 |   font-size: 12px;
181 |   font-weight: 600;
182 |   text-transform: uppercase;
```

```
183 | color: var(--text-secondary);
184 | letter-spacing: 0.5px;
185 | }
186 |
187 | .kpi-value {
188 |   font-size: 26px;
189 |   font-weight: 700;
190 |   margin-top: 8px;
191 |   color: var(--text-primary);
192 | }
193 |
194 | .kpi-sub {
195 |   font-size: 12px;
196 |   color: var(--text-secondary);
197 |   margin-top: 4px;
198 | }
199 |
200 | /* Tabs */
201 | .tabs-nav {
202 |   display: flex;
203 |   gap: 8px;
204 |   border-bottom: 1px solid var(--border-color);
205 |   padding-bottom: 8px;
206 |   overflow-x: auto;
207 | }
208 |
209 | .tab-btn {
210 |   background: transparent;
211 |   border: none;
212 |   color: var(--text-secondary);
213 |   padding: 8px 16px;
214 |   font-size: 13px;
215 |   font-weight: 600;
216 |   border-radius: var(--radius-md);
217 |   cursor: pointer;
218 |   transition: all 0.2s;
219 |   white-space: nowrap;
220 | }
```

```
221 |
222 | .tab-btn:hover {
223 |   color: var(--text-primary);
224 |   background: var(--bg-card);
225 | }
226 |
227 | .tab-btn.active {
228 |   color: var(--accent-blue);
229 |   background: rgba(56, 189, 248, 0.1);
230 |   border-bottom: 2px solid var(--accent-blue);
231 | }
232 |
233 | .tab-content {
234 |   display: none;
235 | }
236 |
237 | .tab-content.active {
238 |   display: block;
239 | }
240 |
241 | /* Live Status & Controls Bar */
242 | .live-status-bar {
243 |   display: flex;
244 |   justify-content: space-between;
245 |   align-items: center;
246 |   background: var(--bg-card);
247 |   border: 1px solid var(--border-color);
248 |   border-radius: var(--radius-lg);
249 |   padding: 12px 20px;
250 |   margin-bottom: 18px;
251 |   box-shadow: var(--shadow-sm);
252 |   flex-wrap: wrap;
253 |   gap: 12px;
254 | }
255 |
256 | .status-badge-container {
257 |   display: flex;
258 |   align-items: center;
```

```
259 | gap: 12px;
260 | }
261 |
262 | .pulse-dot {
263 |     width: 10px;
264 |     height: 10px;
265 |     background-color: var(--accent-green);
266 |     border-radius: 50%;
267 |     box-shadow: 0 0 8px var(--accent-green);
268 |     animation: pulse-animation 2s infinite;
269 | }
270 |
271 | @keyframes pulse-animation {
272 |     0% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(16, 18
273 |         5, 129, 0.7); }
274 |     70% { transform: scale(1); box-shadow: 0 0 0 8px rgba(16, 18
275 |         5, 129, 0); }
276 |     100% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(16,
277 |         185, 129, 0); }
278 | }
279 |
280 | .status-badge-text {
281 |     font-size: 14px;
282 |     font-weight: 600;
283 | }
284 |
285 | .status-badge-sub {
286 |     font-size: 11px;
287 |     color: var(--text-secondary);
288 |     margin-top: 2px;
289 | }
290 |
291 | .live-controls {
292 |     display: flex;
293 |     align-items: center;
294 |     gap: 12px;
295 | }
```

```
294 | .toggle-label {
295 |     display: flex;
296 |     align-items: center;
297 |     gap: 6px;
298 |     font-size: 12px;
299 |     color: var(--text-secondary);
300 |     cursor: pointer;
301 | }
302 |
303 | .toggle-label input {
304 |     cursor: pointer;
305 | }
306 |
307 | /* Current KPI Grid */
308 | .current-kpi-grid {
309 |     display: grid;
310 |     grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
311 |     gap: 14px;
312 |     margin-bottom: 20px;
313 | }
314 |
315 | .current-kpi-card {
316 |     background: var(--bg-card);
317 |     border: 1px solid var(--border-color);
318 |     border-radius: var(--radius-md);
319 |     padding: 14px 16px;
320 | }
321 |
322 | .current-kpi-header {
323 |     display: flex;
324 |     justify-content: space-between;
325 |     align-items: center;
326 | }
327 |
328 | .current-kpi-icon {
329 |     font-size: 18px;
330 | }
331 |
```

```
332 | .mini-meter-bg {
333 |     width: 100%;
334 |     height: 4px;
335 |     background: rgba(255, 255, 255, 0.1);
336 |     border-radius: 2px;
337 |     margin-top: 8px;
338 |     overflow: hidden;
339 | }
340 |
341 | .mini-meter-fill {
342 |     height: 100%;
343 |     background: var(--accent-blue);
344 |     transition: width 0.3s ease;
345 | }
346 |
347 | /* Charts Grid */
348 | .charts-grid {
349 |     display: grid;
350 |     grid-template-columns: repeat(auto-fit, minmax(440px, 1fr));
351 |     gap: 20px;
352 | }
353 |
354 | .chart-card {
355 |     background: var(--bg-card);
356 |     border: 1px solid var(--border-color);
357 |     border-radius: var(--radius-lg);
358 |     padding: 18px 20px;
359 |     box-shadow: var(--shadow-sm);
360 | }
361 |
362 | .chart-header {
363 |     margin-bottom: 14px;
364 | }
365 |
366 | .chart-title {
367 |     font-size: 15px;
368 |     font-weight: 600;
369 |     color: var(--text-primary);
```

```
370 }
371
372 .chart-canvas-wrapper {
373     position: relative;
374     height: 280px;
375     width: 100%;
376 }
377
378 /* Split Grid */
379 .split-grid {
380     display: grid;
381     grid-template-columns: 1.2fr 1fr;
382     gap: 20px;
383     margin-top: 20px;
384 }
385
386 /* Tables */
387 .table-wrapper {
388     background: var(--bg-card);
389     border: 1px solid var(--border-color);
390     border-radius: var(--radius-lg);
391     overflow: hidden;
392     box-shadow: var(--shadow-sm);
393 }
394
395 .table-header-bar {
396     padding: 14px 20px;
397     border-bottom: 1px solid var(--border-color);
398     display: flex;
399     justify-content: space-between;
400     align-items: center;
401 }
402
403 table {
404     width: 100%;
405     border-collapse: collapse;
406     font-size: 13px;
407     text-align: left;
```

```
408 }
409
410 th, td {
411     padding: 10px 16px;
412     border-bottom: 1px solid var(--border-color);
413 }
414
415 th {
416     background: rgba(11, 15, 25, 0.4);
417     color: var(--text-secondary);
418     font-size: 11px;
419     text-transform: uppercase;
420     font-weight: 600;
421 }
422
423 tr:hover {
424     background: var(--bg-card-hover);
425 }
426
427 .pagination-bar {
428     display: flex;
429     justify-content: flex-end;
430     gap: 8px;
431     padding: 12px 20px;
432     border-top: 1px solid var(--border-color);
433 }
434
435 .search-input {
436     background: var(--bg-main);
437     border: 1px solid var(--border-color);
438     color: var(--text-primary);
439     padding: 6px 12px;
440     border-radius: var(--radius-md);
441     font-size: 13px;
442     outline: none;
443     min-width: 280px;
444 }
445
```



```
446 | .search-input:focus {
447 |   border-color: var(--accent-blue);
448 | }
449 |
450 | /* Hypotheses Grid */
451 | .hypotheses-grid {
452 |   display: grid;
453 |   grid-template-columns: repeat(auto-fit, minmax(380px, 1fr));
454 |   gap: 16px;
455 | }
456 |
457 | .hypothesis-card {
458 |   background: var(--bg-card);
459 |   border: 1px solid var(--border-color);
460 |   border-radius: var(--radius-lg);
461 |   padding: 18px 20px;
462 |   box-shadow: var(--shadow-sm);
463 | }
464 |
465 | .hyp-header {
466 |   display: flex;
467 |   justify-content: space-between;
468 |   align-items: center;
469 |   margin-bottom: 8px;
470 | }
471 |
472 | .hyp-status {
473 |   padding: 2px 8px;
474 |   border-radius: 4px;
475 |   font-size: 11px;
476 |   font-weight: 700;
477 |   text-transform: uppercase;
478 | }
479 |
480 | .status-confirmed {
481 |   background: rgba(16, 185, 129, 0.2);
482 |   color: var(--accent-green);
483 | }
```

```
484 |
485 | .status-rejected {
486 |   background: rgba(239, 68, 68, 0.2);
487 |   color: var(--accent-red);
488 | }
489 |
490 | /* Correlations */
491 | .correlation-list {
492 |   display: flex;
493 |   flex-direction: column;
494 |   gap: 10px;
495 |   margin-top: 10px;
496 | }
497 |
498 | .corr-item {
499 |   display: flex;
500 |   justify-content: space-between;
501 |   align-items: center;
502 |   padding: 10px 14px;
503 |   background: var(--bg-main);
504 |   border: 1px solid var(--border-color);
505 |   border-radius: var(--radius-md);
506 |   font-size: 13px;
507 | }
508 |
509 | .corr-coeff {
510 |   font-weight: 700;
511 |   font-size: 14px;
512 |   padding: 2px 8px;
513 |   border-radius: 4px;
514 | }
515 |
516 | .corr-strong {
517 |   color: var(--accent-amber);
518 |   background: rgba(245, 158, 11, 0.15);
519 | }
520 |
521 | .corr-med {
```

```
522 | color: var(--accent-blue);
523 | background: rgba(56, 189, 248, 0.15);
524 | }
525 |
526 | /* Activity Feed */
527 | .activity-feed-list {
528 |   list-style: none;
529 |   display: flex;
530 |   flex-direction: column;
531 |   gap: 8px;
532 |   max-height: 380px;
533 |   overflow-y: auto;
534 |   padding-right: 6px;
535 | }
536 |
537 | .activity-feed-item {
538 |   display: flex;
539 |   justify-content: space-between;
540 |   align-items: flex-start;
541 |   padding: 10px 12px;
542 |   background: rgba(11, 15, 25, 0.6);
543 |   border: 1px solid var(--border-color);
544 |   border-radius: var(--radius-md);
545 |   font-size: 12px;
546 | }
547 |
548 | .activity-feed-item:hover {
549 |   background: var(--bg-card-hover);
550 | }
551 |
552 | .activity-feed-left {
553 |   display: flex;
554 |   flex-direction: column;
555 |   gap: 2px;
556 | }
557 |
558 | .activity-tag {
559 |   display: inline-block;
```

```
560 | padding: 2px 6px;
561 | border-radius: 4px;
562 | font-size: 10px;
563 | font-weight: 600;
564 | text-transform: uppercase;
565 | }
566 |
567 | .activity-tag-device {
568 |   background: rgba(56, 189, 248, 0.15);
569 |   color: var(--accent-blue);
570 | }
571 |
572 | .activity-tag-system {
573 |   background: rgba(168, 85, 247, 0.15);
574 |   color: var(--accent-purple);
575 | }
576 |
577 | /* -----
   | -- */
578 | /* AI Response Presentation Card & Dynamic Visualization
   | */
579 | /* -----
   | -- */
580 | .ai-response-card {
581 |   background: linear-gradient(180deg, #162038 0%, #101728 100%
   | );
582 |   border: 1px solid #2a3d66;
583 |   border-radius: var(--radius-lg);
584 |   padding: 22px 26px;
585 |   box-shadow: var(--shadow-lg);
586 |   margin-top: 10px;
587 |   margin-bottom: 20px;
588 |   position: relative;
589 |   animation: fadeInDown 0.3s ease;
590 | }
591 |
592 | @keyframes fadeInDown {
593 |   from { opacity: 0; transform: translateY(-10px); }
```

```
594 |   to { opacity: 1; transform: translateY(0); }
595 | }
596 |
597 | .ai-response-header {
598 |   display: flex;
599 |   justify-content: space-between;
600 |   align-items: center;
601 |   border-bottom: 1px solid #233355;
602 |   padding-bottom: 14px;
603 |   margin-bottom: 16px;
604 | }
605 |
606 | .ai-response-title-group {
607 |   display: flex;
608 |   align-items: center;
609 |   gap: 12px;
610 | }
611 |
612 | .ai-badge {
613 |   background: rgba(56, 189, 248, 0.15);
614 |   color: var(--accent-blue);
615 |   border: 1px solid rgba(56, 189, 248, 0.3);
616 |   font-size: 11px;
617 |   font-weight: 700;
618 |   padding: 4px 10px;
619 |   border-radius: 20px;
620 |   text-transform: uppercase;
621 |   letter-spacing: 0.5px;
622 | }
623 |
624 | .ai-response-title {
625 |   font-size: 17px;
626 |   font-weight: 700;
627 |   color: var(--text-primary);
628 | }
629 |
630 | .btn-icon {
631 |   background: transparent;
```

```
632 | border: none;
633 | color: var(--text-secondary);
634 | font-size: 18px;
635 | cursor: pointer;
636 | padding: 4px 8px;
637 | border-radius: 6px;
638 | transition: all 0.2s;
639 | }
640 |
641 | .btn-icon:hover {
642 |   color: var(--accent-red);
643 |   background: rgba(239, 68, 68, 0.15);
644 | }
645 |
646 | .ai-response-content {
647 |   display: flex;
648 |   flex-direction: column;
649 |   gap: 16px;
650 | }
651 |
652 | .ai-chart-wrapper {
653 |   background: rgba(11, 15, 25, 0.7);
654 |   border: 1px solid var(--border-color);
655 |   border-radius: var(--radius-md);
656 |   padding: 16px 18px;
657 | }
658 |
659 | .ai-canvas-container {
660 |   position: relative;
661 |   height: 320px;
662 |   max-width: 800px;
663 |   margin: 0 auto;
664 | }
665 |
666 | .ai-metrics-row {
667 |   display: flex;
668 |   flex-wrap: wrap;
669 |   gap: 10px;
```

```
670 | }
671 |
672 | .ai-metric-pill {
673 |   background: rgba(11, 15, 25, 0.6);
674 |   border: 1px solid var(--border-color);
675 |   border-radius: var(--radius-md);
676 |   padding: 8px 14px;
677 |   font-size: 12px;
678 |   display: flex;
679 |   align-items: center;
680 |   gap: 6px;
681 | }
682 |
683 | .ai-metric-pill strong {
684 |   color: var(--accent-blue);
685 |   font-size: 14px;
686 | }
687 |
688 | .ai-reply-body {
689 |   font-size: 14px;
690 |   line-height: 1.6;
691 |   color: var(--text-primary);
692 | }
693 |
694 | .ai-reply-body h3 {
695 |   font-size: 16px;
696 |   margin-bottom: 8px;
697 |   color: var(--accent-blue);
698 | }
699 |
700 | .ai-reply-body h4 {
701 |   font-size: 14px;
702 |   margin: 12px 0 6px 0;
703 |   color: var(--text-primary);
704 | }
705 |
706 | .ai-reply-body p {
707 |   margin-bottom: 8px;
```

```
708 }
709
710 .ai-reply-body ul, .ai-reply-body ol {
711     padding-left: 20px;
712     margin-bottom: 10px;
713 }
714
715 .ai-reply-body li {
716     margin-bottom: 4px;
717 }
718
719 .ai-reply-body table {
720     margin: 10px 0;
721     border-radius: 6px;
722     overflow: hidden;
723 }
724
725 .ai-reply-body code {
726     background: rgba(0, 0, 0, 0.3);
727     padding: 2px 6px;
728     border-radius: 4px;
729     color: var(--accent-blue);
730     font-family: Consolas, Monaco, monospace;
731     font-size: 12px;
732 }
733
734 .ai-sql-details {
735     background: rgba(11, 15, 25, 0.8);
736     border: 1px solid var(--border-color);
737     border-radius: var(--radius-md);
738     padding: 10px 14px;
739     font-size: 12px;
740     cursor: pointer;
741 }
742
743 .ai-sql-details summary {
744     font-weight: 600;
745     color: var(--accent-amber);
```



```
746 | outline: none;
747 | }
748 |
749 | .ai-sql-code {
750 |     margin-top: 10px;
751 |     padding: 10px;
752 |     background: #070a12;
753 |     border-radius: 6px;
754 |     font-family: Consolas, Monaco, monospace;
755 |     font-size: 12px;
756 |     color: #38bdf8;
757 |     overflow-x: auto;
758 |     white-space: pre-wrap;
759 | }
760 |
761 | .ai-followup-row {
762 |     display: flex;
763 |     align-items: center;
764 |     flex-wrap: wrap;
765 |     gap: 8px;
766 |     margin-top: 6px;
767 | }
768 |
769 | .ai-chips-list {
770 |     display: flex;
771 |     flex-wrap: wrap;
772 |     gap: 8px;
773 | }
774 |
775 | /* -----
776 |    -- */
777 | /* Bottom Fixed AI Query Bar
778 |    */
779 | /* -----
780 |    -- */
781 |
782 | .bottom-ai-bar {
783 |     position: fixed;
784 |     bottom: 0;
```

```
781 | left: 0;
782 | right: 0;
783 | background: rgba(19, 27, 46, 0.95);
784 | backdrop-filter: blur(12px);
785 | -webkit-backdrop-filter: blur(12px);
786 | border-top: 1px solid #2a3a5e;
787 | padding: 14px 28px;
788 | z-index: 999;
789 | box-shadow: 0 -8px 24px rgba(0, 0, 0, 0.5);
790 | display: flex;
791 | flex-direction: column;
792 | gap: 10px;
793 | }
794 |
795 | .ai-quick-chips {
796 |   display: flex;
797 |   gap: 8px;
798 |   overflow-x: auto;
799 |   padding-bottom: 2px;
800 | }
801 |
802 | .prompt-chip {
803 |   background: rgba(11, 15, 25, 0.7);
804 |   border: 1px solid var(--border-color);
805 |   color: var(--text-secondary);
806 |   font-size: 12px;
807 |   font-weight: 500;
808 |   padding: 6px 12px;
809 |   border-radius: 20px;
810 |   cursor: pointer;
811 |   white-space: nowrap;
812 |   transition: all 0.2s ease;
813 | }
814 |
815 | .prompt-chip:hover {
816 |   background: rgba(56, 189, 248, 0.15);
817 |   border-color: var(--accent-blue);
818 |   color: var(--text-primary);
```

```
819 |   transform: translateY(-1px);
820 | }
821 |
822 | .ai-query-form {
823 |   display: flex;
824 |   gap: 12px;
825 |   max-width: 1400px;
826 |   width: 100%;
827 |   margin: 0 auto;
828 | }
829 |
830 | .ai-input-wrapper {
831 |   position: relative;
832 |   flex: 1;
833 |   display: flex;
834 |   align-items: center;
835 | }
836 |
837 | .ai-input-icon {
838 |   position: absolute;
839 |   left: 14px;
840 |   font-size: 16px;
841 |   pointer-events: none;
842 | }
843 |
844 | .ai-query-input {
845 |   width: 100%;
846 |   background: #0b0f19;
847 |   border: 1px solid #2a3a5e;
848 |   color: var(--text-primary);
849 |   padding: 12px 42px 12px 42px;
850 |   border-radius: 28px;
851 |   font-size: 14px;
852 |   outline: none;
853 |   box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.4);
854 |   transition: all 0.2s ease;
855 | }
856 |
```

```
857 | .ai-query-input:focus {
858 |   border-color: var(--accent-blue);
859 |   box-shadow: 0 0 0 3px rgba(56, 189, 248, 0.25);
860 |   background: #0e1424;
861 | }
862 |
863 | .btn-clear {
864 |   position: absolute;
865 |   right: 14px;
866 |   background: transparent;
867 |   border: none;
868 |   color: var(--text-secondary);
869 |   font-size: 14px;
870 |   cursor: pointer;
871 | }
872 |
873 | .btn-clear:hover {
874 |   color: var(--text-primary);
875 | }
876 |
877 | .btn-ai-send {
878 |   border-radius: 28px;
879 |   padding: 10px 24px;
880 |   font-size: 14px;
881 |   font-weight: 700;
882 |   display: flex;
883 |   align-items: center;
884 |   justify-content: center;
885 |   min-width: 130px;
886 | }
887 |
888 | .spinner {
889 |   width: 16px;
890 |   height: 16px;
891 |   border: 2px solid rgba(11, 15, 25, 0.3);
892 |   border-top-color: #0b0f19;
893 |   border-radius: 50%;
894 |   animation: spin 0.8s linear infinite;
```

```
895 | }
896 |
897 | @keyframes spin {
898 |   to { transform: rotate(360deg); }
899 | }
900 |
901 | @media (max-width: 768px) {
902 |   .ai-quick-chips {
903 |     display: none;
904 |   }
905 |   .app-header {
906 |     flex-direction: column;
907 |     gap: 12px;
908 |     align-items: flex-start;
909 |   }
910 |   .charts-grid {
911 |     grid-template-columns: 1fr;
912 |   }
913 | }
914 |
915 | /* Gemini Evaluations Grid & Cards */
916 | .gemini-evals-grid {
917 |   display: grid;
918 |   grid-template-columns: repeat(auto-fill, minmax(340px, 1fr))
919 |   ;
920 |   gap: 16px;
921 |   margin-top: 14px;
922 | }
923 | .gemini-card {
924 |   background: var(--bg-surface);
925 |   border: 1px solid var(--border-color);
926 |   border-radius: var(--radius-md);
927 |   padding: 16px;
928 |   display: flex;
929 |   flex-direction: column;
930 |   gap: 10px;
931 |   position: relative;
```

```
932 |     transition: transform 0.15s ease, border-color 0.15s ease;
933 | }
934 |
935 | .gemini-card:hover {
936 |     transform: translateY(-2px);
937 |     border-color: var(--accent-purple);
938 | }
939 |
940 | .gemini-card-header {
941 |     display: flex;
942 |     justify-content: space-between;
943 |     align-items: flex-start;
944 |     gap: 8px;
945 | }
946 |
947 | .gemini-card-title {
948 |     font-size: 15px;
949 |     font-weight: 700;
950 |     color: var(--text-primary);
951 | }
952 |
953 | .gemini-verdict-badge {
954 |     padding: 4px 10px;
955 |     border-radius: 12px;
956 |     font-size: 11px;
957 |     font-weight: 700;
958 |     text-transform: uppercase;
959 | }
960 |
961 | .gemini-verdict-normal {
962 |     background: rgba(16, 185, 129, 0.15);
963 |     color: var(--accent-green);
964 |     border: 1px solid rgba(16, 185, 129, 0.3);
965 | }
966 |
967 | .gemini-verdict-risk {
968 |     background: rgba(239, 68, 68, 0.15);
969 |     color: var(--accent-red);
```

```
970 | border: 1px solid rgba(239, 68, 68, 0.3);
971 | }
972 |
973 | .gemini-card-metrics {
974 |   display: flex;
975 |   gap: 10px;
976 |   flex-wrap: wrap;
977 |   font-size: 12px;
978 |   color: var(--text-secondary);
979 | }
980 |
981 | .gemini-metric-tag {
982 |   background: var(--bg-base);
983 |   padding: 2px 8px;
984 |   border-radius: 4px;
985 |   border: 1px solid var(--border-color);
986 | }
987 |
988 | .gemini-card-analysis {
989 |   font-size: 13px;
990 |   line-height: 1.45;
991 |   color: var(--text-primary);
992 |   background: rgba(15, 23, 42, 0.4);
993 |   padding: 10px;
994 |   border-radius: var(--radius-sm);
995 |   border-left: 3px solid var(--accent-blue);
996 | }
997 |
998 | .gemini-card-recs {
999 |   font-size: 12px;
1000 |   color: var(--text-secondary);
1001 |   padding-left: 16px;
1002 |   margin: 0;
1003 | }
1004 |
1005 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Init
5 # Description:
6 #   Пакет исследования логов телеметрии, визуализации и FastAPI
7 #   Web GUI.
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.telemetry_research
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-04 07:47:00
17 # =====
18
19 from __future__ import annotations
20 """Пакет исследования логов телеметрии, визуализации и FastAPI
21 Web GUI."""
22
23 from .analyzer import TelemetryResearcher
24 from .charts import TelemetryChartGenerator
25 from .extractor import TelemetryDataExtractor
26 from .models import (
27     AnomalyEvent,
28     ChartConfig,
29     ClientProgramResource,
30     ClientResourceSummary,
31     CorrelationMatrixItem,
```



```
31     DeepResearchReport,
32     DeviceEventSummary,
33     HypothesisResult,
34     MetricPoint,
35     MetricStats,
36     ResearchScenarioRequest,
37     TelemetryResearchReport,
38     TimeSeriesDataset,
39     UnknownProgramEvaluation,
40 )
41 from .client_resource_analyzer import ClientResourceAnalyzer
42 from .diagnostic_engine import DiagnosticEngine, SystemDiagnosticEngine
43 from .deep_diagnostics import DeepDiagnosticsEngine
44 from .incident_detector import IncidentDetector
45 from .reboot_analyzer import WindowsRebootAnalyzer
46 from .aggregator import TelemetryAggregator
47 from .grouped_telemetry import (
48     GroupedTelemetryBuilder,
49     GroupDiagnoseRequest,
50     GroupDiagnosticResult,
51     SynthesisRequest,
52     SynthesisDiagnosticResult,
53 )
54 from .compactor import TelemetryCompactor, compute_percentile
55 from .hardware_history_manager import HardwareHistoryManager
56 from .hardware_auditor import HardwareAuditor
57 from .audit_startup_checker import AuditStartupChecker, StartupAuditResult, run_startup_audit
58
59 __all__ = [
60     "TelemetryResearcher",
61     "TelemetryChartGenerator",
62     "TelemetryDataExtractor",
63     "TelemetryResearchReport",
64     "DeepResearchReport",
65     "ChartConfig",
66     "MetricPoint",
```

```
67 | "MetricStats",
68 | "AnomalyEvent",
69 | "DeviceEventSummary",
70 | "TimeSeriesDataset",
71 | "CorrelationMatrixItem",
72 | "HypothesisResult",
73 | "ResearchScenarioRequest",
74 | "DiagnosticEngine",
75 | "SystemDiagnosticEngine",
76 | "DeepDiagnosticsEngine",
77 | "IncidentDetector",
78 | "WindowsRebootAnalyzer",
79 | "TelemetryAggregator",
80 | "GroupedTelemetryBuilder",
81 | "GroupDiagnoseRequest",
82 | "GroupDiagnosticResult",
83 | "SynthesisRequest",
84 | "SynthesisDiagnosticResult",
85 | "TelemetryCompactor",
86 | "compute_percentile",
87 | "HardwareHistoryManager",
88 | "HardwareAuditor",
89 | "AuditStartupChecker",
90 | "StartupAuditResult",
91 | "run_startup_audit",
92 | "ClientResourceAnalyzer",
93 | "ClientProgramResource",
94 | "ClientResourceSummary",
95 | "UnknownProgramEvaluation",
96 | ]
97 |
```